

순차적 SMT 구조 및 그룹화 방안에
관한 연구

연세대학교 대학원
전기전자공학과
문 병 인

순차적 SMT 구조 및 그룹화 방안에
관한 연구

지도 이 용 석 교수

이 논문을 박사 학위논문으로 제출함

2001년 12월 일

연세대학교 대학원
전기전자공학과
문 병 인

문병인의 박사 학위논문을 인준함

심사위원 이 용 석 인

심사위원 이 문 기 인

심사위원 이 광 엽 인

심사위원 강 성 호 인

심사위원 최 린 인

연세대학교 대학원

2001년 12월 일

감사의 글

지난 7년의 대학원 생활 동안 아낌없는 격려와 인자하신 모습으로 모든 어려움을 헤쳐나갈 수 있는 지혜와 용기를 길러주신 이용석 교수님께 진심으로 감사드립니다. 교수님께서 보여주신 공학인의 참된 모습은 저의 일생에 큰 지침이 될 것입니다. 부족한 저의 논문에 세심한 지도와 많은 관심을 보여주신 이문기 교수님, 이광엽 교수님, 강성호 교수님, 최린 교수님께 깊이 감사드립니다. 또한 학부와 대학원 과정동안 가르침을 주신 김봉열 교수님, 박규태 교수님, 이상배 교수님, 차일환 교수님, 강창언 교수님, 박민용 교수님, 윤대회 교수님, 김재희 교수님, 이재용 교수님, 김재석 교수님, 홍대식 교수님, 송홍엽 교수님, 이철희 교수님, 강문기 교수님, 최우영 교수님, 이충용 교수님, 한건희 교수님, 김영용 교수님께 감사드립니다.

학문적인 열의와 안목을 가르쳐주신 최병운, 서광수, 남승현, 이승호, 손승일, 이보나, 정갑중, 정한힘, 백종섭 선배님께 감사의 마음을 전합니다. 그리고 연구실 생활에서 많은 도움을 주신 오승호, 정태식, 김근희, 최한준, 김홍규, 형상준, 이회진, 임의철, 김경환, 이종익, 곽승호, 양훈모 선배님께도 감사드립니다. 형님처럼 포근한 유동열 선배님, 대학원 생활의 길잡이가 되어 주신 이용환 선배님, 엄한 선배로서 연구실을 이끌어주신 안상준 선배님, 많은 대화를 나누지 못해 아쉬웠던 멋있는 이태영 선배님, 항상 열심히 올바른 길을 가는 좋은 남자 안종근 선배님, 대학원 신입 시절 힘들어하는 저를 다독여 주신 이원 선배님께 진심으로 감사드립니다. 학부부터 대학원까지 11년간 같이 생활하면서 동고동락했던 졸업 동기 문상국, 이승우 님과 연구실에서 기쁨과 슬픔을 함께 하며 서로에게 격려와 자극이 되었던 권경환, 권성우, 이동환, 오홍철, 허상경, 박종환, 홍종욱, 최대봉 님께 감사드립니다. 부족한 저를 선배로 잘 따라준 권호경, 조영록, 고재명, 이성연, 김영준, 정재원, 최근일 후배님께 감사드립니다. 대학원 생활의 즐거움과 괴로움을 같이 한 김유근, 조용상, 이명순, 이재욱, 최창순, 이성주, 황상운, 조용권 님께도 감사드립니다. 영원한 입생이 이용주, 항상 열심히 김도균, 밝은 웃음의 소유자 김상철, 무

슨 일을 시켜도 믿을 수 있는 구군서, 썰렁한 유머가 특기인 장서봉 후배님과 귀여운 매력당이 김승철 님께 감사드리면서 연구실이 더욱 발전할 수 있도록 노력해 주기를 바랍니다. 지난 6년 동안 많은 기쁨과 자극을 주었던 믿음직한 정우경 후배님께 감사드리면서 하루속히 좋은 결실이 있기를 바랍니다. 저의 졸업 논문을 위한 연구를 전력으로 도와준 김문경, 홍인표, 박재민, 이승필, 김재익 후배님께 특별한 감사의 마음을 전하면서 아울러 김문경, 홍인표 후배님의 향후 연구가 훌륭한 결실을 맺기 바랍니다. 그리고 연구실 생활이 원만하게 이루어질 수 있게 도와주신 장수진 님께 감사드립니다.

모든 생활에서 기쁨과 슬픔을 나누는 오랜 친구들인 재영, 우진, 근무, 상진, 상학, 완수, 충남, 범이와 연락이 안되지만 우리 친구들이 그리워하는 영훈에게 고마운 마음을 전합니다. 그리고 이런 친구들의 반려자이자 이제는 저의 친구들이기도 한, 은영, 혜숙, 유미, 윤희, 여진에게도 진심으로 감사의 마음을 전하면서 아울러 아직도 혼자인 친구들은 좋은 배필을 만나기를 기원합니다.

오늘이 있기까지 한없는 사랑으로 보살피 주신 저의 소중한 아버지, 어머니의 은혜에 감사드리며 이 작은 결실을 바칩니다. 그리고 따뜻한 사랑과 격려를 아끼지 않으신 장인, 장모님께 깊은 감사를 드립니다. 어려울 때마다 힘이 되어준 형님, 형수님, 큰 누님, 큰 매형, 작은 누님, 작은 매형, 그리고 소중하고 귀여운 족하들 다현, 서현, 재서, 시윤, 하준, 하영, 처제 계영, 처남 인렬과 이 작은 기쁨을 함께 하고 싶습니다.

마지막으로 결혼한 이후로 바쁘다는 이유로 학교에서만 생활하고 임신 후에 그리고 출산 후에 옆에 있어 주지 못한 남편을 큰 사랑으로 내조해 준 고마운 아내 양진 그리고 제 생애 가장 큰 기쁨인 아들 수열과 이 작은 결실을 나누고자 합니다.

2001년 12월

수열 아빠 **정병인** 올림

차 례

그 림 차 례	iii
표 차 례	vi
국 문 요 약	vii
제 1 장 서 론	1
제 2 장 순차적 수퍼스칼라 구조	10
2-1절 전체적인 순차적 수퍼스칼라 구조	10
2-2절 순차적 수퍼스칼라 구조의 파이프라인.....	14
2-3절 명령어 페치 구조	19
2-4절 디코드 유닛	34
2-5절 레지스터 파일과 스코어보드 어레이	36
2-6절 이슈 유닛	42
2-7절 기능 유닛과 데이터 메모리 액세스	51
2-8절 분기 및 예외의 처리	58
제 3 장 순차적 SMT 구조	64
3-1절 전체적인 순차적 SMT 구조	64
3-2절 세부적인 순차적 SMT 구조	69
제 4 장 그룹화 방안의 적용	82
4-1절 그룹화 방안의 정의	82
4-2절 그룹화 1의 적용	88
4-3절 그룹화 2의 적용	90

4-4절 그룹화 3의 적용	92
제 5 장 시뮬레이션 환경	94
5-1절 시뮬레이터	94
5-2절 시뮬레이터의 구조 변수들	103
5-3절 입력 프로그램들과 시뮬레이션 방법	109
제 6 장 성능 평가 및 결과	116
6-1절 성능 평가 방법	116
6-2절 다양한 SMT 구성에 대한 성능 측정	118
6-3절 최적의 SMT 구조 제안	150
제 7 장 기존 SMT 구조와의 비교	155
제 8 장 결 론	161
참 고 문 헌	165
부 록	177
영 문 요 약	200

그 립 차 례

그림 1-1 순차적 실행과 다중스레디드 실행의 비교	3
그림 1-2 CMT, FMT 및 SMT의 명령어 이슈 비교	4
그림 1-3 수퍼스칼라, CMP 및 SMT 구조의 명령어 이슈 비교	6
그림 2-1 전체적인 순차적 수퍼스칼라 구조의 블록 다이어그램	11
그림 2-2 순차적 수퍼스칼라 구조의 파이프라인	15
그림 2-3 기능 유닛들의 파이프라인 단계별 동작	17
그림 2-4 순차적 수퍼스칼라 구조의 명령어 페치	19
그림 2-5 페치 유닛의 PC 갱신	22
그림 2-6 BTB의 구조	24
그림 2-7 방향 카운터 변경의 흐름도	26
그림 2-8 BHB의 구조	27
그림 2-9 ITLB의 구조	28
그림 2-10 명령어 페치 큐의 구조	32
그림 2-11 디코드 유닛의 명령어 디코딩	35
그림 2-12 레지스터 파일 구조	38
그림 2-13 스코어보드 어레이의 구조	40
그림 2-14 명령어 이슈 큐의 구조	43
그림 2-15 잘못된 순서로 레지스터에 결과를 쓰는 예	45
그림 2-16 다중 사이클 명령어로 인한 후속 명령어의 이슈 지연	46
그림 2-17 스코어보드 엔트리의 변경을 보여주는 예	49
그림 2-18 파이프라인의 각 단계에서의 ALU 동작 예	52
그림 2-19 다수의 액세스를 동시에 수행하는 데이터 메모리 액세스 구조	54
그림 2-20 데이터 메모리 액세스 과정	57
그림 2-21 분기 예측 오류의 검사 과정	59

그림 3-1 전체적인 순차적 SMT 구조의 블록 다이어그램	65
그림 3-2 순차적 SMT 구조의 파이프라인	66
그림 3-3 순차적 SMT 구조의 명령어 폐치	70
그림 3-4 다수의 BHB들 (스레드당 한 개의 BHB)	71
그림 3-5 단일 스레드의 플러쉬로 인한 명령어 폐치 큐中间的 빈 엔트리들	72
그림 3-6 유효한 엔트리들만을 명령어 이슈 큐에 저장하는 모습	75
그림 3-7 명령어 이슈 큐의 압축	76
그림 3-8 순차적 SMT 구조의 레지스터 파일	78
그림 3-9 순차적 SMT 구조의 스코어보드 어레이	79
그림 4-1 그룹화를 적용하지 않은 레지스터 읽기	84
그림 4-2 그룹화에 의한 레지스터 읽기의 간략화	85
그림 4-3 순차적 SMT 구조에서의 세 가지 그룹화	87
그림 4-4 그룹화 1의 예	89
그림 4-5 그룹화 2의 예	91
그림 4-6 그룹화 3의 예	93
그림 5-1 명령어-기본 시뮬레이션의 수행 과정	96
그림 5-2 사이클-기본 시뮬레이션의 수행 과정	97
그림 5-3 시뮬레이터 본체의 파이프라인 기술	102
그림 5-4 입력 프로그램들을 각각의 스레드에 할당하는 모습	114
그림 5-5 시뮬레이션 과정	115
그림 6-1 스레드 수에 따른 2-issue 구조의 성능 변화	121
그림 6-2 스레드 수에 따른 4-issue 구조의 성능 변화	121
그림 6-3 스레드 수에 따른 8-issue 구조의 성능 변화	122
그림 6-4 스레드 수에 따른 16-issue 구조의 성능 변화	122
그림 6-5 스레드 수에 따른 4-issue 구조의 캐쉬 미스율 변화	123
그림 6-6 스레드 수에 따른 8-issue 구조의 캐쉬 미스율 변화	123
그림 6-7 캐쉬 way 수에 따른 4-issue, 4-thread 구조의 성능 변화	125
그림 6-8 캐쉬 way 수에 따른 4-issue, 4-thread 구조의 캐쉬 미스율	125

그림 6-9 캐쉬 way 수에 따른 4-issue, 8-thread 구조의 성능 변화	126
그림 6-10 캐쉬 way 수에 따른 4-issue, 8-thread 구조의 캐쉬 미스율	126
그림 6-11 캐쉬 way 수에 따른 4-issue, 16-thread 구조의 성능 변화	127
그림 6-12 캐쉬 way 수에 따른 4-issue, 16-thread 구조의 캐쉬 미스율	127
그림 6-13 캐쉬 way 수에 따른 8-issue, 4-thread 구조의 성능 변화	128
그림 6-14 캐쉬 way 수에 따른 8-issue, 4-thread 구조의 캐쉬 미스율	128
그림 6-15 캐쉬 way 수에 따른 8-issue, 8-thread 구조의 성능 변화	129
그림 6-16 캐쉬 way 수에 따른 8-issue, 8-thread 구조의 캐쉬 미스율.....	129
그림 6-17 캐쉬 way 수에 따른 8-issue, 16-thread 구조의 성능 변화	130
그림 6-18 캐쉬 way 수에 따른 8-issue, 16-thread 구조의 캐쉬 미스율	130
그림 6-19 캐쉬 크기에 따른 4-issue, 4-thread 구조의 성능 변화	132
그림 6-20 캐쉬 크기에 따른 4-issue, 4-thread 구조의 캐쉬 미스율 변화	132
그림 6-21 캐쉬 크기에 따른 8-issue, 8-thread 구조의 성능 변화	133
그림 6-22 캐쉬 크기에 따른 8-issue, 8-thread 구조의 캐쉬 미스율 변화	133
그림 6-23 스레드 수에 따른 4-issue 구조의 분기 예측 오류 빈도	134
그림 6-24 스레드 수에 따른 8-issue 구조의 분기 예측 오류 빈도	134
그림 6-25 BTB 엔트리 수에 따른 4-thread 구조의 성능 변화	135
그림 6-26 BTB 엔트리 수에 따른 4-thread 구조의 분기 예측 오류 빈도	136
그림 6-27 BTB 엔트리 수에 따른 8-thread 구조의 성능 변화	136
그림 6-28 BTB 엔트리 수에 따른 8-thread 구조의 분기 예측 오류 빈도	137
그림 6-29 명령어 캐쉬 포트 수에 따른 4-issue, 4-thread 구조의 성능	138
그림 6-30 명령어 캐쉬 포트 수에 따른 8-issue, 8-thread 구조의 성능	138
그림 6-31 명령어 페치 큐 엔트리 수에 따른 4-issue, 4-thread 구조의 성능 ...	142
그림 6-32 명령어 페치 큐 엔트리 수에 따른 8-issue, 8-thread 구조의 성능 ...	142
그림 6-33 명령어 이슈 큐 엔트리 수에 따른 4-issue, 4-thread 구조의 성능 ...	143
그림 6-34 명령어 이슈 큐 엔트리 수에 따른 8-issue, 8-thread 구조의 성능 ...	143

표 차 례

표 2-1 PC의 정렬 조건과 순차적 다음 PC 값	23
표 2-2 ITLB 엔트리의 영역들	29
표 2-3 지원되는 페이지의 종류와 페이지 크기	30
표 2-4 프로세서 모드들과 사용 가능한 레지스터들	37
표 2-5 명령이 이슈 기준	48
표 2-6 예외의 종류와 백터 주소	62
표 4-1 그룹화의 3가지 종류와 각 그룹화와 관련된 자원들	86
표 5-1 시뮬레이터의 구성	99
표 5-2 구조 변수들	104
표 5-3 페치 및 이슈 우선권 정책의 종류	108
표 5-4 성능 평가를 위한 입력 프로그램들	109
표 5-5 SWI 시스템 호출들	111
표 6-1 성능 평가를 위한 구조 변수들의 값	119
표 6-2 페치 우선권 정책에 따른 성능 변화	139
표 6-3 이슈 우선권 정책에 따른 성능 변화	140
표 6-4 기능 유닛의 수에 따른 성능 변화	144
표 6-5 그룹화에 따른 8-issue, 8-thread 구조의 성능 변화	148
표 6-6 제안된 내장형 순차적 SMT 구조	151
표 6-7 제안된 고성능 순차적 SMT 구조	153
표 6-8 제안된 고성능 SMT 구조를 사용한 네트워크 프로세서의 성능	154
표 7-1 기존의 SMT 구조들과 본 논문의 제안된 SMT 구조들의 비교	160

국 문 요 약

순차적 SMT 구조와 그룹화 방안에 관한 연구

본 논문은 효율적인 SMT(Simultaneous MultiThreading) 구조를 제안한다. SMT 구조는 ILP(Instruction Level Parallelism)와 TLP(Thread Level Parallelism)를 이용하여 명령어 처리 능력을 향상시킨다. 제안된 SMT 구조는 같은 스레드에 속하는 명령어들을 프로그램의 순서대로 이슈하고 실행한다. 이러한 정책은 비순차적으로 명령어들을 이슈하고 실행하는 구조와 비교해서 본 구조의 설계 복잡도와 하드웨어 비용을 줄일 수 있게 한다. 그런데, 서로 다른 스레드에 속하는 명령어들에 대해서는 정해진 순서가 존재하지 않기 때문에, 이러한 명령어들에 대해서는 이슈 및 실행 순서를 명령어 페치 순서와 다르게 할 수 있다. 본 논문의 SMT 구조는 ILP 및 TLP를 활용하고 자원들을 스레드들 사이에 동적으로 공유시키면서 다수의 스레드로부터의 명령어들을 동시에 기능 유닛들로 이슈하고 실행한다. 이러한 병렬 수행으로 SMT 구조는 슈퍼스칼라 구조와 비교해서 추가적인 하드웨어 비용은 최소화하면서 성능을 매우 많이 높일 수 있다.

SMT 구조는 프로세서의 활용도 및 성능을 향상시키는 놀라운 장점을 가지고 있지만 설계가 너무 복잡하다는 단점이 있다. 이러한 이유로 본 논문은 그룹화 방안을 제시한다. 그룹화가 적용되지 않은 SMT 구조에서는 모든 스레드들이 대부분의 프로세서 자원들을 동적으로 공유한다. 반면에, 그룹화가 적용된 SMT 구조에서는 설계의 복잡도를 줄일 목적으로, 자원들과 스레드들이 몇 개의 그룹으로 나뉘고 자원들과 같은 그룹에 속하는 스레드들만이 해당 자원들을 공유한다.

본 논문에서 SMT 구조를 상세히 모델링한 시뮬레이터가 구현된다. 이 시뮬레이터는 실행-구동 및 사이클-기본 방식을 사용하고 C 언어로 쓰여진다. 다양한 구성의 SMT 프로세서들을 시뮬레이션하기 위해서 시뮬레이터는 다양한 구조 변수들을 가지고 있다. 시뮬레이터는 32비트 RISC(Reduced Instruction Set Computer) 명령어 세트 구조를 채택하였다. 그리고 SPEC CPU2000 벤치마크 프

로그래밍과 ADS(ARM Developer Suite)의 예제 프로그램들이 시뮬레이션을 위한 입력 프로그램들로 사용하였다.

시뮬레이션 결과에 의해서, 지원되는 스레드의 수가 단일 사이클에 이슈될 수 있는 명령어의 수와 같을 때 SMT 프로세서는 가장 효율적임을 알 수 있다. 그리고 스레드간의 충돌 미스를 최소화하기 위해서 캐쉬의 way 수가 지원되는 스레드 수의 반 이상으로 설정되어야 하고, 적당한 그룹화 방안을 적용할 경우 성능 저하는 거의 없으면서 설계 복잡도를 상당히 줄일 수 있음을 알 수 있다.

본 논문은 시뮬레이션 결과를 사용하여 내장형 응용을 위한 SMT 구조와 고성능 응용을 위한 SMT 구조를 제안한다. 제안된 두 구조는 적은 하드웨어 비용 추가로 슈퍼스칼라 구조와 비교해서 3배 이상의 IPC(Instructions Per Cycle)를 보인다. 또한 고성능 응용을 위해 제안된 SMT 구조를 채택한 네트워크 프로세서는 이미 출시된 다른 네트워크 프로세서들과 비교해서 충분한 경쟁력을 가질 것으로 예상된다.

핵심되는 말 : 다중스레딩, SMT, 슈퍼스칼라, ILP, TLP, 동적 자원 공유, 순차적 이슈 및 실행, 그룹화

제 1 장 서 론

최근 10여년 동안 마이크로프로세서(microprocessor)는 성능이 크게 발전하였으며, 거의 모든 전자 기기의 가장 핵심적인 역할을 하고 있다. 따라서 마이크로프로세서의 성능 향상은 이를 채택한 전자 기기의 성능을 좌우할 뿐 아니라, 새로운 소프트웨어와 전자 기기의 개발에 상당한 파급 효과를 지니고 있다. 많은 컴퓨터들이 인터넷과 네트워크를 통해 연결되어 대량의 정보를 고속으로 교환, 처리 및 저장한다. OS(Operating System), 응용 프로그램 및 데이터의 대용량화와 고속의 데이터 처리 요구로 인해 개인용 및 산업용 컴퓨터들은 계속해서 더 높은 성능의 마이크로프로세서를 탑재하도록 요구되고 있다. 또한 최근 고속 인터넷 통신망의 발전과 사용자의 요구는 네트워크 데이터를 더 빠르고 융통성 있게 처리할 수 있는 프로세서를 요구하게 되었고, 이 요구에 부응하여 네트워크 프로세서(network processor)^[1]에 대한 연구가 활발히 이루어지고 있다^[2-11]. 이와 같은 마이크로프로세서의 성능 향상에 대한 꾸준한 요구로 인해 성능 향상을 위한 다각적인 연구 개발이 이루어져 왔으며, 현재도 많은 연구소와 주요 회사들에서 경쟁적인 연구 개발을 수행하고 있다.

마이크로프로세서의 성능 향상은 우선 이를 제작하기 위한 반도체 공정을 향상시킴으로써 이루어진다. 즉, 더욱 미세한 공정을 사용함으로써 마이크로프로세서의 동작 주파수를 높게 하여 더 많은 명령어(instruction)들을 단위 시간 내에 처리하도록 할 수 있다. 반도체 공정의 발전은 매년 거의 일정한 속도로 향상되고 있으나, 이것만으로는 마이크로프로세서의 성능 향상에 대한 요구를 충족시키기 힘들다^[12]. 따라서 다른 각도에서의 성능 향상을 위한 방법인 마이크로프로세서 구조의 발전이 꾸준히 추구되어 왔다. 마이크로프로세서는 초기의 CISC(Complex Instruction Set Computer)^[13] 방식으로부터 명령어를 단순화하고 파이프라인(pipeline) 개념을 도입한 RISC(Reduced Instruction Set Computer)^[13-16] 방식으로 발전했다. 그리고 단일 사이클에 여러 명령어들을 동시에 수행할 수 있는 슈퍼스

칼라(superscalar)^{[13][17-19]} 방식이 상용화되어 현재 널리 사용되고 있으며, 다수의 동작(operation)을 단일 명령어군으로 하여 실행하도록 하는 VLIW(Very Long Instruction Word)^{[20][21]} 방식도 연구되어 왔다. 이 외에 서버급 컴퓨터를 위한 다수의 프로세서 노드(processor node)를 가지는 구조^[22]에 대한 연구도 활발히 이루어지고 있다.

수퍼스칼라 구조의 마이크로프로세서는 최근 10여년 동안 꾸준히 인기를 누리고 있고, 현재 마이크로프로세서의 주류를 형성하고 있다. 그러나 수퍼스칼라 마이크로프로세서는 구조적인 측면에서 한계에 도달했다는 것이 많은 마이크로프로세서 설계자들의 공통된 의견이다. 비순차적(out-of-order) 명령어 실행^[23-26] 등 다양한 연구를 바탕으로 수퍼스칼라 마이크로프로세서의 성능을 높이기 위한 연구 개발이 이루어졌지만, 일반적인 프로그램의 ILP(Instruction Level Parallelism)^[27]의 한계로 인해 수퍼스칼라 마이크로프로세서가 2 IPC(Instruction Per Cycle) 이상의 성능을 보이기 힘들다는 것이 현실이다^{[28][29]}. 이러한 성능의 한계는 수퍼스칼라 마이크로프로세서의 이슈 폭(issue width)을 넓히고 기능 유닛들의 수를 늘리더라도 나타난다. 최근 반도체 공정의 발전으로 단일 칩에 1억 개 이상의 트랜지스터(transistor)들을 집적할 수 있게 되었고^[12], 이 수는 계속 증가하고 있다. 단일 칩에 엄청난 수의 트랜지스터 수를 집적할 수 있는 공정은 수퍼스칼라 마이크로프로세서 이상의 복잡도(complexity)를 지닌 마이크로프로세서의 구현을 가능하게 했다. 그 결과 마이크로프로세서 설계자들은 다음 세대의 마이크로프로세서 구조를 연구하게 되었고, 많은 마이크로프로세서 구조 설계 기술자들은 다중스레딩(multithreading)^{[30][31]} 구조를 제시하고 있다^[32].

다중스레딩 구조는 다수의 독립적인 스레드(thread)들이 프로세서의 자원을 공유하면서 수행되는 구조를 가진다. 구체적으로, 다중스레딩 구조에서는 어떤 스레드로부터 명령어들을 수행하다가 ILP의 부족으로 그 스레드로부터 명령어 수행이 이루어질 수 없을 때 다른 스레드의 명령어들을 수행함으로써, 프로세서 자원의 활용도(utilization)를 높이고 전체적인 명령어 처리 능력(throughput)을 향상시킨다. 즉, 개별적인 스레드의 ILP 한계를 스레드간의 TLP(Thread Level Parallelism)^[33]로 극복하는 것이 다중스레딩 구조의 목적이다.

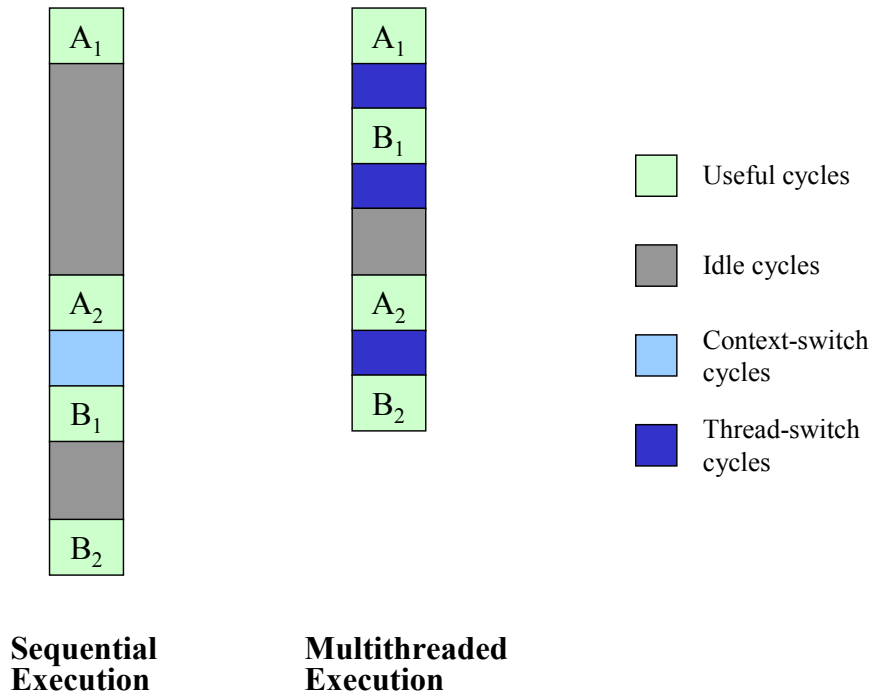


그림 1-1 순차적 실행과 다중스레디드 실행의 비교

Fig. 1-1 Comparison of sequential and multithreaded execution

그림 1-1은 보통의 프로세서(conventional processor) 및 다중스레디드 프로세서(multithreaded processor)에서 스레드 A와 B가 수행되는 모습을 보여준다. 보통의 프로세서에서는 스레드 A가 A₁만큼 수행된 후에 데이터 의존성(data dependency)^[13] 같은 문제로 스레드 A를 수행할 수 없는 경우에, 스레드 A를 수행할 수 있을 때까지 기다린 후에 스레드 A의 나머지 부분(A₂)을 수행하고 스레드 B도 같은 방식으로 수행한다. 반면에 다중스레디드 프로세서는 스레드 A를 수

행할 수 없을 때 스레드 B로의 스레드 스위치(thread switch)^[32] 후에 스레드 B를 수행하고, 스레드 B를 수행할 수 없을 때에는 스레드 A로 다시 스위치하여 주어진 작업을 수행해 나간다. 이와 같은 방식으로 다중스레디드 프로세서에서는 프로세서의 활용도가 높으며, 결과적으로 프로세서의 처리 능력을 향상시킬 수 있다. 그림 1-1에서 두 개의 스레드 A와 B가 다중스레디드 프로세서에서 더 빨리 처리되는 것을 볼 수 있다.

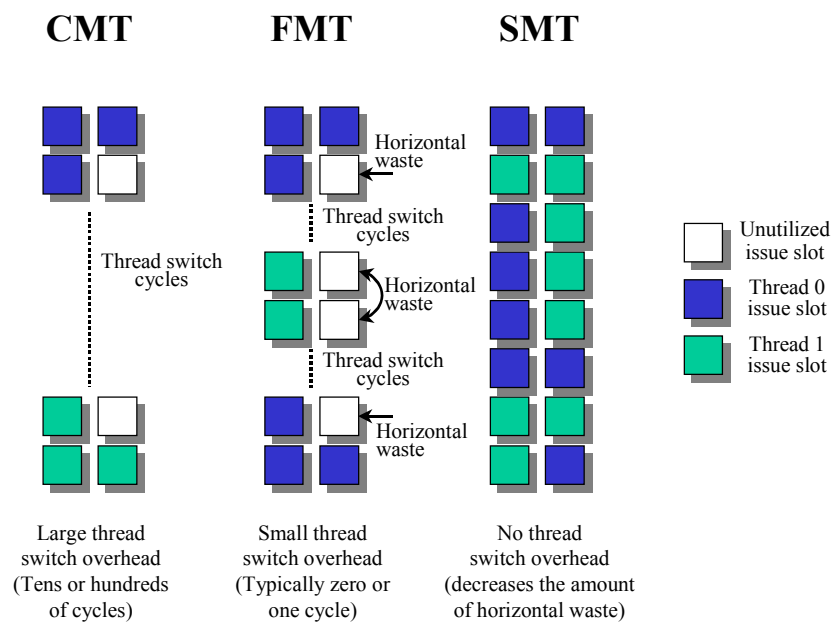


그림 1-2 CMT, FMT 및 SMT의 명령어 이슈 비교

Fig. 1-2 Comparison of the instruction issue of CMT, FMT and SMT

다중스레딩 구조는 CMT(Coarse MultiThreading)^[32], FMT(Fine MultiThreading)^{[32][34]} 및 SMT(Simultaneous MultiThreading)^[34-37]의 세 가지로 구분된다. CMT는 단일 스레드만이 프로세서의 실행 파이프라인에 존재하는 구조를 가지고 있다. CMT에서는 스레드 전환(thread switching)에 많은 클럭 사이클(clock cycle)이 소요되고, 캐쉬 미스(cache miss)^[38]와 같이 큰 지연(latency)이 수반되는 경우에만 스레드 전환을 수행한다. FMT는 다수의 스레드들이 프로세서의 실행 파이프라인에 존재할 수 있는 구조를 가지므로, 나눗셈 연산(divide operation)과 같은 작은 지연의 경우에도 스레드 전환을 수행하여 이득을 볼 수 있다. 그러나, FMT에서는 단일 사이클에 단일 스레드의 명령어들을만 이슈(issue)한다는 점에서 SMT와 차별된다. SMT는 다수의 스레드가 실행 파이프라인에 존재하면서 다수의 스레드로부터의 명령어들을 동시에 이슈하는 구조를 가진다. 그리고 그림 1-2가 보여주듯이, SMT는 CMT 및 FMT와 비교해서 가장 높은 성능을 보인다. 설계의 복잡도 때문에 SMT 구조를 채택한 상용 마이크로프로세서가 현재는 없지만, SMT 마이크로프로세서는 차세대 마이크로프로세서로서 주목받고 있으며 가까운 미래에 마이크로프로세서 시장에서 큰 부분을 차지할 것이다.

여분의 트랜지스터를 이용해서 CMP(Chip MultiProcessor)^{[39][40]}를 구현하고, 독립적인 다수의 스레드들을 CMP 안의 다수의 프로세서(processor)에 분배하여 수행하도록 하는 방법에 대한 연구도 활발히 이루어지고 있다. 그러나, CMP 안의 개개의 프로세서들은 슈퍼스칼라 마이크로프로세서들이 지닌 한계를 가지고 있으며, 그림 1-3에서 보여주듯이 CMP에서 수행되는 독립적인 스레드들은 각자에 할당된 프로세서 이외의 자원을 사용할 수 없다는 단점이 있다. 따라서 CMP에서는 스레드들의 ILP가 불균형적일 때에는 자원의 낭비가 크고 성능이 낮다. 반면에 SMT는 프로세서 자원을 스레드들이 공유해서 사용하므로 프로세서 자원의 활용도가 크고 성능도 높게 나타난다. 다중스레딩 및 CMP 외에 VLIW 구조가 슈퍼스칼라 구조의 대안으로서 연구되고 있지만, VLIW는 낮은 코드 밀도(code density)와 구조 변경 시 호환성 유지가 어렵다는 결정적인 단점을 가지고 있다^[13].

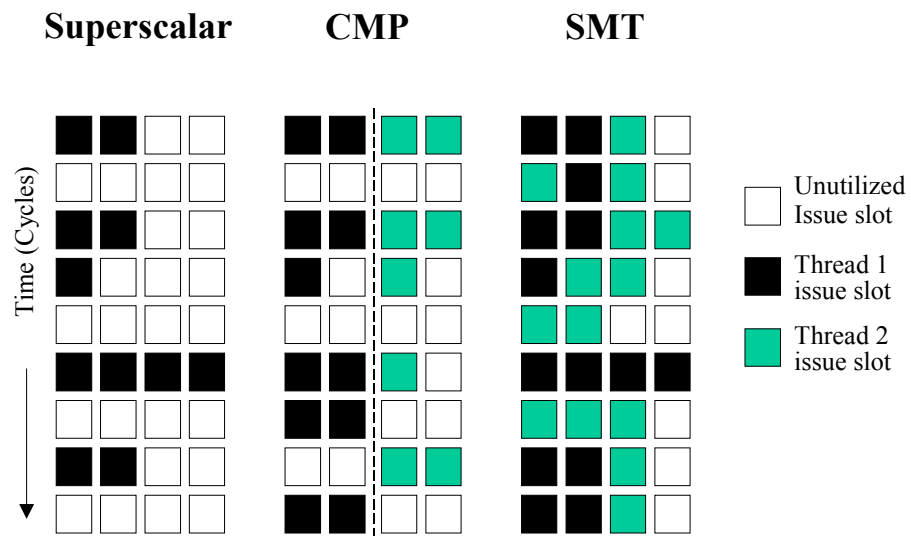


그림 1-3 슈퍼스칼라, CMP 및 SMT 구조의 명령어 이슈 비교

Fig. 1-3 Comparison the instruction issue of Superscalar, CMT and SMT architectures

SMT 구조는 순차적(in-order) 또는 비순차적(out-of-order) 명령어 이슈 방법^[13]을 사용한다. 순차적 SMT 구조에서는 같은 스레드에 속하는 명령어들은 프로그램 순서(program sequence)대로 이슈한다. 반면에 비순차적 SMT 구조에서는 같은 스레드에 속하는 명령어들을 프로그램 순서와 상관없이 이슈한다. 비순차적 SMT는 순차적 SMT보다 더 높은 성능을 보이지만, 비순차적 수퍼스칼라 구조에서 이루어지는 레지스터 리네이밍과 순차적 상태(in-order state)의 추적 및 복구라는 복잡한 동작이 필요하며^[13], 이것은 비순차적 SMT 구조를 더욱 복잡하게 한다. 더구나 SMT에서는 ILP의 한계를 TLP를 사용하여 극복할 수 있기 때문에 비순차적 명령어 이슈로 인한 성능 향상은 복잡성의 증가에 비해서 작다.

본 논문에서는 순차적 SMT 마이크로프로세서 구조를 연구하고, 이를 바탕으로 최적의 구조를 제안한다. 본 연구는 순차적 명령어 이슈를 사용해서 구조의 복잡도를 최소화하면서, TLP를 이용해서 성능은 최대화하는 것에 역점을 둔다. 본 연구는 기반이 되는 순차적 수퍼스칼라 구조의 연구에서 시작해서, 이 구조를 순차적 SMT 구조로 확장하는 연구로 이어진다. 연구된 구조를 C 언어로 기술된 시뮬레이터(simulator)로 작성함으로써 검증 및 성능 평가를 수행한다. 시뮬레이터로 구현되는 수퍼스칼라 및 SMT 구조는 32비트 RISC 명령어 세트 구조(instruction set architecture)^[41-43]를 채택하였다. 성능 평가 과정은 SMT 구조의 다양한 구성 인자(configuration parameter)들을 변화시키면서 수행되며, 특히 스레드간의 자원 공유 정도가 구조의 복잡도와 성능에 미치는 영향을 주의 깊게 분석한다. 본 논문은 이러한 성능 평가 결과에 근거해서 하드웨어 비용과 성능을 고려한 최적의 SMT 구조를 제안한다.

SMT 구조는 기본적으로 모든 스레드들이 프로세서의 자원을 공유함으로써, ILP뿐 아니라 TLP를 충분히 활용하는 구조이다. 그러나 이러한 공유 철학은 SMT의 설계를 어렵게 만드는 요인이기도 하다. CMP 구조는 프로세서 자원을 스레드간에 정적으로 배분(static allocation)하는 이유로 SMT에 비해 프로세서의 활용도 및 성능이 낮지만 CMP 안의 각 프로세서 및 전체 칩의 설계가 SMT 구조에 비해 간단하다. 본 논문은 SMT와 CMP의 이러한 장단점을 고려하여 SMT에 CMP의 정적인 자원 배분 방식을 도입함으로써 SMT 구조의 설계 복잡도를 줄이

면서 성능을 유지하기 위한 방법인 그룹화를 제안한다. 본 논문에서 제안하는 그룹화는 자원과 스레드들을 몇 개의 그룹으로 나누고 특정 자원을 같은 그룹에 속하는 스레드들 사이에서만 공유하는 방식이다. 그룹화를 적용할 경우에 모든 스레드들이 전체 자원을 공유하는 방식과 비교해서 설계의 복잡도를 줄일 수 있다는 장점이 있다. 반면에 그룹화로 인해서 자원의 활용도 및 성능이 저하되는 경향이 생길 수 있다. 그러나 적절한 그룹화의 적용은 성능에 거의 영향을 미치지 않으면서 설계의 복잡도를 줄일 수 있는데, 본 논문은 시뮬레이션을 통해서 이를 증명한다. SMT 구조의 설계가 매우 복잡하다는 것을 고려하여 SMT 프로세서를 설계할 때에는 그룹화를 적용하기 위한 방법을 다양하게 모색해야 한다. 본 논문의 그룹화에 대한 제안은 그러한 연구의 기반을 마련하기 위한 것이다.

현재, 네트워크 상의 패킷(packet)^[44-46]들을 라우팅(routing)^[47-49]하기 위한 네트워크 프로세서^[50-52]에 대한 연구가 활발히 이루어지고 있는데, SMT 구조는 이러한 네트워크 프로세서에 적당한 구조이다^{[53][54]}. 이것은 SMT 구조의 네트워크 프로세서는 각 스레드에 패킷을 대응하여 독립적인 패킷들을 병렬적으로 처리할 수 있기 때문이다. 이러한 이유로 본 논문은 일반적인 프로세서의 코어 구조를 가지지만 약간의 변경 및 추가 유닛의 사용으로 네트워크 프로세서에서도 사용될 수 있는 SMT 코어의 구조를 제시한다. 아울러 본 논문은 제안된 SMT 구조가 네트워크 프로세서의 코어로 사용되었을 때 어느 정도의 성능을 보일지를 제시한다.

본 논문은 다음과 같이 구성되어 있다. 제 2 장은 SMT 구조의 기초가 되는 슈퍼스칼라 구조에 대한 전반적인 설명과 그 구성 유닛들에 대한 자세한 설명을 다룬다. 제 3 장에서는 제 2 장의 슈퍼스칼라 구조를 바탕으로 SMT 구조를 제시하고, SMT 구조와 그 구성 요소들이 슈퍼스칼라 구조와 비교해서 어떠한 차이점이 있는지를 논한다. 제 4 장에서는 SMT 구조의 설계 복잡도를 줄이기 위한 방법으로서 세 가지의 그룹화 방안을 제안한다. 제 5 장에서는 슈퍼스칼라 및 SMT 구조의 시뮬레이터를 작성하고, 성능 평가를 위한 시뮬레이션(simulation) 방법 및 과정에 대해서 설명한다. 제 6 장에서는 시뮬레이션 결과를 토대로 성능 평가 및 분석을 수행하여, SMT 구조의 성능을 슈퍼스칼라 구조의 성능과 비교하고 다양한 구성 인자들의 변화로 인한 성능 및 하드웨어 비용의 증감을 고려하여 최적의

SMT 구조를 제안한다. 그리고 제 7 장에서는 SMT에 관한 이전의 연구들을 소개하고, 본 논문이 이러한 연구들과 어떠한 차이점을 가지는지를 설명한다. 마지막으로, 제 8 장에서는 결론으로 본 논문의 끝을 맺는다.

제 2 장 순차적 수퍼스칼라 구조

본 장에서는 순차적 SMT 구조의 기반이 되는 순차적 수퍼스칼라 구조의 전체적인 구성과 특징에 대해서 설명하고, 그 구성 요소들에 대한 자세한 설명을 제시한다.

2-1절 전체적인 순차적 수퍼스칼라 구조

본 논문의 SMT 마이크로프로세서 구조에 관한 연구는 수퍼스칼라 마이크로프로세서 구조에 관한 연구에서부터 시작한다. 즉, 본 논문은 수퍼스칼라 마이크로프로세서 구조를 제시하고 이를 기반으로 SMT 마이크로프로세서 구조를 제시하는 연구 방법을 사용한다.

본 논문의 SMT 마이크로프로세서는 순차적 명령어 이슈 및 완료(in-order instruction issue and completion)^[13]를 기본 정책(policy)으로 하기 때문에, SMT의 기반이 되는 수퍼스칼라 마이크로프로세서도 순차적 명령어 이슈 및 완료에 근거하는 구조를 가지고 있다. 그림 2-1은 SMT 구조의 기반이 되는 수퍼스칼라 마이크로프로세서의 전체적인 구조를 보여주는 블록 다이어그램(block diagram)이며, 4개의 명령어를 이슈 폭으로 하는 경우에 대한 것이다.

그림 2-1에서 페치 유닛(fetch unit)은 명령어를 명령어 캐쉬(instruction cache)^[55]로부터 페치하여 명령어 페치 큐(instruction fetch queue)로 저장하는 동작을 제어하는 부분이다. 명령어 페치와 동시에, 페치 유닛 안의 BTB(Branch Target Buffer)^{[56][57]}는 분기(branch)를 수반하는 명령어들에 대한 분기 예측(branch prediction)^{[58][59]}을 수행한다. IMMU(Instruction Memory Management Unit)^{[60][61]}는 페치되는 명령어의 32비트 가상주소(virtual address)^[38]를 32비트 실제주소(physical address)^[38]로 바꿔주는 역할을 수행하며, 이렇게 얻어진 실제주소를 가지고 명령어 캐쉬를 액세스(access)하여 명령어들을 읽어낸다.

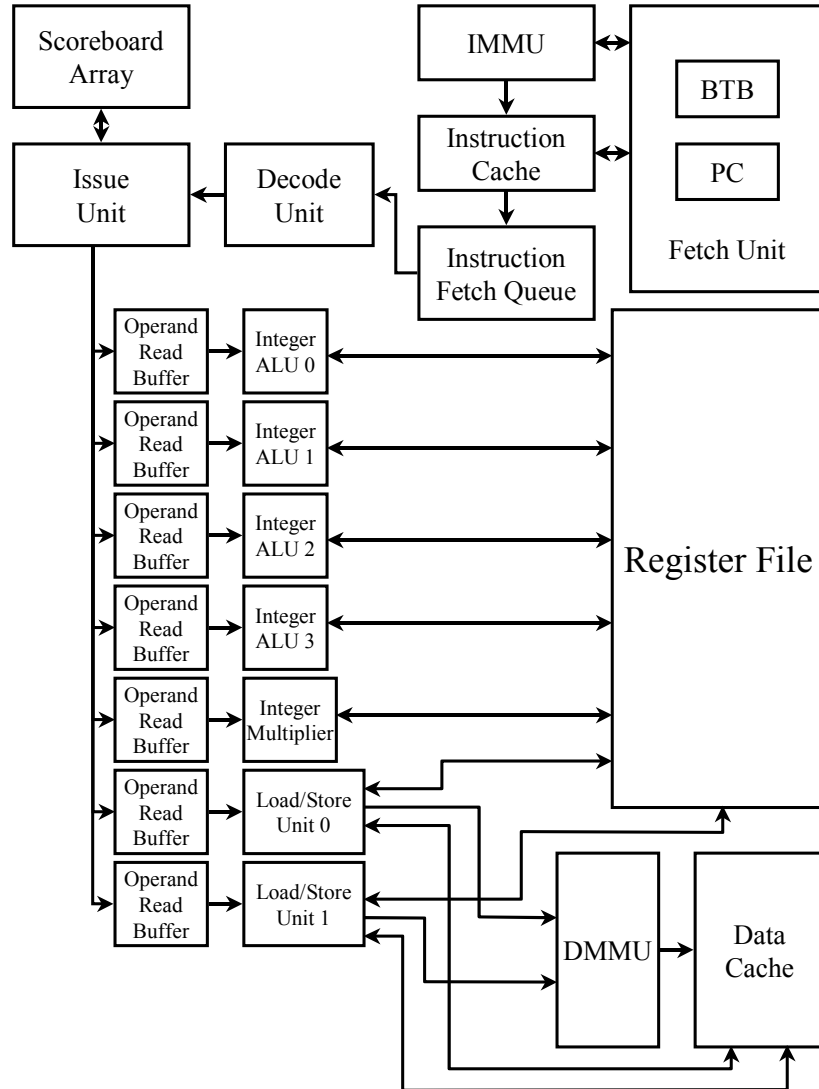


그림 2-1 전체적인 순차적 슈퍼스칼라 구조의 블록 다이어그램

Fig. 2-1 Block diagram of the overall in-order superscalar architecture

디코드 유닛(decode unit)은 명령어 페치 큐의 명령어들 중에서 가장 오래된 명령어들을 읽어서 해석하고, 이슈 유닛(issue unit) 안에 존재하는 명령어 이슈 큐(instruction issue queue)에 해석된 명령어들을 저장한다. 이슈 유닛은 명령어 이슈 큐에 존재하는 명령어들 중에서 가장 오래된 명령어들, 즉 프로그램 순서에 가장 앞에 있는 명령어들을 기능 유닛(functional unit)들로 이슈한다. 이슈 유닛이 명령어를 이슈하는 과정에서는 이슈하고자 하는 명령어에 대해서 데이터의 의존성 및 자원 충돌(resource conflict)^[13]을 검사한다. 그리고 이러한 검사를 통해서 데이터 의존성과 자원 충돌이 없는 명령어들만이 해당 기능 유닛으로 이슈된다. 명령어의 데이터 의존성을 검사하기 위해서 이슈 유닛은 스코어보드 어레이(scoreboard array)^[13]를 사용한다. 스코어보드 어레이는 일반 레지스터들(register)과 조건 코드 플래그(condition code flag)들에 대해서 최신의 갱신(update) 동작이 존재하는 기능 유닛과 파이프라인 단계(pipeline stage)에 대한 정보를 가지고 있다.

기능 유닛의 종류에는 ALU(Arithmetic Logic Unit)^[62], 곱셈 유닛(multiply unit)^[63] 및 로드/스토어 유닛(load/store unit)^[64]의 세 가지가 존재한다. 본 논문의 연구는 부동소수점(floating-point) 연산^{[65][66]}이 필요 없는 네트워크 프로세서의 코어로서 사용될 수 있는 SMT 구조를 제안하는 것이 그 목적의 하나이기 때문에, 기본이 되는 슈퍼스칼라 구조는 부동소수점 연산을 위한 기능 유닛은 따로 존재하지 않는다. ALU는 산술(arithmetic) 및 논리(logic) 연산 명령어들을 실행하기 위한 것이다. 그리고 본 구조에서는 분기 유닛(branch unit)이 따로 존재하지 않기 때문에 분기 명령어들도 ALU에서 실행되며, 또한 CLZ(Count Leading Zeros) 명령어들도 ALU에서 실행된다. 곱셈 유닛은 곱셈(multiply) 및 MAC(Multiply-and-Accumulate) 명령어들과 DSP(Digital Signal Processing) 확장 명령어^[41]들을 실행하는 역할을 한다. 그 외에 로드/스토어 유닛은 로드(load)와 스토어(store)를 포함하는 모든 메모리 액세스 명령어들을 실행한다. 기능 유닛으로 이슈된 명령어들은 오퍼랜드 읽기 버퍼(operand read buffer)에서 필요한 오퍼랜드(operand)들을 레지스터 파일(register file)^[62]로부터 읽거나 또는 데이터 바이패싱(data forwarding)^[67]을 통해 얻는다. 오퍼랜드를 획득한 명령어들은 해당 기

능 유닛에서 실행되고, 그 결과를 레지스터 파일에 쓴다.

메모리 액세스 명령어의 데이터 메모리 가상주소는 로드/스토어 유닛에서 계산된다. 이렇게 계산된 가상주소는 DMMU(Data Memory Management Unit)에 의해 실제주소로 바뀌게 되고, 실제주소를 사용하여 데이터 캐쉬를 액세스하여 데이터를 쓰거나 읽는 동작을 수행한다. 데이터 메모리의 가상주소와 실제주소는 명령어 주소의 경우와 같이 32비트로 이루어졌다.

상기의 것들 외에 시스템의 중요 사항 중에는 little-endian 메모리 시스템^[42]을 사용한다는 것이 있다. 그리고 본 논문의 구조 연구에서 채택한 명령어 세트 구조의 특성상 각 명령어의 길이는 32비트, 즉 4바이트(byte)로 고정되었다.

본 절에서는 수퍼스칼라 시스템의 전반적인 구조에 대해서 대략적인 설명만을 하였다. 수퍼스칼라 시스템의 파이프라인 구조와 구성 유닛들에 대한 자세한 설명은 다음의 절들에서 제시된다.

2-2절 순차적 슈퍼스칼라 구조의 파이프라인

마이크로프로세서 구조에서 파이프라인 구조는 프로세서의 성능을 좌우하는 큰 요소가 된다. 특히 파이프라인 단계의 수를 정하는 데는 여러 가지 장단점을 고려해야 한다. 파이프라인 단계의 수가 작을 경우에는 각 단계에서 해야 할 일이 많아지기 때문에 프로세서의 동작 주파수가 낮아지는 단점이 있지만 파이프라인 제어가 간단해 지기 때문에 설계 복잡도가 줄어드는 장점이 있다. 특히 분기 예측 오류(branch misprediction)^{[58][59]}로 인한 손실(penalty)이 적기 때문에 정확한 분기 예측에 대한 부담이 매우 적다. 반면에 파이프라인 단계의 수를 증가시키면 설계 복잡도가 증가한다. 뿐만 아니라 분기 예측 오류 또는 예외(exception)^{[68][69]} 발생의 경우에 파이프라인에 존재하는 많은 명령어들이 플러쉬(flush)되기 때문에 결과적으로 많은 사이클들을 낭비하게 된다. 그러나 세분된 파이프라인 구조로 각 단계에서 수행해야 할 일이 단순해져서 프로세서의 동작 주파수가 높아지고 성능이 향상된다. 따라서 고성능 프로세서는 파이프라인 단계를 증가시키고 분기 예측 오류의 발생을 최소화하기 위해서 BTB와 같은 하드웨어(hardware)를 추가한다. 그런데 파이프라인을 세분화하여 주파수를 높일 수 있지만, 그에 따르는 파이프라인 제어의 복잡도 증가는 동작 주파수의 증가를 억제하는 역할을 한다. 따라서 파이프라인 단계의 수는 적정 수준에서 결정되어야 한다.

슈퍼스칼라 파이프라인 구조는 상기 사항들과 SMT 구조로의 확장성을 고려하여 결정되었으며, 그림 2-2는 슈퍼스칼라 파이프라인 구조의 대략적인 모습을 보여준다. 그림 2-2에서 보여주듯이 슈퍼스칼라 파이프라인 구조는 F(Fetch), D(Decode), I(Issue), R(Read), E(Execute), M(Memory), W(Write)의 단계들로 구성되어 있으며, 각 단계에서의 구체적인 동작은 다음과 같다.

F 단계에서는 PC(program counter)의 주소를 이용하여 명령어 캐쉬로부터 명령어를 읽어서 명령어 페치 큐에 저장하는 동작이 이루어진다. 구체적으로 우선 PC의 가상주소가 IMMU에 의해서 실제주소로 바뀌고, 실제주소로 명령어 캐쉬를 액세스하여 명령어들을 읽어서 명령어 페치 큐의 최상위 포인터(top pointer)가 가

리키는 위치에 저장하게 된다. 만약에 IMMU에 의한 주소 변환(address translation) 과정에서 ITLB(Instruction Translation Lookaside Buffer)^{[60][61]} 미스가 발생하거나 명령어 캐쉬에서 미스가 발생하면, 명령어 페치 동작은 정지(stall) 되고 미스 처리가 끝난 후에 다시 명령어 페치 동작이 이루어진다.

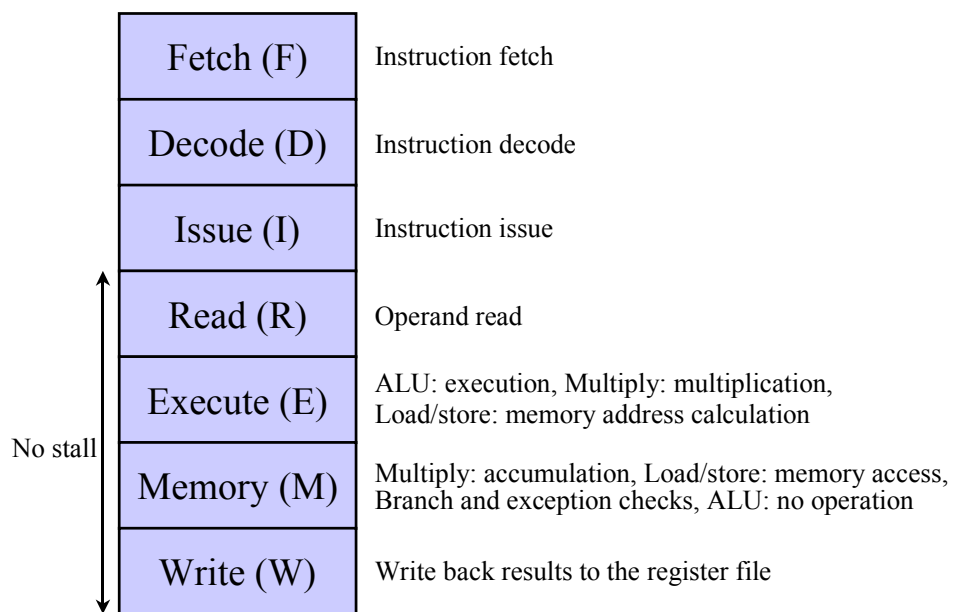


그림 2-2 순차적 슈퍼스칼라 구조의 파이프라인

Fig. 2-2 pipeline structure of the in-order superscalar architecture

D 단계에서는 디코드 유닛이 명령어 패치 큐로부터의 명령어들을 디코드한다. 구체적으로 D 단계에서는 명령어 패치 큐의 최하위 포인터(bottom pointer)가 가리키는 위치로부터 명령어들을 읽어서 디코드하고, 디코드된 명령어들은 명령어 패치 큐로부터 제거한다. 디코드 유닛은 명령어 디코드에 의해서 각 명령어에 대해서 내부적으로 사용하는 오프 코드(op code), 소스 레지스터 주소(source register address), 목적 레지스터 주소(destination register address), 상수 데이터(immediate data) 및 명령어 실행 기능 유닛의 종류 등의 정보를 분석하고, 이 명령어 정보를 명령어 이슈 큐의 최상위 포인터가 가리키는 위치로 저장한다.

I 단계에서는 이슈 유닛이 명령어 이슈 큐의 최하위 포인터가 가리키는 위치로부터의 명령어들을 적절한 기능 유닛들로 이슈한다. 명령어 이슈는 순차적으로 이루어지기 때문에 이전의 명령어가 이슈되지 못하면 후속하는 명령어들도 이슈되지 못한다. 명령어 이슈 과정에서 이슈 유닛은 명령어를 이슈할 수 있는지를 결정하기 위해서 데이터 의존성과 자원 충돌에 대한 검사를 수행한다. 이슈 유닛은 이러한 검사를 통해 이슈되지 않은 이전의 명령어가 존재하지 않고, 데이터 의존성과 자원 충돌이 없는 것으로 확인된 명령어들을 해당 기능 유닛들로 이슈한다. 이슈된 명령어들은 R 단계부터는 정지 없이 계속 수행되어 나가는데, 이를 위한 조정은 명령어 이슈 과정에서 이루어진다.

기능 유닛으로 이슈된 명령어들은 명령어의 종류에 따라서 다른 동작이 이루어진다. 명령어의 종류는 명령어를 실행하는 기능 유닛의 종류와 밀접한 관계를 가지므로 기능 유닛의 종류에 따라서 명령어 실행의 단계들에서 이루어지는 동작들이 다르며, 그림 2-3은 기능 유닛의 종류에 따라 파이프라인별로 이루어지는 동작들을 간략하게 보여준다.

그림 2-3에서 알 수 있듯이 ALU로 이슈되는 명령어들은 R 단계에서 오퍼랜드를 읽은 후에 E 단계에서는 주어진 동작을 수행한다. M 단계에서 ALU 명령어들은 아무런 동작을 수행하지 않는다. 즉 ALU 명령어에 있어서 M 단계는 다른 종류의 명령어들과의 동기화를 위해서 존재하는 단계로서 볼 수 있다. 그리고 최종적으로 W 단계에서 ALU 명령어의 실행 결과가 레지스터 파일로 기록된다.

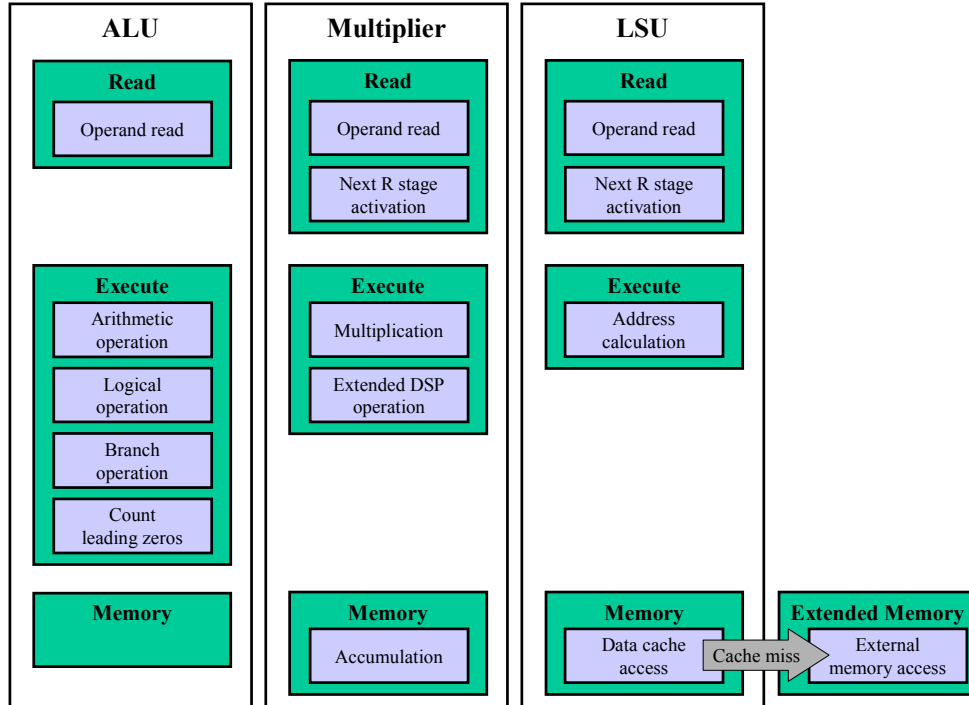


그림 2-3 기능 유닛들의 파이프라인 단계별 동작

Fig. 2-3 Operations in the pipeline stages of functional units

곱셈 유닛을 사용하여 실행되는 명령어들이 R 단계에서 오퍼랜드를 읽는 것은 ALU 명령어의 경우와 같다. 그러나, 곱셈 유닛의 명령어들 중에는 다중 사이클 실행(multi-cycle execution) 명령어들이 존재하는데, 이러한 다중 사이클 명령어들에 대해서는 첫째 R 단계가 E 단계로 가면서 동시에 둘째 R 단계가 활성화된다는 차이점이 있다. 즉 곱셈 유닛의 다중 사이클 명령어의 첫째 동작이 E 단계에 있을 때에는 그 명령어의 둘째 동작이 R 단계에 존재하게 된다. 곱셈 유닛의 명령어는 E 단계에서는 곱셈 동작을 실행하고 M 단계에서는 누적(accumulation) 동작

이 이루어진다. 마지막으로 W 단계에서는 실행 결과를 레지스터 파일의 목적 레지스터에 저장한다.

로드/스토어 유닛을 사용하는 명령어들은 R 단계에서 곱셈 유닛 명령어들의 R 단계 동작과 같이 오퍼랜드 읽기와 새로운 R 단계의 활성화를 수행한다. 그리고, M 단계에서는 데이터 캐쉬를 액세스한다. M 단계에서 데이터 캐쉬 미스가 발생하거나 데이터 액세스가 uncached 영역에 대한 것일 경우에는, EM(Extended Memory) 단계로 가서 데이터 캐쉬 미스를 처리하거나 외부 메모리를 액세스한다. 데이터 캐쉬 히트(cache hit)이면, M 단계에서 데이터 캐쉬로부터 데이터를 읽거나 데이터 캐쉬에 데이터를 쓰는 동작이 이루어진다. W 단계에서 로드 명령어는 메모리로부터 읽은 데이터를 레지스터 파일의 해당 목적 레지스터에 쓰지만, 스토어 명령어는 어떤 동작도 수행하지 않는다.

2-3절 명령어 페치 구조

2-3-1 전체적인 페치 구조

명령어 페치와 관련된 동작들은 페치 유닛, IMMU, 명령어 캐쉬 및 명령어 페치 큐에 의해서 이루어진다. 페치 유닛은 PC와 BTB를 포함하며 명령어 페치 주소를 관리한다. 그리고 IMMU는 명령어의 가상주소를 실제주소로 변환하며, 명령어 캐쉬는 명령어의 실제주소를 입력으로 해서 해당 주소의 명령어들을 명령어 페치 큐에 전달하는 역할을 수행한다. 그림 2-4는 명령어 페치 관련 유닛들에 의해서 명령어가 명령어 캐쉬로부터 명령어 페치 큐로 전달되는 모습을 보여준다.

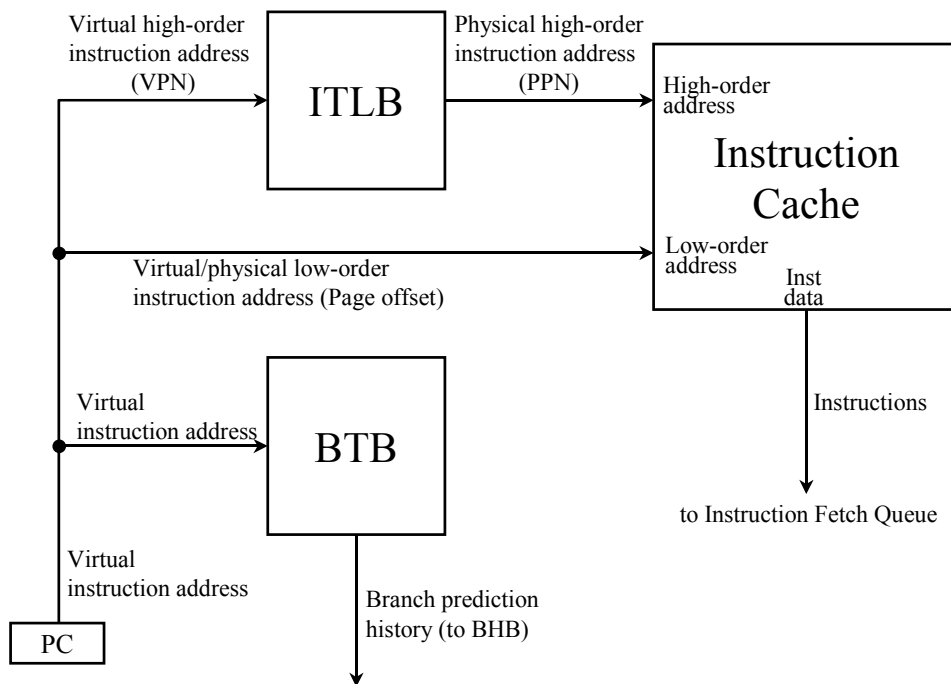


그림 2-4 순차적 슈퍼스칼라 구조의 명령어 페치

Fig. 2-4 Instruction fetch structure of the in-order superscalar architecture

그림 2-4에서 PC는 페치할 명령어의 가상주소를 가지고 있다. PC로부터의 주소 중에서 상위 비트들(high-order bits)은 VPN(Virtual Page Number, 가상 페이지 수)^[38]에 해당하며, IMMU는 VPN을 PPN(Physical Page Number, 실제 페이지 수)^[38]으로 변환한다. PC의 하위 비트들(low-order bits)은 가상주소의 페이지 오프셋(page offset)에 해당하는데, 이것은 실제주소의 페이지 오프셋과 같다. 따라서 실제주소는 PPN과 페이지 오프셋을 연결한 비트들에 의해서 구성되며, 이것은 실제주소 캐쉬(physical cache)^[38]인 명령어 캐쉬를 액세스하는데 사용된다. 그런데 본 구조에서는 페이지 오프셋의 비트 수가 명령어 캐쉬를 인덱싱(indexing)^[38]하기 위해서 필요한 비트들의 수보다 크도록 설정했다. 따라서 IMMU에 의한 명령어 주소 변환과 동시에 캐쉬 액세스가 이루어지며, 이는 명령어 페치를 단일 사이클에 수행할 수 있도록 해 준다. 이와 같이 PC의 주소를 사용하여 IMMU와 명령어 캐쉬에 의해서 읽혀진 명령어들은 명령어 페치 큐로 전달되어 저장된다.

페치되는 명령어 중에서 분기를 수반하는 명령어가 존재하는 경우에 다음 사이클에는 분기 타겟(branch target)^{[56][57]}으로부터 명령어를 페치해야 올바른 동작이 이루어진다. 그러나 명령어 페치 중에는 어떤 명령어에서 분기가 이루어지는지, 그리고 분기 타겟 주소가 무엇인지 알 수가 없다. 그림 2-4의 BTB는 페치되는 명령어의 가상주소를 사용해서 해당 명령어에 의한 분기 여부와 분기 타겟 주소를 예측하는 역할을 수행하는 것이다. 이와 같이 BTB는 분기 예측을 통해서 프로시저 의존성(procedural dependency)^[13]을 최소화하는 역할을 수행하며, 분기 예측이 맞았을 경우에는 프로시저 의존성으로 인한 사이클 낭비를 완전히 없앨 수 있다.

2-3-2 페치 유닛

페치 유닛은 PC의 적절한 갱신을 통해 명령어 페치를 담당하는 유닛이다. 그림 2-5는 단일 사이클에 페치할 수 있는 명령어의 수, 즉 페치 폭(fetch width)이 4개의 명령어인 경우에 대한 페치 유닛의 PC 갱신 동작을 보여주며, 다음의 설명들은 4개의 명령어의 페치 폭을 가정한 경우에 해당한다. 단일 명령어는 4바이트로 구성되므로 단일 사이클에 페치하는 명령어 데이터를 바이트 단위로 환산하면

(4×4)바이트, 즉 16바이트가 된다. 따라서 분기가 없이 순차적으로(sequentially) 명령어 페치가 이루어지는 경우에는 다음 사이클에 페치할 명령어 주소는 일반적으로 현재 사이클의 PC 값에 16을 더한 값이 된다. 그러나 PC의 값이 16바이트 경계(boundary)로 정렬(aligned)되어 있지 않은 경우에는 다음의 PC 값은 16을 더한 값이 아니게 된다. PC가 16바이트 경계로 정렬되지 않았다는 것은 PC의 최하위 4비트 중에서 0이 아닌 것이 존재한다는 것이다. 그런데 단일 명령어는 4바이트(1워드)이므로 PC의 최하위 2비트는 항상 0이 되며, 그렇지 않을 경우에는 명령어 페치가 더 이상 제대로 이루어질 수 없다. 따라서 PC가 16바이트 경계로 정렬되지 않은 경우는 PC의 비트 3과 비트 2 중에서 0이 아닌 것이 존재하는 경우 ($PC[3:2] \neq 00_{(2)}$)에 발생하며, 이것은 4워드 경계로 보았을 때 첫째 명령어 이외의 명령어로 분기가 일어난 경우에 발생한다. 예를 들어 4워드 경계의 둘째 명령어로 분기가 일어난 경우 $PC[3:2]$ 는 $01_{(2)}$ 이 되고 현재 사이클에 유효하게 페치되는 명령어의 수는 3개이며, 다음 PC는 현재 PC의 값에 12만을 더한 값이 된다. 이와 같이 PC의 값이 4워드 경계로 정렬되지 않은 경우에 페치되는 명령어의 수가 줄어드는 것은 명령어 캐쉬가 4워드 경계로 4개의 명령어를 제공하기 때문이다. 즉 명령어 캐쉬는 단지 $PC[31:4]$ 만을 고려하여 4개의 명령어들을 제공되는데, 제공되는 명령어들의 주소는 $PC[31:4]$ 를 상위 28비트로 가지면서 하위 4비트는 각기 $0000_{(2)}$, $0100_{(2)}$, $1000_{(2)}$, $1100_{(2)}$ 이 된다. 따라서 PC가 $0100_{(2)}$ 에 해당하는 주소를 가진다면 첫째 명령어인 $0000_{(2)}$ 에 해당하는 명령어는 필요 없는 명령어가 되며 페치 과정에서 버려진다. 그리고 다음 사이클의 페치가 순차적인 경우에는 다음 사이클의 PC는 현재 사이클의 $PC[31:4]$ 에 1을 더한 것과 $0000_{(2)}$ 을 연결한 것을 다음 사이클의 PC의 값으로 하게 된다. 상기의 순차적 PC 갱신 동작에 대한 설명은 페치 폭이 네 명령어가 아닌 경우에도 약간의 변경으로 적용할 수 있으며, 표 2-1은 다양한 페치 폭에 대해서 정렬된 명령어 주소의 조건과 순차적 다음 사이클 PC 값을 정리한 것이다.

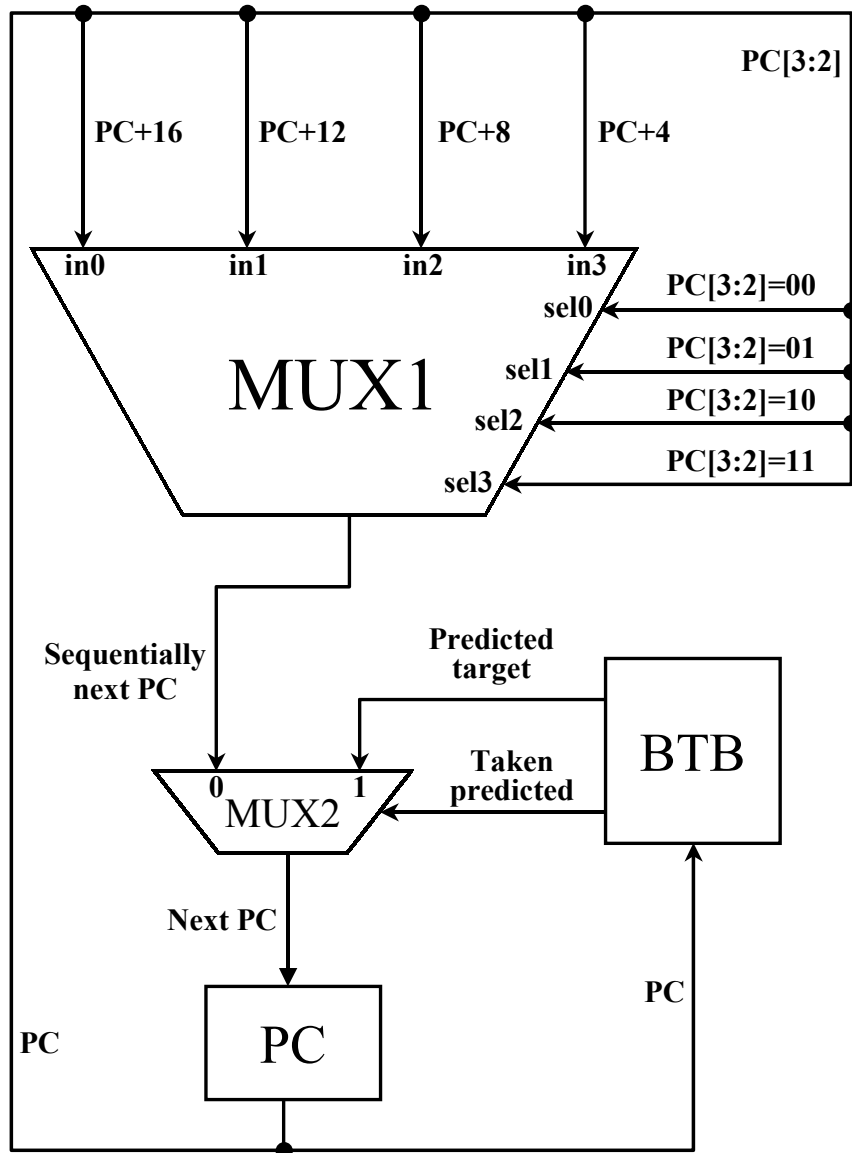


그림 2-5 페치 유닛의 PC 갱신

Fig. 2-5 PC update of the fetch unit

표 2-1 PC의 정렬 조건과 순차적 다음 PC 값

Table 2-1 Qualification for aligned PC and the next sequential value of PC

페치 대역폭	PC의 정렬 조건	순차적 다음 PC 값
1 instruction	PC[1:0] = 00 ₍₂₎	{(PC[31:2]+1),00 ₍₂₎ }
2 instructions	PC[2:0] = 000 ₍₂₎	{(PC[31:3]+1),000 ₍₂₎ }
4 instructions	PC[3:0] = 0000 ₍₂₎	{(PC[31:4]+1),0000 ₍₂₎ }
8 instructions	PC[4:0] = 00000 ₍₂₎	{(PC[31:5]+1),00000 ₍₂₎ }
16 instructions	PC[5:0] = 000000 ₍₂₎	{(PC[31:6]+1),000000 ₍₂₎ }

명령어 페치가 이루어지는 동안에 BTB는 현재의 PC를 사용해서 분기 예측을 수행한다. 분기 예측에 의해서 현재 사이클에 페치되는 명령어 중에서 분기 동작을 수반하는 것이 존재하는 것으로 예측될 경우에는 다음 사이클의 페치는 분기의 타겟으로부터 이루어지고, 다음 사이클의 PC는 분기 타겟 주소가 된다. 즉, 분기 예측에 따라서 그림 2-5의 MUX2는 두 개의 입력 중에서 한 개를 다음 사이클의 PC 값으로 선택한다.

수퍼스칼라 구조에서는 단일 사이클에 페치되는 명령어가 다수이므로 다수의 명령어에 대한 분기 예측이 동시에 이루어져야 한다. 그런데 이것은 BTB가 다수의 읽기 포트(read port)를 가지고 있어야 한다는 것을 의미하며, BTB의 하드웨어 비용을 증가시킨다. 이러한 점을 고려하여, 본 구조의 BTB는 페치되는 각 명령어에 대해 분기 예측을 수행하는 것이 아니라, 단일 사이클에 페치되는 연속적인 명령어 열에 대한 분기 예측을 수행하도록 하였다. 연속적인 명령어 열을 단일한 것으로 보고 그 열에 대한 분기 예측을 수행함으로써, BTB의 읽기 포트 수를 1개로 줄이는 방법을 사용한다. 이러한 정책에 따라서 BTB 구조는 페치 폭이 2^N 개의 명

령어인 경우 그림 2-6과 같이 구성된다.

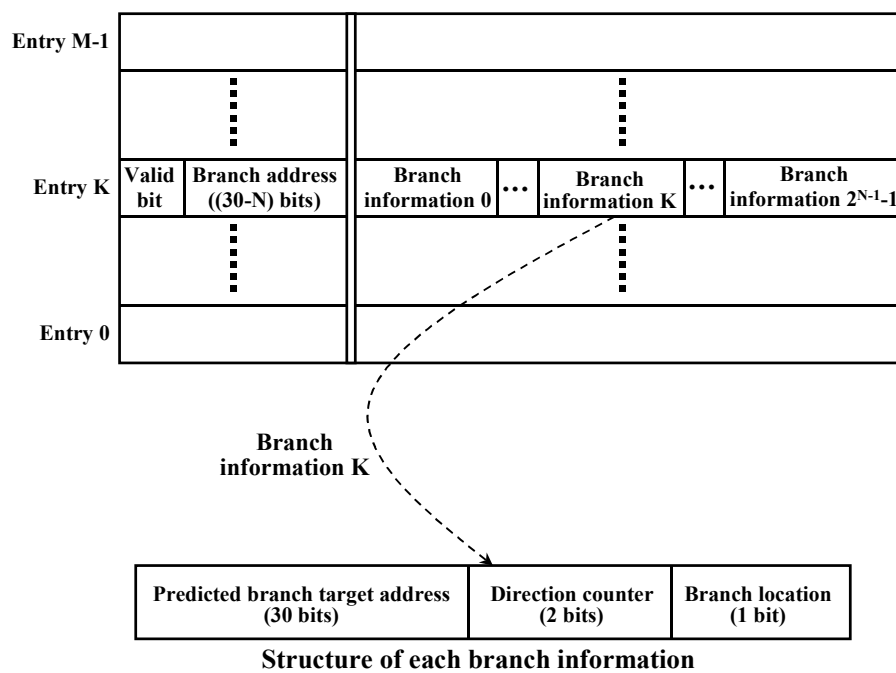


그림 2-6 BTB의 구조

Fig. 2-6 BTB structure

BTB의 각 엔트리는 분기 주소(branch address)와 분기 예측 정보(branch information)로 구성된다. BTB의 각 엔트리는 2^N 개의 명령어 열에 대응하는 것이기 때문에 단일 BTB 엔트리를 액세스하기 위한 분기 명령어 주소는 명령어 주소의 하위 비트들인 $(N+2)$ 비트들을 제외한 비트 31부터 비트 $(N+2)$ 까지의 $(30-N)$ 비트로 구성된다. 분기 예측 정보는 2개의 명령어마다 한 개씩 존재하고, 따라서 BTB의 단일 엔트리는 $2^N/2=2^{N-1}$ 개의 분기 예측 정보를 가지고 있다. 그리고, 각 분기 예측 정보는 하위 2비트를 제외한 분기 타겟 주소(predicted branch target address), taken 또는 not taken의 분기 방향을 결정하는 2비트의 방향 카운터(direction counter)^[59] 및 분기 위치가 2개의 명령어 중 어떤 명령어인지를 알려주는 1비트의 분기 위치(branch location)를 구성 요소로 한다.

상기의 BTB를 액세스 할 때에는 PC[31:2]가 사용되며, fully-associative^[38]로 구성된 BTB의 엔트리 중에서 PC[31:N+2]와 같은 분기 주소를 가지고 있는 엔트리가 존재하지 않은 경우에는 폐치되는 명령어 열에 의한 분기가 없는 것으로 예상된다. 반대로 주소가 부합하는 엔트리가 존재하는 경우에는 그 엔트리의 분기 정보를 사용하여 분기 예측을 수행한다. 그런데 단일 BTB 엔트리에는 여러 개의 분기 예측 정보가 있으므로 어떤 분기 예측 정보를 사용할 지를 결정해야 한다. 이것과 관련된 것이 N 비트의 PC[N+1:2]이다. 예를 들어서 N 이 3이고 PC[3+1:2], 즉 PC[4:2]가 $011_{(2)}$ 인 경우에는 주소[4:2]가 $000_{(2)}$, $001_{(2)}$ 및 $010_{(2)}$ 인 명령어들은 폐치에서 제외되는 명령어들이다. 따라서, 주소[4:2]가 $000_{(2)}$ 및 $001_{(2)}$ 에 대응하는 첫째 분기 예측 정보는 무시되며, 둘째 분기 예측 정보부터 사용되어야 한다. 또한 둘째 분기 예측 정보가 taken으로 예측하더라도, 분기 위치가 2개의 명령어 중에서 첫째 명령어(주소[4:2]가 $010_{(2)}$ 인 명령어)인 경우에는 그 예측은 무시된다. 전체적으로 분기 예측 정보들은 순서대로 분기 예측에 대한 우선권을 가지게 된다. 즉 둘째 분기 예측 정보에서 taken으로 분기 예측이 이루어진 경우에는 셋째 및 넷째 분기 예측 정보는 의미가 없다. 즉, 마지막 분기 예측 정보가 사용되기 위해서는 마지막을 제외한 모든 분기 예측 정보에 의한 예측이 not taken으로 이루어져야 한다. 이 경우, 마지막 분기 예측 정보도 not taken일 경우에는 현재 사이클에 폐치되는 명령어 열에 대해서는 분기 동작이 없는 것으로 보게 되고, 다음 사이클에

는 순차적으로 다음 명령어부터 패치를 시작하게 된다.

분기 예측 정보의 구성 요소인 2비트 카운터는 분기의 방향을 예측한다. 즉 방향 카운터 값이 $00_{(2)}$ 또는 $01_{(2)}$ 인 경우에는 not taken으로 예측하며, $10_{(2)}$ 또는 $11_{(2)}$ 인 경우에는 taken으로 예상한다. 방향 카운터의 값은 그림 2-7과 같이 갱신된다. 즉 실제로 taken 될 때마다 값이 1씩 증가하고, not taken 될 때에는 1씩 감소하며, 방향 카운터 값이 $11_{(2)}$ 일 때 taken 되거나 $00_{(2)}$ 일 때 not taken 되는 경우에는 그 값의 변화가 없다. 이와 같이 방향 카운터를 1비트가 아닌 2비트로 한 것은 프로그램의 루프(loop) 특성을 고려했기 때문이다^[70].

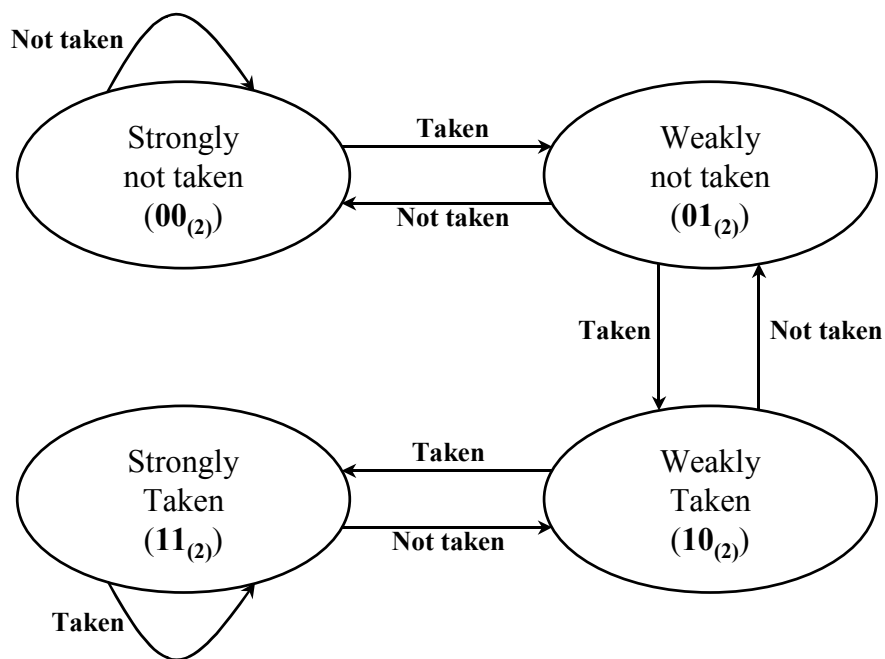


그림 2-7 방향 카운터 변경의 흐름도

Fig. 2-7 Flow of direction counter update

분기 명령어 실행에 의해서 분기 예측의 히트 또는 미스를 검사하기 위해서는 분기 예측의 내용들, 즉 분기 예측 히스토리(branch prediction history)를 저장해야 한다. 그림 2-8의 BHB(Branch History Buffer)는 분기 예측 히스토리를 저장하는 것이며, taken으로 예상된 분기에 대해서 분기 주소와 분기 타겟 주소를 선입선출(FIFO) 방식으로 저장하고 있다. BHB는 최상위 포인터와 최하위 포인터에 의해서 선입선출 방식으로 구성되며, 최상위 포인터는 새로운 분기 예측 히스토리를 저장할 엔트리를 가리키며, 최하위 포인터는 가장 오래된 분기 예측 히스토리를 저장하고 있는 엔트리를 가리킨다.

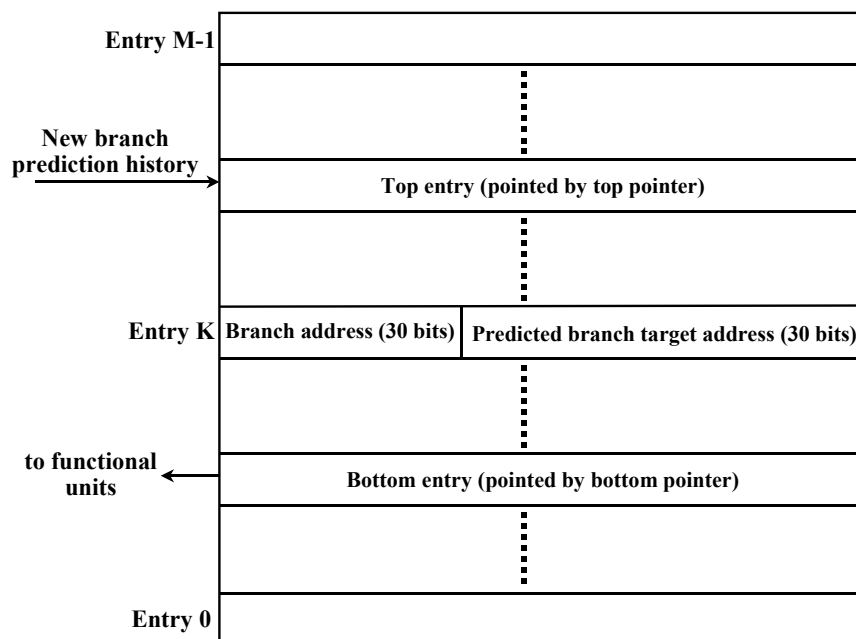


그림 2-8 BHB의 구조

Fig. 2-8 BHB structure

2-3-2 IMMUE와 명령어 캐쉬

명령어 페치의 성능과 관련해서 가장 중요한 역할을 하는 것이 IMMUE와 명령어 캐쉬이다. 즉 IMMUE의 ITLB 또는 명령어 캐쉬에서 미스가 발생한다면 명령어 페치는 중단되고, 미스 처리가 이루어진 후에 페치가 다시 시작한다. 이것은 상당한 사이클 낭비를 초래하기 때문에 ITLB 또는 명령어 캐쉬에서의 미스를 최소화하고, 또한 액세스 속도를 빠르게 하도록 구성되어야 한다.

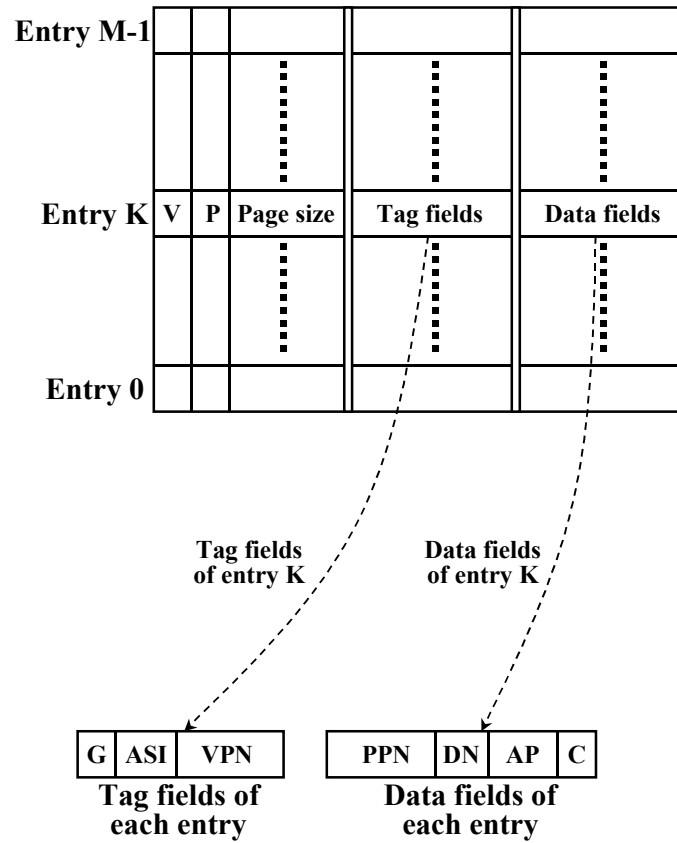


그림 2-9 ITLB의 구조

Fig. 2-9 ITLB structure

표 2-2 ITLB 엔트리의 영역들

Table 2-2 Fields of ITLB entries

영역(field)	설명
V	해당 엔트리의 유효성을 나타내는 비트
P	해당 엔트리가 보호(protect)됨을 나타내는 비트
Page size	페이지의 크기를 가리키는 영역으로서, VPN과 ITLB 엔트리의 가상주소 비교 시에 참고되는 영역
G	해당 엔트리는 모든 ASI의 페이지에 적용되는 것을 의미하는 비트, 즉, 비트 G가 1일 때에는 ASI의 비교는 무시됨
ASI	해당 엔트리의 주소 공간(address space)을 가리키는 수
VPN	가상 페이지 수(virtual page number)
PPN	실제 페이지 수(physical page number)
DN	도메인 수(domain number), 해당 엔트리의 domain을 가리키는 것으로서, AP와 함께 액세스 권한(access permission)을 제어하는데 사용됨
AP	액세스 권한을 제어하는데 사용되는 영역
C	해당 엔트리의 페이지를 명령어 캐쉬에 넣을 수 있음(cachability)을 알려주는 비트

IMMU는 명령어의 가상주소를 실제주소로 바꾸는 역할을 수행하는데, 이러한 명령어 주소의 변환은 메모리에 있는 페이지 테이블(page table)^[38]에 저장되어 있다. 그런데 주소 변환 시마다 항상 페이지 테이블에서 주소 변환 정보를 가져온다면, 명령어 페치는 단일 사이클에 이루어질 수 없다. 따라서 최근에 또는 자주 사용되는 주소 변환 정보를 빠르게 액세스할 수 있는 곳에 저장해 놓고서 그 정보

를 이용하여 주소 변환을 수행하는데, ITLB가 이러한 정보를 저장하고 있다. 본 구조의 ITLB는 fully-associative 방식으로 그림 2-9와 같이 구성되고 각 영역(field)에 대한 설명은 표 2-2와 같다. 주소 변환의 단위인 페이지의 크기는 컴퓨터 시스템마다 다르다. 본 구조에서는 표 2-3과 같이 4가지의 페이지 크기를 모두 지원하도록 하였는데, 이것은 본 논문의 명령어 세트 구조의 규정에 의한 것이다. 따라서 ITLB의 각 엔트리들은 페이지 크기에 대한 정보를 가지고 있다. 그리고 VPN은 명령어 주소의 비트 10부터 비트 31까지의 22비트로 구성되어 있는데, 이것은 가장 작은 페이지 크기인 1KB에 대한 것이다. 따라서 페이지 크기가 1KB보다 큰 경우에는 VPN의 하위 비트들은 입력되는 주소와의 비교 과정에서 사용되지 않는다.

표 2-3 지원되는 페이지의 종류와 페이지 크기

Table 2-3 Types of supported pages and their sizes

Type of page	Page size
Section	1 MB
Large page	64 KB
Small page	4 KB
Tiny page	1 KB

페이지 크기는 명령어 캐쉬의 사이즈와 관련해서 매우 중요한 변수이다. 즉 페이지 크기에 따라서 페이지 오프셋의 비트 수가 결정되는데 이 비트 수가 명령어 캐쉬를 인덱싱하는데 사용되는 명령어 주소 비트들의 수, 즉 명령어 캐쉬의 인덱

스(index) 비트 수와 바이트 오프셋(byte offset) 비트 수의 합보다 작은 경우에는 명령어 캐쉬의 액세스와 ITLB 액세스가 동시에 이루어질 수 없다. 이 경우 ITLB 액세스 후에 명령어 캐쉬를 액세스해야 하므로 명령어 페치의 지연 시간이 길어진다. 이러한 문제점 때문에 캐쉬의 크기는 페이지 크기에 의해서 제한된다. set-associative 캐쉬^[38]를 사용하여 캐쉬의 way^[38] 수를 크게 함으로써 캐쉬의 크기를 늘릴 수는 있으나, 이것은 근본적인 해결책이 아니다. 그래서 본 구조에서는 OS가 페이지 컬러링(page coloring)^{[38][71]}을 사용하여 명령어 캐쉬의 인덱싱을 위한 하위 주소 비트들에 대해서는 가상주소와 실제주소가 서로 같도록 보장한다는 것을 전제로 하였다.

명령어 캐쉬는 최근에 또는 자주 사용되는 명령어들을 저장하고 있는 것으로서, 명령어 페치를 빠르게 할 수 있도록 한다. 본 구조는 슈퍼스칼라 방식을 사용하기 때문에 단일 사이클에 페치되는 명령어는 다수이며, 일반적으로 페치 폭만큼의 명령어들을 출력하는 역할을 한다. 그러나, 항상 페치 폭만큼의 명령어들을 출력하는 것은 아니다. 예를 들어서 PC가 페치 폭에 의해 정해진 경계로 정렬되지 않은 경우에는 PC 이전의 주소에 해당하는 명령어들은 의미가 없다. 또는 현재 페치되는 명령어 열에 대해서 taken 분기 예측이 이루어지면 예측된 분기 주소 이후의 명령어들도 의미가 없는 것이다. 이와 같이, 명령어 캐쉬는 항상 정렬된 명령어 주소에 대해 페치 폭만큼의 명령어들을 제공하지만, 경우에 따라서 그 중의 일부분은 무효한(invalid) 명령어이며 명령어 페치 큐에 저장되지 않는다.

2-3-3 명령어 페치 큐

명령어 페치 큐는 명령어 캐쉬로부터의 명령어들을 저장하는 것으로서, 명령어의 디코드와 이슈에서의 정지 시에도 명령어 페치를 계속할 수 있도록 한다. 만약에 명령어 페치 큐가 없는 경우에는 명령어 디코드나 이슈에서의 정체가 발생할 경우에 명령어 페치를 수행할 수 없게 되고, 차후에 명령어 페치가 정체되면 명령어 디코드 및 이슈도 정체되기 쉽다. 그러나, 명령어 페치 큐를 사용함으로써, 명령어 페치의 정체 시에도 명령어 디코드와 이슈는 이미 페치된 명령어 페치 큐의 명령어들을 사용하여 이루어질 수 있다.

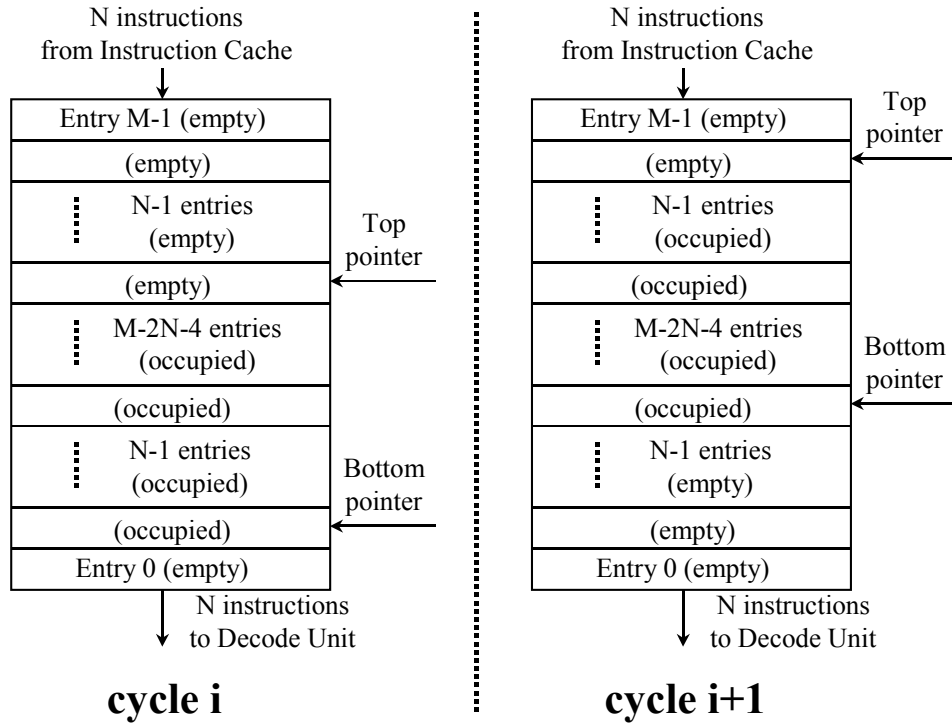


그림 2-10 명령어 페치 큐의 구조

Fig. 2-10 Structure of the instruction fetch queue

명령어 페치 큐는 그림 2-10과 같이 최상위 포인터와 최하위 포인터에 의한 선입선출 방식의 큐(queue)이다. 명령어 캐쉬로부터의 명령어들은 최상위 포인터가 가리키는 최상위 엔트리(top entry)와 그 다음의 엔트리들에 저장되며, 최상위 포인터의 값은 페치된 명령어의 수만큼 증가하게 된다. 명령어 캐쉬로부터의 명령어들을 명령어 페치 큐에 저장할 때에는 유효한(valid) 명령어들만이 저장되어야 한다. PC의 주소가 정렬되어 있지 않거나 페치되는 명령어 열에 taken 분기

예측이 존재할 경우에는 명령어 중에서 유효한 것들만을 저장해야 한다. 명령어의 유효성(validity)은 각 명령어에 수반되어 입력되는 유효 비트(valid bit)를 사용하여 이루어진다. 한편으로는, 최하위 포인터가 가리키는 최하위 엔트리(bottom entry)부터 디코드 폭(decode width)만큼의 명령어들이 디코드 유닛으로 보내지며, 유효하게 디코드된 명령어들만큼이 명령어 페치 큐로부터 빠져나가고 최하위 포인터의 값은 그 수만큼 증가한다. 명령어 페치 큐의 각 엔트리는 명령어와 그 명령어 주소를 포함한다. 명령어 주소는 명령어와 함께 입력되는 명령어 주소를 같이 저장한 것이다. 이와 같이 명령어 주소를 명령어 페치 큐에 저장하는 것은 명령어 주소가 명령어의 입력 오퍼랜드로 사용되는 경우가 존재하고, 또한 분기 예측 오류나 예외의 발생 때에 PC를 적절하게 복구하기 위해서이다. 따라서 명령어 페치 큐의 명령어 주소는 페치 이후의 모든 파이프라인 단계에서 명령어와 함께 움직이고, 디코드 유닛은 명령어 페치 큐로부터 명령어 뿐 아니라 명령어 주소도 같이 입력으로 받아들인다.

2-4절 디코드 유닛

디코드 유닛은 그림 2-11과 같이 명령어 페치 큐로부터의 명령어들을 디코드 해서 명령어 이슈 큐로 전달하는 역할을 수행한다. 이러한 명령어의 디코드 작업은 디코드 폭만큼의 명령어들에 대해서 동시에 이루어지며, 명령어 디코드 작업에서는 다음과 같은 것들이 이루어진다.

우선, 명령어 디코드에 의해서는 명령어가 어떤 동작을 수행하는 것인지에 대한 확인이 이루어지며, 이것은 내부적으로 사용되는 오퍼 코드(op code)로 해석된다. 그리고 명령어들의 소스 레지스터 주소들, 목적 레지스터 주소들, 명령어가 사용하는 기능 유닛의 종류 및 명령어의 상수 데이터 등의 정보를 해석한다. 이러한 디코드 작업에 의해서 해석된 단일 명령어에 대한 정보들의 집합을 디코드된 명령어(decoded instruction)로 볼 수 있다. 디코드 유닛은 명령어들을 디코드한 후에 디코드된 명령어들을 명령어 이슈 큐로 전달한다. 이 과정에서 명령어 주소는 디코드된 명령어와 같이 명령어 이슈 큐로 전달되어 저장된다.

명령어의 디코드 작업은 항상 디코드 폭만큼의 명령어들에 대해서 이루어 질 수 있는 것은 아니다. 명령어 페치 큐에 존재하는 명령어의 수가 디코드 폭보다 작거나, 명령어 이슈 큐의 빈(empty) 엔트리 수가 디코드 폭보다 작은 경우에 명령어 디코드는 디코드 폭 이하로 이루어진다. 디코드되는 명령어의 수는 명령어 페치 큐의 최하위 포인터의 값과 명령어 이슈 큐의 최상위 포인터의 값에 영향을 주어서, 디코드되는 명령어 수만큼 명령어 페치 큐의 최하위 포인터와 명령어 이슈 큐의 최상위 포인터가 증가한다.

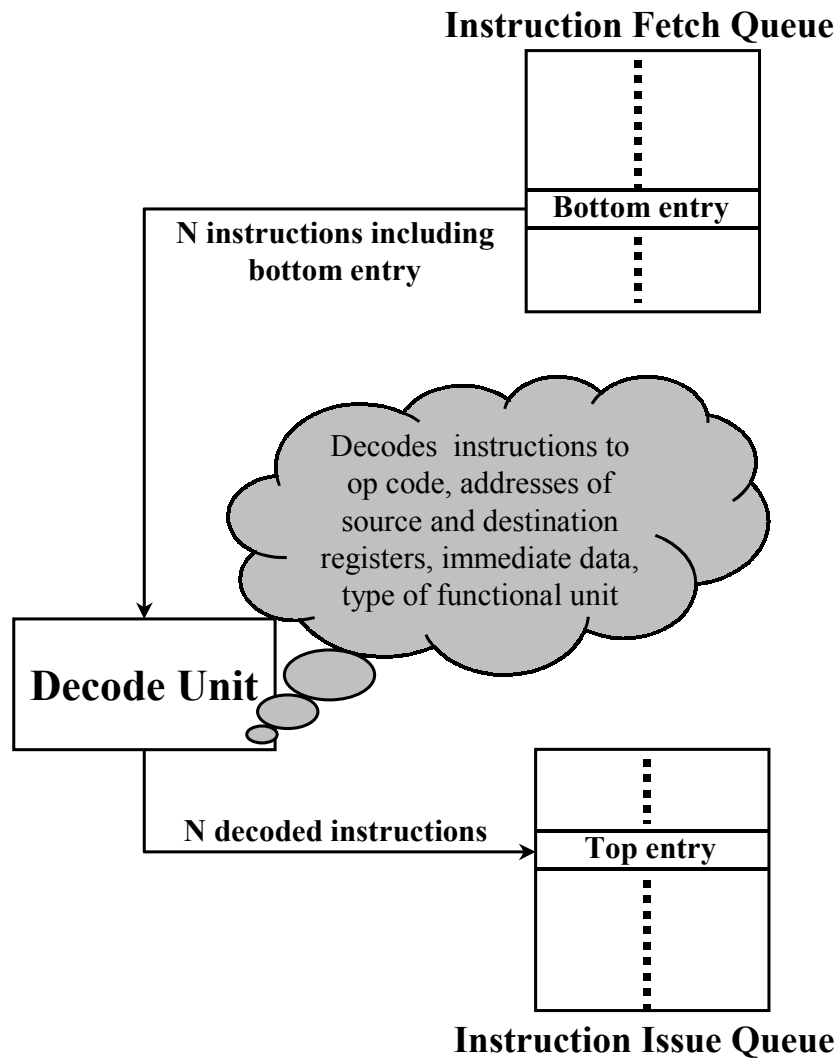


그림 2-11 디코드 유닛의 명령어 디코딩

Fig. 2-11 Instruction decoding of the decode unit

2-5절 레지스터 파일과 스코어보드 어레이

2-5-1 레지스터 파일

본 논문의 명령어 세트 구조에서는 4비트의 레지스터 주소를 사용한다. 따라서 보통의 경우 각 명령어들은 $16(2^4)$ 개의 일반 레지스터들 중에서 일부분을 소스 레지스터와 목적 레지스터로 사용한다. 그러나, 실제로 존재하는 레지스터의 수는 16개보다 더 많은데, 그 이유는 표 2-4와 같이 7개의 프로세서 모드(processor mode)가 제공되기 때문이다. 그 중에서 User 모드는 응용 프로그램이 정상적인 상황에서 수행되는 모드로서 보호된(protected) 시스템 상태를 변경할 수 없는 모드이다. 대부분의 경우에는 User 모드에서 프로그램이 수행된다. System 모드는 시스템 프로그램이 보호된 시스템 자원을 액세스하는 작업을 수행할 때 사용하는 모드이다. 그 외의 5개의 모드들은 예외와 관련되어 있는 것들이다. FIQ(Fast Interrupt reQuest) 모드는 FIQ 예외 발생에 의해서 프로세서가 들어가는 모드이고 FIQ 예외에 대한 처리는 이 모드에서 이루어진다. 그리고 IRQ(Interrupt ReQuest) 모드는 IRQ 예외, Supervisor 모드는 Reset 또는 SWI(SoftWare Interrupt) 예외, Abort 모드는 Prefetch Abort 또는 Data Abort 예외, Undefined 모드는 Undefined 예외와 각각 관련된 모드들이다. 이러한 7개의 모드 중에서 User 모드를 제외한 모드들에서는 모든 시스템 자원을 액세스하고 변경할 수 있다. 즉 User 모드를 제외한 모든 모드들은 privileged 모드로서 정의된다.

각각의 모드들은 그 모드만의 레지스터들을 가지고 있다. 레지스터 주소는 R0부터 R15까지 16개이지만 레지스터 주소가 같더라도 어떤 모드이냐에 따라서 실제 레지스터가 달라지며, 그림 2-12와 같은 레지스터 구조를 가진다. User 및 System 모드는 프로그램의 정상적인 수행을 위한 레지스터들인 R0부터 R14까지의 레지스터들과 R15(PC)를 사용한다. 그러나 FIQ 모드에서는 R0부터 R7까지의 레지스터들 및 R15는 User 모드의 레지스터들을 공유해서 사용하지만 R8부터 R14까지의 레지스터들에 대해서는 FIQ 모드에서만 액세스되는 고유의 레지스터들을 가지고 있다. 다른 예외 관련 모드들도 R13(stack pointer) 및 R14(link

register)에 대해서는 고유의 레지스터들을 가지고 있고, 그 외의 레지스터들에 대해서는 User 모드의 레지스터들을 공유한다. 레지스터 파일은 R15를 제외한 R0부터 R14까지의 모든 레지스터들을 포함하는 것으로서, 총 30개의 레지스터로 구성된다.

표 2-4 프로세서 모드들과 사용 가능한 레지스터들

Table 2-4 Processor modes and their accessible registers

프로세서 모드	관련된 예외	사용 가능한 레지스터들
User	없음	PC, R14 to R0, CPSR
System	없음	PC, R14 to R0, CPSR
FIQ	FIQ (fast interrupt)	PC, R14_fiq to R8_fiq, R7 to R0, CPSR, SPSR_fiq
IRQ	IRQ (interrupt)	PC, R14_irq to R13_irq, R12 to R0, CPSR, SPSR_irq
Supervisor	Reset SWI	PC, R14_svc to R13_svc, R12 to R0, CPSR, SPSR_svc
Abort	Prefetch Abort Data Abort	PC, R14_abt to R13_abt, R12 to R0, CPSR, SPSR_abt
Undefined	Undefined	PC, R14_und to R13_und, R12 to R0, CPSR, SPSR_und

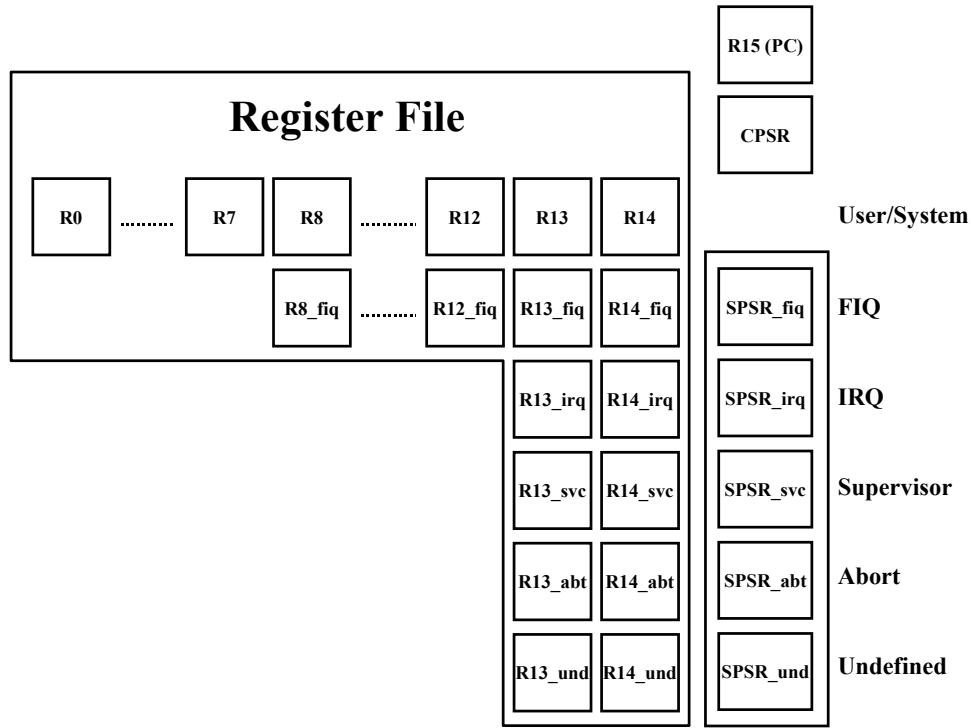


그림 2-12 레지스터 파일 구조

Fig. 2-12 Register File structure

그림 2-12의 CPSR(Current Program Status Register)은 현재의 프로그램 상태를 저장하는 것으로서, 조건 코드 플래그들(N, Z, V, C, Q), 인터럽트 디세이블 비트들(interrupt disable bits) 및 현재의 프로세서 모드를 포함한다. 이러한 CPSR은 단지 1개만이 존재하며, 모든 프로세서 모드에서 액세스가 가능하다. SPSR(Saved Program Status Register)은 User 및 System 모드를 제외한 모드들 즉, 예외와 관련된 모드들마다 1개씩 존재하며, 예외 발생 시 CPSR을 저장하는데

사용된다. 따라서 User 및 System 모드에서는 SPSR를 액세스할 수 없으며, 그 외의 모드들에서는 각 모드의 SPSR를 액세스할 수 있다. CPSR, 5개의 SPSR 및 PC(R15)는 일반 레지스터와는 다른 특별한 의미가 존재하고 언제든지 액세스할 수 있어야 하기 때문에, 레지스터 파일 밖에 플립플롭(flip-flop) 형태의 레지스터로서 존재한다.

2-5-2 스코어보드 어레이

스코어보드 어레이는 이슈 유닛이 명령어를 이슈할 때 데이터 의존성 검사를 수행하고 바이패싱 정보를 만들어내기 위해서 사용되는 것으로서, 각 엔트리는 대응하는 레지스터에 대한 최신의 갱신 상태를 알려준다. R15(PC)의 값, 즉 명령어 주소 값은 모든 파이프라인 단계들에서 각 명령어에 수반하여 존재하므로, R15에 대한 의존성 검사는 필요가 없으며 분기 예측 및 분기 예측 오류 검사에 의해서 PC에 대한 의존성 문제는 모두 해결된다. 그리고, 조건 코드 플래그들을 제외한 CPSR 및 SPSR들을 변경하는 명령어는 매우 드물기 때문에 이 레지스터들에 대해서는 별도의 의존성 검사 방법을 사용한다. 따라서 스코어보드 어레이에는 레지스터 파일의 30개의 레지스터와 조건 코드 플래그들에 대한 갱신 정보만 있으면 된다. 따라서, 5개의 조건 코드 플래그들을 1개의 레지스터로 보고 스코어보드 엔트리 1개(Scnd)를 할당하여 스코어보드 어레이에는 총 31개의 엔트리가 존재하게 되며, 각 엔트리는 대응하는 레지스터에 대한 최신의 갱신 정보를 가지고 있다.

각 스코어보드 엔트리는 그림 2-13과 같이 구성된다. 그림 2-13에서 FU 영역은 해당 레지스터의 최신 갱신이 어떤 기능 유닛에 존재하는 지를 알려주는 것이며, 레지스터 파일에 최신의 갱신 값이 존재하는 경우에 이 영역은 의미가 없다. 그리고, RUS 영역은 최신의 갱신이 어떤 파이프라인 단계에 존재하는지를 알려주며, 그 값으로는 R, E, M, W 및 레지스터 파일에 최신의 갱신 값이 존재하는 것을 의미하는 C(Committed) 중 한 개를 취한다. RUS가 C가 아닌 경우에는 데이터 바이패싱이 가능한지를 알 수 있어야 한다. 즉, 그 레지스터에 대한 결과는 나왔으나 레지스터 파일에 쓰여지기 전의 파이프라인 단계에 최신의 갱신이 존재하는 경우에는 해당 레지스터의 결과를 바이패싱하는 것이 가능하다. BPS 영역은

해당 레지스터의 결과 값을 바이패싱할 수 있는 파이프라인 단계를 의미하며, BPS의 값이 M인 경우에는 해당 레지스터를 오퍼랜드로 사용하는 명령어의 이슈 사이클에서 RUS가 E 또는 M인 경우에는 데이터 바이패싱이 가능하며, RUS가 W 또는 C인 경우에는 레지스터 파일에서 오퍼랜드를 읽는다. 반면에 그 외의 경우에는 데이터 의존성에 의해서 해당 명령어는 이슈되지 못한다. BPS와 RUS의 이와 같은 관계는 명령어 이슈 다음 사이클에 오퍼랜드 읽기를 수행하기 때문이다.

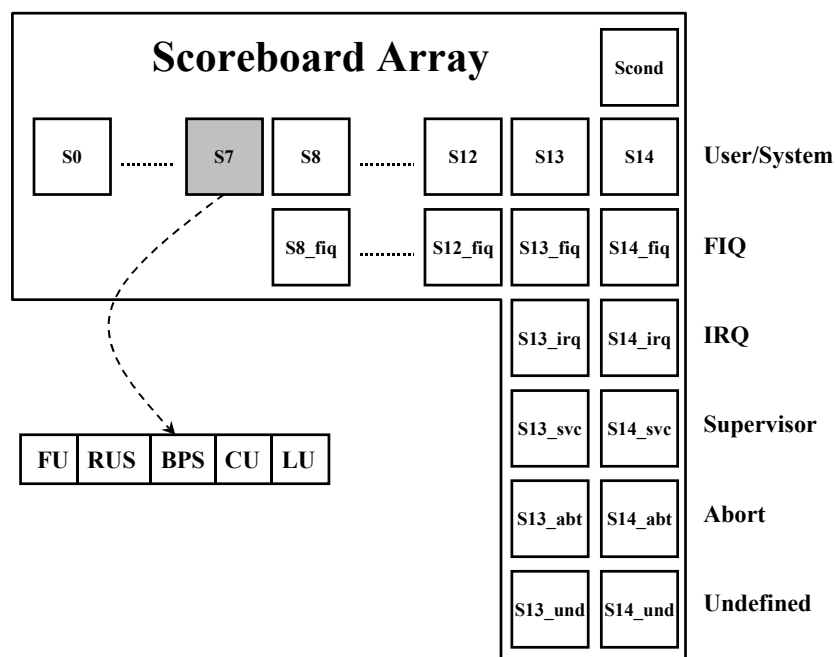


그림 2-13 스코어보드 어레이의 구조
Fig. 2-13 Scoreboard array structure

데이터 바이패싱이 가능한 경우에 바이패싱 정보는 FU와 RUS를 이용하여 만들어진다. 그 외의 영역으로는, 해당 레지스터에 대한 갱신이 조건부 수행 명령어에 의한 것임을 알려주는 CU 비트 및 로드 동작에 의해서 갱신되는 것임을 알려주는 LU 비트가 존재한다. CU 비트는 차후에 조건부 수행 명령어를 취소할 때에 그 명령어에 의한 레지스터 갱신 상태도 취소하기 위해서 존재하는 것이며, LU는 로드 명령어의 데이터 메모리 액세스 동작이 EM 단계로 넘어갈 경우를 대비한 것이다.

2-6절 이슈 유닛

이슈 유닛은 명령어 이슈 큐의 명령어들을 순차적으로 기능 유닛들로 이슈하고, 프로세서 전반의 파이프라인 제어 정보를 동적으로 만들어내는 유닛이다. 이슈 유닛은 명령어 이슈 큐 및 명령어 이슈 제어 로직(instruction issue control logic)으로 구성된다.

2-6-1 명령어 이슈 큐

명령어 이슈 큐는 디코드된 명령어들을 저장하고 있는 큐로서, 명령어 이슈는 명령어 이슈 큐의 명령어들을 사용하여 이루어진다. 명령어 이슈 큐는 그림 2-14와 같이 최상위 포인터와 최하위 포인터를 이용한 선입선출 방식의 큐이다. 그림 2-14에서 알 수 있듯이, 명령어 이슈 큐는 명령어 페치 큐와 같은 방식으로 동작한다. 최상위 포인터는 새로이 디코드된 명령어들을 저장할 엔트리의 위치를 가리키며, 이러한 엔트리를 최상위 엔트리라고 한다. 최하위 포인터는 명령어 이슈 큐의 엔트리들 중에서 가장 오래된 유효한 엔트리, 즉 프로그램 순서로 가장 앞의 명령어를 저장하고 있는 명령어 이슈 큐 엔트리의 위치를 가리키고, 이러한 엔트리를 최하위 엔트리라고 한다. 명령어 이슈는 순차적으로 이루어지기 때문에 최하위 엔트리의 명령어와 그 다음의 순차적인 명령어들만이 명령어 이슈의 고려 대상이 된다. 그리고 순서상 앞의 명령어가 이슈되지 못하면, 그 뒤의 명령어들은 다른 조건들로 보았을 때 이슈가 가능하더라도 명령어 이슈를 수행할 수 없다. 예를 들어서, 최하위 엔트리의 명령어가 이슈되지 못하면, 그 사이클에는 모든 명령어들은 이슈될 수 없다.

명령어 이슈 큐의 각 엔트리들은 디코드 유닛으로부터 전달되는 디코드된 명령어들을 저장하고 있으며, 디코드된 명령어는 오퍼 코드, 소스 레지스터 정보, 목적 레지스터 정보, 상수 데이터 및 명령어 수행에 사용되는 기능 유닛의 종류 등의 정보를 포함한다.

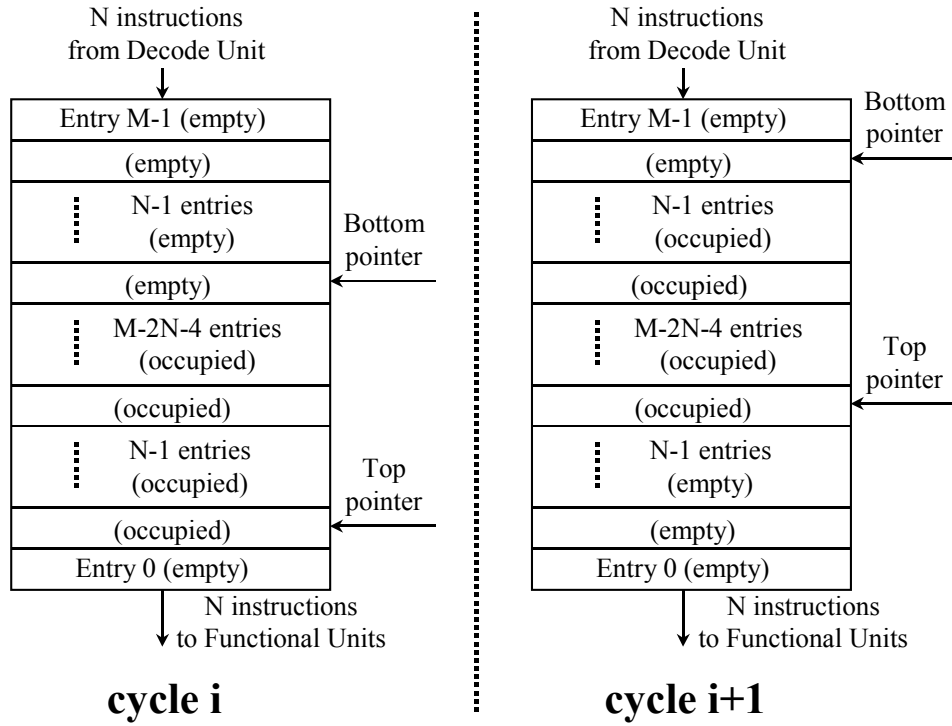


그림 2-14 명령어 이슈 큐의 구조

Fig. 2-14 Structure of the instruction issue queue

오피 코드는 명령어가 수행할 동작, 명령어를 수행하기 위해서 소비되는 사이클 수에 대한 정보 및 조건부 수행에 대한 정보를 포함하고 있다. 소스 레지스터 정보는 몇 개의 소스 레지스터를 오퍼랜드로 사용하는지에 대한 정보와 각 소스 레지스터의 주소 등을 포함한다. 소스 레지스터 정보는 조건 코드 플래그에 대한 정보를 제외하면 최대 4개의 소스 레지스터에 대한 정보를 포함할 수 있다. 목적 레지스터 정보는 명령어가 몇 개의 유효한 목적 레지스터를 가지고 있는지에 대한 정보와 각 목적 레지스터의 주소 등을 포함한다. 목적 레지스터는 조건 코드

플래그들에 대한 정보를 제외하면 최대 2개의 목적 레지스터에 대한 정보를 가지고 있을 수 있다. 상수 데이터는 보통의 경우에 명령어의 일부 영역을 사용해서 인코딩(encoding)된 상수 데이터이다. 그러나 LDM 또는 STM 명령어에서는 다른 의미를 가진다. LDM은 최대 16개까지의 로드를 수행하는 명령어로서, 이 명령어의 경우에 상수 데이터는 로드의 목적 레지스터 주소들을 비트맵(bit-map) 형식으로 인코딩한 것이다. 그리고, STM은 최대 16개까지의 스토어를 수행하는 명령어이며, 이 명령어의 경우에 상수 데이터는 스토어의 소스 레지 주소들을 비트맵 형식으로 인코딩한 것이다.

2-6-2 명령어 이슈

명령어 이슈 제어 로직은 명령어 이슈 큐의 명령어들을 검사하여서 최대 명령어 이슈 폭만큼의 명령어들을 기능 유닛으로 이슈하고, 이슈된 명령어들을 명령어 이슈 큐에서 제거하는 역할을 수행한다. 그리고 명령어 이슈 로직은 이슈되는 명령어들의 파이프라인 정보들을 만들어서 이것을 기능 유닛들로 전달하는 역할을 수행한다. 그 외에 명령어 이슈 로직은 이슈된 명령어들이 R 단계부터는 파이프라인 상에서 정지되는 일이 없도록 조정하는 역할과 스코어보드 엔트리들의 값을 갱신하는 역할을 수행한다. 다음과 같은 이슈 기준들이 명령어 이슈에 전반적으로 적용된다.

첫째, RAW(Read-After-Write)^[13] 데이터 의존성에 의한 이슈 제한이 있다. 명령어의 소스 레지스터 값을 레지스터 파일로부터 읽을 수도 없고 데이터 바이패싱에 의해서도 읽을 수 없는 경우에 해당 명령어는 이슈되지 못한다. 예를 들어서 명령어 A가 레지스터 R1을 소스 오퍼랜드로 사용하고, 레지스터 R1이 목적 레지스터인 명령어 B의 결과가 아직 나오지 않은 경우에는 레지스터 R1의 값을 바이패싱할 수도 없고 레지스터 파일에서 읽을 수도 없다. 이러한 경우에 명령어 A는 이슈되지 못한다.

둘째, WAW(Write-After-Write)^[13] 데이터 의존성에 의한 이슈 제한이 있다. 본 구조에서는 순차적으로 명령어들을 이슈하고 완료하기 때문에 WAW 의존성이 발생하는 경우가 원칙적으로는 거의 없다. 그러나 로드 명령어로 인해서 예외가

생긴다. 로드 명령어가 M 단계에서 데이터 캐쉬 미스가 발생하거나 또는 외부 메모리 로드인 경우에는 M 단계 후에 W 단계가 아닌 EM 단계로 가서 데이터 캐쉬 미스를 처리하거나 외부 메모리를 액세스한 후에 로드된 데이터를 레지스터 파일에 쓴다. 따라서 로드 명령어 이후에 로드 명령어와 같은 목적 레지스터를 가지는 명령어가 이슈되기 위해서는 그 명령어에 의한 레지스터 파일로의 쓰기가 로드 명령어에 의한 레지스터 파일 쓰기보다 나중에 이루어질 수 있도록 보장된 후에야 이슈될 수 있다. 그러므로 로드 명령어가 M 단계에서 데이터 캐쉬 히트가 나거나 EM 단계에서의 동작이 끝나는 것이 확실해진 이후에야 같은 목적 레지스터를 갖는 후속 명령어가 이슈될 수 있다. 후속 명령어를 미리 이슈하고서 선행하는 로드 명령어에서 데이터 캐쉬 미스와 같은 사건이 발생하면 실행 파이프라인을 정지시키는 방법도 생각할 수 있으나, 본 구조에서는 이슈된 명령어들을 실행 파이프라인에서 정지 없이 계속 수행한다는 것을 기본 정책으로 하기 때문에 이와 같은 이슈 제한을 가한 것이다. 만약에 이와 같은 WAW 데이터 의존성 검사 없이 로드 명령어의 후속 명령어를 이슈할 경우에는 그림 2-15와 같이 후속하는 명령어에 의한 레지스터 쓰기가 먼저 이루어지는 상황이 발생할 수 있다.

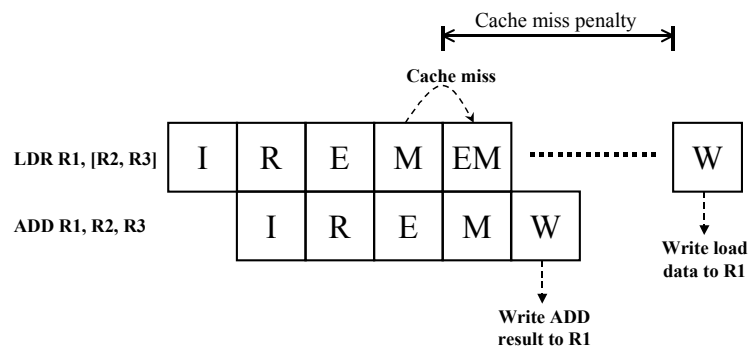


그림 2-15 잘못된 순서로 레지스터에 결과를 쓰는 예

Fig. 2-15 Example of the incorrect sequence of register writes

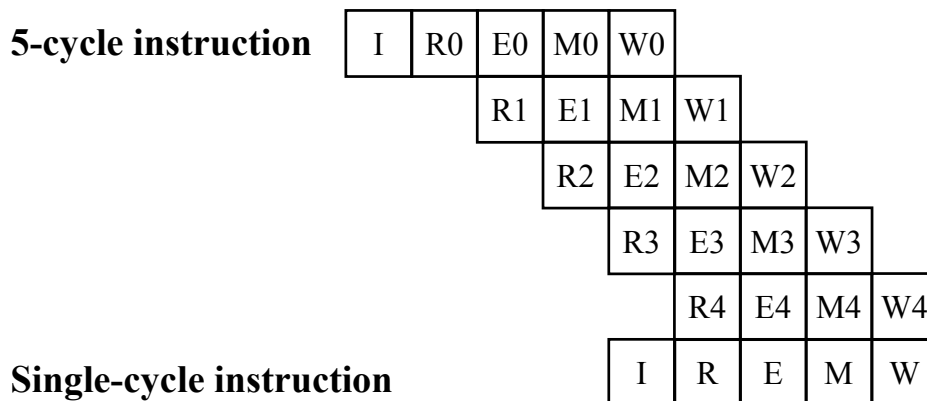


그림 2-16 다중 사이클 명령어로 인한 후속 명령어의 이슈 지연

셋째, 다중 사이클 명령어로 인한 이슈 제한이 존재한다. 다중 사이클 명령어는 이슈된 후에 R 단계에서 E 단계로 넘어가면서 또 다른 R 단계를 만들어 낸다. 이렇게 새로 만들어진 R 단계는 처음의 R 단계처럼 E, M 및 W 단계들을 거쳐서 파이프라인을 흘러가게 된다. 이와 같이 다중 사이클 명령어들은 다수의 파이프라인 흐름을 자체적으로 형성하면서 다수의 동작들을 계속적으로 수행하는 구조를 가진다. 이러한 점에서 첫째 R 단계를 R0 단계로 볼 수 있으며, 둘째 R 단계를 R1 단계라고 할 수 있다. 일반적으로 다중 사이클 명령어의 (N+1) 번째 R 단계를 RN 단계라고 하며, E, M 및 W 단계에 대해서도 같은 명칭들이 사용된다. 본 구

조는 순차적으로 명령어들을 이슈하고 완료하는 구조이다. 그런데 다중 사이클 명령어에 후속하는 명령어들이 다중 사이클 명령어와 같이 이슈된다면 다중 사이클 명령어보다 먼저 이슈되고 완료되는 효과를 보게 된다. 따라서 본 구조에서 다중 사이클 명령어의 마지막 R 단계보다 후속하는 명령어의 첫째 R 단계가 먼저 올 수 없도록 제한을 가한다. 그림 2-16은 다섯 사이클 명령어에 후속하는 명령어가 이슈되는 모습을 예로서 보여준다.

넷째, 자원 충돌로 인한 이슈 제한이 존재한다. 마이크로프로세서에서 자원이라 함은 여러 가지가 존재하지만 명령어 이슈와 관련되어서 자원이라 함은 레지스터 파일의 포트와 기능 유닛을 의미한다. 이슈된 명령어들은 R 단계에서 레지스터 파일 읽기를 수행한다. 그런데 레지스터 파일에서 제공하는 읽기 포트의 수보다 더 많은 읽기가 존재한다면 일부 명령어들은 R 단계에서의 읽기 동작을 수행하지 못하고 정지되어야 한다. 이것은 R 단계 이후로는 명령어 수행의 정지가 존재하지 않는다는 본 구조의 정책과 어긋나게 된다. 따라서 레지스터 파일의 읽기 포트 수에 의해서 명령어 이슈가 제한된다. 레지스터 파일로의 쓰기 동작에도 같은 원칙이 적용된다. 같은 사이클에 이슈된 명령어들은 같은 사이클에 W 단계에 도달하게 되어서 레지스터 파일로의 쓰기를 동시에 한다. 따라서 같은 사이클에 이슈되는 명령어들의 쓰기 동작 수와 레지스터 파일의 쓰기 포트(write port) 수를 고려해서 명령어 이슈를 제한하여야 한다. 또한 기능 유닛에 의해서도 명령어 이슈가 제한된다. 예를 들어서 ALU의 수가 4개인 경우에는 ALU를 사용하는 명령어는 단일 사이클에 4개까지만 이슈될 수 있는 것이다. 자원 충돌로 인한 명령어 이슈 제한에는 이전에 이슈된 다중 사이클 명령어도 같이 고려해야 한다. 이전에 이슈된 다중 사이클 명령어가 새로이 이슈되는 명령어들과 같은 사이클에 R 단계에 존재하게 된다면 다중 사이클 명령어와 이슈된 명령어들은 같은 사이클에 같은 단계의 자원들을 사용하기 때문에 자원 충돌이 발생할 수 있기 때문에, 이미 이슈된 다중 사이클 명령어도 자원 충돌에서 고려의 대상이 되는 것이다..

마지막으로, 이슈 폭에 의해서 명령어 이슈가 제한된다. 예를 들어서 이슈 폭이 4개의 명령어인 경우에는 단일 사이클에 이슈할 수 있는 명령어의 수는 4개로 제한된다.

표 2-5 명령어 이슈 기준

Table 2-5 Instruction issue rules

이슈 제한 원인	설명
순차적 이슈	이슈되지 않은 명령어 이후의 명령들을 이슈할 수 없음
프로세서 모드 변경	프로세서의 모드를 변경하는 명령어가 실행 중인 동안에는 후속 명령어들을 이슈할 수 없음
다중 사이클 명령어	다중 사이클 명령어 이후의 명령어들은 다중 사이클 명령어의 마지막 R 단계에 맞추어서 또는 그 이후에 이슈할 수 있음
CPSR 쓰기	CPSR을 변경하는 명령어 다음에 CPSR을 읽는 명령어가 올 경우에 두 명령어를 같은 사이클에 이슈할 수 없음
SPSR 쓰기	SPSR을 변경하는 명령어 다음에 SPSR을 읽는 명령어가 올 경우에 두 명령어를 같은 사이클에 이슈할 수 없음
기능 유닛의 부족	같은 종류의 기능 유닛을 사용하는 명령어들을 단일 사이클에 이슈할 수 있는 최대 수는 그 종류의 기능 유닛의 수에 의해서 제한됨
레지스터 파일의 읽기 포트 부족	단일 사이클에 이슈되는 명령어들에 의한 레지스터 파일 읽기 동작은 레지스터 파일의 읽기 포트 수보다 클 수 없으며, 이로 인해 명령어 이슈가 제한됨
레지스터 파일의 쓰기 포트 부족	단일 사이클에 이슈되는 명령어들에 의한 레지스터 파일 쓰기 동작은 레지스터 파일의 쓰기 포트 수보다 클 수 없으며, 이로 인해 명령어 이슈가 제한됨
RAW 데이터 의존성	명령어 실행 결과가 나오지 않은 명령어에 대해서 RAW 데이터 의존성을 가지는 명령어들을 이슈할 수 없음
WAW 데이터 의존성	M 단계 이전 또는 EM 단계에 존재하는 load 명령어와 같은 목적 레지스터를 가지는 명령어를 이슈할 수 없음
명령어 이슈 폭	단일 사이클에 이슈할 수 있는 명령어들의 최대 수는 명령어 이슈 폭에 의해서 제한됨

상기의 이슈 기준 중에 WAR(Write-After-Read)^[13] 데이터 의존성으로 인한 명령어 이슈 제한은 존재하지 않는데, 본 구조에서는 명령어 이슈가 순차적으로 이루어지기 때문이다. 상기의 명령어 이슈 기준들은 대략적인 것이며 세부적으로는 더 복잡한데, 이는 표 2-5에 정리되어 있다.

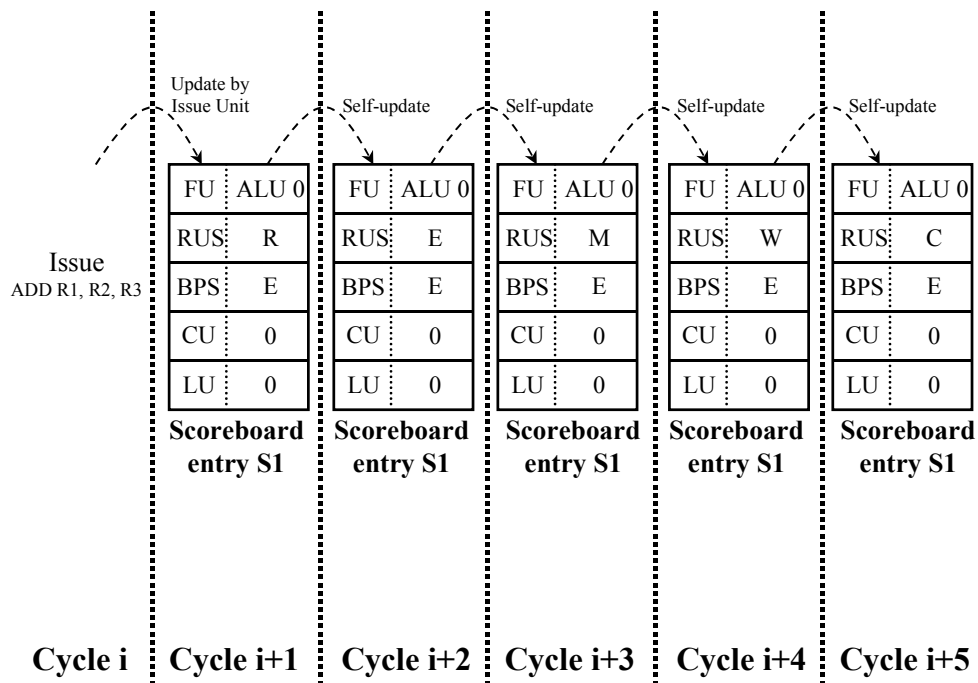


그림 2-17 스코어보드 엔트리의 변경을 보여주는 예

Fig. 2-17 Update example of a scoreboard entry

명령어 이슈 제어 로직은 명령어를 이슈하면서 관련된 스코어보드 엔트리들을 갱신한다. 이슈되는 명령어의 목적 레지스터에 대응하는 스코어보드 엔트리는 이슈되는 명령어에 의한 새로운 레지스터 쓰기를 반영하여 엔트리의 값들을 갱신해야한다. 명령어 이슈에 의해서 갱신된 스코어보드 엔트리는 해당 레지스터에 대한 새로운 쓰기 명령어가 이슈될 때까지는 자체적으로 그 값을 갱신한다. 그림 2-17은 명령어 이슈에 의한 스코어보드 엔트리의 갱신과 자체적인 갱신의 예를 보여주는 것이다.

2-7절 기능 유닛과 데이터 메모리 액세스

2-7-1 기능 유닛

이슈 유닛에 의해서 기능 유닛으로 이슈된 명령어들은 할당된 기능 유닛의 오퍼랜드 읽기 버퍼에 저장되고 R 단계의 동작을 수행한다. 그리고 R 단계부터 W 단계까지의 동작은 기능 유닛에서 이루어지며, 기능 유닛의 종류에 따라 각 단계에서 이루어지는 동작은 그림 2-3과 같다. 기능 유닛의 종류에는 다음과 같이 ALU, 곱셈 유닛 및 로드/스토어 유닛이 존재한다.

ALU는 산술 및 논리 연산 명령어와 분기 명령어 및 CLZ 명령어들을 실행하는 유닛이다. 또한 로드/스토어 명령어의 베이스 갱신(base update) 동작이 ALU에서 수행된다. 따라서 베이스 갱신 동작을 수반하는 로드/스토어 명령어는 ALU와 로드/스토어 유닛을 동시에 사용한다. ALU로 이슈되어 오퍼랜드 읽기 버퍼에 저장된 명령어들은 R 단계에서 오퍼랜드 읽기를 수행한다. 오퍼랜드는 레지스터 파일에서 직접 읽거나 또는 현재 수행 중인 명령어로부터의 바이패싱에 의해서 얻는다. 오퍼랜드를 얻은 명령어는 ALU의 실행 자원을 사용해서 실행되며, 그 결과를 M 단계 버퍼에 저장하는 것까지가 E 단계의 동작에 해당한다. ALU의 M 단계는 다른 기능 유닛들과의 동기화를 위한 단계로서 아무런 동작도 수행하지 않는다. 따라서 ALU의 M 단계는 W 단계로 명령어와 명령어 결과를 전달하는 것으로 끝난다. W 단계는 명령어의 결과를 레지스터 파일에 쓰면서 명령어의 수행을 마친다. 그림 2-18은 ALU의 각 실행 단계들에서 이루어지는 동작들을 예로서 보여주고 있다.

곱셈 유닛은 곱셈 명령어, MAC 명령어 및 DSP 확장 명령어들을 실행하는 기능 유닛이다. 곱셈 유닛이 각 파이프라인 단계에서 수행하는 동작은 전반적으로 ALU와 비슷하다. 그러나 실행하는 명령어의 종류가 ALU와 다르기 때문에 그로 인한 차이점을 보인다. 곱셈 유닛의 E 단계에서는 곱셈기를 이용한 곱셈이 이루어지며 M 단계에서는 누적 동작이 이루어진다. 그런데 곱셈 동작이나 누적 동작을 한 번씩만 수행하여 해당 명령어를 실행할 수 있는 경우도 있지만, 어떤 명령어들

에 대해서는 이러한 동작을 두 번씩 수행해야 명령어의 전체 동작을 실행한다. 이러한 명령어는 두 사이클 명령어이며, 첫째 R 단계인 R0 단계가 E0 단계로 전달 되면서, 새로운 R 단계인 R1 단계가 형성되고, R1은 차후에 E1, M1 및 W1 단계를 통해서 전달되면서 할당된 동작들을 수행한다. 따라서 곱셈 유닛의 오퍼랜드 읽기 버퍼에는 새로운 R 단계를 활성화시키기 위한 로직이 추가적으로 필요하다.

ADD R1, R2, R3

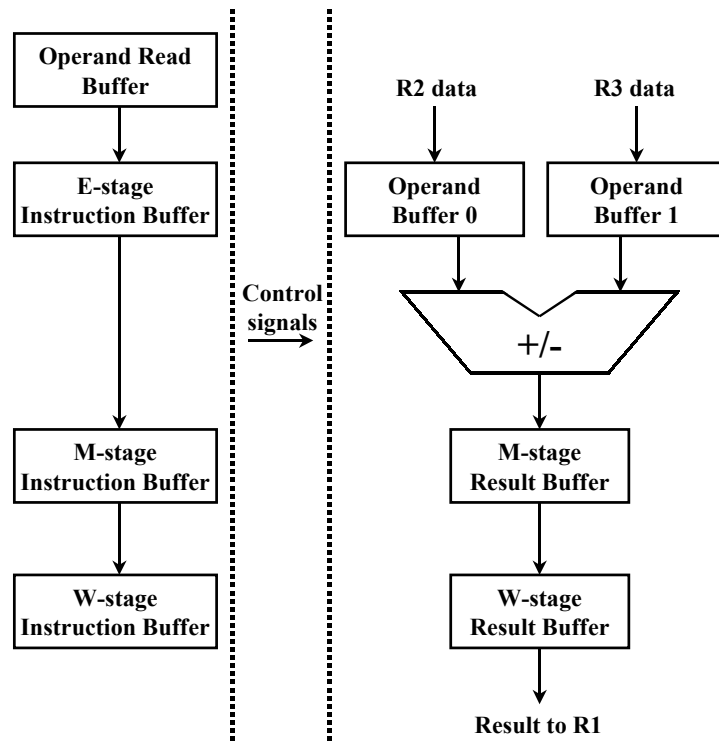


그림 2-18 파이프라인의 각 단계에서의 ALU 동작 예

Fig. 2-18 Example of ALU operation in each pipeline stage

로드/스토어 유닛은 로드 및 스토어 명령어를 포함해서 데이터 메모리를 액세스하는 모든 명령어들을 실행하기 위한 기능 유닛이다. 로드/스토어 유닛이 R 단계에서 수행하는 동작은 곱셈 유닛의 동작과 비슷하며 오퍼랜드 읽기와 함께 다중 사이클 명령어의 경우에 새로운 R 단계의 활성화 동작을 동시에 수행한다. 다만 곱셈 유닛 명령어의 최대 수행 사이클 수는 2 사이클이고, 로드/스토어 유닛의 경우 LDM이나 STM의 명령어에 대해서 최대 수행 사이클 수가 16 사이클이기 때문에 새로운 R 단계의 활성화가 더욱 반복적으로 수행된다는 차이점이 있다. 로드/스토어 유닛의 E 단계에서는 데이터 메모리 주소를 계산하고, 이 주소를 사용하여 M 단계에서는 DMMU와 데이터 캐쉬를 사용한 로드 또는 스토어 동작을 수행한다. W 단계는 로드 명령어에 대해서 레지스터 파일로의 쓰기 동작이 이루어지고, 스토어 명령어의 경우에는 아무런 동작 없이 끝난다. M 단계에서 데이터 캐쉬 미스가 발생하거나 또는 해당 데이터 액세스가 uncached 영역을 액세스하는 경우에는 데이터 액세스는 EM 단계로 넘겨지고, EM 단계에서 여러 사이클에 걸쳐 데이터 캐쉬 미스를 처리하거나 외부 메모리 영역을 액세스한 후에 W 단계로 넘어가게 된다. 따라서 EM 단계를 거치는 명령어에 의한 프로세서 상태의 변화는 후속하는 명령어에 의한 것보다 나중에 이루어지므로, 이러한 명령어들에 대한 예외의 처리를 precise^{[68][69]}하게 처리할 수 없다. 따라서 본 구조에서는 데이터 캐쉬 미스를 처리하거나 외부 메모리를 액세스하는 중에 발생하는 예외에 대해서는 precise하지 않아도 된다는 규정을 정하였다. DTLB(Data Translation Lookaside Buffer) 미스가 발행하면 메모리 액세스가 단일 사이클에 이루어질 수 없다. 그런데 DTLB 미스는 데이터 캐쉬 미스처럼 처리할 수가 없다. DTLB 미스는 프로그램의 정상적인 수행 시에 발생하는 것이기 때문에 DTLB 미스가 난 명령어에 의한 예외를 precise하지 않게 처리하는 것은 명령어 세트 구조 자체의 규정에 어긋나는 것이다. 따라서 본 구조에서는 DTLB 미스 시 후속하는 명령어들을 모두 플러쉬하고, DTLB 미스 처리 후에 해당 명령어부터 다시 페치하여 수행하도록 하고 있다. 이러한 동작은 프로세서의 사이클을 많이 낭비하는 것이지만, 보통 DTLB 미스율(miss rate)이 매우 작다는 것을 고려하면 프로세서의 성능에 별 영향을 미치지 못한다.

2-7-2 데이터 메모리 액세스

데이터 메모리 액세스와 관련된 유닛으로는 로드/스토어 유닛, DMMU 및 데이터 캐쉬가 있다. 본 절에서는 이들의 상호 작용에 의한 데이터 메모리 액세스 동작을 설명한다.

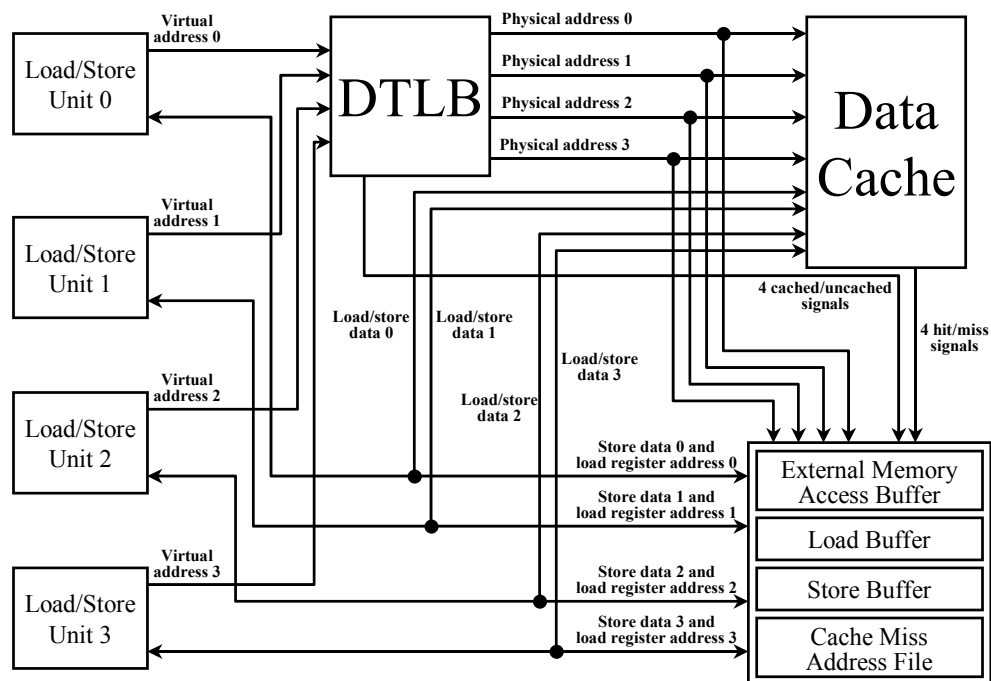


그림 2-19 다수의 액세스를 동시에 수행하는 데이터 메모리 액세스 구조

Fig. 2-19 Data memory access structure supporting simultaneously multiple accesses

로드/스토어 유닛의 M 단계에서 데이터 메모리에 대한 액세스는 다음과 같은 순서로 이루어진다. 우선 DMMU를 액세스하여 데이터 가상주소를 데이터 실제주소로 변환하고, 실제주소를 이용하여 실제주소 캐쉬인 데이터 캐쉬를 액세스함으로써 데이터 메모리 액세스가 끝난다. 본 구조에서는 로드/스토어 유닛의 수가 여러 개 존재할 수 있다. 이러한 경우에는 다수의 데이터 메모리 액세스가 동시에 발생할 수 있는데, 다수의 데이터 메모리 액세스 요구를 동시에 처리하기 위해서 DMMU는 로드/스토어 유닛의 수만큼 동시에 주소 변환을 수행할 수 있어야 한다. 이를 위해서 DTLB의 읽기 포트 수는 로드/스토어 유닛의 수만큼 존재해야 한다. 데이터 캐쉬 액세스도 로드/스토어 유닛의 수만큼 동시에 이루어지기 때문에 데이터 캐쉬의 액세스 포트 수도 로드/스토어 유닛의 수만큼 존재해야 한다. 그리고 데이터 캐쉬는 write-back, write-allocate^[38] 캐쉬로 구성된다. 그림 2-19는 로드/스토어 유닛의 수가 4개인 경우 데이터 메모리 액세스가 동시에 이루어지는 모습을 보여주는 것이다.

그림 2-19에서 보면 외부 메모리 액세스 버퍼(external memory access buffer), 로드 버퍼(load buffer), 스토어 버퍼(store buffer) 및 캐쉬 미스 주소 파일(cache miss address file)이 존재하는 것을 볼 수 있다. 이것들은 데이터 액세스가 uncached 영역에 대한 것이거나 캐쉬 미스가 발생할 경우를 위한 것이다. 로드/스토어 유닛의 M 단계에서 DTLB 미스가 발생하면 해당 명령어와 후속하는 명령어들을 파이프라인 상에서 모두 플러쉬하고, DTLB 미스를 처리한 후에 DTLB 미스가 발생했던 명령어부터 다시 페치를 시작해서 명령어 수행을 계속해 나간다. DTLB에서 히트가 발생한 경우에는 cached 영역인지 또는 uncached 영역인지에 대한 검사를 수행하여서 uncached 영역인 경우에는 해당 메모리 액세스 요구를 외부 메모리 액세스 버퍼에 등록한다. 그리고 로드인 경우에는 목적 레지스터 주소를 포함한 로드 관련 정보들을 로드 버퍼에 등록하고, 스토어인 경우에는 스토어 데이터를 포함한 스토어 관련 정보들을 스토어 버퍼에 등록한다. cached인 경우에는 데이터 캐쉬를 액세스하여 캐쉬 히트이면 M 단계에서 메모리 액세스가 끝나지만, 캐쉬 미스인 경우에는 좀 더 복잡한 동작을 보인다. 캐쉬 미스인 경우에는 미스 처리를 위한 데이터 액세스 요구를 외부 메모리 액세스 버퍼에

등록하고, 캐쉬 미스 처리를 위한 정보를 캐쉬 미스 주소 파일에 저장한다. 그리고 캐쉬 미스 발생한 메모리 액세스가 로드인 경우에는 로드 버퍼에 로드 관련 정보를 저장하여 차후에 목적 레지스터에 대한 쓰기 작업을 할 수 있도록 한다. 이와 같이 외부 메모리 액세스를 위해서 등록된 것들은 등록된 순서대로 외부 메모리를 액세스하거나 캐쉬 미스를 처리한다. 그리고 로드인 경우에는 레지스터 파일의 쓰기 포트 중에 사용 가능한 것이 존재할 때에 목적 레지스터로 쓰기를 수행한다. 그림 2-20은 데이터 메모리 액세스를 위한 일련의 작업 흐름을 보여주는 것이다.

DTLB와 데이터 캐쉬와의 관계는 ITLB와 명령어 캐쉬와의 관계와 비슷하다. 우선 DTLB의 주소 변환과 데이터 캐쉬 액세스가 동시에 이루어질 수 있도록 OS는 필요한 경우 데이터 메모리 할당에서 페이지 컬러링을 사용한다. 따라서 DTLB와 데이터 캐쉬에서 히트가 난 경우에는 메모리 액세스가 단일 사이클에 모두 수행될 수 있다. DTLB의 구조는 그림 2-9의 ITLB 구조와 비슷하지만 다수의 액세스 포트를 가지고 있다는 차이점을 가진다. 데이터 캐쉬도 다수의 액세스 포트를 가지고 있다는 점에서 명령어 캐쉬와 다르다. 또한 데이터 캐쉬의 경우 캐쉬 미스가 발생하더라도 캐쉬 미스를 처리하면서 새로운 데이터 액세스를 처리할 수 있다는 점에서 명령어 캐쉬와 다르다. 즉 데이터 캐쉬는 hit-under-miss를 지원하는 non-blocking^[72] 캐쉬이다. 이러한 이유로 데이터 메모리 액세스는 명령어 메모리 액세스보다 전반적으로 복잡한 구조를 가진다. 데이터 메모리 액세스를 더욱 복잡하게 하는 것으로서 로드 바이패싱(load bypassing)^[13]이 있다. 로드 바이패싱을 사용하면, uncached 영역으로의 로드가 존재하는 경우에 그것을 외부 메모리 액세스를 위해 등록하기 전에 스토어 버퍼의 엔트리들을 검사해서, 같은 주소의 스토어가 스토어 버퍼에 존재하는 경우에는 그 엔트리의 스토어 데이터를 읽어서 레지스터 파일에 쓴다. 이것은 uncached 영역으로의 로드를 캐쉬 히트의 경우처럼 처리할 수 있다는 점에서 성능 향상에 많은 도움을 주지만 데이터 액세스를 복잡하게 만드는 요소이기도 하다.

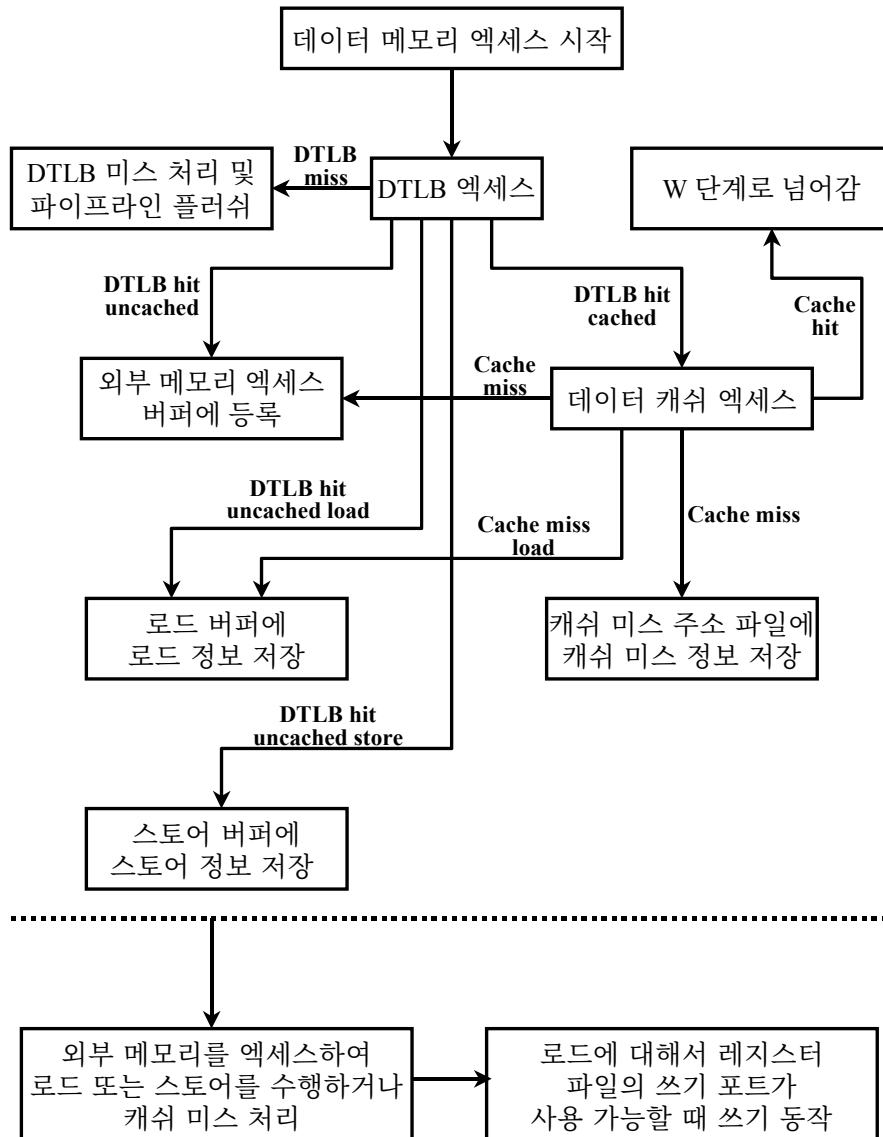


그림 2-20 데이터 메모리 액세스 과정

Fig. 2-20 Flow of data memory accesses

2-8절 분기 및 예외의 처리

2-8-1 분기 처리

분기 처리는 BTB에 의한 분기 예측과 BHB로의 분기 예측 히스토리의 저장 및 BHB의 분기 예측 히스토리를 이용한 분기 명령어의 실행에 의해서 이루어진다. 이 중에서 분기 예측과 분기 예측 히스토리 저장에 대해서는 2-3절에서 설명하였으므로, 본 절에서는 분기 명령어의 실행에 대해서만 설명한다.

분기 명령어의 실행은 BHB의 분기 예측 히스토리 정보를 이용한다. BHB에는 taken으로 예측된 분기 예측 히스토리들이 저장되어 있는데 BHB의 최하위 엔트리에는 이러한 분기 예측 히스토리 중 가장 오래된 것, 즉 프로그램 실행 순서에서 가장 빠른 것이 저장되어 있다. 분기 예측 오류 검사는 BHB의 최하위 엔트리의 분기 주소와 실행되는 분기 명령어의 주소를 비교하는 것에 기반을 두고 있다. 실행되는 분기 명령어 주소는 파이프라인 상에서 각 명령어에 수반되는 명령어 주소로부터 쉽게 알 수 있다. 본 논문에서 채택한 명령어 세트 구조에서는 분기 명령어로 분류되는 명령어 외에도 일반 명령어들이 PC를 목적 레지스터로서 사용할 수 있고 이러한 명령어들도 분기를 수반한다. 본 논문에서 분기 명령어라고 함은 분류상 분기 명령어뿐 아니라 분기를 수반하는 모든 명령어들을 포함하는 것이다. 분기 예측 오류에 대한 검사 및 그에 따르는 동작은 기능 유닛의 M 단계에서 이루어진다. 분기 예측 오류 검사의 과정이 그림 2-21에 설명되어 있으며 각 경우에 따르는 동작은 다음과 같다.

첫째, 실행되는 분기 명령어의 주소와 BHB의 최하위 엔트리의 분기 주소가 같고, 실제로 분기 명령어가 taken되고 실제 분기 타겟 주소와 BHB의 최하위 엔트리의 분기 타겟 주소가 같은 경우이다. 이 경우는 해당 분기 명령어에 대해서 taken으로 분기 예측이 되었고 분기 명령어가 실제로도 taken되었으며, 분기 타겟 주소도 올바르게 예측한 경우이다. 즉 분기 예측 히트의 경우이다. 이 경우에는 BHB의 최하위 포인터를 1 증가시켜 이전의 최하위 엔트리를 BHB로부터 제거하고, 해당 BTB 엔트리의 방향 카운터의 값이 11₍₂₎이 아닌 경우 그 값을 1 증가시

킴으로서 분기 명령어의 수행은 끝난다. 이와 같이 간단한 동작에 의해서 분기 명령어의 실행이 완료되는 것은 프로세서의 명령어 실행 흐름이 올바른 것이기 때문이다.

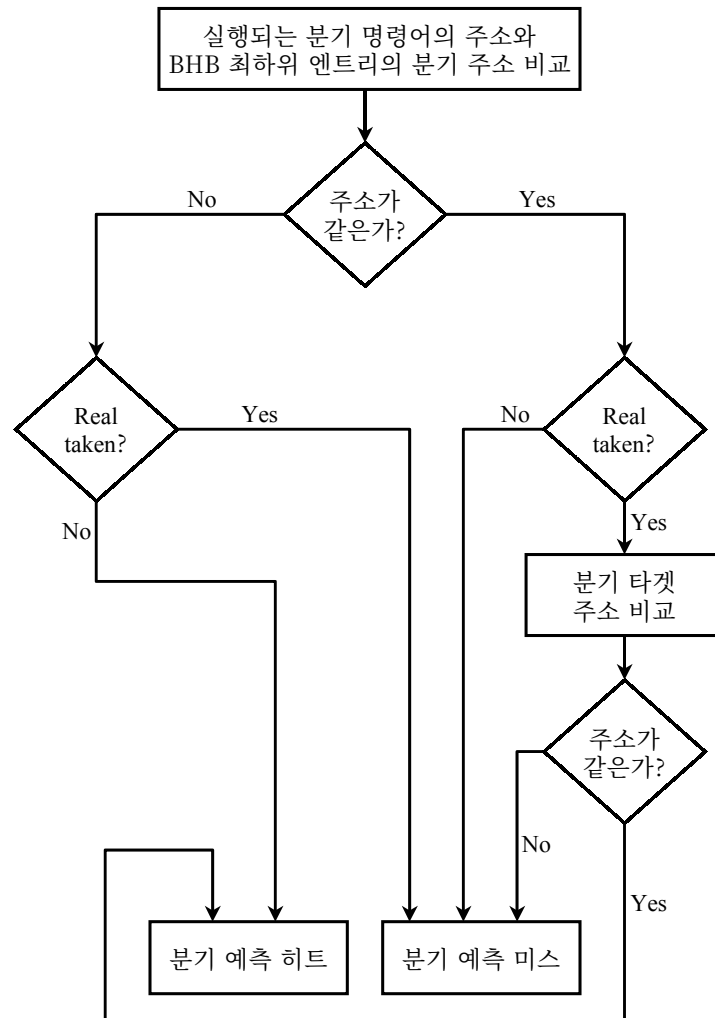


그림 2-21 분기 예측 오류의 검사 과정

Fig. 2-21 Flow of branch misprediction check

둘째, 실행되는 분기 명령어의 주소와 BHB의 최하위 엔트리의 분기 주소가 같고, 실제로 분기 명령어가 taken되었으나 실제 분기 타겟 주소와 BHB의 최하위 엔트리의 분기 타겟 주소가 다른 경우이다. 이 경우는 해당 분기 명령어에 대해서 taken으로 분기 예측되었고 분기 명령어가 실제로도 taken되었으나, 예측된 분기 타겟 주소와 실제 분기 타겟 주소가 다른 경우이다. 따라서 분기 예측 오류에 해당하는 경우이며, 분기 명령어에 후속하는 모든 명령어들을 파이프라인 상에서 플러쉬하고 명령어 폐치를 위한 PC를 실제 분기 타겟 주소로 설정하여 올바른 흐름에서 다시 명령어들을 수행할 수 있도록 한다. 그리고 BTB의 해당 엔트리에서 분기 타겟 주소를 실제 분기 타겟 주소로 설정하고, 방향 카운터의 값이 $11_{(2)}$ 이 아닌 경우 그 값을 1 증가시킨다. 마지막으로 BHB의 전체 엔트리들을 플러쉬함으로써 분기 명령어의 실행을 완료한다.

셋째, 실행되는 분기 명령어의 주소와 BHB의 최하위 엔트리의 분기 주소가 같으나, 분기 명령어가 실제로는 not taken인 경우이다. 이 경우는 해당 분기 명령어에 대해서는 taken.으로 분기 예측하였으나, 실제로는 not taken된 분기 예측 오류에 해당한다. 따라서 분기 명령어에 후속하는 모든 명령어들을 파이프라인 상에서 플러쉬하고 명령어 폐치를 위한 PC를 순차적으로 분기 명령어 주소 다음으로 설정하여 올바른 명령어 수행 흐름을 복구하도록 한다. 그리고 해당 BTB 엔트리의 방향 카운터 값을 1 감소시키고 BHB의 전체 엔트리들을 플러쉬함으로써 분기 명령어의 실행을 완료한다.

넷째, 실행되는 분기 명령어의 주소와 BHB의 최하위 엔트리의 분기 주소가 다르나, 분기 명령어가 실제로 taken인 경우이다. 이 경우는 해당 분기 명령어에 대해서 not taken으로 분기 예측하였으나, 실제로는 taken된 분기 예측 오류에 해당한다. 따라서 분기 명령어에 후속하는 모든 명령어들을 파이프라인 상에서 플러쉬하고 명령어 폐치를 위한 PC를 실제 분기 타겟 주소로 설정한다. 그리고 해당 BTB 엔트리가 존재하는 경우에는 방향 카운터의 값을 1 증가시킨다. 해당 BTB 엔트리가 존재하는 않는 경우에는 BTB 엔트리를 새로이 할당하고 방향 카운터의 값은 $10_{(2)}$ 으로 설정한다. 그리고 이 경우는 분기 예측 오류에 해당하기 때문에, 둘째 및 셋째 경우와 같이 BHB의 모든 엔트리들을 플러쉬함으로써 분기 명령어의 실행을 완료한다.

행을 완료한다.

다섯째, 실행되는 분기 명령어의 주소와 BHB의 최하위 엔트리의 분기 주소가 다르고, 분기 명령어가 실제로 not taken인 경우이다. 이 경우는 해당 분기 명령어에 대해서 not taken으로 분기 예측하고, 실제로도 not taken된 분기 예측 히트에 해당한다. 따라서 프로세서는 현재의 명령어 실행 흐름을 계속 유지한다. 그리고, 해당 BTB 엔트리가 존재하는 경우에는 방향 카운터의 값이 $00_{(2)}$ 이 아닌 경우 그 값을 1 감소시킴으로써 분기 명령어의 실행을 완료한다.

본 논문의 분기 처리는 상기와 같이 매우 간단하며, 분기 예측 오류 처리는 단지 후속 명령어들의 플러쉬와 명령어 폐치를 위한 PC의 설정에 의해서 이루어진다. 분기 예측 오류 처리가 간단한 것은 본 슈퍼스칼라 구조가 순차적 명령어 이슈 및 완료 정책을 사용하기 때문이다.

2-8-1 예외 처리

본 논문의 슈퍼스칼라 구조에서는 채택된 명령어 세트 구조에서 정의된 모든 예외 종류들을 지원하며, 표 2-6은 지원되는 예외들을 정리한 것이다. 예외의 종류에 따라서 예외가 발생하는 방식이나 예외를 검출하는 파이프라인 단계들이 다르다. Reset, IRQ 및 FIQ 예외는 프로세서의 프로그램 실행과 비동기적으로(asynchronously) 발생하는 것이다. Prefetch Abort는 명령어 폐치 단계인 F 단계에서 검출되고, Undefined 예외와 BKBT(Breakpoint) 명령어에 의한 Prefetch Abort는 명령어 디코드 단계인 D 단계에서 검출된다. 그 외의 예외들은 모드 M 단계에서 검출된다. 예외의 발생 방식이나 검출 단계가 다르더라도 예외의 처리는 동기적으로(synchronously) 이루어지는데, Reset을 제외하고는 모두 M 단계에서 처리되는 방식을 취한다. 따라서 F 또는 D 단계에서 검출되는 예외들은 일단 버퍼에 저장되었다가 이전의 모든 명령어들이 M 단계까지 정상적으로 완료된 후에 예외를 처리한다. IRQ 또는 FIQ 예외의 처리는 예외를 처리하는 사이클에 E 단계를 포함해서 이전 단계에 존재하는 모든 명령어들을 플러쉬하지만, M 및 W 단계에 존재하는 명령어들은 정상적으로 완료하도록 함으로써 프로세서의 명령어 실행과 예외 처리를 동기화 시킨다. M 단계에서 검출되는 예외들에 대해서는 검출

과 동시에 예외 처리를 수행하면서 예외 발생 명령어를 포함하여 후속하는 명령어들을 모두 플러쉬하고, 예외 발생 명령어보다 앞서는 M 또는 W 단계의 명령어들은 정상적으로 명령어 수행을 완료하도록 한다. Reset은 프로세서의 상태를 초기화하기 때문에 특정 파이프라인 단계와 동기적으로 처리되지 않는다.

표 2-6 예외의 종류와 벡터 주소

Table 2-6 Exception types and vector addresses

예외 종류	관련된 프로세서 모드	예외 벡터 주소
Reset	Supervisor	0x00000000
Undefined	Undefined	0x00000004
SWI	Supervisor	0x00000008
Prefetch Abort	Abort	0x0000000C
Data Abort	Abort	0x00000010
IRQ	IRQ	0x00000018
FIQ	FIQ	0x0000001C

이와 같이 예외 처리가 파이프라인의 M 단계에서 이루어지기 때문에, 예외를 precise하게 처리하기 위해서 명령어들은 M 단계 이전에는 프로세서의 상태를 변경시키면 안 된다. 단, 예외의 처리 시의 플러쉬 동작에 의해서 복구되는 상태에 대해서는 명령어에 의한 상태 변화가 가능하다. 예외 처리는 기본적으로 명령어들의 플러쉬 및 예외의 종류에 따라 다른 예외 벡터 주소(exception vector address)로 명령어 페치 PC를 설정하는 것에 의해서 이루어진다. 이 외에는 예외의 종류

에 따라 프로세서 모드를 변경하고 및 CPSR을 SPSR로 복사한다. 이와 같이 예외 처리가 간단한 것은 분기 예측 오류 처리가 간단한 것과 같은 이유인 순차적 명령어 이슈 및 완료 정책 때문이다.

제 3 장 순차적 SMT 구조

본 논문의 SMT 구조는 슈퍼스칼라 구조로부터 약간의 구조 변경과 하드웨어 추가에 의해서 구성될 수 있다. 본 장에서는 우선 전체적인 구조 측면에서 SMT 구조가 슈퍼스칼라 구조와 다른 점들과 각 구성 요소들의 세부적인 변화에 대해서 설명한다. 그리고 본 장의 SMT 구조를 바탕으로 다음 장에서는 SMT 구조의 설계 복잡도를 줄이기 위한 그룹화(grouping) 방안을 제시한다.

3-1절 전체적인 순차적 SMT 구조

본 절에서는 SMT 마이크로프로세서의 전체적인 구조를 슈퍼스칼라 마이크로프로세서 구조와의 비교를 통해서 제시한다. 본 논문의 SMT 구조는 다수의 독립적인 스레드들로부터의 명령어들을 동시에 이슈하고 실행하는 구조를 가진다. SMT 구조는 슈퍼스칼라 구조의 ILP 부족을 TLP로 극복하여 프로세서 자원들의 활용도를 높이면서 전체적인 성능을 향상시킨다^[33-36]. SMT 구조는 2장에서 제시한 슈퍼스칼라 구조처럼 순차적 명령어 이슈 및 완료 정책을 사용한다. 따라서 같은 스레드에 속하는 명령어들은 프로그램의 순서대로 이슈되고 실행된다. 그러나, 서로 다른 스레드들에 속하는 명령어들 상호간에는 순서가 존재하지 않는다. 따라서 서로 다른 스레드들에 속하는 명령어들에 대해서는 이슈 및 실행 순서에 대한 제한이 존재하지 않으며, 명령어들은 페치 순서와 다르게 이슈되고 실행될 수 있다.

그림 3-1은 4개의 스레드를 지원하는 SMT 구조의 전체적인 모습을 보여주는 블록 다이어그램이다. 그림 3-1에서 보면 전체적인 SMT 구조는 슈퍼스칼라 구조와 거의 차이가 없다. 그림 3-1에서 보이는 차이는 단지 페치 유닛 안에 스레드 선택기(thread selector)가 존재한다는 것과 다수의 PC가 존재한다는 것 및 레지스터 파일 안에 다수의 레지스터 세트가 존재한다는 것이다.

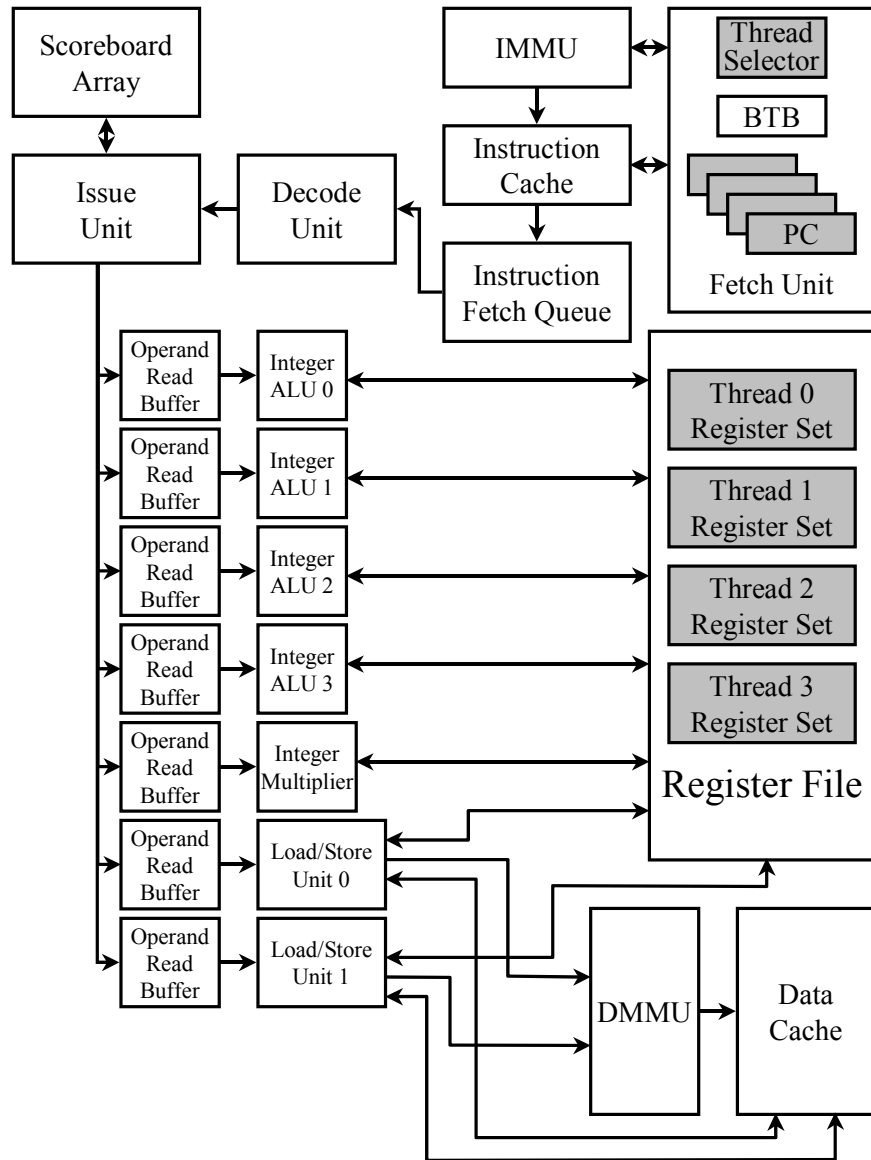


그림 3-1 전체적인 순차적 SMT 구조의 블록 다이어그램

Fig. 3-1 Block diagram of the overall in-order SMT architecture

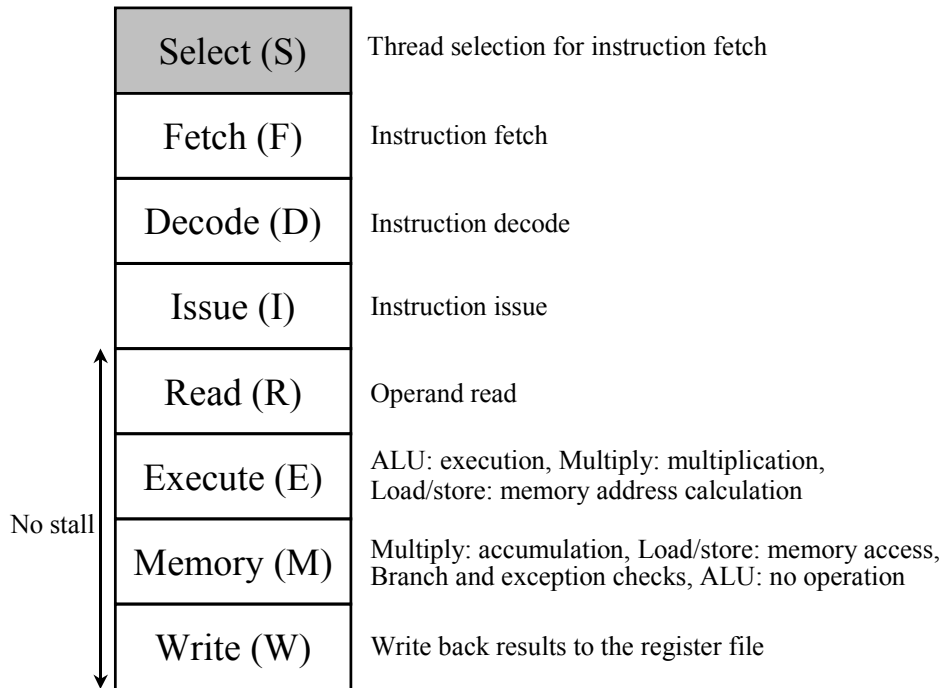


그림 3-2 순차적 SMT 구조의 파이프라인

Fig. 3-2 pipeline structure of the in-order SMT architecture

그러나 세부적인 측면에서 보았을 때에는 좀 더 많은 차이점을 보인다. 그 예로서, 그림 3-2의 SMT 파이프라인 구조는 슈퍼스칼라 파이프라인 구조와 차이가 있다. 즉 S(Select) 단계가 F 단계 앞에 추가된다. SMT 구조에서는 다수의 스레드들로부터 명령어들을 페치하여 명령어 페치 큐에 저장하는 구조를 가진다. 따라서 매 사이클마다 어떤 스레드들로부터 명령어들을 페치할 지를 결정해야 한다. 지원되는 스레드 수와 같은 수만큼 명령어 캐쉬의 포트들을 제공하고 매 사이클 모든 스레드들로부터 명령어들을 페치할 수도 있으나, 이 방법은 명령어 캐쉬

에 대한 하드웨어 오버헤드(hardware overhead)가 너무 크고, 추가되는 하드웨어 비용에 비해서 페치 성능 향상이 이루어지는 것도 아니다. 따라서 명령어 캐쉬는 제한된 포트 수를 가지며, 전체 스레드들 중에서 일부 스레드들로부터만 명령어들을 페치한다. 이 경우에 어떤 스레드들로부터 명령어들을 페치할 것인지에 대한 선택을 해야 하는데, F 단계에서 스레드 선택과 명령어 페치를 같이 하면 F 단계의 지연 시간(delay time)이 너무 커지는 문제점이 있다. 따라서 F 단계 앞에 S 단계를 두고 S 단계에서 다음 사이클에 어떤 스레드들로부터 명령어들을 페치할지를 결정한다. 스레드 선택은 스레드 선택기에 의해서 이루어지며, 주어진 페치 우선권 정책(fetch priority policy)을 스레드 선택 시에 적용한다.

F 단계에서는 이전 사이클에 S 단계에서 선택된 스레드들로부터 명령어들을 페치하여 명령어 페치 큐에 저장한다. 단일 사이클에 몇 개의 스레드들로부터 명령어들을 페치할 수 있는지에 대한 SMT 구조의 선택 사양(option)이며, 단일 사이클에 2개 이상의 스레드들로부터 명령어들을 페치하는 구조인 경우, 명령어 캐쉬, ITLB 및 BTB의 액세스 포트 수들이 2개 이상 존재하게 된다. SMT 구조의 D 단계는 슈퍼스칼라 구조의 D 단계와 차이점이 거의 없으며, 명령어 페치 큐로부터의 명령어들을 해석해서 명령어 이슈 큐에 저장하는 동작이 주요한 것이다. I 단계에서는 명령어 이슈 큐에 저장된 명령어들을 검사하여 기능 유닛들로 명령어들을 이슈한다. SMT의 명령어 이슈가 슈퍼스칼라와 비교해서 다른 점은 명령어들간의 의존성 검사를 수행할 때, 같은 스레드에 속하는 명령어들간에만 의존성을 검사해야 한다는 것이다. 따라서 각 명령어들이 어떤 스레드에 속하는지를 알 필요가 있다. 명령어 이슈 큐의 각 엔트리는 스레드 식별자(thread identifier) 영역을 포함하고 있으며, 이슈 유닛은 스레드 식별자를 사용하여 각 명령어들이 속한 스레드들을 구별하여서 같은 스레드에 속하는 명령어들 간에만 의존성을 검사한다. 서로 다른 스레드에 속한 명령어들 간에는 명령어 이슈 순서가 페치 순서, 즉 명령어 이슈 큐 상의 순서와 다를 수 있다. 따라서 슈퍼스칼라 구조와 달리 SMT 구조에서는 명령어 이슈 큐의 순서에서 뒤에 있는 명령어들을 먼저 이슈할 수 있다. 이러한 이유로 명령어 이슈 큐의 중간에 무효한, 즉 비어있는(empty) 엔트리들이 존재할 수 있고, SMT 구조에서는 이러한 비어있는 엔트리들을 제거하기 위

한 특별한 방법을 사용한다. 기능 유닛으로 이슈된 명령어들은 R 단계에서 오퍼랜드를 얻은 후에, E, M 및 W 단계를 거쳐서 명령어의 수행을 완료한다. 스레드 식별자들은 R, E, M 및 W 단계들을 거치면서 계속 존재하는데, 이것은 스레드별 명령어 플러쉬를 위한 것이다. 즉, 분기 예측 오류 또는 예외를 처리할 때에 명령어들을 플러쉬할 때에 분기 예측 오류나 예외가 발생한 스레드에 속하는 명령어들만을 플러쉬하기 위해서 파이프라인의 실행 단계들에 스레드 식별자가 존재한다.

3-2절 세부적인 순차적 SMT 구조

본 절에서는 SMT 마이크로프로세서의 세부 구조가 수퍼스칼라 구조와 관련해서 어떠한 차이점이 있는지를 설명한다. 이하의 설명에서 SMT 구조에서 지원되는 스레드를 수는 T 이며, 페치 폭, 디코드 폭 및 이슈 폭은 N 이다. 그리고, 단일 사이클에 페치할 수 있는 스레드의 수는 M 이다. 이와 같은 가정은 특별한 설명이 없는 경우에는 항상 적용된다.

SMT 구조가 수퍼스칼라 구조와 가장 많은 차이점을 보이는 부분은 페치 구조인데, 그림 3-3은 SMT의 페치 구조를 보여주는 것이다. 스레드 선택기는 페치 우선권 정책에 의해서 페치할 스레드들을 선택한다. 페치 유닛 안에는 지원되는 스레드들의 수만큼 다수의 페치용 PC들이 존재하는데, 스레드 선택기에 의해서 선택된 스레드들에 속하는 PC들의 명령어 주소들을 사용한 ITLB와 명령어 캐쉬 및 BTB에 대한 액세스가 매 사이클 이루어진다. 단일 사이클에 몇 개의 스레드들로부터 명령어 페치를 수행할 수 있는지는 SMT 구조의 사양이며, 최소 1개부터 최대로는 지원되는 모든 스레드들로부터 명령어 페치를 매 사이클 수행할 수 있다. 단일 사이클에 다수의 스레드들로부터 명령어들을 페치할 수 있을 경우에는 명령어 캐쉬, ITLB 및 BTB도 다수의 액세스 포트를 가지고 있어야 한다. 즉, 명령어 캐쉬는 M 개의 읽기 포트를 가지고 있어야 하며 각 읽기 포트에서는 N/M 개의 명령어들이 출력된다. 명령어 캐쉬는 다수의 포트를 가지기 위해서 다수의 내부적인 बैं크(bank)^[73]들을 가지고 있는 다중 बैं크 구조로 이루어진다. 여기서 बैं크의 수는 M 의 배수가 되어야 한다. 수퍼스칼라 구조에서는 명령어 페치에서 캐쉬 미스가 발생하면 캐쉬 미스를 처리한 후에 페치를 다시 시작한다. 따라서 수퍼스칼라 구조에서는 명령어 캐쉬가 hit-under-miss를 지원할 필요가 없다. 반면에, SMT 구조에서는 어떤 스레드의 명령어 페치에서 명령어 캐쉬 미스가 발생하였을 때에, 캐쉬 미스를 처리하면서 다른 스레드로부터 명령어 페치를 계속할 수 있어야 SMT의 장점을 최대한 살릴 수 있다. 따라서 SMT의 명령어 캐쉬는 hit-under-miss를 지원하는 non-blocking 캐쉬이다.

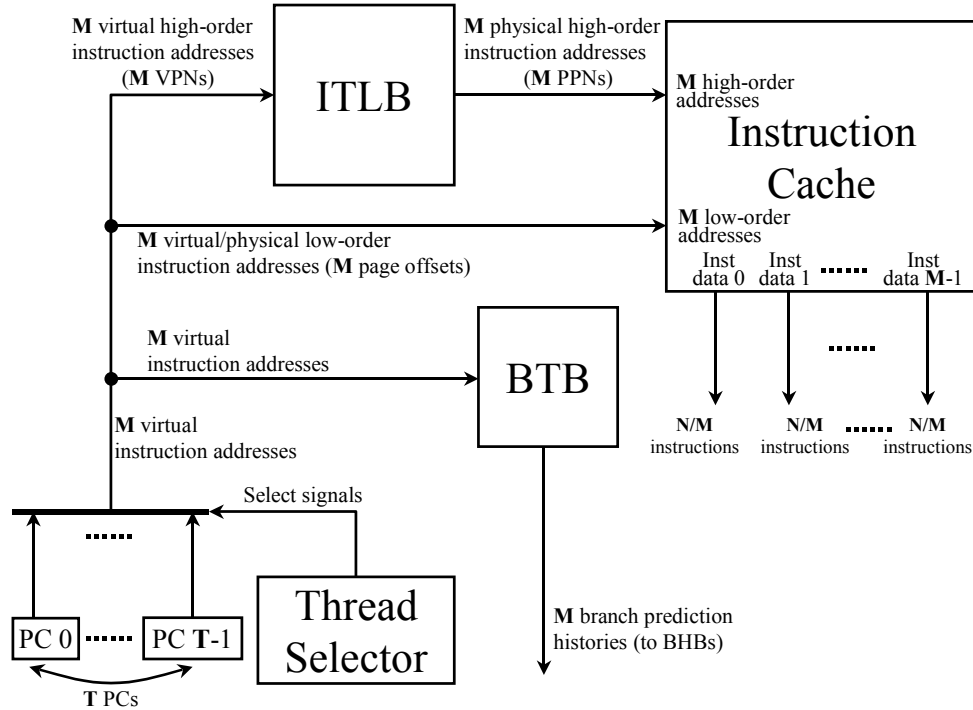


그림 3-3 순차적 SMT 구조의 명령어 페치

Fig. 3-3 Instruction fetch structure of the in-order SMT architecture

ITLB도 다수의 명령어 페치를 동시에 지원하기 위해서 다수의 포트를 가지고 있어야 한다. 즉 ITLB는 M 개의 액세스 포트를 사용하여 M 개의 주소 변환 동작을 동시에 지원할 수 있어야 한다. 그리고 단일 사이클에 M 개의 스레드들로부터 M 개의 명령어 열이 페치되기 때문에, M 개의 명령어 열에 대한 분기 예측이 동시에 이루어져야 한다. 따라서 BTB 역시 M 개의 액세스 포트를 가지고 있어야 한다.

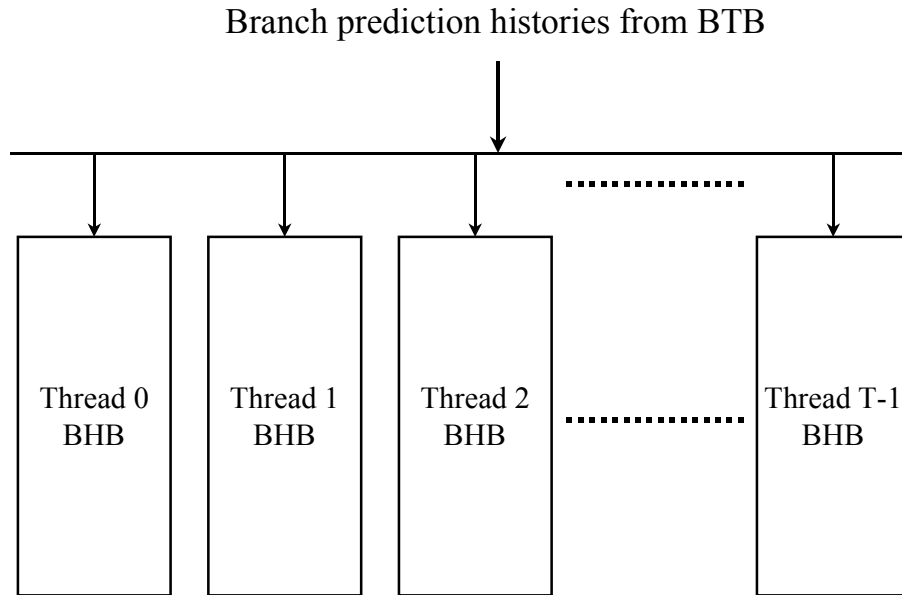


그림 3-4 다수의 BHB들 (스레드당 한 개의 BHB)

Fig. 3-4 Multiple BHBs (one BHB per thread)

SMT 구조에서는 다수의 독립적인 스레드들이 존재하며, 각 스레드들의 프로그램의 흐름이 서로 다르다. 따라서 같은 가상주소를 가지는 분기 명령어라고 하더라도 스레드가 다르면 분기의 경향도 다르다. 이러한 이유로 분기 예측을 수행할 때에 다른 스레드의 BTB 엔트리를 사용하지 않도록 해야 하는데, 이를 위해서 BTB의 각 엔트리에는 스레드 식별자가 존재한다. 분기 예측 히스토리를 저장하고 있는 BHB는 선입선출 방식으로 구성이 되어야 올바르게 분기 예측 오류를 검사할 수 있다. 따라서 BHB는 스레드별로 따로 존재하며, 각 BHB는 슈퍼스칼라 구조의 BHB처럼 선입선출 방식의 구조를 가진다. BHB는 그림 3-4와 같이 분기 예

측에 대한 히스토리를 저장할 때에 어떤 스레드에 대한 분기 예측인지를 판단하여 해당 BHB의 최상위 엔트리에 저장해야 한다. 분기 예측 히스토리가 속하는 스레드는 명령어 페치 시에 각 명령어 열에 대해서 결정되는 스레드 식별자를 사용하여 구별한다.

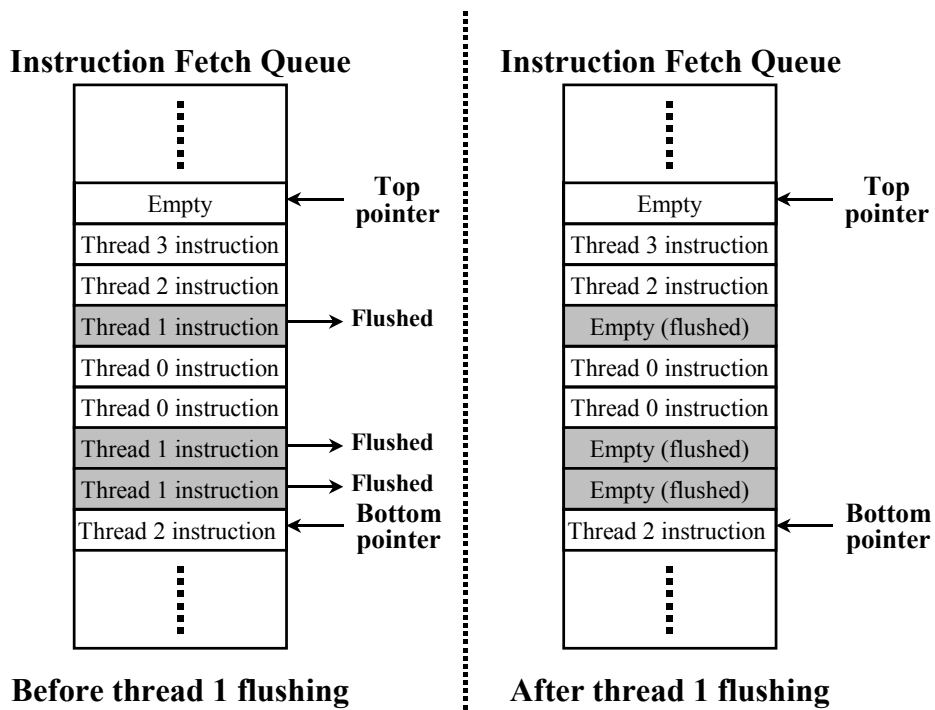


그림 3-5 단일 스레드의 플러쉬로 인한 명령어 페치 큐 중간의 빈 엔트리들
 Fig. 3-5 Empty entries between the top and bottom entries of the instruction fetch queue caused by flushing one thread

SMT의 명령어 페치 큐는 슈퍼스칼라 구조의 명령어 페치 큐와 같이 최상위 포인터와 최하위 포인터를 이용한 선입선출 구조를 가진다. 그러나 스레드별 명령어 플러쉬로 인해서 그 동작에 있어서 약간의 차이를 보이는데, 분기 예측 오류나 예외의 처리 시에 해당 스레드에 속하는 명령어들만 파이프라인 상에서 모두 플러쉬된다. 명령어 페치 큐에는 여러 스레드로부터의 명령어들이 페치 순서대로 저장되어 있으므로, 특정 스레드의 명령어들만 플러쉬될 경우에는 그림 3-5와 같이 중간에 비어있는 엔트리들이 존재하게 된다. 이러한 비어 있는 엔트리들은 명령어 페치 큐의 효율적인 사용을 저해하며 명령어 페치 큐의 적체 현상을 유발하여 명령어 페치의 성능에 나쁜 영향을 미친다. 비어있는 엔트리들을 없애기 위해서는 명령어 페치 큐를 매 사이클 압축(compressing)^[13]할 필요가 있다. 그러나, 명령어 페치 큐의 압축은 하드웨어 비용과 설계 복잡도를 늘릴 뿐만 아니라 지연 시간을 증가시키는 요인이 된다. 또한 명령어 페치 큐의 모든 엔트리들이 채워져서 페치가 중단되는 적체 현상은 중간에 비어있는 엔트리들로 인해서 발생하기보다는 명령어 이슈 큐의 적체 때문에 발생하기가 쉽다. 또한 중간에 비어있는 엔트리들로 인해서 명령어 페치가 중단되더라도, 명령어 페치 큐에 유효하게 존재하는 명령어들과 명령어 이슈 큐의 엔트리들을 이용하여 명령어 이슈는 계속 이루어질 수 있으며, 이와 함께 명령어 페치 큐의 적체 요인이 없어지면서 페치가 계속되기 쉽다. 따라서 명령어 페치 큐에 대해서는 압축을 하지 않는다. 다만 각 엔트리는 유효한 명령어를 가지고 있는지를 유효 비트를 사용하여 알려준다는 점이 슈퍼스칼라 구조와의 차이점이다.

디코드 유닛은 슈퍼스칼라 구조와 별 차이점이 없으며, 명령어 페치 큐의 최하위 엔트리로부터의 명령어들을 해석해서 스레드 식별자와 함께 명령어 이슈 큐의 최상위 엔트리 위치로 저장한다. 다만, 명령어 페치 큐와 관련해서 차이점이 있다. 슈퍼스칼라 구조에서는 명령어 페치 큐의 최하위 엔트리부터 최상위 엔트리까지 모두 유효한 명령어들을 가지고 있다. 따라서 명령어 페치 큐에 충분한 명령어들이 존재하는 경우에는 명령어 페치 큐의 최하위 엔트리를 시작으로 디코드 폭만큼의 엔트리들은 모두 유효한 명령어들을 가지고 있으며, 이들 명령어들을 디코드해서 모두 명령어 이슈 큐에 저장한다. 그러나, SMT 구조에서는 명령어 페치 큐

의 중간에 유효하지 않은 엔트리들이 존재하고, 따라서 명령어 페치 큐의 최하위 엔트리를 시작으로 디코드 폭만큼의 엔트리들 중에는 유효하지 않는 것들이 있다. 이러한 엔트리들로부터의 디코드는 무효하며, 해당 디코드로 만들어진 디코드된 명령어들을 명령어 이슈 큐에 저장해서는 안 된다. 디코드 유닛은 명령어 페치 큐로부터 디코드 폭만큼의 명령어들을 디코드하면서, 각 명령어의 유효 비트를 사용하여 유효하지 않은 명령어들에 대해서는 실질적으로 디코드를 수행하지 않으며, 또한 그 명령어들을 명령어 이슈 큐에 저장하지도 않는다. 그림 3-6은 디코드 폭이 4개의 명령어인 경우에 디코드 유닛이 유효한 명령어들만을 명령어 이슈 큐에 저장하는 모습을 보여 준다.

SMT의 이슈 유닛은 페치 구조와 함께, 동작에 있어서 슈퍼스칼라와 많은 차이점을 보여주는 부분이다. SMT의 이슈 유닛은 명령어 이슈에서 사용하는 이슈 원칙들 중에서, 명령어들 상호간의 관계에 대한 것들은 같은 스레드에 속하는 명령어들 사이에만 적용한다. 즉, 이슈 유닛은 같은 스레드에 속하는 명령어들 사이에만 의존성을 검사한다. 예를 들어서, 스레드 0에 속하는 명령어 A가 레지스터 R1을 목적 레지스터로 사용하고 스레드 1에 속하는 후속 명령어 B가 레지스터 R1을 소스 레지스터로 사용하더라도 명령어 B는 명령어 A에 대해서 데이터 의존성을 가지고 있지 않다. 다른 차이점으로는, SMT의 이슈 유닛이 이슈할 명령어의 선택에서 있어서 슈퍼스칼라의 것과 비교해서 자유롭다는 것이다. 슈퍼스칼라에서는 항상 명령어 이슈 큐의 최하위 엔트리부터 명령어를 이슈해야 하며, 최하위 엔트리가 이슈될 수 없을 때에는 그 이후의 명령어들도 이슈될 수 없다. 그러나, SMT 구조에서는 서로 다른 스레드들에 속하는 명령어들 사이에는 명령어 이슈의 순서가 존재하지 않기 때문에, 최하위 엔트리의 명령어가 이슈되지 못하더라도 그 엔트리와 다른 스레드에 속하는 명령어들을 얼마든지 이슈할 수 있으므로, 명령어 이슈의 선택 폭이 더 넓다. 이것은 SMT 구조가 ILP의 부족을 TLP로 극복할 수 있기 때문이며, 이런 이유로 SMT 구조가 슈퍼스칼라 구조보다 더 좋은 성능을 보일 수 있는 것이다. SMT에서 이슈 선택의 폭이 넓은 이유로, 이슈할 명령어를 선택하는데 있어서 일정한 정책이 필요한데, 이를 이슈 우선권 정책(issue priority policy)이라고 한다. 페치 유닛의 스레드 선택기가 페치 우선권 정책을 사용하여

페치할 스레드들을 선택하는 것처럼, 이슈 유닛도 주어진 이슈 우선권 정책을 사용하여 이슈할 명령어들을 선택하는 기능이 있어야 한다.

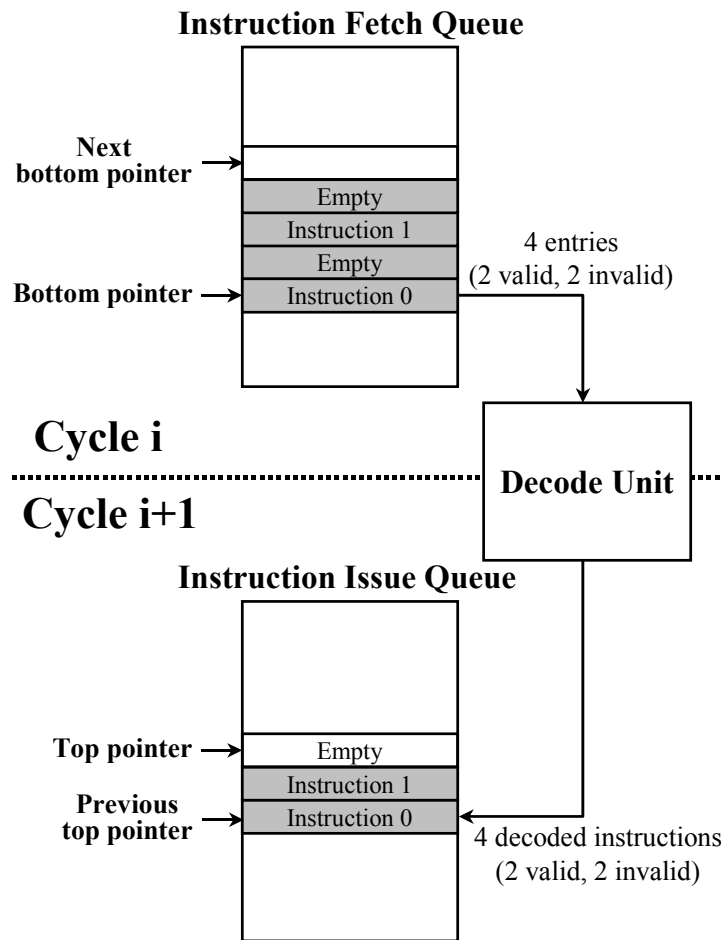


그림 3-6 유효한 엔트리들만을 명령어 이슈 큐에 저장하는 모습
Fig. 3-6 Storing only valid entries to the instruction issue queue

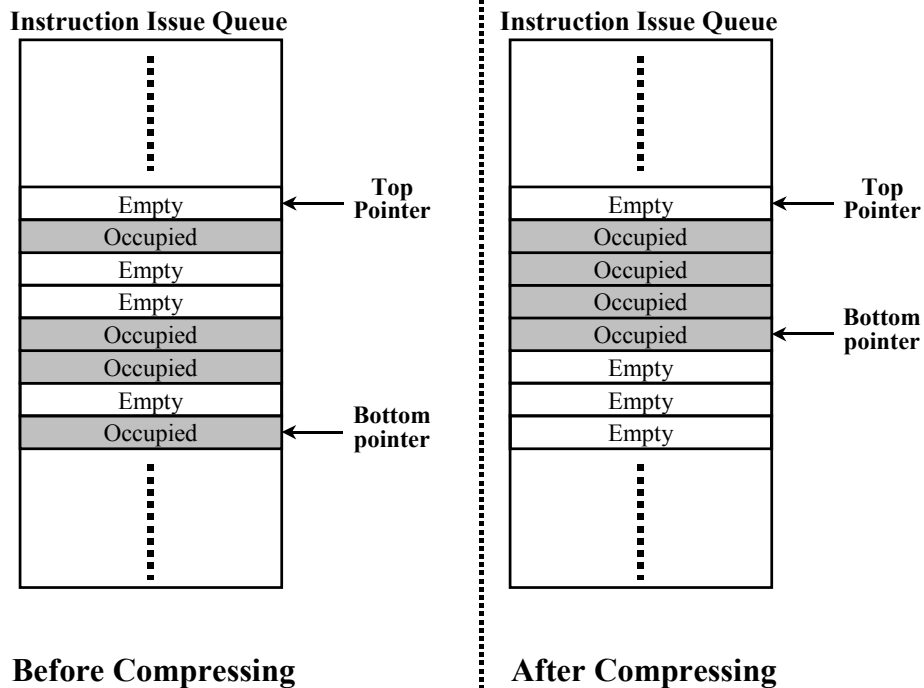


그림 3-7 명령어 이슈 큐의 압축

Fig. 3-7 Instruction issue queue compressing

명령어 이슈 큐의 최하위 엔트리가 이슈되지 않았을 때 다른 엔트리들의 명령어들을 이슈할 수 있다는 점과 스레드별로 명령어들을 플러쉬한다는 점은 명령어 이슈 큐의 중간 엔트리들에 비어있는 엔트리를 발생시킨다. 이것은 명령어 페치 큐의 경우와 같다. 그런데, 명령어 이슈 큐의 적체는 곧바로 이슈율(issue rate)의 저하로 이어진다. 현재 명령어 이슈 큐에 스레드 0, 1, 2의 명령어들만이 존재하는데 이 중에서 이슈할 수 있는 명령어들이 없고, 현재 명령어 페치 큐에 존재하는 스레드 3의 명령어들에 대해서 이슈의 가능성이 있다고 하자. 이 경우 명령어 이

슈 큐의 적체로 인해서 스레드 3의 명령어들이 명령어 이슈 큐에 저장될 수 없다면, 그렇지 않은 경우와 비교해서 명령어 이슈를 그만큼 못하게 되는 것이다. 명령어 이슈율의 저하는 마이크로프로세서의 명령어 처리 능력(instruction throughput)과 직결되므로, 명령어 이슈 큐의 적체는 최소화시켜야 한다. 따라서 본 논문의 SMT 구조는 그림 3-7과 같은 압축을 명령어 이슈 큐에 매 사이클 적용시킴으로써 비어있는 엔트리들을 없앤다. 이러한 압축 동작은 명령어 이슈와 동시에 이루어지기 때문에, 설계의 복잡성을 높이지만 성능에 미치는 영향을 고려했을 때 꼭 필요한 동작으로 볼 수 있다.

SMT의 레지스터 파일과 스코어보드 어레이 관련해서는 설계의 복잡성이 늘어나지는 않지만, 마이크로프로세서의 면적이라는 측면에서는 하드웨어 비용이 상당히 늘어난다. SMT 마이크로프로세서의 각 스레드는 자신만의 레지스터 세트(register set)를 가지며, 각각의 레지스터 세트들은 슈퍼스칼라 마이크로프로세서의 레지스터 파일에 해당한다. 따라서 그림 3-8과 같이 T개의 스레드를 지원하는 SMT 구조의 레지스터 파일은 슈퍼스칼라 구조와 비교해서 T배의 레지스터들을 가지게 되며, 이것은 마이크로프로세서의 면적 증가로 직결된다. 또한 레지스터 파일로의 액세스 속도를 느리게도 한다. 스코어보드 어레이의 각 엔트리는 해당 레지스터에 대한 최신의 갱신 상태를 가지고 있다. 즉, 레지스터 한 개당 스코어보드 엔트리 한 개가 존재한다. 따라서 그림 3-9와 같이 스코어보드 어레이는 레지스터 파일과 같은 이유로 엔트리 수가 T배가 된다.

SMT의 기능 유닛들과 데이터 메모리 관련 유닛들에 대해서는 슈퍼스칼라 구조와 거의 차이가 없다. 다만, 기능 유닛의 M 단계에서 이루어지는 분기 예측 오류 및 예외에 대한 검사와 처리가 스레드별로 이루어져야 한다는 것이다. 따라서 명령어들도 스레드별로 플러쉬되어야 하며, 이를 위해서 스레드 식별자가 기능 유닛들의 파이프라인 단계들에 존재한다.

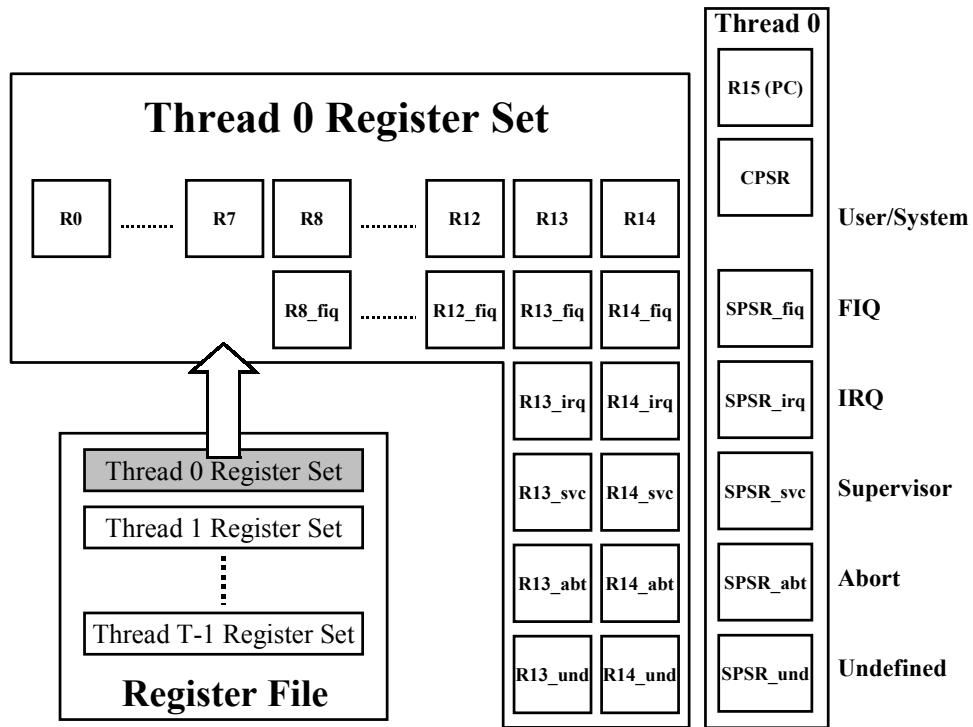


그림 3-8 순차적 SMT 구조의 레지스터 파일

Fig. 3-8 Register file structure of the in-order SMT architecture

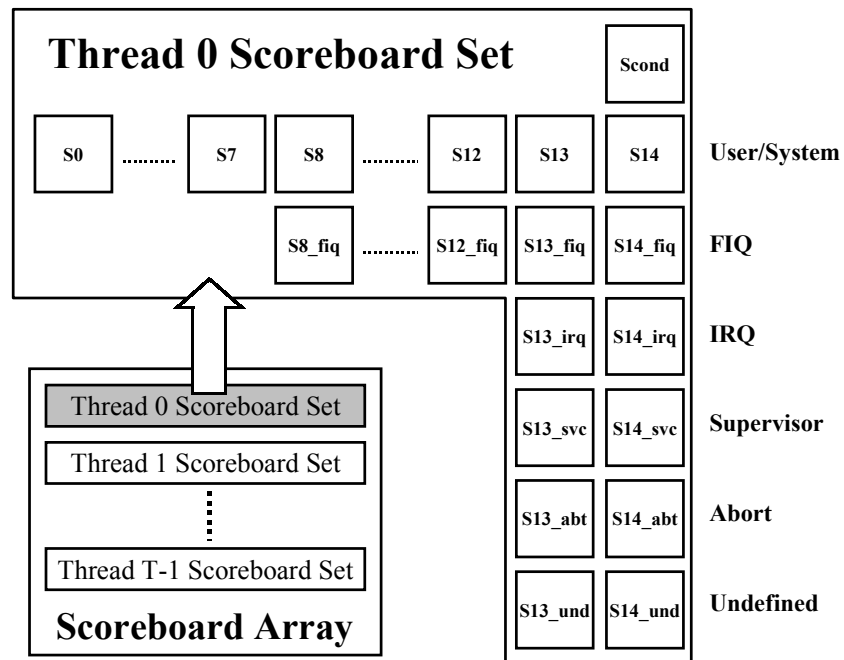


그림 3-9 순차적 SMT 마이크로프로세서의 스코어보드 어레이
Fig. 3-9 Scoreboard array structure of the in-order SMT architecture

본 논문의 SMT 구조가 슈퍼스칼라 구조와 다른 점들은 다음과 같이 요약될 수 있다.

- 명령어 폐치를 위한 PC가 스레드별로 한 개씩 존재해야 한다. 즉, 다수의 PC가 존재한다.
- 어떤 스레드들로부터 명령어들을 폐치할 것인지를 선택할지를 선택하는 스레드 선택기가 존재한다.
- S 단계가 명령어 폐치를 위한 스레드 선택을 위해서 파이프라인의 F 단계 앞에 존재한다.
- 단일 사이클에 다수의 스레드들로부터 명령어 폐치를 수행할 수 있는 SMT 구조에서는 명령어 캐쉬가 다수의 뱅크들과 읽기 포트들을 가지고 있어야 한다.
- 명령어 캐쉬가 hit-under-miss를 지원하는 non-blocking 캐쉬이다.
- 단일 사이클에 다수의 스레드들로부터 명령어 폐치를 수행할 수 있는 SMT 구조에서는 ITLB와 BTB가 다수의 액세스 포트들을 가지고 있어야 한다.
- BTB의 각 엔트리에 스레드 식별자가 존재한다.
- 스레드별로 BHB가 존재한다. 즉 다수의 BHB가 존재한다.
- 파이프라인 상의 명령어들에 스레드 식별자가 수반된다.
- 폐치 우선권 정책 및 이슈 우선권 정책이 필요하다.
- 명령어 이슈 큐의 효율적인 사용을 위해서 큐를 압축하는 동작이 이루어진다.
- 명령어 의존성 검사는 같은 스레드에 속하는 명령어들 간에만 이루어진다.
- 분기 예측 오류와 예외의 처리는 스레드별로 이루어져야 한다. 따라서 특정 스레드의 명령어들만을 플러쉬하는 동작이 이루어진다.
- 레지스터 파일 안에는 스레드당 한 개씩의 레지스터 세트가 존재한다. 따라서 레지스터 파일의 크기는 지원되는 스레드들의 수만큼 배가된다.
- 레지스터 파일과 비슷한 이유로, 스코어보드 어레이의 크기는 지원되는 스레드들의 수만큼 배가된다.

상기의 차이점들 중에서 레지스터 파일과 스코어보드 어레이에 관한 것을 제외하면 면적 증가는 미미하다. 다만 동작의 복잡도 증가로 인해서 SMT 마이크로프로세서의 설계는 슈퍼스칼라의 것보다 복잡해진다. 그러나 이러한 설계의 복잡도 증가는 예상되는 성능 증가와 비교해 볼 때 미미하다. 마이크로프로세서의 면적 증가와 직결되는 것은 레지스터 파일과 스코어보드 어레이의 크기 증가이다. 그러나, 레지스터 파일과 스코어보드 어레이가 전체 면적에서 차지하는 비중이 10% 미만이라고 본다면 이것 역시 심각한 오버헤드는 아니다.

제 4 장 그룹화 방안의 적용

4-1절 그룹화 방안의 정의

SMT 구조는 수퍼스칼라 구조를 기반으로 약간의 변경을 가함으로써 만들어질 수 있다. 그러나, SMT 구조의 설계 복잡도가 수퍼스칼라 구조에 비해서 상당히 높다는 것은 SMT 마이크로프로세서의 실용화에 큰 장애가 된다. 그래서 본 논문에서는 SMT 구조의 복잡도를 줄이기 위한 방안으로서 그룹화를 제안한다.

그룹화는 자원과 스레드들을 몇 개의 그룹으로 나누고 특정 자원을 같은 그룹에 속하는 스레드들 사이에서만 공유하는 방식이다. 그룹화를 적용할 경우에 모든 스레드들이 전체 자원을 공유하는 방식과 비교해서 자원의 활용도 및 성능이 낮아지나, 설계의 복잡도를 줄일 수 있다는 장점이 있다.

그림 4-1과 4-2는 그룹화를 적용하지 않은 경우와 레지스터 파일의 포트들과 기능 유닛들을 2개의 그룹(group)으로 나누는 그룹화를 적용한 경우를 비교하기 위한 것이다. 그림 4-1과 4-2는 4개의 스레드를 지원하는 SMT 구조를 가정하고 있다. 그림 4-1에서 보면 그룹화를 적용하지 않았을 때에는 모든 스레드들로부터의 명령어들이 모든 레지스터 파일 읽기 포트들과 기능 유닛들을 공유한다. 따라서 레지스터 파일의 읽기 포트들과 기능 유닛들에 대한 활용도를 높일 수 있지만 레지스터 파일의 설계 및 레지스터 주소의 선택이 매우 복잡하다. 레지스터 파일에서 읽혀진 오퍼랜드들을 기능 유닛들로 보내는 패스(path)도 복잡하다. 반면에 그림 4-2에서처럼 그룹화를 적용할 경우에는 레지스터 파일로부터 데이터를 읽어내는 구조가 훨씬 간단하다. 우선, 레지스터 파일의 포트들을 서로 다른 그룹의 스레드들이 배타적으로 사용하기 때문에 레지스터 파일을 2개로 나누고, 각 레지스터 파일의 포트 수는 전체 포트 수의 반이 된다. 그림 4-2에서 레지스터 파일 0은 스레드 0 및 1에 대한 레지스터 세트들을 가지고 있으며, 레지스터 파일 1은 스레드 2 및 3에 대한 레지스터 세트들을 가지고 있다. 따라서 레지스터의 전체 수에

는 차이가 없다. 그러나, 단일 레지스터 파일의 읽기 포트 수가 그림 4-1의 8개의 반인 4개이다. 레지스터 파일의 크기는 그 안의 레지스터 수뿐 아니라 포트 수에 의해서도 많은 차이를 보인다는 점을 고려할 때, 그림 4-2의 두 레지스터 파일의 전체 면적은 그림 4-1의 레지스터 파일의 면적보다 작다. 또한 단일 레지스터에 속하는 레지스터의 수와 포트 수를 고려할 때 그림 4-2의 레지스터 파일들에 대한 액세스 속도는 그림 4-1의 것보다 빠르다. 뿐만 아니라 레지스터 파일의 주소 입력들을 선택하는 구조도 그림 4-2의 것이 더 간단하다. 그리고 레지스터 파일로부터의 오퍼랜드들을 기능 유닛들로 전달하는 패스도 그림 4-2에서 훨씬 간단하다는 것을 알 수 있다. 마지막으로 데이터 바이패싱을 적용할 경우에는 바이패싱을 위한 패스가 같은 그룹에 속하는 기능 유닛들의 파이프라인 단계들 사이에서만 이루어지게 되므로, 전체적으로 바이패싱을 위한 패스와 제어 로직이 매우 간단해진다.

이와 같이 프로세서의 자원들과 스레드들을 몇 개의 그룹으로 나누는 그룹화는 SMT 마이크로프로세서의 설계를 간단하게 한다. 그러나, 그룹화로 인해 특정 그룹의 스레드들에 속하는 명령어들은 다른 그룹의 자원들을 사용할 수 없기 때문에 자원의 활용도 및 성능이 낮아지는 문제점이 있다. 이것은 CMP 구조가 SMT 구조보다 낮은 성능을 보이는 것과 같은 이유이다. 따라서 적당한 그룹화를 통해서 성능 감소를 최소화하면서 설계의 복잡도를 줄일 수 있도록 해야 한다.

본 논문에서 그룹화는 그룹으로 나뉘어지는 자원의 종류들에 의해서 표 4-1의 세 가지로 분류된다. 그룹화 1은 명령어 폐치와 관련된 자원들에 대한 그룹화이며, 그룹화 2는 기능 유닛으로 명령어들을 이슈하기 위해서 명령어들을 정렬하는 자원들에 관한 것이며, 그룹화 3은 명령어 이슈 및 실행과 관련된 자원들에 관한 것이다. 그룹화 1, 2 및 3은 그림 4-3처럼 서로 다른 자원들에 대한 것이기 때문에 SMT 구조에 동시에 적용될 수 있으며, 이 경우에 개개의 스레드들은 세 가지 그룹화 각각에 의한 그룹들을 할당받는다. 예를 들어서 스레드 3은 그룹화 1에 의한 그룹 2에 속하면서, 그룹화 2에 의한 그룹 1과 그룹화 3에 의한 그룹 0에 속할 수 있다.

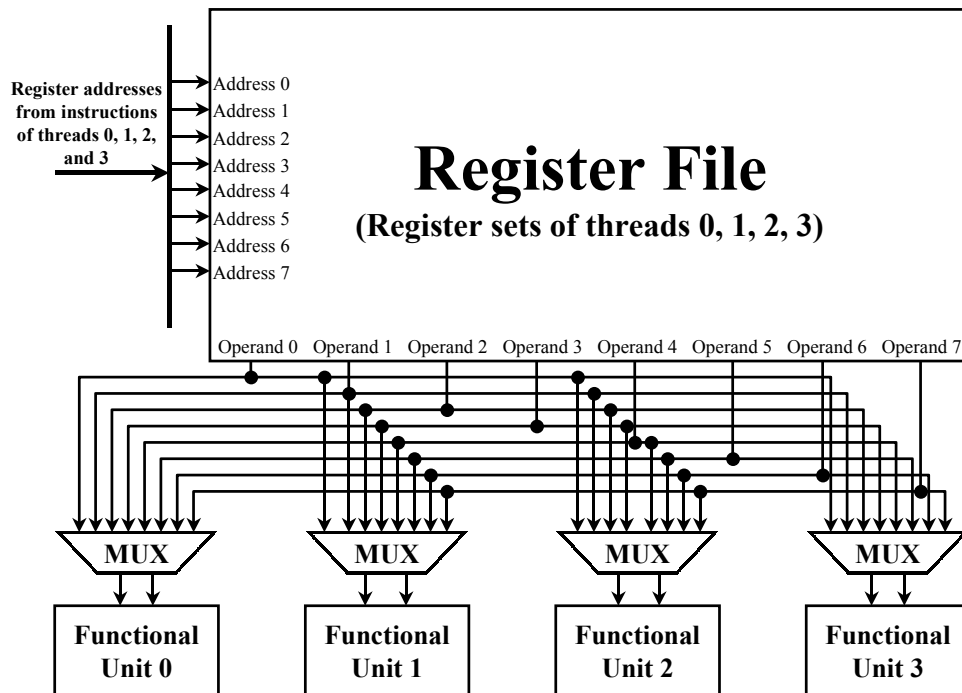


그림 4-1 그룹화를 적용하지 않은 레지스터 읽기

Fig. 4-1 Register read without grouping

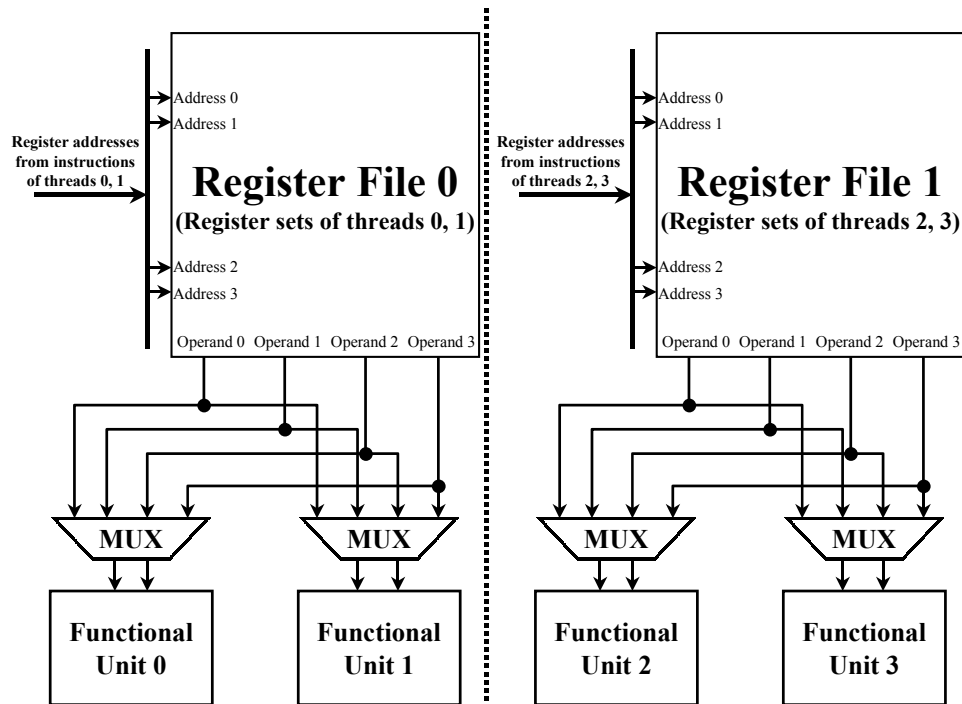


그림 4-2 그룹화에 의한 레지스터 읽기의 간략화

Fig. 4-2 Simplification of register reads by grouping

표 4-1 그룹화의 3가지 종류와 각 그룹화와 관련된 자원들

Table 4-1 Three types of grouping and their related resources

그룹화 종류	관련된 자원
그룹화 1	스레드 선택기 ITLB 포트 명령어 캐쉬 포트 BTB 포트
그룹화 2	명령어 페치 큐 디코드 슬롯 명령어 이슈 큐
그룹화 3	명령어 이슈 제어 로직 레지스터 파일 포트 스코어보드 어레이 포트 기능 유닛 DTLB 포트 데이터 캐쉬 포트

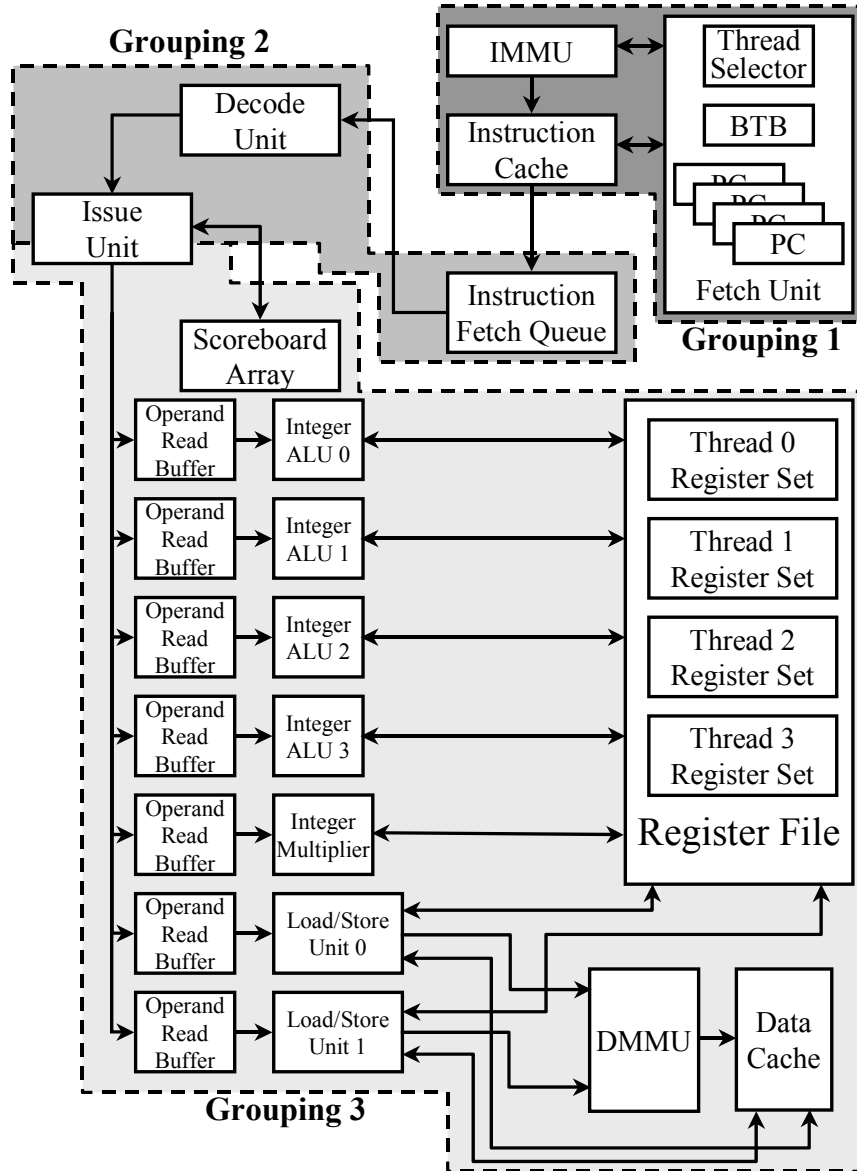


그림 4-3 순차적 SMT 구조에서의 세 가지 그룹화

Fig. 4-3 Three types of grouping in the in-order SMT architecture

4-2절 그룹화 1의 적용

그룹화 1은 명령어 페치와 분기 예측과 관련된 자원들에 대한 것이며, 그림 4-4는 2개의 그룹으로 나누는 그룹화 1에 의해서 명령어 페치 및 분기 예측의 패스가 간략해지는 모습을 보여준다. 그림 4-4에서는 단일 사이클에 최대 2개의 스레드들로부터 명령어 페치를 수행할 수 있고, 4개의 스레드가 존재하는 SMT 구조를 가정하고 있다. 이러한 SMT 구조에서 4개의 스레드가 페치 자원을 모두 공유한다면, 스레드 선택기는 4개의 스레드 중에서 2개의 스레드를 선택하여 이것을 각각 2개의 명령어 캐쉬 포트, ITLB 포트 및 BTB 포트에 할당하여야 한다. 반면에, 그림 4-4처럼 페치 자원을 2개의 그룹으로 나눈다면 스레드 선택기는 2개의 스레드 중에 1개의 스레드를 선택하는 동작 2개를 병렬적으로 수행함으로써, 스레드 선택이 이루어지고 ITLB, 명령어 캐쉬 및 BTB의 포트 할당은 이와 함께 자동적으로 이루어진다. 따라서 스레드를 선택하는 과정도 간단해지며, 선택된 스레드들에 페치 자원을 할당하는 것 또한 간단해진다. 전체적으로도 명령어 페치 및 분기 예측의 패스가 간략해진다. 이와 같은 그룹화 1에 의해서 설계의 복잡도는 간단해지지만 명령어 페치 자원의 활용은 그룹화 1에 의한 경우보다 효율적이지 못하다. 예를 들어서 그룹화 1에 의해 그룹 0에 속하는 스레드인 스레드 0과 스레드 1은 모두 명령어 페치가 가능하고, 반면에 그룹 1에 속하는 스레드 2와 스레드 3은 명령어 캐쉬 미스로 인해 페치가 불가능 할 경우에, 스레드 0 또는 스레드 1 중 1개에 대해서만 명령어 페치가 이루어진다. 반면에 그룹화를 적용하지 않았다면 스레드 0과 스레드 1 모두에 대해 명령어 페치가 이루어진다. 전체적으로 그룹화 1은 명령어 페치와 관련하여 설계의 복잡도를 현저히 줄일 수 있지만 명령어 페치 성능을 감소시킴으로서 전체적인 성능 및 자원의 활용도에 부정적인 영향을 미친다. 따라서 그룹화 1의 적용은 요구되는 성능과 설계의 복잡도를 고려하여 이루어져야 한다.

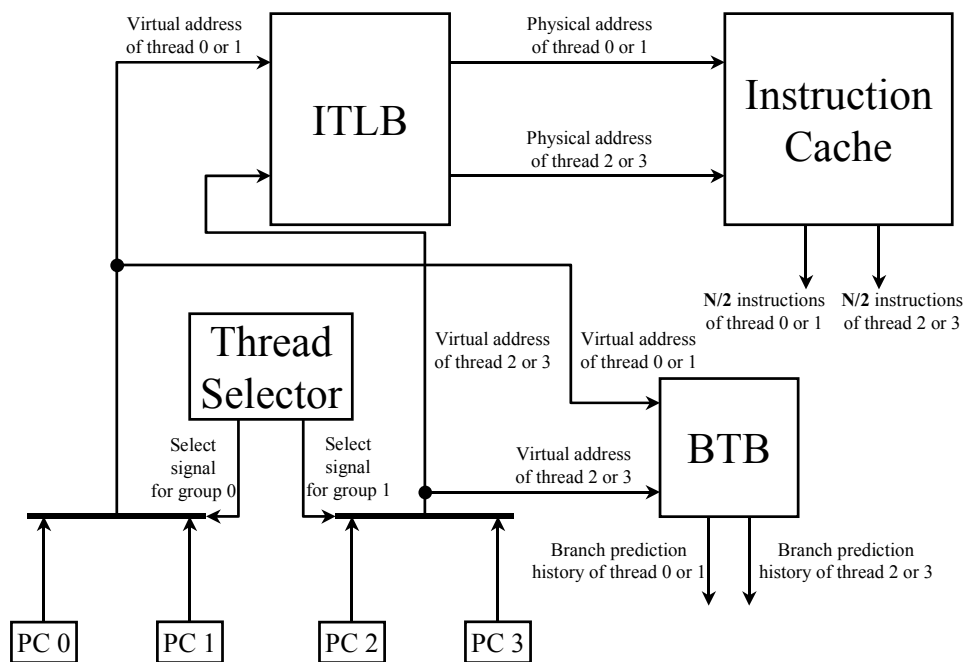


그림 4-4 그룹화 1의 예
Fig. 4-4 Example of grouping 1

4-3절 그룹화 2의 적용

그룹화 2는 명령어 페치 큐, 디코드 슬롯(decode slot) 및 명령어 이슈 큐에 대한 것이며, 그림 4-5는 이러한 자원들을 2개의 그룹으로 나눈 구조를 보여준다. 그림 4-5에서 보듯이 그룹화 2를 적용할 경우에는 각 그룹별로 명령어 페치 큐, 디코드 유닛 및 명령어 이슈 큐가 개별적으로 존재한다. 따라서 단일 사이클에 각 큐로 입력되는 또는 큐로부터 출력되는 명령어의 수가 그룹화에 의해서 줄어들기 때문에 각 큐의 관리가 간단해진다. 그리고, 그룹화 1에 의해서 명령어 페치 패스가 그룹화되었다면 명령어 페치 큐의 입력 부분은 더욱 간단해질 수 있다. 예를 들어서 그룹화 1과 그룹화 2에 의한 그룹 수가 2개라면 그룹화 2에 의해 그룹 0에 속하는 명령어 페치 큐는 그룹화 1에 의해 그룹 0에 속하는 명령어 페치 패스로부터만 명령어를 받으면 된다. 그리고 그룹화 2에 의한 각 그룹에 속하는 스레드의 수가 1개이면, 그룹화 2에 의한 그룹 수는 전체 스레드의 수와 같아진다. 이 경우 명령어 페치 큐와 명령어 이슈 큐는 스레드별로 존재하기 때문에 개개의 명령어 페치 큐와 명령어 이슈 큐는 슈퍼스칼라의 것과 같아진다. 따라서 명령어 페치 큐와 명령어 이슈 큐에 존재하는 명령어들에 대한 스레드별 명령어 플러쉬는 해당되는 명령어 페치 큐와 명령어 이슈 큐의 전체 엔트리들을 플러쉬하는 것이 된다. 따라서 명령어 플러쉬로 인해 명령어 페치 큐의 중간에 비어있는 엔트리가 발생하는 현상이 없다. 또한 각 명령어 이슈 큐의 엔트리들은 큐 상의 순서대로 이슈되기 때문에 명령어 이슈 큐의 중간에 비어있는 엔트리가 발생하지 않으므로, 명령어 이슈 큐에 압축을 적용할 필요가 없다. 그룹화 2는 명령어들을 디코드하고 저장하는 구성 요소의 설계를 간단히 할 수 있지만 그룹화 2의 적용은 성능에는 좋지 않은 영향을 미칠 수 있다. 예를 들어서 그룹 0의 명령어 페치 큐가 모두 채워지고, 그룹 1의 명령어 페치 큐는 많은 여유 엔트리들을 가지고 있을 때에, 전체적으로 여유 엔트리가 존재함에도 불구하고, 그룹 0에 속하는 스레드로부터는 명령어를 페치할 수 없다. 그리고 명령어 이슈 큐에 대해서도 이와 같은 현상이 발생한다. 이것은 그룹화 2를 적용하지 않았을 경우에는 발생하지 않는 현상이다.

따라서 그룹화 2를 적용할 경우에는 각 그룹별로 명령어 페치 큐와 명령어 이슈 큐의 엔트리 수를 충분한 값으로 하여야 한다. 즉, 큐의 전체 엔트리 수가 충분하더라도 각 큐의 엔트리 수가 부족할 경우에 전체적인 페치 성능에 부정적인 영향을 미친다. 따라서 그룹화 2는 그룹화 1처럼 적절한 수준에서 적용되어야 하며, 그룹화 2의 적용 시에는 각 명령어 페치 큐 또는 명령어 이슈 큐의 엔트리 수를 적절히 증가시켜 주어야 한다.

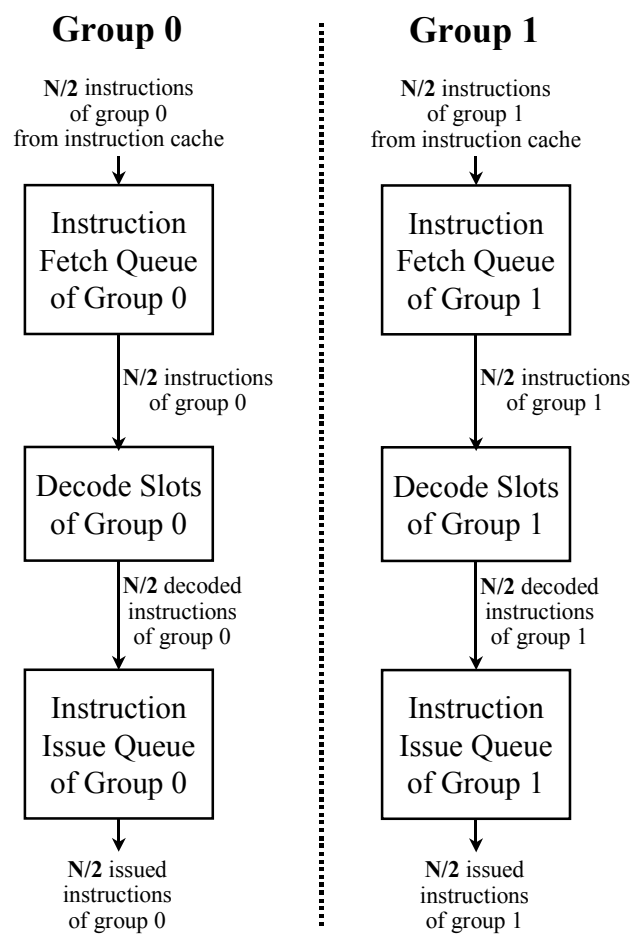


그림 4-5 그룹화 2의 예

Fig. 4-5 Example of grouping 2

4-4절 그룹화 3의 적용

그룹화 3은 명령어 이슈 및 실행 자원에 대한 그룹화이며, 그림 4-6은 그 예를 보여준다. 그룹화 3에 의한 각 그룹은 각각의 기능 유닛 종류에 대해서 서로 같은 수의 기능 유닛들을 가지고 있다. 예를 들어서 ALU의 전체 수가 4개이고, 그룹 수가 2개이면 각 그룹은 2개씩의 ALU들을 포함한다. 그룹화 3은 3가지 종류의 그룹화 중에서 설계의 간략화 효과가 가장 큰 것이다. 그룹화 3을 적용할 경우에는 읽기 포트 및 쓰기 포트를 그룹별로 배타적으로 사용할 수 있기 때문에 레지스터 파일을 그룹별로 따로 존재하게 할 수 있다. 이로 인해 단일 레지스터의 크기를 줄일 수 있을 뿐 아니라 각 레지스터 파일의 포트 수가 작아지기 때문에 레지스터 파일의 설계가 간단해 지면서 레지스터 파일로의 액세스 시간도 매우 단축된다. 또한 레지스터 파일의 크기는 포트 수에 의해서도 많은 영향을 받는다는 것을 고려하면 분할된 레지스터 파일들의 전체 면적은 그룹화 3이 적용되지 않은 단일 레지스터의 크기보다 작다. 또한 그룹화 3에 의해서 명령어 이슈 로직이 간단해 진다. 즉 명령어 이슈 로직은 그룹별로 따로 존재하게 되고, 각 이슈 로직은 해당 그룹에 속하는 스레드들에 속하는 명령어들에 대해서만 이슈를 관리하게 되므로 그룹화 3이 적용되지 않았을 경우보다 더욱 간단해 지는 것이다. 그리고 각 명령어의 실행은 그 명령어와 같은 그룹에 속하는 기능 유닛 및 레지스터 파일을 사용해서 이루어져야 하므로, 이슈되는 명령어들에 대해서 기능 유닛들 및 레지스터 파일의 포트들을 할당하는 것은 그룹화 3에 의한 그룹 수만큼 간편해 진다. 그리고, 레지스터 파일에서 소스 오퍼랜드들을 읽어서 기능 유닛들에 제공하는 패스가 간단해지고, 기능 유닛으로부터 레지스터 파일에 결과 데이터를 쓰는 과정도 간략해진다. 또한 그룹화 3에 의해 데이터 바이패싱은 같은 그룹에 속하는 기능 유닛들의 파이프라인 단계들 사이에서 이루어지게 되므로 바이패싱 로직 및 패스가 간단해진다. 마지막으로, 그룹화 3은 스레드별 명령어 플러쉬 동작 시에 플러쉬할 명령어들을 특정 그룹에 속하는 기능 유닛들에서만 찾을 수 있도록 한다. 이와 같이 그룹화 3은 설계의 복잡도를 현저히 줄일 수 있는 방법이지만 이로 인한 성능 저

하를 고려해야 한다. 예를 들어서 그룹화 3의 적용에 의해서 각 그룹에는 ALU의 수가 2개인 경우, 그룹 0에는 3개의 이슈될 수 있는 ALU 명령어가 있고, 그룹 1에는 이슈될 수 있는 명령어가 존재하지 않을 때, 전체적으로 이슈되는 ALU 명령어의 수는 단지 2개이다. 이 경우 그룹 1에 여분의 ALU가 존재함에도 불구하고 그룹 0의 ALU 명령어 1개는 이슈되지 못하는 것이다. 이와 같이 그룹화 3은 실행 자원을 고정적으로 분할함으로써 자원의 사용 효율과 성능에 부정적인 영향을 미친다. 따라서 그룹화 3의 적용은 상기의 그룹화 1 및 2처럼 그 영향을 분석하고 적절히 적용되어야 한다.

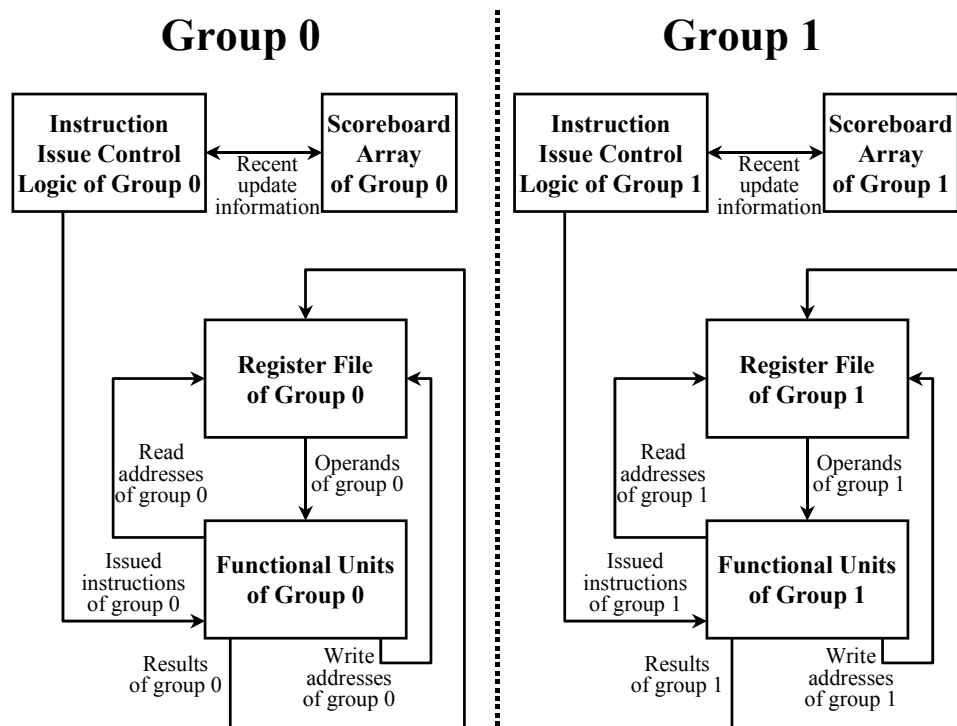


그림 4-6 그룹화 3의 예

Fig. 4-6 Example of grouping 3

제 5 장 시뮬레이션 환경

5-1절 시뮬레이터

마이크로프로세서의 구조를 설계할 때에 구조를 정하고 이의 성능을 예측하는 과정은 필수적이다. 이러한 과정을 위해서 마이크로프로세서의 시뮬레이터가 제작되며 이 시뮬레이터에 의해 각 구조 항목에 대한 tradeoff의 계량화로 최적화된 구조 설계를 도출할 수 있게 된다. 이와 같은 방법으로 경험이나 추측에 의한 구조 설계가 아닌 정확한 성능 데이터를 바탕으로 세부적인 구조를 정할 수 있다. 또한 시뮬레이터는 차후의 실제적인 마이크로프로세서 설계 과정 중 구조의 이해를 돕는 역할과 함께 설계를 마친 후에 설계 검증에 유용하게 사용된다. 같은 테스트 프로그램을 시뮬레이터와 설계된 마이크로프로세서에 모두 입력하고 시뮬레이터를 사용한 시뮬레이션 결과와 설계된 마이크로프로세서에서의 결과가 같은지 확인함으로써 검증을 수행할 수 있다. 그리고 실제적인 마이크로프로세서 설계 이전에 상용화된 프로그램들을 입력으로 사용한 시뮬레이션을 통해서 다른 프로세서와의 대략적인 성능 비교(benchmarking)도 가능하게 한다. 그리고 마이크로프로세서 시스템의 최종 개발 이전에 시뮬레이터를 이용하여 응용 프로그램을 개발하는 것이 가능하다.

시뮬레이터는 대략 트레이스-구동(trace-driven) 시뮬레이터와 실행-구동(execution-driven) 시뮬레이터로 구분할 수 있다^[74]. 트레이스-구동 시뮬레이터가 동작하려면 우선 마이크로프로세서가 프로그램을 수행하면서 사용하는 명령어 주소, 데이터 메모리 주소, 데이터 등의 정보를 모아 놓은 트레이스(trace)^[75]가 필요하다. 이러한 트레이스는 다른 시뮬레이터로부터 얻을 수 있고 실제 컴퓨터로부터 수집할 수도 있다. 트레이스가 생성되면 이 트레이스는 시뮬레이터에 가해져 성능 평가를 수행할 수 있다. 트레이스-구동 시뮬레이션은 페이징 알고리즘(paging algorithm), 캐쉬 운용 알고리즘, TLB 성능 등을 평가하는데 주로 사용된다. 이

시뮬레이터의 장점은 시뮬레이터를 비교적 단순하게 만들 수 있으며, 시뮬레이션 속도가 빠르다는 것이다. 반면에 단점은 비슷한 구조의 시스템에서만 유효하다는 점, 시스템의 동적인 구성이 불가능하다는 점, 시스템의 일부분만을 시뮬레이션할 수 있다는 점 등을 들 수다. 또한 여기에 트레이스의 크기도 문제가 되는데 트레이스가 큰 경우에는 몇 십 GB 이상을 필요로 하기 때문에 이 경우 저장하거나 다루기가 어렵다.

실행-구동 시뮬레이터는 이에 반해 트레이스를 사용하지 않고 실제 실행 프로그램의 입력으로 사용하여 독립적인 시뮬레이션을 수행한다. 독립적인 시뮬레이션이 가능하기 위해서 실행-구동 시뮬레이터는 마이크로프로세서가 갖추어야 할 모든 부분들을 모델링(modeling)해야 한다. 실행-구동 시뮬레이터의 장점은 트레이스 없이 독자적인 시뮬레이션이 가능하다는 점, 시스템을 동적으로 구성할 수 있다는 점, 정확한 성능 평가 결과를 얻을 수 있다는 점, 마이크로프로세서의 다양한 구성 요소에 대해서 성능 평가를 수행할 수 있다는 점 등을 들 수 있다. 반면에 트레이스-구동 시뮬레이터에 비해 시뮬레이터 작성이 어렵고 시뮬레이션의 속도도 다소 느리다는 단점이 있다.

시뮬레이터의 다른 분류 방법으로서, 명령어-기본(instruction-based) 시뮬레이터와 사이클-기본(cycle-based) 시뮬레이터로 구분할 수 있다^[76]. 명령어-기본 시뮬레이터는 입력 프로그램을 명령어 단위로 실행하는 시뮬레이터이다. 따라서 명령어-기본 시뮬레이터에서는 입력 프로그램의 한 명령어의 모든 동작을 실행한 후에야 다음 명령어를 받아들여 실행하는 형태를 취한다. 즉, 그림 5-1과 같이 단일 명령어의 수행을 끝낸 후에 시스템 레지스터와 메모리의 값을 갱신하고 다음 명령어를 수행한다. 그림 5-1에서 한 번의 루프 반복(loop iteration)은 단일 명령어의 수행에 해당한다. 이러한 명령어-기본 시뮬레이터는 응용 프로그램을 개발할 때에 주로 사용된다. 명령어-기본 시뮬레이터의 장점은 시뮬레이션 속도가 빠르다는 점과 시뮬레이터의 작성이 쉽다는 점 등을 들 수 있다. 반면에 단점은 마이크로프로세서의 사이클 단위 모습을 보여주지 않는다는 점, 내부 구성 요소들에 대한 시뮬레이션을 수행할 수 없다는 점, 마이크로프로세서의 성능 평가에 사용될 수 없다는 점 등이다.

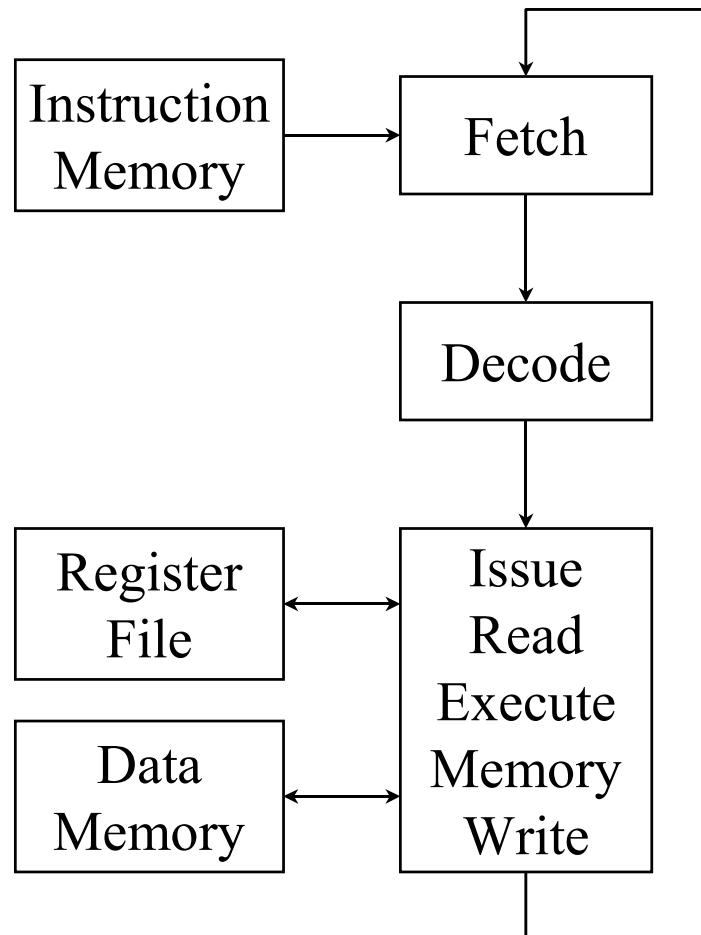


그림 5-1 명령어-기본 시뮬레이션의 수행 과정

Fig. 5-1 Flow of instruction-based simulation

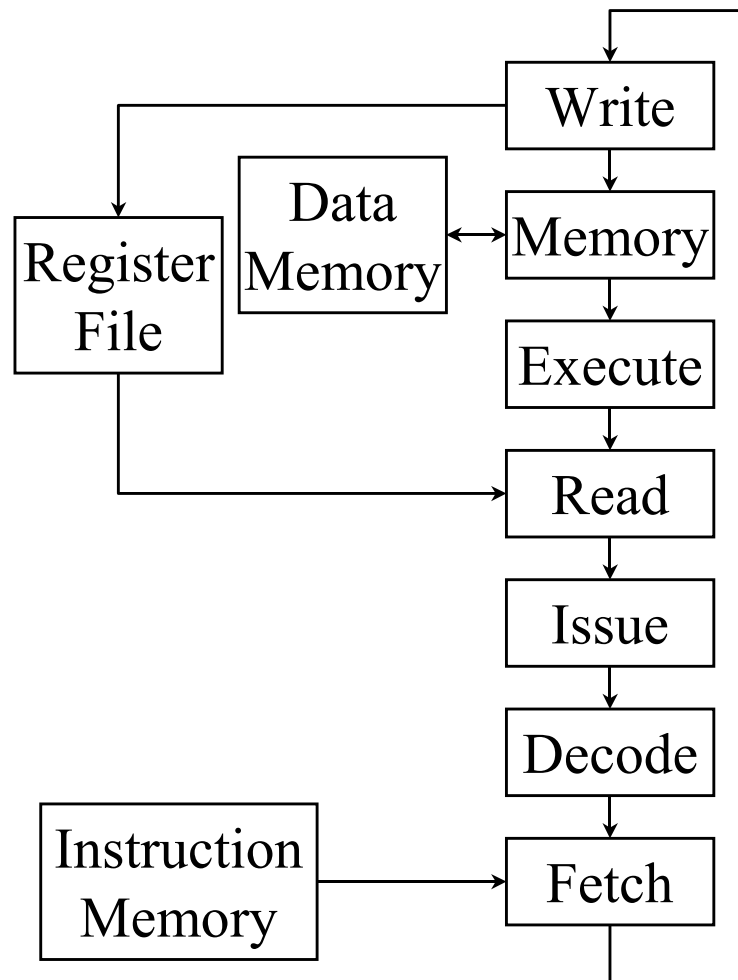


그림 5-2 사이클-기본 시뮬레이션의 수행 과정

Fig. 5-2 Flow of cycle-based simulation

반면에 사이클-기본 시뮬레이터는 입력 프로그램을 사이클 단위로 수행한다. 따라서 사이클-기본 시뮬레이터는 한 사이클의 동작을 수행한 후에 다음 사이클의 동작을 수행하는 방식을 취한다. 파이프라이닝(pipelining)이 마이크로프로세서에 적용되어 있다면 단일 사이클의 수행은 다수의 파이프라인 단계들을 한 번씩 수행함으로써 이루어진다. 파이프라인 단계들의 수행 순서에 있어서는 각 단계들 사이의 의존성을 고려해야 한다. 명령어 페치 동작은 명령어 디코드 동작에 의존성을 가지기 때문에 명령어 디코드 동작을 수행한 후에 그 결과에 바탕을 두고 명령어 페치를 수행된다. 같은 이유로 명령어 디코드 동작은 명령어 이슈 동작의 결과를 바탕으로 수행되며, 이와 같은 의존성 관계는 다른 파이프라인 단계들 사이에도 적용된다. 파이프라인 단계들 사이의 의존성 때문에 그림 5-2와 같이 사이클-기본 시뮬레이터는 각 파이프라인 단계들을 마이크로프로세서의 파이프라인 구조에서 정의된 순서의 역으로 수행하며, 그림 5-2에서 한 번의 루프 반복은 단일 사이클의 수행에 해당한다. 사이클-기본 시뮬레이터는 주로 마이크로프로세서의 구조 검증 또는 성능 평가에 사용된다. 사이클-기본 시뮬레이터의 장점은 실제 마이크로프로세서의 상태를 사이클마다 보여준다는 점, 사이클 단위를 기본으로 하는 성능 평가를 수행할 수 있다는 점, HDL(Hardware Description Language)^{[77][78]} 기술(description) 및 기타 설계 과정에서 검증을 위한 모델로서 사용될 수 있다는 점 등을 들 수 있다. 반면 단점으로는 시뮬레이터의 기술이 어렵고 시뮬레이션 속도가 비교적 느리다는 점 등이다.

본 논문에서는 슈퍼스칼라 및 SMT 구조의 마이크로프로세서를 C 언어^[79]를 사용하여 작성하고 IBM의 x86 계열 PC 기반의 Microsoft Windows 환경의 Microsoft Visual C++^[80] 컴파일러(compiler)를 사용하여 컴파일(compile)한 후에 컴파일러의 내장된 디버깅(debugging) 기능을 사용하여 검증을 수행하였다. 시뮬레이터를 HDL로 기술할 수도 있지만 이럴 경우 시뮬레이션 속도가 매우 느리고 마이크로프로세서의 구조를 완전히 묘사해야 하기 때문에 부담이 따른다. 따라서 보통의 경우 C 언어에 의한 시뮬레이터 기술과 시뮬레이션을 통해 마이크로프로세서에 구조에 대한 검증 및 성능 평가가 이루어진 후에 이를 기반으로 HDL을 이용한 시뮬레이터를 기술한다.

표 5-1 시뮬레이터의 구성

Table 5-1 Simulator structure

구분	시뮬레이터 루틴	설명
시뮬레이터 본체	파이프라인 제어 루틴	파이프라인의 S 단계부터 I 단계까지를 관리하 는 루틴
	기능 유닛 루틴	R 단계부터 W 단계까지의 기능 유닛 동작들 을 파이프라인 단계별로 수행하는 루틴
	메모리 엑세스 루틴	ITLB, DTLB, 명령어 캐쉬 및 데이터 캐쉬에 대한 엑세스 요구를 처리해 주고, TLB 또는 캐쉬에서의 미스를 처리해 주는 루틴
시뮬레이터 보조 루틴	초기화 루틴	하드웨어의 구성 요소들을 만들고 전원이 처음 켜졌을 때의 상태로 하드웨어 상태를 설정하는 루틴
	프로그램 로더	프로그램을 로드하고 개개의 프로그램에 맞게 하드웨어 상태를 초기화시키는 루틴
	시스템 호출 처리기	SWI 명령어에 의한 시스템 호출을 처리해 주 는 루틴
	트레이스 발생기	매 사이클 하드웨어의 다양한 상태 값들을 파 일로 출력해 주는 루틴
	결과 요약기	시뮬레이션이 끝날 때 해당 마이크로프로세서 의 성능과 관련된 결과 데이터들을 파일로 출 력해주는 루틴

본 논문의 시뮬레이터는 실행-구동 및 사이클-기본 방식을 취해서, 수퍼스칼라 및 SMT 구조에 대한 세밀한 모델링이 시뮬레이터 기술 시에 이루어진다. 따라서 시뮬레이션을 통해서 다양한 사이클 기본의 정확한 성능 평가가 가능하며, 마이크로프로세서 구조의 각 구성 요소들에 대한 성능 평가도 가능하다. 또한 매 사이클 각 구성 요소들의 상태를 정밀하게 볼 수 있기 때문에 그 정보를 바탕으로 성능 개선을 위한 구조 변경을 수행할 수 있다. 시뮬레이터에서 실제 마이크로프로세서 구조와 다른 점이 있다면 명령어 캐쉬와 데이터 캐쉬에서의 뱅크 충돌(bank conflict)^[73]을 무시하고 있다는 점 정도다. 시뮬레이터는 마이크로프로세서의 각 구성 요소들의 구조, 배치, 크기 및 제어 방법 등을 고정된 것으로 하지 않고 다양한 구조 변수를 사용하여 설정할 수 있게 하고 있다. 이것은 마이크로프로세서 구조를 다양하게 구성시키면서 성능 평가를 수행할 수 있도록 하고, 각 설계 항목에 대한 tradeoff를 정확히 예측할 수 있도록 한다.

본 논문의 시뮬레이터는 표 5-1과 같이 시뮬레이터 본체(simulator body)와 시뮬레이터 보조 루틴(simulator assistant routine)으로 나눌 수 있다. 시뮬레이터 본체는 실제 마이크로프로세서의 구조가 세밀하게 모델링되어 있는 것으로서, 마이크로프로세서에서 이루어지는 동작에 대한 시뮬레이션이 이루어진다. 시뮬레이터 보조 루틴은 시뮬레이터 본체의 실행을 돕고 결과를 분석하기 위한 것이다. 부록 1은 이러한 시뮬레이터의 main 함수를 보여준다.

본 논문의 수퍼스칼라 및 SMT 구조의 파이프라인 구조는 각각 그림 2-2 및 3-2와 같다. 그런데 시뮬레이터는 사이클 단위로 시뮬레이션을 수행하기 때문에 그림 5-2와 같이 시뮬레이터 본체는 파이프라인 단계들을 그림 2-2 및 3-2의 역순으로 실행한다. 그림 5-3은 SMT 구조의 시뮬레이터 본체가 파이프라인 단계들을 역순으로 실행하는 시뮬레이터 기술의 간략한 모습을 보여준다. 먼저 W 단계의 writeback 함수는 이전 사이클에 EM 또는 M 단계로부터 W 단계로 넘어온 결과를 바탕으로 레지스터 파일에 그 결과를 쓴다. EM 단계의 ext_memory 함수는 이전 사이클까지 M 단계에서 캐쉬 미스가 발행하였거나 또는 uncached 영역으로 판별되어서 EM 단계로 넘겨진 데이터 메모리 액세스들에 대한 동작을 수행한다. M 단계의 memory 함수는 이전 사이클에 E 단계에서 이루어진 주소 계산

등의 동작을 바탕으로 M 단계의 동작을 수행한다. E 단계의 execute 함수는 이전 사이클의 R 단계에서 읽은 레지스터 오퍼랜드들을 사용하여 ALU, 곱셈 및 주소 계산 등의 동작을 수행한다. R 단계의 reg_read 함수는 이전 사이클에 이슈된 명령어들의 오퍼랜드들을 레지스터 파일에서 읽거나 또는 바이패싱을 통해서 오퍼랜드 값을 얻는다. I 단계의 issue 함수는 이전 사이클까지 디코드되어서 명령어 이슈 큐에 저장된 명령어들 중의 일부분을 기능 유닛으로 이슈한다. D 단계의 decode 함수는 이전 사이클까지 명령어 페치에 의해서 명령어 페치 큐에 저장된 명령어들 중에서 최하위 엔트리부터의 명령어들을 디코드한 후에 명령어 이슈 큐의 최상위 엔트리 위치로 저장한다. F 단계의 fetch 함수는 이전 사이클의 S 단계에서 선택된 스레드들의 명령어들을 명령어 캐쉬에서 페치하여 명령어 페치 큐의 최상위 엔트리 위치로 저장한다. 마지막으로, S 단계의 fetch_select 함수는 다음 사이클의 F 단계에서 명령어들을 페치할 스레드들을 선택한다. 이와 같은 전 과정에 의해서 단일 사이클의 동작이 수행되며, 이 과정이 매 사이클마다 루프(loop)로 반복되도록 함으로써 시뮬레이터 본체의 동작이 이루어진다.

```
While (!end)
{
    writeback();           // W stage function
    ext_memory();          // EM stage function
    memory();              // M stage function
    execute();             // E stage function
    read();                // R stage function
    issue();               // I stage function
    decode();              // D stage function
    fetch();               // F stage function
    fetch_sel();           // S stage function
}
```

그림 5-3 시뮬레이터 본체의 파이프라인 기술

Fig. 5-3 Pipeline description of the simulator body

5-2절 시뮬레이터의 구조 변수들

시뮬레이터를 기술할 때에 마이크로프로세서의 내부 구성 요소들의 구성, 배치, 크기 및 제어 방법 등을 고정된 것으로 만들 경우에는 마이크로프로세서의 단일 구성에 대해서만 시뮬레이션 및 성능 평가를 수행할 수 있다. C 언어로 기술되는 시뮬레이터의 가장 주요한 목적은 다양한 시뮬레이션을 통해서 가장 적합한 구조를 제안하기 위한 것이기 때문에 고정된 구조의 시뮬레이터는 적당하지 않다. 따라서 본 논문의 시뮬레이터는 표 5-2와 같이 다양한 구조 변수를 포함하고 있다. 부록 2는 이러한 구조 변수들의 값을 정하는 헤더(header) 파일의 기술을 보여주는 것이다. 구조 변수들은 시뮬레이터를 컴파일하는 과정에서 그 값이 정해진다. 따라서 구조 변수들의 값을 다양하게 변화시켜 가면서 컴파일을 수행함으로써, 다양한 구성의 시뮬레이터를 만들 수 있다. 그리고 이러한 시뮬레이터들을 사용하여 시뮬레이션과 성능 평가를 수행하여 최적의 구조를 제안하기 위한 결과들을 얻어 낸다.

먼저 4장의 그룹화에 대한 구조 변수인 GRP1_NUM, GRP2_NUM 및 GRP3_NUM은 각각 그룹화 1에서 정의되는 그룹 수, 그룹화 2에서 정의되는 그룹 수 및 그룹화 3에서 정의되는 그룹 수를 정하기 위한 것이다. THREAD_NUM은 SMT 구조에서 지원되는 스레드의 수를 정하기 위한 것이다. 본 논문에서는 단일 시뮬레이터 기술이 수퍼스칼라 구조와 SMT 구조에 적용되는데, THREAD_NUM을 1로 정하며 시뮬레이터는 수퍼스칼라 구조를 위한 것이고 THREAD_NUM이 2 이상인 경우 시뮬레이터는 SMT 구조에 해당한다. FETCH_WIDTH, DECODE_WIDTH 및 ISSUE_WIDTH는 각기 페치 폭, 디코드 폭 및 이슈 폭에 해당하는 명령어 수를 정하기 위한 것이다.

IFQ_SIZE와 IIQ_SIZE는 각각 명령어 페치 큐와 명령어 이슈 큐의 엔트리 수를 정하기 위한 것이다. GRP2_NUM이 2 이상이면 다수의 개별적인 명령어 이슈 큐와 명령어 페치 큐가 존재하게 되며, 이 경우 IFQ_SIZE와 IIQ_SIZE는 전체 엔트리 수를 정의하는 것이다.

표 5-2 구조 변수들

Table 5-2 Configurations parameters

구조 변수	설명
GRP1_NUM	그룹화 GRP 1에 의해서 정해진 그룹 수
GRP2_NUM	그룹화 GRP 2에 의해서 정해진 그룹 수
GRP3_NUM	그룹화 GRP 3에 의해서 정해진 그룹 수
THREAD_NUM	지원되는 스레드의 수
FETCH_WIDTH	명령어 페치 폭
DECODE_WIDTH	명령어 디코드 폭
ISSUE_WIDTH	명령어 이슈 폭
IFQ_SIZE	명령어 페치 큐의 엔트리 수
IIQ_SIZE	명령어 이슈 큐의 엔트리 수
REGR_NUM	레지스터 파일의 읽기 포트 수
REGW_NUM	레지스터 파일의 쓰기 포트 수
ALUF_NUM	ALU의 수
MULF_NUM	곱셈 유닛의 수
LSUF_NUM	로드/스토어 유닛의 수
ICP_NUM	명령어 캐쉬의 읽기 포트 수
IC_SIZE	명령어 캐쉬의 크기
ICWAY_NUM	명령어 캐쉬의 way 수
ICBLK_SIZE	명령어 캐쉬의 블록 크기
DC_SIZE	데이터 캐쉬의 크기
DCWAY_NUM	데이터 캐쉬의 way 수
DCBLK_SIZE	데이터 캐쉬의 블록 크기
BTB_SIZE	BTB의 엔트리 수
ITLB_SIZE	ITLB의 엔트리 수
DTLB_SIZE	DTLB의 엔트리 수
CACHEREP	명령어 캐쉬 및 데이터 캐쉬의 대체 정책
BTBREP	BTB의 대체 정책
TLBREP	ITLB 및 DTLB의 대체 정책
CFILL_CYC	명령어 캐쉬 및 데이터 캐쉬의 단일 블록을 외부 메모리로부터 채우거나 외부 메모리로 내보내는데 소요되는 사이클 수
FETCH_PRI	페치 우선권 정책
ISSUE_PRI	이슈 우선권 정책

레지스터 파일과 관련해서는 REGR_NUM과 REGW_NUM이 존재한다. REGR_NUM은 레지스터 파일의 읽기 포트 수를 정하기 위한 것이고 REGW_NUM은 레지스터 파일의 쓰기 포트 수를 정하는 변수이다. GRP3_NUM이 2 상인 경우 REGR_NUM과 REGW_NUM은 개개의 레지스터 파일에 대한 포트 수에 대한 것이 아니라 전체 포트 수를 정하는 것이다. 레지스터 파일 안에 존재하는 레지스터들의 수는 스레드 수에 의해서 정해지기 때문에 구조 변수에 포함되지 않는다. 스코어보드 어레이의 포트 수는 대략 레지스터 파일의 포트 수와 비슷한 값을 갖는 것이 좋다. 그런데 마이크로프로세서 구조에서 스코어보드 어레이의 포트 수는 중요한 요인이 아니기 때문에 스코어보드 어레이의 포트 수로 인해서 성능이 제한되는 경우가 없도록 충분히 많은 포트가 존재한다고 가정한다. ALUF_NUM은 기능 유닛 중에서 ALU의 수에 대한 것이고, MULF_NUM과 LSUF_NUM은 각기 곱셈 유닛과 로드/스토어 유닛의 수에 대한 변수이다.

ICP_NUM은 명령어 캐쉬의 읽기 포트 수를 정하기 위한 것으로서 단일 사이클에 폐치할 수 있는 스레드 수를 위한 것이기도 하므로 THREAD_NUM보다 큰 값을 가질 수 없다. 명령어 캐쉬의 단일 포트로부터 단일 사이클에 읽을 수 있는 명령어의 수는 FETCH_WIDTH를 ICP_NUM으로 나눈 수이다. 데이터 캐쉬의 포트 수는 로드/스토어 유닛의 수와 같기 때문에 구조 변수에 포함되지 않는다. IC_SIZE와 DC_SIZE는 각기 명령어 캐쉬와 데이터 캐쉬의 크기를 의미한다. ICWAY_NUM과 DCWAY_NUM은 각기 명령어 캐쉬와 데이터 캐쉬를 set-associative 캐쉬로 구성할 때 캐쉬의 way 수를 정하기 위한 것이고 direct-mapped 캐쉬인 경우에 이 변수들은 각기 1이 된다. ICBLK_SIZE와 DCBLK_SIZE는 각기 명령어 캐쉬와 데이터 캐쉬의 블록 크기(block size)에 대한 것이다. 캐쉬 세트(set)들의 수는 캐쉬의 크기와 way 수 및 블록 크기에 의해서 계산되며, 그 값이 1인 경우에 fully-associative 캐쉬에 해당한다. 명령어 캐쉬와 데이터 캐쉬에서의 뱅크 충돌은 고려되지 않기 때문에 뱅크의 수에 대한 구조 변수는 존재하지 않는다.

BTB는 fully-associative로 구성되며 BTB_SIZE는 BTB의 엔트리 수를 정하기 위한 것이다. BTB의 읽기 포트 수는 명령어 캐쉬의 읽기 포트 수와 같기 때문에

구조 변수로서 정의되지 않는다. BHB는 마이크로프로세서 구조에서 중요한 유닛이 아니고 엔트리 수를 적당히 하면 BHB의 엔트리 부족으로 인한 정체 현상이 거의 발생하지 않기 때문에 BHB의 엔트리 수가 성능에 미치는 영향이 없다. 따라서 BHB의 엔트리 수에 대해서는 제한을 두지 않았다. ITLB_SIZE는 full-associative로 구성된 ITLB의 엔트리 수를 정한다. ITLB의 포트 수는 명령어 캐쉬의 포트 수와 같기 때문에 구조 변수에 포함되지 않는다. DTLB도 fully-associative로 구성되었으며 DTLB_SIZE는 DTLB의 엔트리 수를 정하기 위한 것이다. DTLB의 포트 수는 로드/스토어 유닛의 수와 같기 때문에 구조 변수로서 정의되지 않는다.

캐쉬, BTB 및 TLB는 새로운 엔트리를 할당할 때에 어떤 엔트리를 대체할 지를 결정하는 대체 정책(replacement policy)이 필요하다. CACHEREP, BTBREP 및 TLBREP는 각기 2개의 캐쉬들, BTB 및 2개의 TLB들에 대한 대체 정책을 정하기 위한 것이다. 대체 정책으로서 선택할 수 있는 것에는 랜덤(random) 방식과 LRU(Least Recently Used)^[38] 방식이 있다. 본 논문의 시뮬레이터는 little-endian 메모리 시스템을 사용하는 x86 계열의 PC에서 시뮬레이션을 수행하기 때문에 시뮬레이터 기술의 편의를 위해서 시뮬레이터도 little-endian 메모리 시스템을 사용한다. 그 외에 메모리와 관련해서 고려해야 될 사항으로 메모리 액세스의 지연 시간에 대한 것이다. 이와 관련해서 캐쉬 히트 시 캐쉬 액세스를 위해서 소요되는 사이클 수는 1사이클로 정했다. 이것은 요즈음에 대부분의 마이크로프로세서에서도 적용되는 사이클 수이다. 그리고, 캐쉬의 블록을 외부 메모리로부터 채우는(refill) 데 소요되는 사이클 수 또는 캐쉬의 블록을 외부 메모리로 내보내는데(evict) 소요되는 사이클 수는 CFILL_CYC에 의해서 정의된다. 입력 프로그램들의 모든 주소 영역(address space)은 시뮬레이터에서 cached 영역으로 처리된다. 따라서 외부 메모리에 대한 액세스는 항상 캐쉬를 통해서 이루어지기 때문에 외부 메모리를 액세스하기 위한 사이클 수는 따로 정의되지 않는다. 시뮬레이터는 입력 프로그램들을 페이지로 나눌 때 페이지의 크기를 가능한 크게(최대 1 MB) 한다. 그리고 시뮬레이터로 입력되는 프로그램의 크기는 비교적 작다. 이런 이유로 시뮬레이션 중에 TLB 미스는 거의 발생하지 않으며 대부분이 시뮬레이션 초기에 발

생하기 때문에 TLB 미스로 인한 성능 변화는 거의 없다. 그리고, TLB 미스의 수는 OS의 페이징 기법에 의해서 많은 영향을 받기 때문에 본 논문의 시뮬레이션에 의한 성능 평가 항목이 아니다. 따라서 시뮬레이터에서 TLB 미스로 인한 성능 변화를 최소화하기 위해서 TLB 미스를 처리하는데 소요되는 사이클 수는 1사이클로 정했다.

마지막으로 생각해야 될 구조 변수로서는 페치 우선권 정책 및 이슈 우선권 정책에 대한 것이 있다. FETCH_PRI와 ISSUE_PRI는 각기 페치 우선권 정책과 이슈 우선권 정책을 정의하는 것으로서 표 5-3과 같은 변수 값들이 존재한다. 표 5-3이 보여주듯이 대부분의 우선권 정책들은 파이프라인 내에 존재하는 스레드당 명령어 수에 기초를 두고 있다. 이것은 SMT 마이크로프로세서의 자원들을 스레드들 사이에 가능한 균등하게 분배함으로써 전체적인 성능을 높일 수 있기 때문이다.

표 5-3 페치 및 이슈 우선권 정책의 종류

Table 5-3 Types of fetch and issue priority policies

우선권 정책의 종류	우선권 정책	설명
페치 우선권 정책	RR	Select threads using a round-robin priority scheme
	ICOUNT_IFQ	Give highest priority to those threads with the fewest instructions in the instruction fetch queue(s)
	ICOUNT_Q	Give highest priority to those threads with the fewest instructions in the instruction fetch queue(s) and instruction issue queue(s)
	ICOUNT_ALL	Give highest priority to those threads with the fewest instructions in the instruction fetch queue(s), instruction issue queue(s) and functional units
	ICOUNT_BR	Give highest priority to those threads with the fewest unresolved branches
	ICOUNT_MISS	Give highest priority to those threads that have the fewest outstanding data cache misses
	IIQOL	Give lowest priority to those threads with instructions closest to the bottom(s) of the IIQ(s)
이슈 우선권 정책	OLDEST	Give highest priority to those instructions closest to the head(s) of the instruction issue queue(s)
	ICNT_FU	Give highest priority to instructions belonging to those threads with the fewest instructions in functional units
	ICNT_MISS	Give highest priority to instructions belonging to those threads that have the fewest outstanding data cache misses

5-3절 입력 프로그램들과 시뮬레이션 방법

마이크로프로세서 구조의 성능을 평가하기 위해서는 시뮬레이터의 입력으로 사용할 응용 프로그램이 필요하다. 본 논문의 시뮬레이션은 C 언어로 작성된 SPEC CPU2000^{[81][82]} 벤치마크(benchmark) 프로그램들과 ADS(ARM Developer Suite)^{[83][84]}의 예제 프로그램들을 성능 평가를 위해서 사용하였다. 표 5-4는 시뮬레이션을 위해서 선택된 프로그램들을 보여준다.

표 5-4 성능 평가를 위한 입력 프로그램들

Table 5-4 Input programs for performance evaluation

Benchmark suite	Program	Program description
SPEC CPU2000	parser	Syntactic parser of English, based on link grammar
	twolf	TimberWolfSC placement and global routing package
	vortex	Single-user object-oriented database transaction benchmark
	mcf	Program designed for the solution of single-depot vehicle scheduling problems
ADS	sorts	Benchmark performing and comparing insertion, shell and quick sorts
	Dhrystone	Benchmark used to measure the integer processing performance of a system

시뮬레이터의 입력 프로그램들은 ADS의 내장된 크로스 컴파일러(cross compiler)와 링커(linker)^[85-88]에 의해 컴파일과 링크(link)를 함으로써 시뮬레이터의 입력 벡터로서 사용될 수 있는 실행 파일(executable file)로 변환된다. 그런데 이러한 이유로 테스트 벡터로 입력되어 사용될 응용 프로그램들은 몇 가지 제한을 받는다. 예를 들어 응용 프로그램이 모니터에 그래픽을 디스플레이하는 함수를 사용한다면 컴파일러는 그래픽 관련 라이브러리(library)를 사용하여 응용 프로그램을 컴파일해야 한다. 그런데 ADS의 내장 컴파일러는 그래픽 관련 라이브러리를 가지고 있지 않으며 따라서 이러한 응용 프로그램을 컴파일할 수 없다. 컴파일러가 컴파일을 할 수 있다고 하더라도 시뮬레이터가 그래픽 관련 입출력을 지원하기는 힘들다. 또한 ADS는 보통의 I/O(Input/Output) 관련 함수에 대한 지원만을 하기 때문에 특이한 I/O 관련 함수들을 사용하는 응용 프로그램들을 컴파일하지 못한다. 표 5-4의 응용 프로그램들은 이러한 제약에 저촉되지 않는 프로그램들이다. 파일 입출력 및 메모리 할당과 같이 간단한 부분에 대해서는 ADS의 컴파일러가 라이브러리를 가지고 있기 때문에 컴파일에 문제가 없다. 그런데 이러한 부분들은 예외의 일종인 SWI 명령어에 의한 시스템 호출(system call)에 의해서 이루어진다는 데 어려움이 있다. 이러한 종류의 호출을 제대로 지원하기 위해서는 본 논문의 구조에 적합한 시스템 소프트웨어를 만들고 시뮬레이션을 수행할 때에 응용 프로그램과 함께 시뮬레이터로 로드(load)하고, 응용 프로그램이 SWI 시스템 호출을 할 때에 시스템 루틴 부분으로 분기하여서 요청한 작업을 수행하고 다시 응용 프로그램으로 돌아오는 형식을 취해야 한다. 그러나 본 논문에서는 시스템 소프트웨어를 제공하지 않으며, 이것은 본 논문의 연구 주제와도 상관없는 것이다. 따라서 본 논문의 시뮬레이터는 다른 방법으로 SWI 명령어들을 처리한다. 즉, 시뮬레이션 중간에 SWI 명령어를 만나면 시뮬레이터가 SWI 명령어를 해석하여 시스템 소프트웨어에 의해서 이루어져야 할 일을 똑같이 수행하는 방법을 쓴다. 표 5-1의 시스템 호출 처리기가 이러한 동작을 수행해 주는 것이다. 표 5-5는 ADS에서 지원되는 시스템 호출의 종류를 정리한 것이며, 시뮬레이터의 시스템 호출 처리기는 표 5-5에 열거된 시스템 호출에 대해서 ADS와 같은 동작을 수행한다.

표 5-5 SWI 시스템 호출들

Table 5-5 SWI system calls

SWI	Description
SYS_OPEN	Open a file on the host
SYS_CLOSE	Close a file on the host
SYS_WRITEC	Write a character to the debug channel
SYS_WRITE0	Write a string to the debug channel
SYS_WRITE	Write to a file on the host
SYS_READ	Read the contents of a file into a buffer
SYS_READC	Read a byte from the debug channel
SYS_ISTTY	Check whether a file is connected to an interactive device
SYS_SEEK	Seek to a position in a file
SYS_FLEN	Return the length of a file
SYS_REMOVE	Remove a file from the host
SYS_RENAME	Rename a file on the host
SYS_CLOCK	Number of centiseconds since support code started
SYS_TIME	Number of seconds since January 1, 1970
SYS_ERRNO	Get the value of the C library “errno” variable
SYS_GET_CMDLINE	Get the command-line used to call the executable
SYS_HEAPINFO	Get the system heap parameters
SYS_TICKFREQ	Define a tick frequency

시뮬레이터는 SWI에 의한 시스템 호출을 상기 설명한 것과 같이 수행하지만 그 외의 예외에 대해서는 시스템 프로그램 루틴을 가지고 있지 않고 있다. 따라서 시뮬레이터는 SWI와 다른 종류의 예외가 발생할 경우에는 하드웨어에서 이루어져야 할 동작들, 즉 명령어 플러쉬, 몇 개의 상태 변화 및 예외 처리 루틴의 주소로 PC를 설정하는 동작까지는 수행할 수 있으나, 예외 처리 루틴이 존재하지 않기 때문에 예외 처리는 이루어지지 않는다. 따라서 시뮬레이터는 예외가 발생할 경우에 하드웨어에 의한 동작까지는 처리한 후에 예외가 발생했다는 것을 보고한 후에 시뮬레이션을 종료한다. 실제로 시뮬레이션 중에는 SWI를 제외한 예외는 발생할 수가 없다. 만약에 SWI 이외의 예외가 발생했다면 그것은 시뮬레이터가 잘못 동작하고 있던지 아니면 입력 프로그램에 문제가 있는 것이다. 따라서 SWI를 제외한 예외 발생 때의 시뮬레이션 중단은 시뮬레이터의 검증에 유용하게 사용되며 가장 적당한 처리이다.

SMT 구조의 시뮬레이터는 개개의 독립적인 프로그램들을 독립적인 스레드들에 할당하여 처리한다. 즉, 시뮬레이터는 병렬 처리(parallel processing)가 아닌 다중프로그래밍(multiprogramming) 환경에서 입력 프로그램들을 실행한다. 따라서 본 논문의 시뮬레이션은 동기화 지연(synchronization delay)^[89] 및 비효율적인 병렬화(inefficient parallelization) 요소에 의해서 영향을 받지 않는다.

시뮬레이션 과정에서 입력 프로그램들을 스레드들에 어떻게 할당하는지는 시뮬레이션 결과에 많은 영향을 미친다. 4개의 스레드를 지원하는 SMT 구조를 예로 들어 그 이유를 설명하면 다음과 같다. 4개의 스레드를 지원하는 SMT 구조에서 4개의 스레드 각각에 같은 프로그램을 할당한다면 시뮬레이션 진행 중에 4개의 스레드는 계속해서 입력 프로그램의 비슷한 주소 영역을 실행하게 된다. 같은 가상주소에 대해서는 실제주소의 하위 부분이 같으므로 가상주소가 같은 경우에는 캐쉬에서 같은 세트를 사용하는 경향이 있다. 따라서 비슷한 시기에 같은 주소 영역을 다수의 스레드들이 실행한다면 단일 캐쉬 세트에 대한 요구가 집중적으로 발생하며 이로 인해 캐쉬 미스율이 비정상적으로 커지게 된다. 이러한 인위적인 스레드간의 충돌 미스(inter-thread conflict miss)^[35]를 없애기 위해서는 4개의 스레드에 서로 다른 입력 프로그램을 할당하는 것이 좋다. 그런데 서로 다른 4개의

입력 프로그램으로서 6개의 프로그램 중에 어떤 것들을 선택하는가에 따라서 성능 결과가 다르게 나온다. 6개의 프로그램 중에서 다양한 조합으로 4개의 프로그램을 선택하고 시뮬레이션을 각 조합에 대해서 수행한 후에 그 결과를 평균하여 성능을 평가할 수는 있다. 그러나 이러한 방법은 시뮬레이션에 대한 부담을 너무 크게 한다. 따라서 본 논문에서는 단일 시뮬레이션에서 그림 5-4와 같이 각각의 스레드가 6개의 프로그램들을 모두 연속적으로 수행하도록 하였다. 시뮬레이터는 시뮬레이션 시작 시에 6개의 프로그램을 모두 로드한 후에 단일 스레드는 6개 프로그램 중에 1개를 우선 수행하고, 수행이 끝나면 곧바로 다른 프로그램을 선택해서 수행하는 방식으로 6개의 프로그램을 연속적으로 수행한다. 이러한 연속적인 프로그램 수행은 마이크로프로세서 상의 모든 스레드들에서 이루어진다. 단 캐쉬에서의 비정상적인 스레드간의 충돌 미스를 없애기 위해서 각 스레드는 6개의 프로그램들을 서로 다른 순서로 수행한다.

시뮬레이터가 응용 프로그램을 입력으로 받아서 시뮬레이션을 수행하는 과정을 요약하면 그림 5-5와 같다. 시뮬레이션이 시작되면 초기화 루틴(initializing routine)은 마이크로프로세서의 구성 요소들을 만들고 그 구성 요소들의 상태를 시스템에 전원이 처음 켜졌을 때의 상태로 초기화시킨다. 이러한 동작 후에 응용 프로그램이 시뮬레이터에서 동작하기 위해서는 하드웨어 상태를 시스템에 맞게 초기화하고 시스템 루틴을 로드 및 재배치하는 작업과 시스템 루틴에 의한 응용 프로그램의 로드와 같은 작업이 이루어져야 한다. 그런데 이러한 동작은 부트 로더(boot loader)와 시스템 커널(system kernel)^[90]이라는 시스템 소프트웨어가 맡아 처리한다. 그런데 시뮬레이션에서는 시스템 소프트웨어를 사용하지 않기 때문에 프로그램 로더(program loader)가 이러한 동작을 대신하여 프로그램을 로드하고 하드웨어 상태를 적절하게 초기화시켜 준다. ADS의 내장 컴파일로는 소스 프로그램을 ELF(Executable and Linking Format)^[91] 파일 포맷(format)의 실행 파일로 컴파일하기 때문에 시뮬레이터의 프로그램 로더는 ELF 포맷에 맞춰서 입력 프로그램을 로드한다. 프로그램이 로드된 후에는 시뮬레이터 본체에서의 시뮬레이션이 수행된다. 시뮬레이터 본체에서 그림 5-3의 루프가 매 사이클 반복적으로 수행되는 것과 동시에 트레이스 발생기(trace generator)는 매 사이클 마이크로프로세서

의 상태를 검사하여 여러 트레이스 파일(trace file)들로 출력한다. 부록 3은 트레이스 파일의 예를 보여주는 것이다. 시뮬레이터 본체에서 모든 프로그램에 대한 수행이 끝나면 결과 요약기(result summarizer)가 시뮬레이션 사이클 수, 수행 명령어 수, IPC, 캐쉬 미스율과 같은 데이터들을 결과 파일(result file)로 출력하고 시뮬레이션을 끝낸다. 결과 파일의 예가 부록 4에 나와 있다.

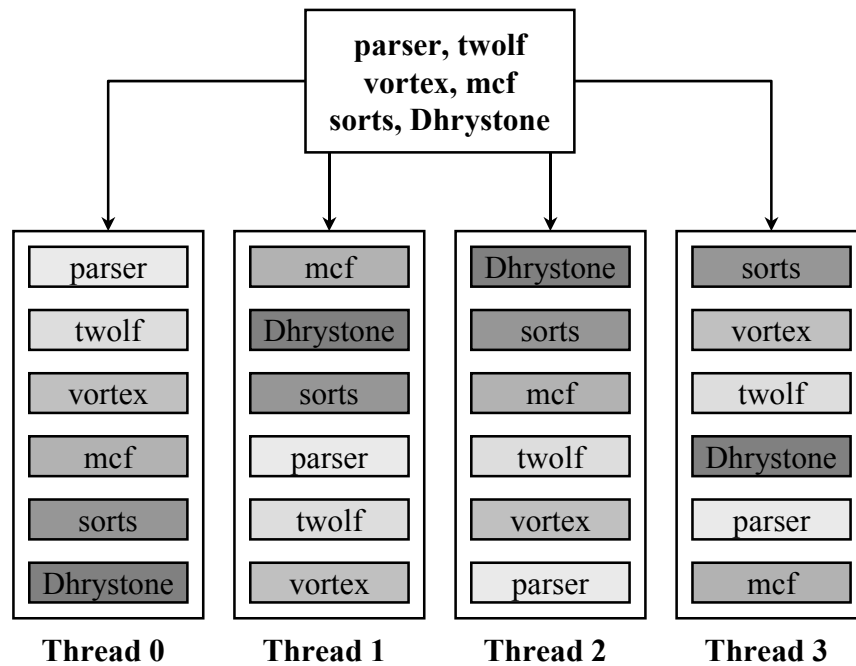


그림 5-4 입력 프로그램들을 각각의 스레드에 할당하는 모습

Fig. 5-4 allocating input programs to each thread

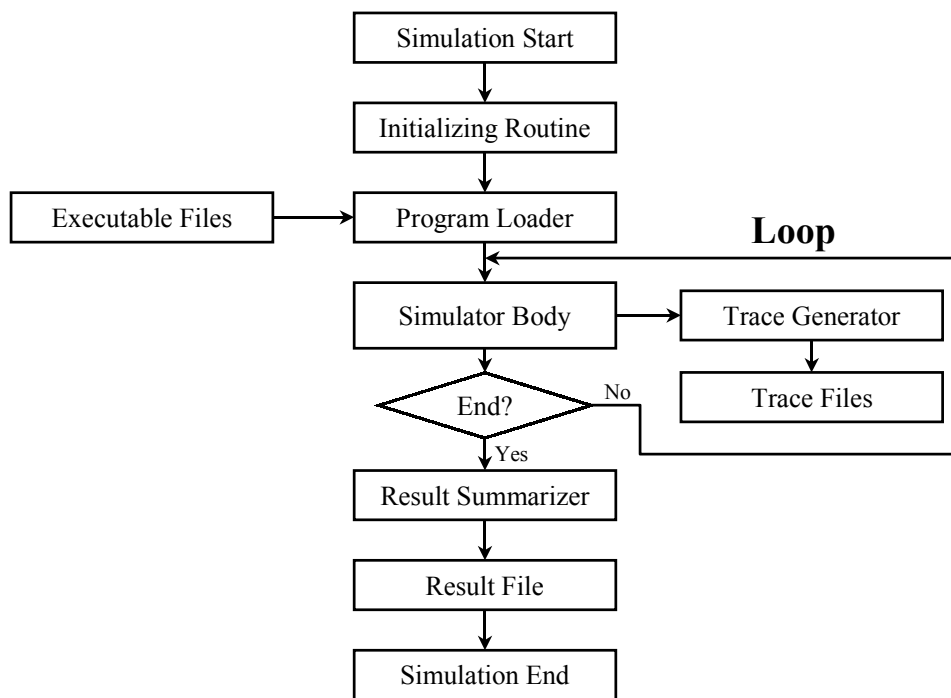


그림 5-5 시뮬레이션 과정
Fig. 5-5 Simulation flow

제 6 장 성능 평가 및 결과

6-1절 성능 평가 방법

C 언어로 작성된 시뮬레이터는 실제 칩과 달리 클럭 주파수가 정해져 있지 않으며, 메모리를 엑세스하는데 걸리는 시간도 가변적이다. 그리고 시뮬레이터에는 프로세서의 코어 부분만이 모델링되어 있으며, 주변 장치(peripheral) 또는 I/O에 대해서는 시뮬레이션을 위한 최소한의 보조 루틴만이 존재한다. 따라서 시뮬레이터를 이용한 시뮬레이션으로는 실제 프로세서의 성능을 알 수 없다. 시뮬레이터는 실제 프로세서 칩에 대한 설계가 아니라, 프로세서의 구조만이 모델링된 것이라는 사실을 고려하면 이것은 당연한 것이다. 이러한 이유로 C 언어로 작성된 시뮬레이터를 사용한 시뮬레이션에서는 대부분의 경우에 성능의 척도로서 IPC 또는 CPI(Clocks Per Instruction)를 사용하고, 본 논문에서도 성능의 척도로서 IPC를 사용한다. IPC와 관련해서는 페치율(fetch rate), 이슈율 및 실행률(execution rate) 세 가지가 존재한다. 페치율은 평균적으로 사이클당 페치하는 명령어의 수를 의미하며, 세 가지 중에서 가장 큰 값을 가진다. 이슈율은 평균적으로 사이클당 이슈하는 명령어 수로서, 페치된 후에 이슈 전에 플러쉬된 명령어는 이슈율의 계산에서 제외되기 때문에 이슈율은 페치율보다 작은 값을 가진다. 실행률은 플러쉬 없이 온전하게 수행이 완료되는 명령어 수가 사이클당 평균 몇 개인지를 측정한 값으로서, 세 가지 중에서 가장 작은 값을 가지고 프로세서 구조의 성능을 결정한다. 본 논문은 다양한 구조 변수들을 변화시켜가면서 다양한 구성에 대한 성능을 측정한 후에 비용과 성능을 고려한 최적의 구조를 제안한다.

다음 절에서 제시되듯이 슈퍼스칼라 구조의 IPC가 SMT에 비해서 현저히 낮게 나온다. 따라서 작성된 시뮬레이터가 슈퍼스칼라 구조에 불리하게 구성된 것이라고 보일 수 있다. 본 논문에서는 이러한 가능성을 제거하기 위해서 같은 명령어 세트 구조를 가지는 실제 프로세서의 IPC와 시뮬레이터가 보이는 IPC를 비교하였

다. 시뮬레이터는 구조 변수의 조정을 통해서 슈퍼스칼라 및 SMT로서 구성될 수 있지만 단순한 스칼라(scalar) 구조로도 변경이 가능하다. 시뮬레이션 결과 스칼라 구조로 구성되고 입력 프로그램으로서 Dhrystone을 사용하였을 경우에 0.71의 IPC를 보였다. 그리고, 같은 명령어 세트 구조의 실제 스칼라 프로세서는 Dhrystone에 대해서 0.67의 IPC를 보인다^[92]. 0.71과 0.67을 비교해 보았을 때, 시뮬레이터는 스칼라 구조로서 불리하게 작성되지 않았음을 알 수 있다. 시뮬레이터에서 스칼라와 슈퍼스칼라의 구조는 단지 페치 및 이슈 폭과 기능 유닛과 같은 자원의 수에 있어서만 차이를 보이지 전체적인 구성은 같다. 따라서 시뮬레이터는 슈퍼스칼라 구조에 불리하게 작성된 것이 아님을 알 수 있다.

가장 적절한 SMT 구조를 제안하기 위해서는 SMT의 구성을 다양하게 변화시켜 가면서 성능을 측정하고, 하드웨어 비용을 성능과 함께 고려하면서 최적의 구조에 접근해 가는 과정이 필요하다. 다음 절이 이러한 과정을 자세히 다루고 있다.

6-2절 다양한 SMT 구성에 대한 성능 측정

본 논문의 첫 번째 성능 평가 작업으로서, 2-issue(명령어 이슈 폭이 2개의 명령어인), 4-issue, 8-issue 및 16-issue 구조들에 대해서 지원되는 스레드의 수를 변화시켜 가면서 성능을 측정하였으며, 표 6-1은 이러한 성능 평가에 사용된 구조 변수들의 값을 정리한 것이다. 표 6-1에는 지원되는 스레드의 수 (THREAD_NUM)에 대한 값이 없는 것은 스레드의 수를 변화시키면서 성능을 평가하기 때문이다. 이 시뮬레이션에서는 그룹화에 의한 효과는 측정하지 않으므로 표 6-1의 그룹화 관련 변수들은 모두 값이 1이다.

그 외의 변수들에 대한 설명은 다음과 같다. 우선 변수 ISSUE_WIDTH는 이슈 폭에 의해서 결정된다. 페치 폭과 디코드 폭은 이슈 폭과 같도록 설정하였는데, 대부분의 경우에 적절한 설정이다. 명령어 페치 큐와 명령어 이슈 큐의 엔트리 수는 2-issue 및 4-issue 구성에 대해서는 32개, 8-issue 및 16-issue 구성에 대해서는 64개로 설정하였는데, 이 값들은 큐 엔트리의 부족으로 인해서 성능에 제한을 받지 않을 만큼 충분한 엔트리들을 제공한다. 레지스터 파일의 읽기 포트 수는 이슈 폭의 2배이고 쓰기 포트 수는 이슈 폭의 값과 같다. ALU의 수는 이슈 폭과 같고 로드/스토어 유닛의 수는 이슈 폭의 반으로 설정하였다. 곱셈 유닛의 수는 항상 1로 고정되었는데, 이것은 본 시뮬레이션의 입력 프로그램들이 곱셈 유닛을 사용하는 빈도가 매우 낮기 때문이다. 명령어 캐쉬의 읽기 포트 수는 페치 폭 및 스레드 수와 관련해서 매우 중요한 변수이다. 예를 들어서 스레드 수가 1개인 경우 (보통의 슈퍼스칼라 구조인 경우)에는 명령어 캐쉬의 포트 수가 1개이면 되고 그 이상은 필요가 없다. 전체적으로 스레드의 수가 늘어나고 페치 폭이 커질수록 명령어 캐쉬의 읽기 포트 수를 늘리는 것이 적당하기 때문에, 명령어 캐쉬의 포트 수는 표 6-1에서 보여 주듯이 다양한 값을 갖는다. 그 외의 변수들은 과하지도 않고 부족하지도 않게 설정되었다. 그런데, 표 6-1은 최적의 구성을 보여주는 것이 아니다. 즉 표 6-1의 값들은 스레드 수 변화에 의한 성능 변화의 대략적인 모습을 보기 위한 것이고, 최적의 구성은 이후의 많은 시뮬레이션을 거쳐서 결정된다.

표 6-1 성능 평가를 위한 구조 변수들의 값

Table 6-1 Values of configurations parameters for performance evaluation

구조 변수	변수 값
GRP1_NUM	1
GRP2_NUM	1
GRP3_NUM	1
FETCH_WIDTH	issue width
DECODE_WIDTH	issue width
ISSUE_WIDTH	issue width
IFQ_SIZE	32(for 2-/4-issue), 64(8-/16-issue)
IIQ_SIZE	32(for 2-/4-issue), 64(8-/16-issue)
REGR_NUM	(issue width)×2
REGW_NUM	issue width
ALUF_NUM	issue width
MULF_NUM	1
LSUF_NUM	(issue width)÷2
ICP_NUM	2-issue: 1 4-issue: 1(for 1-/2-thread), 2(for 4-/8-/16-thread) 8-issue: 1(for 1-/2-thread), 2(for 4-thread), 4(for 8-/16-thread) 16-issue: 1(for 1-/2-thread), 2(for 4-thread), 4(for 8-/16-thread)
IC_SIZE	64 Kbytes
ICWAY_NUM	4
ICBLK_SIZE	32 bytes
DC_SIZE	64 Kbytes
DCWAY_NUM	4
DCBLK_SIZE	32 bytes
BTB_SIZE	128
ITLB_SIZE	128
DTLB_SIZE	128
CACHEREP	RANDOM
BTBREP	RANDOM
TLBREP	RANDOM
CFILL_CYC	5
FETCH_PRI	RR
ISSUE_PRI	OLDEST

그림 6-1부터 6-4까지의 그림들은 각기 2-issue, 4-issue, 8-issue 및 16-issue 구성에 대한 결과이다. 4개의 그림들에서 지원되는 스레드 수가 1인 경우는 SMT가 아닌 수퍼스칼라 구조에 대한 것이다. 4개의 그림에서 공통적으로 볼 수 있는 것은 지원되는 스레드의 수가 이슈 폭과 같을 때까지, 스레드의 수가 증가하면서 전반적으로 폐치율, 이슈율 및 실행률이 빠르게 증가하는 것을 볼 수 있다. 채택된 명령어 세트 구조는 다수의 다중 사이클 명령어들을 제공하고, CPSR과 같은 특수 레지스터들이 존재하고 일반 명령어들이 이러한 특수 레지스터들에 의해서 영향을 받기 때문에 높은 ILP를 구현하기 어려운 구조이다. 그리고 본 논문의 수퍼스칼라 구조는 순차적 이슈 및 완료 정책을 사용하고 있으며, 본 시뮬레이션에서 사용한 입력 프로그램은 스칼라(1-issue) 구조의 프로세서를 위한 것이다. 이러한 이유로 이슈 폭이 큰 수퍼스칼라 구조에서도 IPC가 1을 넘기가 힘들며, 16-issue 수퍼스칼라 구조에서도 IPC가 1.0019에 지나지 않는다. 그러나 스레드 수가 2 이상인 SMT 구조인 경우에는 ILP의 부족을 TLP로 보충하는 현상을 보이게 된다. 각 스레드의 ILP가 1 정도라고 보면 스레드 수가 이슈 폭의 크기와 같아질 때에 SMT 구조는 이슈 폭을 충분히 사용할 수 있게 된다. 그러나 스레드 수가 이슈 폭보다 커지면, ILP와 TLP에 의한 병렬성이 하드웨어가 지원할 수 있는 병렬성보다 커지게 되고, 더 이상 성능 향상을 보이지 않게 되는 것이다. 그림 6-1부터 6-4까지의 그림들을 보면, 스레드 수가 이슈 폭과 같거나 작은 범위에서 스레드 수를 두 배로 늘릴 때 대체로 성능은 거의 두 배로 늘어나고 있다. 그러나, 하드웨어 비용의 증가는 비교적 작은 편이다. 그리고 SMT 구조에서 이슈 폭과 같은 수의 스레드를 지원하는 구성이 성능과 비용을 고려했을 때 가장 적절하다는 것을 알 수 있다. 그런데, 스레드 수가 16개인 경우가 스레드 수가 8개인 경우보다 성능이 낮게 나오는 경향이 시뮬레이션 결과로 나왔다. 원인을 찾기 위해서 4-issue 및 8-issue 구조에서 스레드 수에 따른 캐시 미스율을 측정하였으며, 그림 6-5와 6-6은 이에 대한 결과이다. 두 그림에서 알 수 있듯이, 스레드 수가 늘어나면서 캐시 미스율이 계속 증가하며, 특히 스레드 수가 16개일 때에는 캐시 미스율이 과도하게 크다. 따라서, 이러한 캐시 미스율의 급격한 증가가 16-thread 구조가 낮은 성능을 보인 원인 중의 하나임이 분명하다.

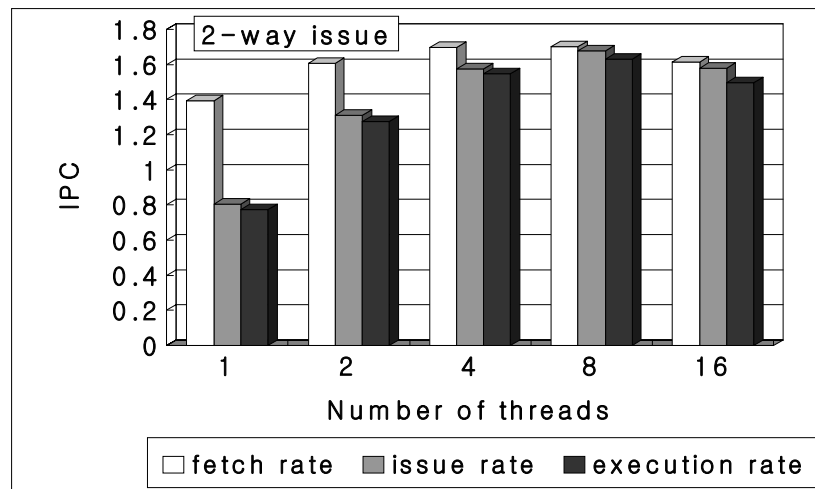


그림 6-1 스레드 수에 따른 2-issue 구조의 성능 변화

Fig. 6-1 Performance of 2-issue architectures as a function of the number of threads

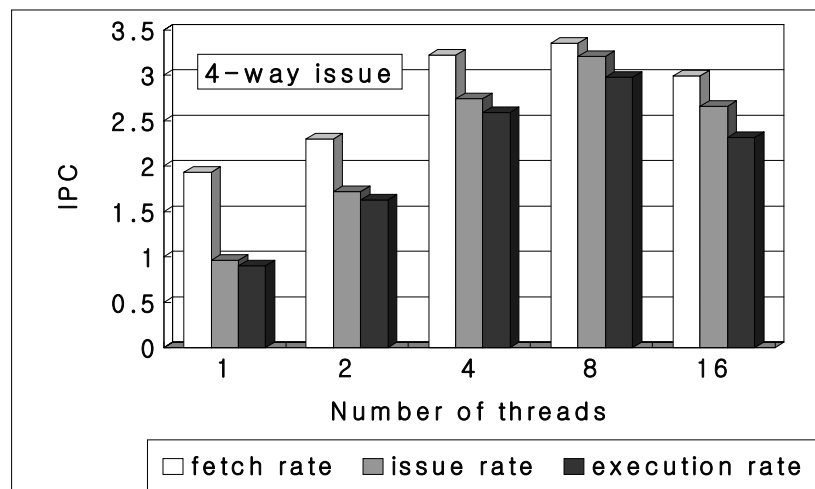


그림 6-2 스레드 수에 따른 4-issue 구조의 성능 변화

Fig. 6-2 Performance of 4-issue architectures as a function of the number of threads

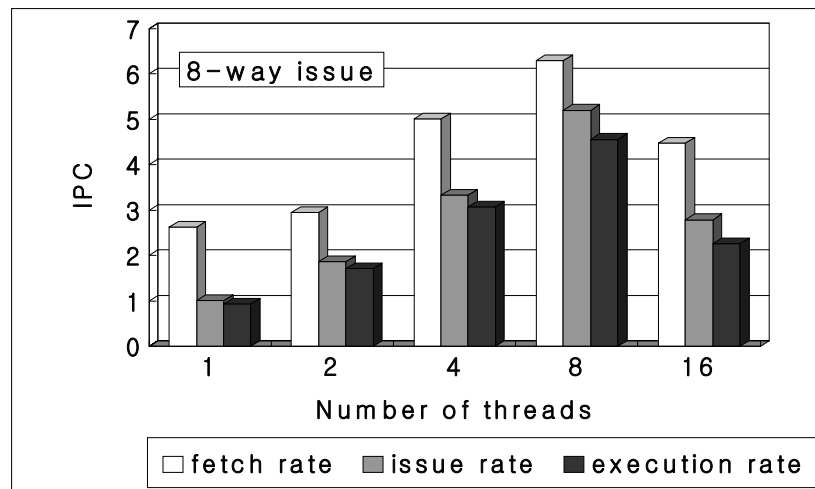


그림 6-3 스레드 수에 따른 8-issue 구조의 성능 변화

Fig. 6-3 Performance of 8-issue architectures as a function of the number of threads

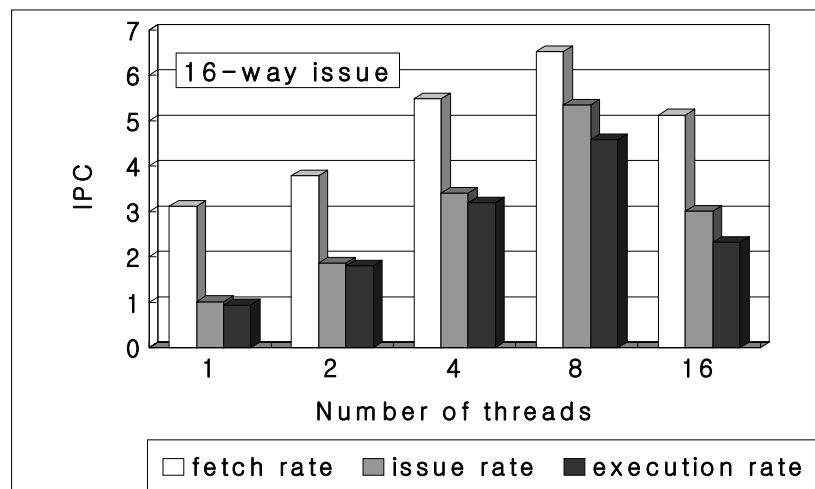


그림 6-4 스레드 수에 따른 16-issue 구조의 성능 변화

Fig. 6-4 Performance of 16-issue architectures as a function of the number of threads

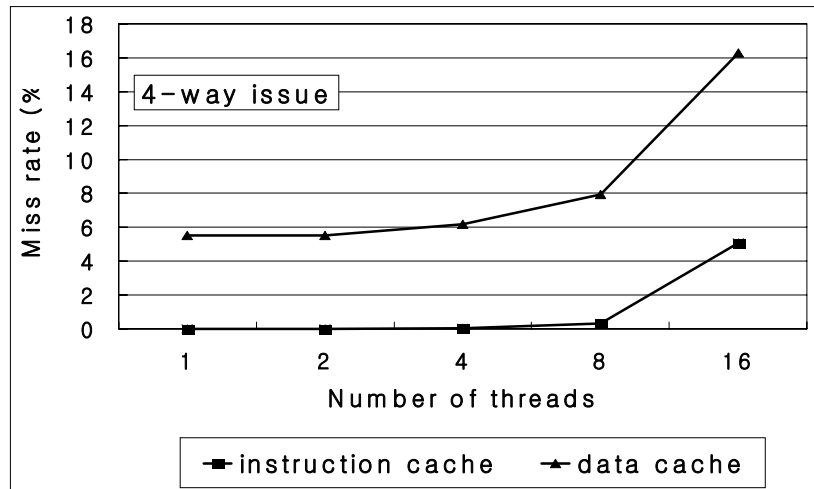


그림 6-5 스레드 수에 따른 4-issue 구조의 캐쉬 미스율 변화

Fig. 6-5 Miss rates of instruction and data caches versus the number of threads for 4-issue architectures

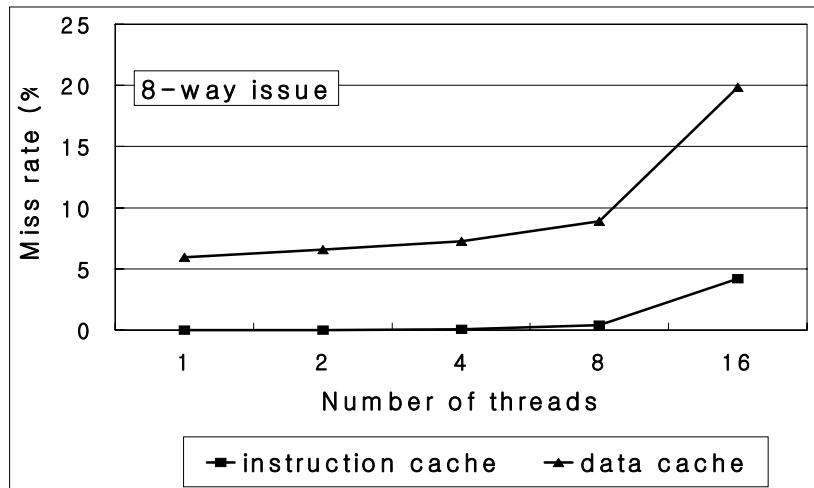


그림 6-6 스레드 수에 따른 8-issue 구조의 캐쉬 미스율 변화

Fig. 6-6 Miss rates of instruction and data caches versus the number of threads for 8-issue architectures

스레드 수가 늘어나면서 캐쉬 미스율이 증가하는 것은 스레드간의 충돌 미스 때문이다. 즉 스레드 수가 늘어나면서, 그 늘어난 수만큼의 주소 공간(address space)이 프로세서 안에서 액세스된다. 이것은 같은 캐쉬 세트에 대한 요구 증가로 이어지고, 캐쉬의 way 수가 한정적일 경우 충돌 미스가 계속 발생하게 된다. 상기 시뮬레이션에서는 캐쉬의 way 수가 4로 고정되었으므로, 16개의 스레드가 같은 세트를 비슷한 시간대에 사용한다면, 해당 세트에서는 계속해서 미스가 발생할 것이다. 이러한 종류의 미스인 스레드간의 충돌 미스는 스레드 수가 늘어날수록 캐쉬의 way 수가 작을수록 증가한다. 상기 시뮬레이션에서 16-thread 구조에서 성능이 낮게 나온 것은 스레드간의 충돌 미스가 16-thread 구조에서 과도하게 많이 발생했기 때문이다. 이러한 스레드간의 충돌 미스로 인한 성능 저하 문제 때문에, SMT 구조에서는 지원되는 스레드 수에 가장 적절한 캐쉬 way 수를 제공해야 한다. 이를 위해서 4-thread, 8-thread 및 16-thread 구조에 대해서 가장 적절한 캐쉬 way 수를 결정하기 위한 시뮬레이션을 수행하였다. 그림 6-7부터 6-18까지는 표 6-1의 구성에서 캐쉬의 way 수를 변화시키면서 수행한 시뮬레이션의 결과를 보여준다. 결과에서 알 수 있듯이, 캐쉬의 way 수가 증가함에 따라서 캐쉬 미스율이 줄어들고 성능이 향상된다. 그림 6-15와 그림 6-17은 캐쉬의 way 수가 16개 일 경우에는 스레드 수가 16개인 구조가 스레드 수가 8개인 구조보다 더 좋은 성능을 가진다는 것을 보여준다. 이것은 캐쉬의 way 수가 4개인 경우에 대한 그림 6-3의 결과와는 반대이다. 그 이유는 그림 6-6과 6-18이 보여주듯이 캐쉬의 way 수가 16개인 16-thread 구조에서의 스레드간의 충돌 미스가 캐쉬 way 수가 4개인 구조와 비교해서 뚜렷이 줄어들었기 때문이다. 전체적으로 캐쉬의 way 수가 스레드 수와 같거나 또는 스레드 수의 반이 된 이후로는 캐쉬의 way 수가 늘어나도 성능 향상이 별로 이루어지지 않는다. 따라서 스레드 1개당 캐쉬의 way 1개 또는 0.5개의 구성이 스레드간의 충돌 미스로 인한 성능 저하를 최소화하는 적절한 구성이다. 캐쉬의 way 수가 그 이상으로 클 경우에는 하드웨어 비용만 증가하고 성능 향상에는 도움을 주지 않기 때문에, 그 이상의 캐쉬 way 수는 권장되지 않는다. 이하의 시뮬레이션들에서는 캐쉬의 way 수가 스레드 수와 같도록 설정하였으며, 그 외에는 표 6-1의 구성과 같다.

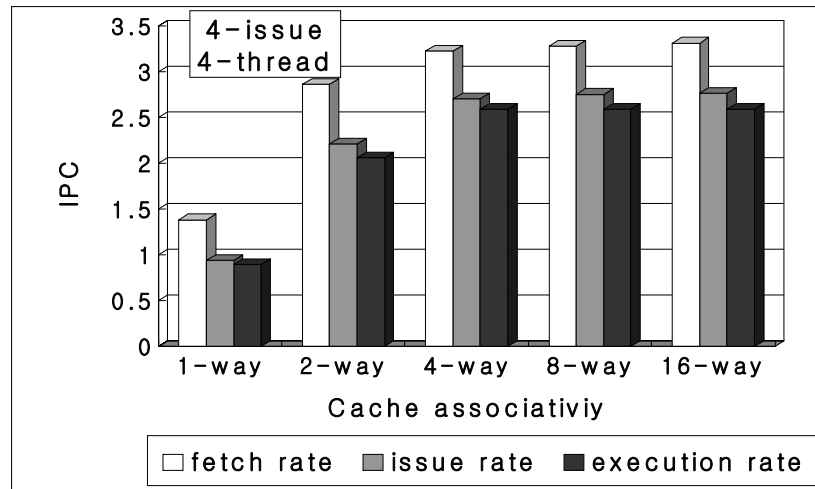


그림 6-7 캐쉬 way 수에 따른 4-issue, 4-thread 구조의 성능 변화

Fig. 6-7 Performance of 4-issue, 4-thread architectures as a function of associativity of instruction and data caches

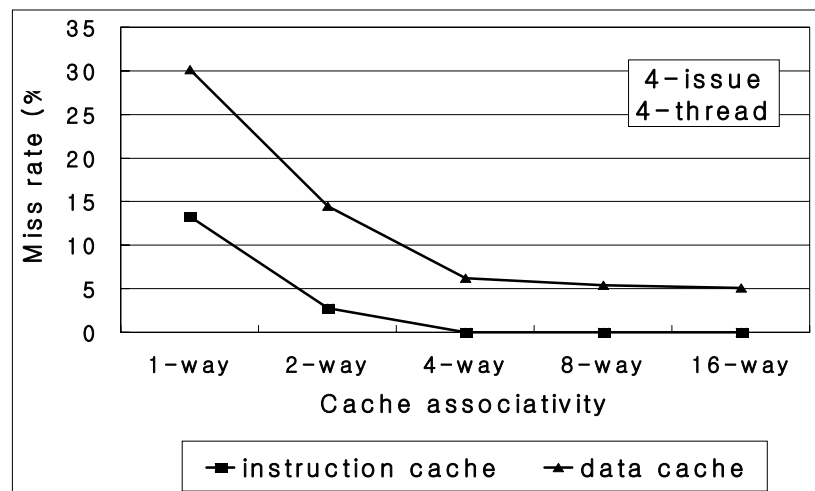


그림 6-8 캐쉬 way 수에 따른 4-issue, 4-thread 구조의 캐쉬 미스율

Fig. 6-8 Miss rates versus associativity of instruction and data caches for 4-issue, 4-thread architectures

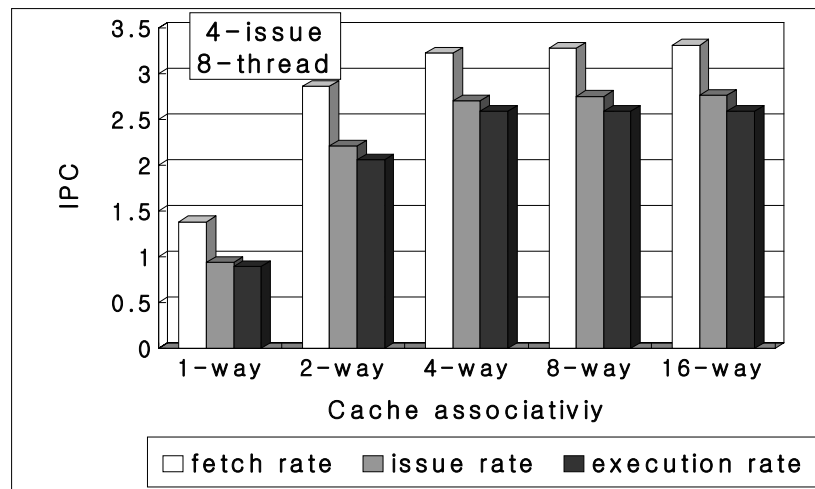


그림 6-9 캐쉬 way 수에 따른 4-issue, 8-thread 구조의 성능 변화

Fig. 6-9 Performance of 4-issue, 8-thread architectures as a function of associativity of instruction and data caches

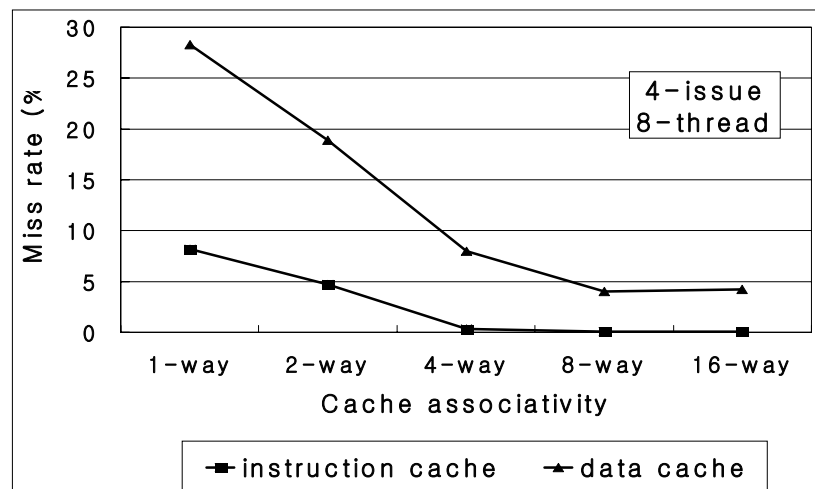


그림 6-10 캐쉬 way 수에 따른 4-issue, 8-thread 구조의 캐쉬 미스율

Fig. 6-10 Miss rates versus associativity of instruction and data caches for 4-issue, 8-thread architectures

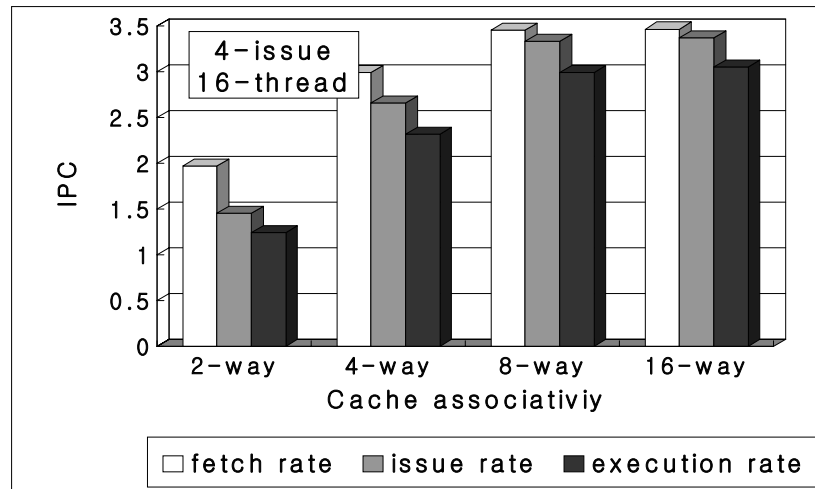


그림 6-11 캐쉬 way 수에 따른 4-issue, 16-thread 구조의 성능 변화

Fig. 6-11 Performance of 4-issue, 16-thread architectures as a function of associativity of instruction and data caches

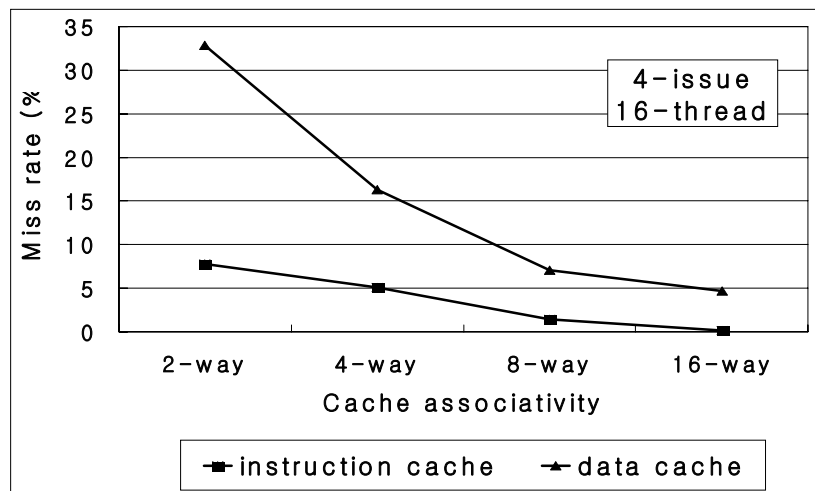


그림 6-12 캐쉬 way 수에 따른 4-issue, 16-thread 구조의 캐쉬 미스율

Fig. 6-12 Miss rates versus associativity of instruction and data caches for 4-issue, 16-thread architectures

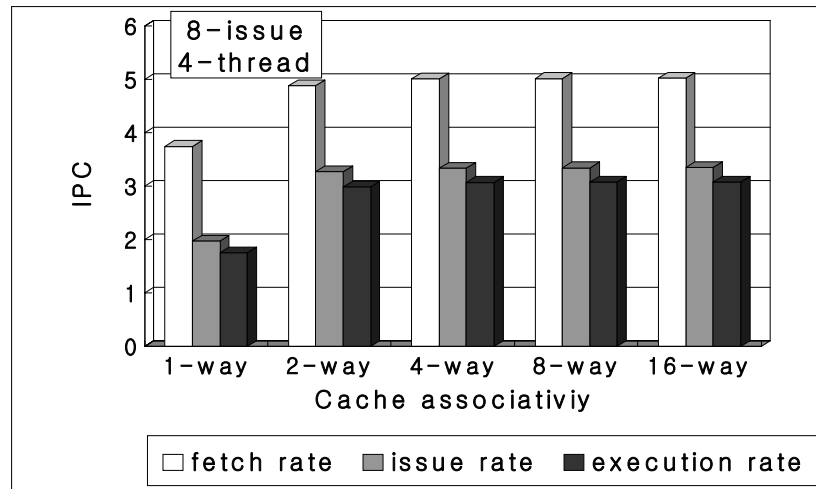


그림 6-13 캐쉬 way 수에 따른 8-issue, 4-thread 구조의 성능 변화
 Fig. 6-13 Performance of 8-issue, 4-thread architectures as a function of associativity of instruction and data caches

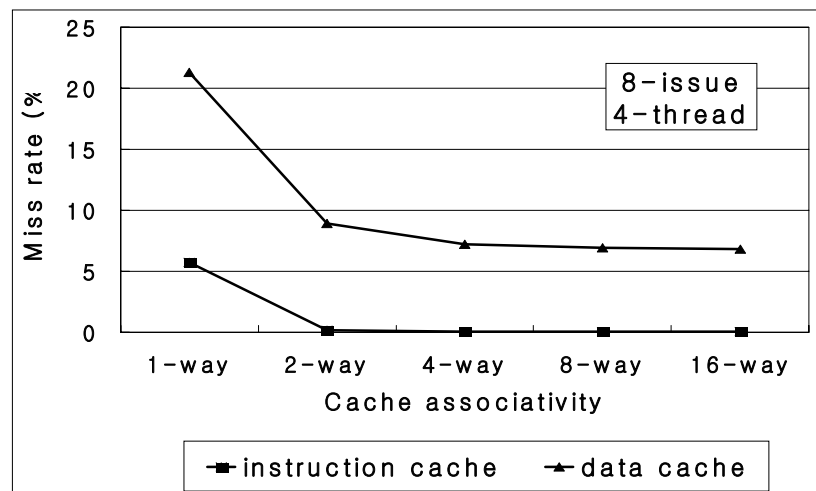


그림 6-14 캐쉬 way 수에 따른 8-issue, 4-thread 구조의 캐쉬 미스율
 Fig. 6-14 Miss rates versus associativity of instruction and data caches for 8-issue, 4-thread architectures

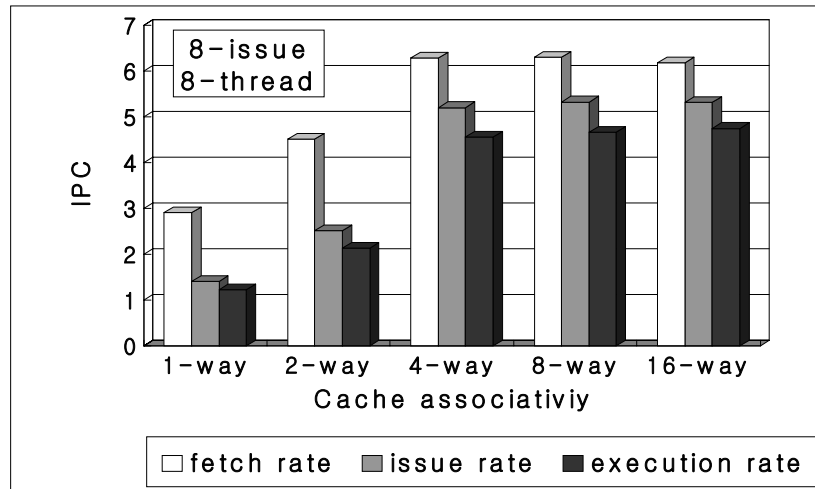


그림 6-15 캐쉬 way 수에 따른 8-issue, 8-thread 구조의 성능 변화
 Fig. 6-15 Performance of 8-issue, 8-thread architectures as a function of associativity of instruction and data caches

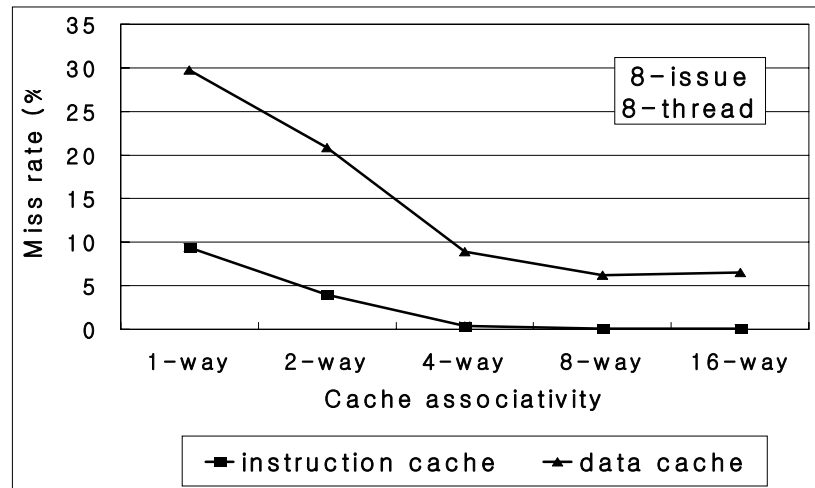


그림 6-16 캐쉬 way 수에 따른 8-issue, 8-thread 구조의 캐쉬 미스율
 Fig. 6-16 Miss rates versus associativity of instruction and data caches for 8-issue, 8-thread architectures

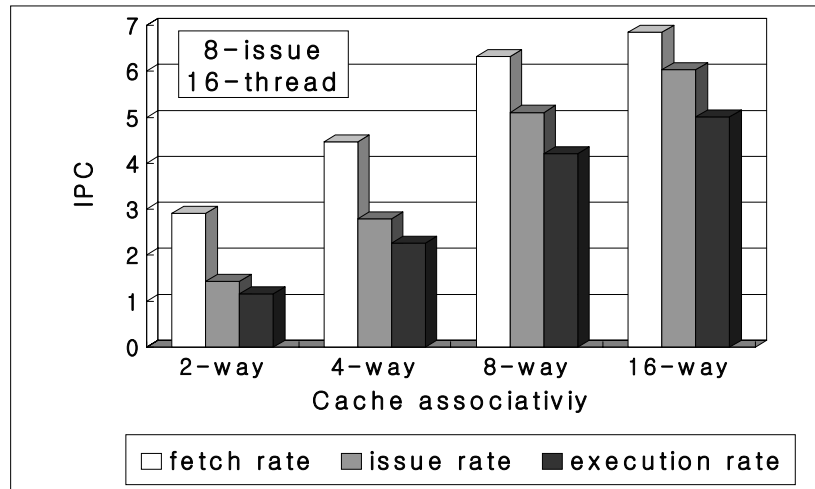


그림 6-17 캐쉬 way 수에 따른 8-issue, 16-thread 구조의 성능 변화

Fig. 6-17 Performance of 8-issue, 16-thread architectures as a function of associativity of instruction and data caches

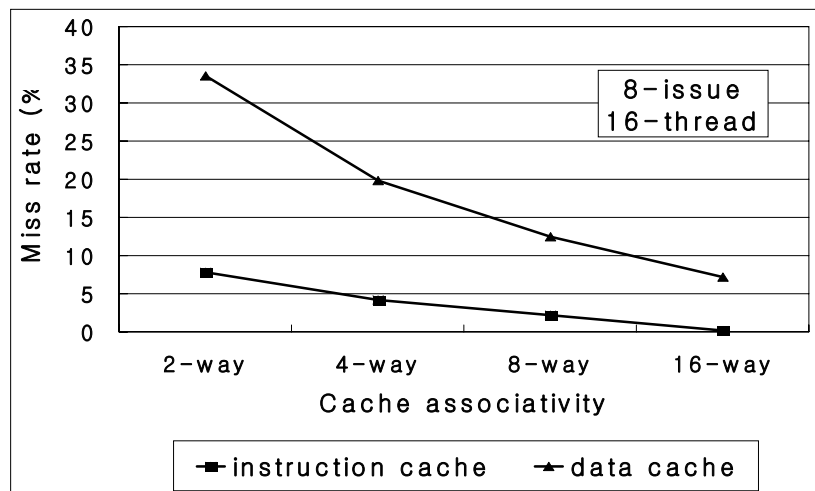


그림 6-18 캐쉬 way 수에 따른 8-issue, 16-thread 구조의 캐쉬 미스율

Fig. 6-18 Miss rates versus associativity of instruction and data caches for 8-issue, 16-thread architectures

캐쉬의 way 수가 성능에 큰 영향을 미친다는 것은 상기의 설명에서 알 수 있었다. 그렇다면 캐쉬의 크기도 성능에 영향을 미치는지를 알아볼 필요가 있다. 특히 스레드 수와 관련해서 적절한 캐쉬 크기를 정하는 작업이 이루어져야 한다. 따라서 표 6-1의 구성에서 캐쉬의 way 수는 스레드 수와 같도록 하고 캐쉬의 크기를 변화시키면서 성능 평가를 수행하였으며, 그림 6-19부터 6-22까지는 그 결과를 보여준다. 그림 6-20과 6-22를 비교하면 8 또는 16 KB의 경우를 제외하고는 같은 캐쉬 크기에 대해서 캐쉬 미스율이 비슷하게 나온다. 이슈 폭과 캐쉬의 미스율은 이론적으로 별 관계가 없기 때문에 8 또는 16 KB에서의 미스율 차이는 스레드 수의 차이에 의한 것이다. 이것은 스레드의 수가 적절한 캐쉬의 크기에 영향을 미치며, 일정한 캐쉬 크기에 대해서 스레드 수가 변화함에 따라서 캐쉬 미스율이 증가한다는 것을 의미한다. 그러나, 그 영향이나 증가 속도는 캐쉬의 way에 대한 것과 비교했을 때 작은 편이다. 그림 6-19와 6-21은 4-issue 4-thread 구조에는 16 KB, 8-issue, 8-thread 구조에서는 32 KB의 크기를 갖는 캐쉬로도 충분하다는 결과를 보여 주는데, 일반적인 슈퍼스칼라 구조에서도 더 큰 캐쉬들을 사용한다는 점을 고려했을 때, 스레드 수의 증가가 필요한 캐쉬 크기에 미치는 영향을 무시할 수 없지만 크지는 않다고 할 수 있다.

BTB의 엔트리 수가 분기 예측 오류 빈도(frequency of branch mispredictions) 및 성능에 미치는 영향도 고려할 사항이다. 이를 측정하기 위해서 표 6-1의 구성에서 스레드 수를 변화시키면서 분기 예측 오류 빈도를 검사하였으며, 그림 6-23과 6-24는 그 결과를 보여준다. 그림 6-24에서 보여주듯이 스레드 수가 늘어날수록 BTB 엔트리가 새로운 엔트리로 대체되는 빈도(frequency of BTB replacements)가 늘어나면서 분기 예측 오류 빈도 또한 증가하는데, 이는 스레드 수가 증가할수록 더 많은 BTB 엔트리가 필요하다는 것을 의미한다. 스레드 수가 늘어날수록 프로그램의 주소 영역은 그만큼 늘어나고 비슷한 시간대에 더욱 다양한 분기들이 프로세서에서 수행된다. 이런 이유로 스레드 수가 늘어날수록 더 많은 양의 분기 예측 정보들을 저장하고 있어야 하기 때문에 필요한 BTB 엔트리 수가 증가하는 것이다.

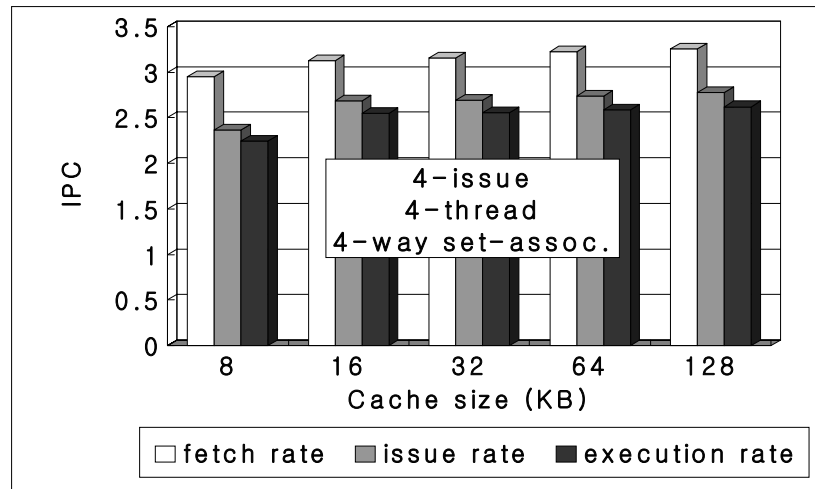


그림 6-19 캐쉬 크기에 따른 4-issue, 4-thread 구조의 성능 변화

Fig. 6-19 Performance of 4-issue, 4-thread architectures as a function of sizes of 4-way set-associative instruction and data caches

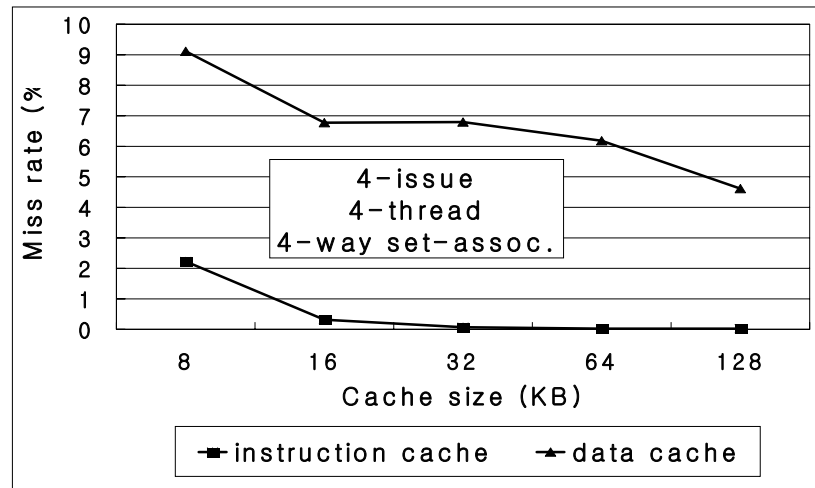


그림 6-20 캐쉬 크기에 따른 4-issue, 4-thread 구조의 캐쉬 미스율 변화

Fig. 6-20 Miss rates versus sizes of 4-way set-associative instruction and data caches for 4-issue, 4-thread architectures

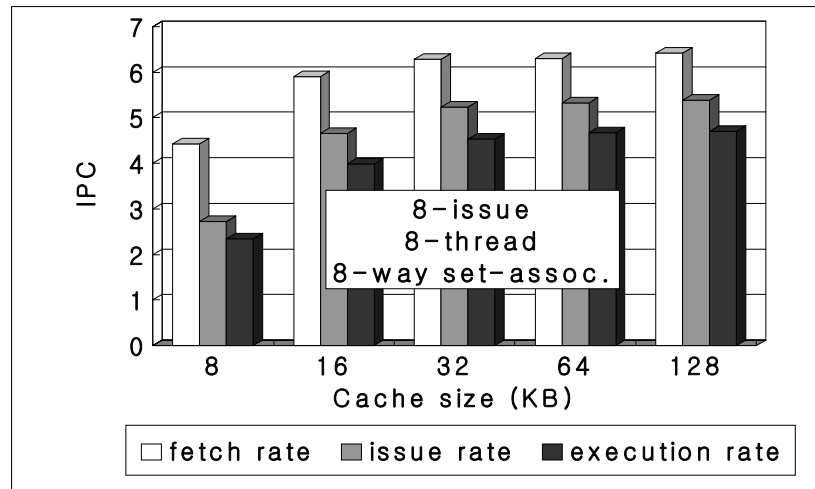


그림 6-21 캐쉬 크기에 따른 8-issue, 8-thread 구조의 성능 변화

Fig. 6-21 Performance of 8-issue, 8-thread architectures as a function of sizes of 8-way set-associative instruction and data caches

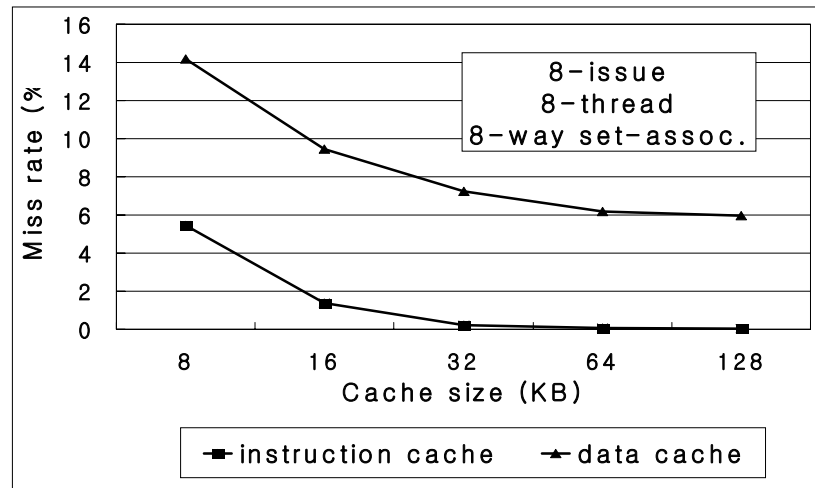


그림 6-22 캐쉬 크기에 따른 8-issue, 8-thread 구조의 캐쉬 미스율 변화

Fig. 6-22 Miss rates versus sizes of 8-way set-associative instruction and data caches for 8-issue, 8-thread architectures

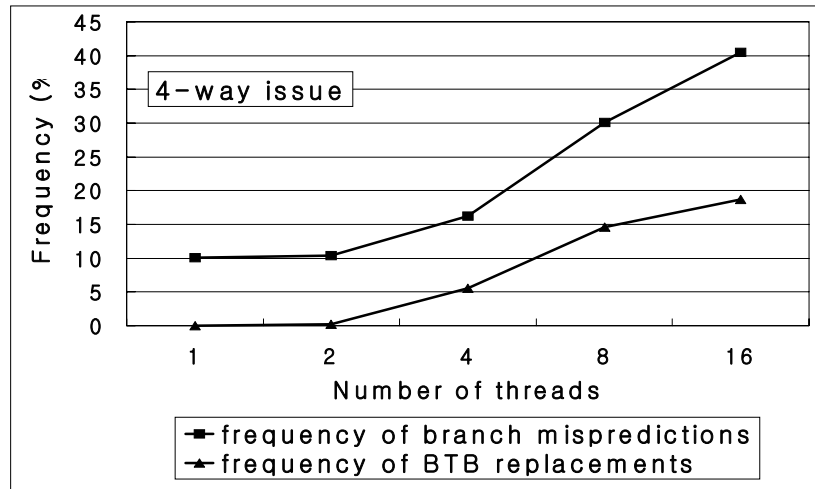


그림 6-23 스레드 수에 따른 4-issue 구조의 분기 예측 오류 빈도

Fig. 6-23 Frequencies of branch mispredictions and BTB replacements versus the number of threads for 4-issue architectures

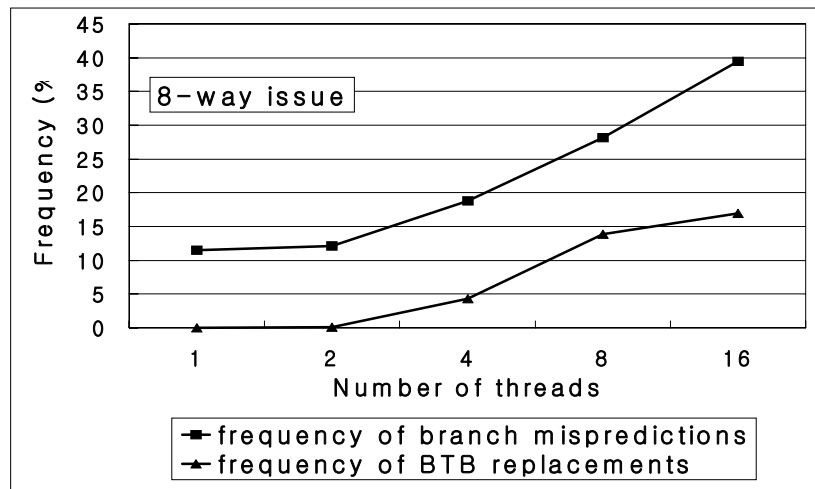


그림 6-24 스레드 수에 따른 8-issue 구조의 분기 예측 오류 빈도

Fig. 6-24 Frequencies of branch mispredictions and BTB replacements versus the number of threads for 8-issue architectures

따라서 스레드 수에 따른 적절한 BTB 엔트리 수를 정할 필요가 있으며, 그림 6-25부터 6-28까지의 그림들은 그 해답을 보여준다. 그림 6-25와 6-27을 보면 BTB의 엔트리 수 변화가 페치율에는 별 영향을 미치는 않지만 실행률에는 많은 영향을 미치는 모습을 볼 수 있다. BTB의 엔트리 수가 작을 때에는 분기 예측 오류 빈도가 높고 따라서 페치된 명령어들이 완료되기 전에 플러쉬되는 경우가 자주 발생하여 실행률이 낮은 반면, BTB의 엔트리 수가 클 경우에는 이러한 플러쉬가 비교적 드물게 이루어지기 때문에 실행률이 페치율과 비슷하게 나오는 것이다. 그림 6-26을 보면 4-issue, 4-thread 구조에서는 BTB 엔트리 수가 128개일 경우에 BTB 엔트리의 대체 빈도가 거의 0의 값을 가진다. 따라서 4-issue, 4-thread 구성에서는 128개의 BTB 엔트리 수가 추천된다. 같은 이유로 8-issue, 8-thread 구성에서는 256개의 BTB 엔트리 수가 적당하다.

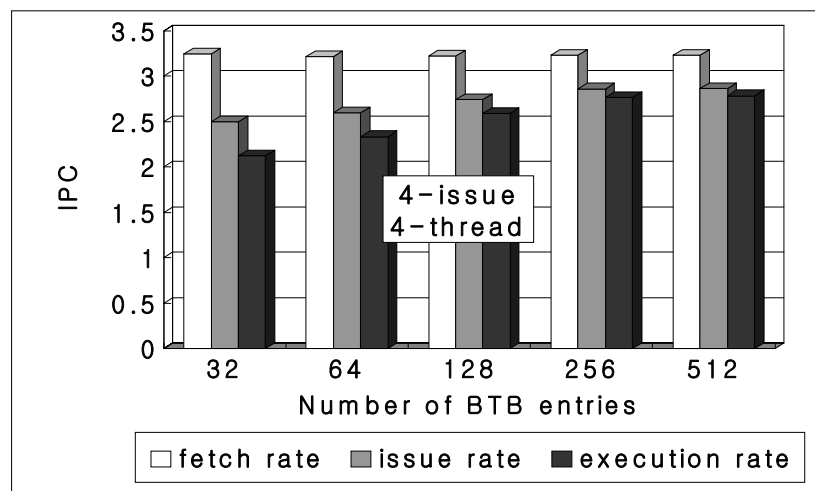


그림 6-25 BTB 엔트리 수에 따른 4-thread 구조의 성능 변화

Fig. 6-25 Performance of 4-issue, 4-thread architectures as a function of the number of BTB entries

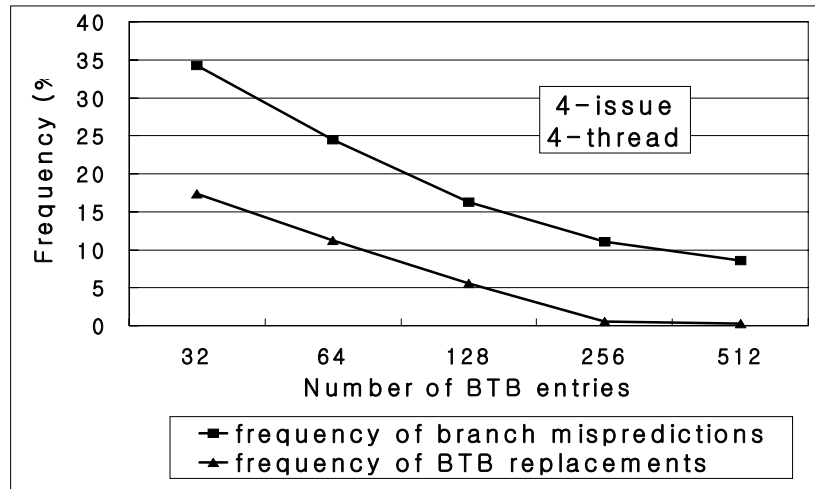


그림 6-26 BTB 엔트리 수에 따른 4-thread 구조의 분기 예측 오류 빈도

Fig. 6-26 Frequencies of branch mispredictions and BTB replacements versus the number of BTB entries for 4-issue, 4-thread architectures

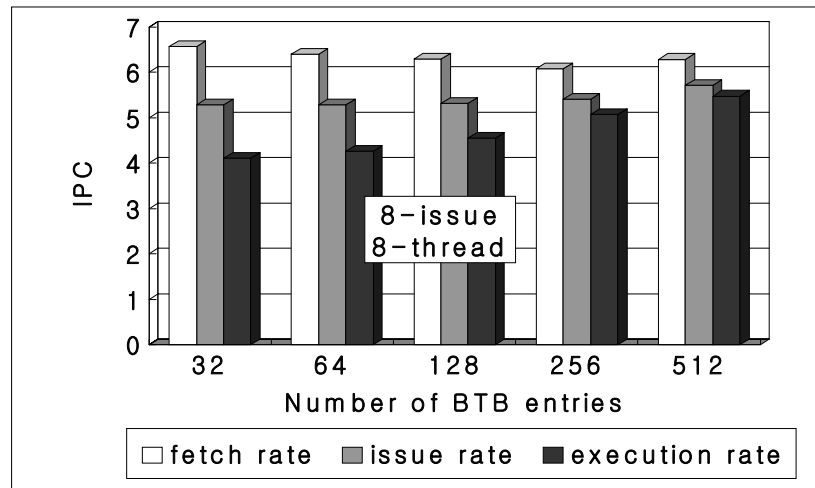


그림 6-27 BTB 엔트리 수에 따른 8-thread 구조의 성능 변화

Fig. 6-27 Performance of 8-issue, 8-thread architectures as a function of the number of BTB entries

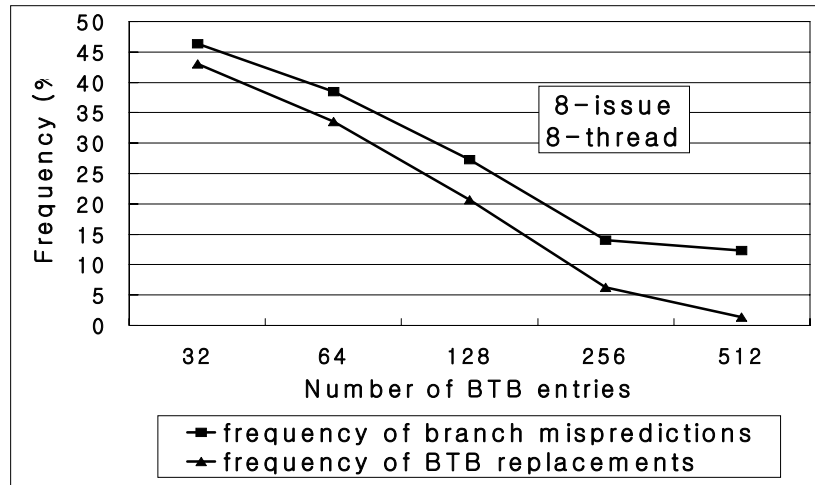


그림 6-28 BTB 엔트리 수에 따른 8-thread 구조의 분기 예측 오류 빈도

Fig. 6-28 Frequencies of branch mispredictions and BTB replacements versus the number of BTB entries for 8-issue, 8-thread architectures

그림 6-29와 6-30은 명령어 캐쉬 포트 수에 따른 성능 변화를 보여준다. 명령어 캐쉬가 더 많은 읽기 포트들을 제공할수록 단일 사이클에 폐치할 수 있는 스레드의 수가 더 많아지고, 따라서 명령어 폐치가 더욱 융통성 있게 이루어진다. 그리고 포트가 많을수록 각 포트의 폭이 줄어들고 PC의 주소 값에 대한 정렬 조건이 완화되기 때문에, 폐치된 명령어 중에서 무효한 명령어의 수가 그만큼 줄어들게 된다. 이러한 이유로 명령어 캐쉬의 읽기 포트 수가 늘어날수록 폐치율이 증가하고, 이것은 실제 성능인 실행률의 증가로 이어진다. 그림 6-29와 6-30으로부터 4-issue, 4-thread 구성에 대해서는 2개의 캐쉬 포트 수가, 8-issue, 8-thread 구성에 대해서는 4개의 포트 수가 적당하다는 것을 보여준다.

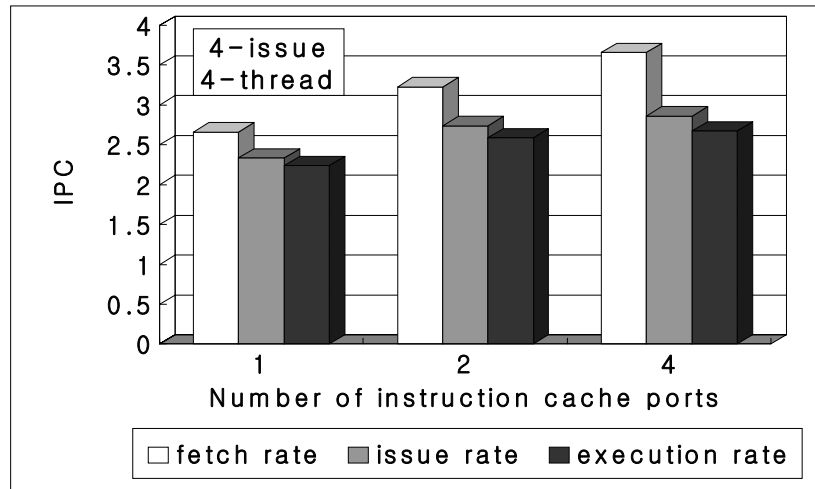


그림 6-29 명령어 캐쉬 포트 수에 따른 4-issue, 4-thread 구조의 성능

Fig. 6-29 Performance of 4-issue, 4-thread architectures as a function of the number of instruction cache ports

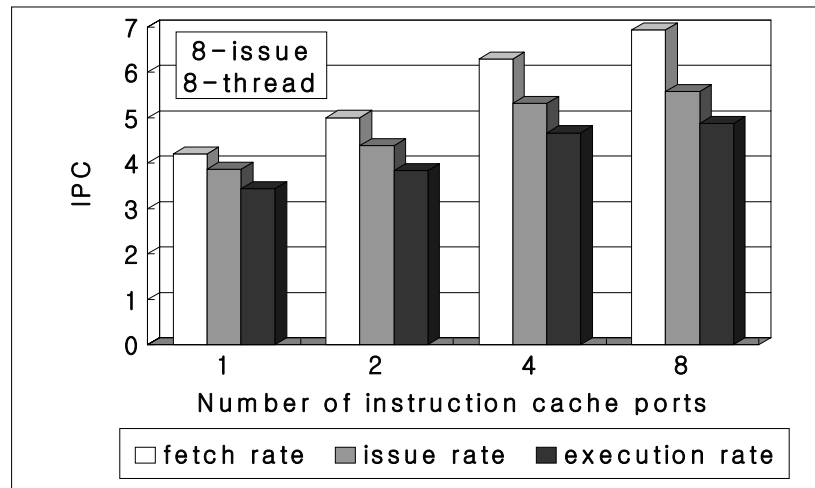


그림 6-30 명령어 캐쉬 포트 수에 따른 8-issue, 8-thread 구조의 성능

Fig. 6-30 Performance of 8-issue, 8-thread architectures as a function of the number of instruction cache ports

표 6-2 페치 우선권 정책에 따른 성능 변화

Table 6-2 Performance as a function of the fetch priority policy

분류	페치 우선권 정책	페치율 (IPC)	이슈율 (IPC)	실행률 (IPC)
4-issue, 4-thread	RR	3.2253	2.7396	2.5888
	ICOUNT_IFQ	3.1864	2.7712	2.6303
	ICOUNT_Q	3.2757	2.8639	2.6965
	ICOUNT_ALL	3.2660	2.8315	2.6708
	ICOUNT_BR	2.6246	1.7086	1.6226
	ICOUNT_MISS	2.7419	1.8134	1.7217
	IIQOL	3.1245	2.6178	2.4700
8-issue, 8-thread	RR	6.2945	5.3211	4.6680
	ICOUNT_IFQ	6.2318	5.1913	4.6911
	ICOUNT_Q	6.3786	5.5108	4.9661
	ICOUNT_ALL	6.7201	5.6951	4.9783
	ICOUNT_BR	5.5374	3.2279	2.8848
	ICOUNT_MISS	5.9460	3.4911	3.0740
	IIQOL	6.5601	5.3966	4.7194

표 6-3 이슈 우선권 정책에 따른 성능 변화

Table 6-3 Performance as a function of the issue priority policy

분류	이슈 우선권 정책	페치율 (IPC)	이슈율 (IPC)	실행률 (IPC)
4-issue, 4-thread	OLDEST	3.2253	2.7396	2.5888
	ICNT_FU	3.2087	2.6865	2.5564
	ICNT_MISS	3.2878	2.7070	2.5467
8-issue, 8-thread	OLDEST	6.2945	5.3211	4.6680
	ICNT_FU	6.1740	5.2031	4.6174
	ICNT_MISS	6.2593	5.2185	4.5083

표 6-2는 페치 우선권 정책에 따른 성능 변화를 보여준다. ICOUNT_IFQ, ICOUNT_Q 및 ICOUNT_ALL의 정책이 RR 정책보다 좋은 성능을 보이는데, 이것은 프로세서의 자원들을 스레드간에 균등하게 배분함으로써 전체적인 성능을 높일 수 있음을 보여준다. ICOUNT_Q가 ICOUNT_IFQ보다 페치율, 이슈율 및 실행률에 있어서 더 높은 값을 가지는데, 페치율에서의 차이보다 이슈율과 실행률에 있어서의 차이가 더 크다. 이것은 명령어 이슈 큐의 엔트리들을 스레드간에 균등하게 배분하는 것이 이슈율 향상에 더 도움이 되기 때문이다. ICOUNT_BR과 ICOUNT_MISS 정책이 RR(Round-Robin) 정책보다 성능이 낮은 것은 특정 동작의 명령어 수에 의존한 페치 우선권 정책은 RR 정책보다 좋지 않기 때문이다. ICOUNT_Q는 명령어 페치 큐와 명령어 이슈 큐에 있는 명령어 수를 세는 것에 의해서 간단히 구현되고 성능도 좋기 때문에 가장 적당한 페치 우선권 정책이다. 그리고 표 6-3은 이슈 우선권 정책이 성능에 미치는 영향을 보여주는데, 3가지 정책 사이에 성능 차이가 거의 없다. OLDEST 정책이 구현이 가장 간단하고 약간이나마 더 좋은 성능을 보이므로, OLDEST 정책이 이슈 우선권 정책으로서 추천된다.

그림 6-31과 6-32는 명령어 페치 큐의 엔트리 수에 따른 성능을 보여준다. 4-issue, 4-thread 구성에서는 명령어 페치 큐의 엔트리 수가 4개인 경우에 페치율의 저하가 이슈율과 실행률의 저하로 이어지며, 그 외에는 페치율, 이슈율 및 실행률에서 비슷한 성능을 보인다. 따라서 4-issue, 4-thread 구성에서는 8개의 엔트리들을 가진 명령어 페치 큐가 적당하다. 같은 이유로 8-issue, 8-thread 구성에서는 16개의 엔트리들을 가진 명령어 페치 큐가 적당하다. 명령어 페치 큐의 엔트리 수가 적어도 문제가 없는 것은 명령어 페치의 정지는 명령어 페치 큐의 엔트리 부족보다는 명령어 이슈 큐의 적체로 인해서 발생하기 쉽기 때문이다.

그림 6-33과 6-34는 명령어 이슈 큐의 엔트리 수와 성능과의 관계를 보여준다. 그림 6-33과 6-34에서 각각 명령어 이슈 큐의 엔트리 수가 8개 또는 16개인 경우, 다른 경우에 비해서 페치율이 낮고 이슈율 및 실행률이 페치율과 많은 차이를 보인다. 명령어 이슈 큐의 엔트리 수가 작을 때에는 명령어 이슈 큐의 적체 현상이 자주 발생하고, 이로 인해 이슈될 수 있는 명령어가 명령어 이슈 큐로 전달되지 못하는 현상이 발생한다. 이것은 이슈율의 저하로 이어지고 또한 분기 명령어의 실행을 그만큼 늦춘다. 따라서 명령어 이슈 큐가 적체된 경우에는 명령어 페치 큐에 명령어들은 계속 쌓이고, 이것은 늦게 실행된 분기 명령어에서 분기 예측 오류가 발생하면 모두 플리쉬된다. 이러한 이유로 페치율이 이슈율 및 실행률과 많은 차이를 보이는 것이다. 명령어 이슈 큐의 엔트리 수로서 적절한 값이 명령어 페치 큐에 대한 것보다 큰데, 이것은 명령어 이슈 큐가 더 적체되기 쉽다는 것을 보여준다. 따라서 명령어 이슈 큐에서의 압축은 필요한 동작이다.

표 6-4는 기능 유닛의 수와 성능과의 관계를 보여준다. 기능 유닛의 수는 성능과 하드웨어 비용을 고려해서 정해져야 한다. 표 6-4의 결과를 고려하면 4-issue, 4-thread 구성은 3개의 ALU, 1개의 곱셈 유닛 그리고 2개의 로드/스토어 유닛이 적당하고, 8-issue, 8-thread 구성은 6개의 ALU, 1개의 곱셈 유닛 그리고 3개의 로드/스토어 유닛이 적당함을 알 수 있다. 표 6-4에서 곱셈 유닛의 수를 1개로 고정한 것은 시뮬레이션의 입력 프로그램들이 곱셈 명령어를 많이 사용하지 않는 일반 프로그램들이기 때문이다. 그리고 특정 응용 프로그램들이 아니면 대부분 곱셈 명령어의 빈도가 매우 낮기 때문에 1개의 곱셈 유닛으로도 충분하다.

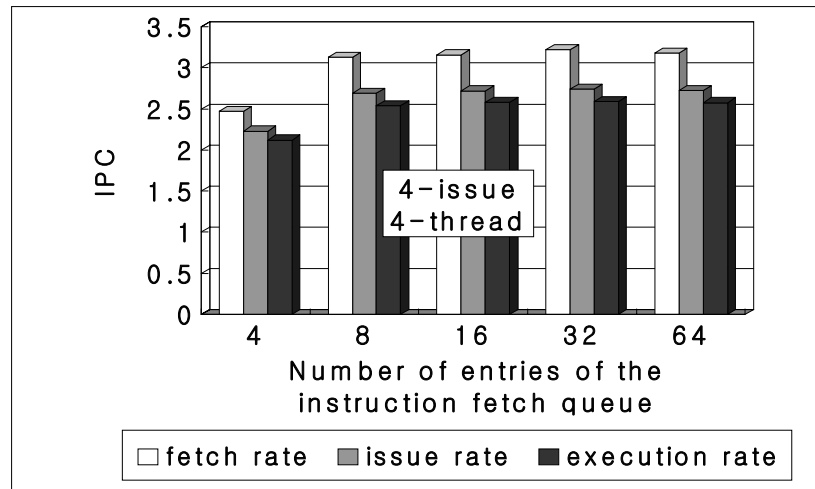


그림 6-31 명령어 페치 큐 엔트리 수에 따른 4-issue, 4-thread 구조의 성능
 Fig. 6-31 Performance of 4-issue, 4-thread architectures as a function of the number of the instruction fetch queue

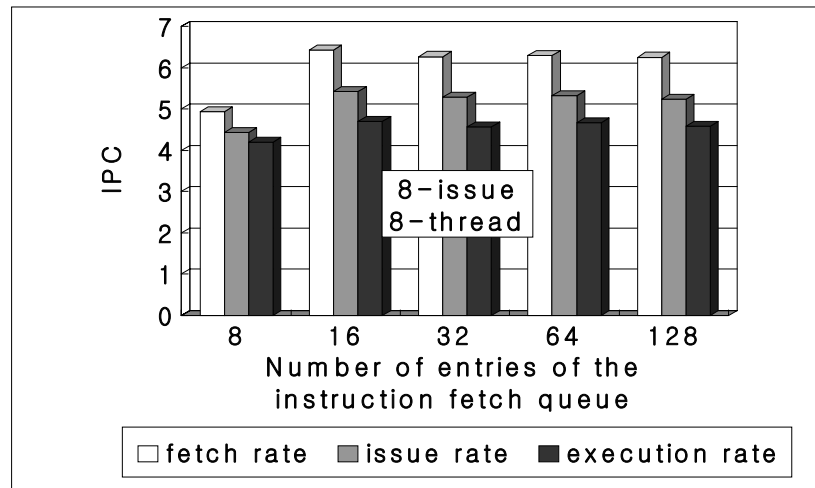


그림 6-32 명령어 페치 큐 엔트리 수에 따른 8-issue, 8-thread 구조의 성능
 Fig. 6-32 Performance of 8-issue, 8-thread architectures as a function of the number of the instruction fetch queue

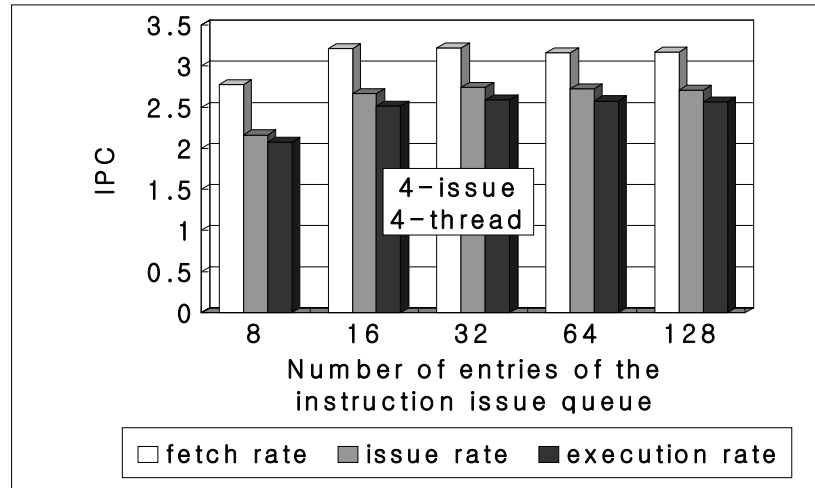


그림 6-33 명령어 이슈 큐 엔트리 수에 따른 4-issue, 4-thread 구조의 성능
 Fig. 6-33 Performance of 4-issue, 4-thread architectures as a function of the number of the instruction issue queue

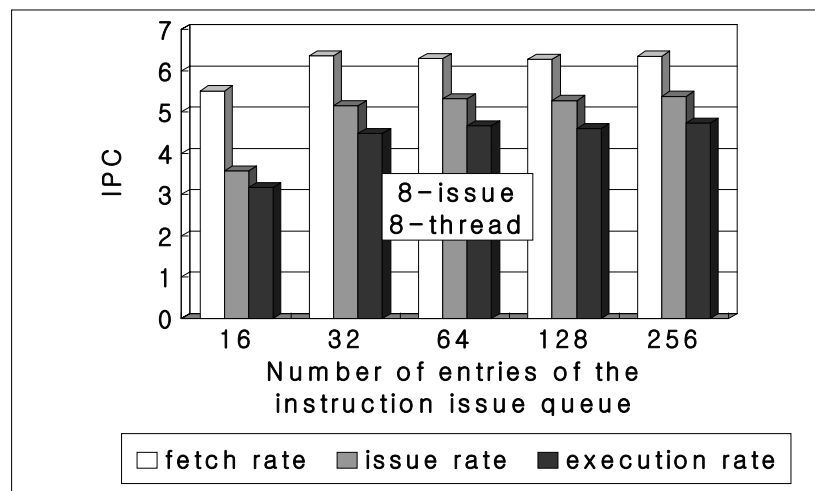


그림 6-34 명령어 이슈 큐 엔트리 수에 따른 8-issue, 8-thread 구조의 성능
 Fig. 6-34 Performance of 8-issue, 8-thread architectures as a function of the number of the instruction issue queue

표 6-4 기능 유닛의 수에 따른 성능 변화

Table 6-4 Performance as a function of the number of functional units

분류	ALU 수 / 곱셈 유닛 수 / 로드/스토어 유닛 수	페치율 (IPC)	이슈율 (IPC)	실행률 (IPC)
4-issue, 4-thread	4 / 1 / 2	3.2253	2.7396	2.5888
	3 / 1 / 2	3.1455	2.6858	2.5572
	2 / 1 / 2	3.0633	2.1972	2.0888
	4 / 1 / 1	3.0276	2.2316	2.1123
	3 / 1 / 1	3.1227	2.2230	2.0744
	2 / 1 / 1	2.8516	1.9496	1.8648
8-issue, 8-thread	8 / 1 / 4	6.2945	5.3211	4.6680
	7 / 1 / 4	6.2043	5.2414	4.5768
	6 / 1 / 4	6.3504	5.3142	4.6859
	5 / 1 / 4	6.1467	4.9223	4.3115
	4 / 1 / 4	6.1602	4.3706	3.8768
	8 / 1 / 3	6.2575	5.2045	4.5390
	8 / 1 / 2	6.3493	4.7932	4.1850
	8 / 1 / 1	5.3739	2.9546	2.6145
	6 / 1 / 3	6.2997	5.1482	4.4733
	6 / 1 / 2	6.1083	4.5448	4.0005
	6 / 1 / 1	5.2634	2.9440	2.6291

상기의 시뮬레이션 결과들은 모두 그룹화를 적용하지 않은 구조에 대한 것이다. SMT 구조는 수퍼스칼라 구조에 비해 성능이 월등히 좋다. 그러나 SMT 구조의 설계 복잡도가 수퍼스칼라 구조에 비해 매우 크기 때문에 SMT의 복잡도를 줄이는 방안으로서 그룹화를 적용하는 것이 필요하다. 표 6-5는 8-issue, 8-thread 구조에서 그룹화의 적용 방식에 따른 성능의 변화를 보여준다. 표 6-5의 시뮬레이션에서 그룹화 3을 적용하기 위해 편의상 곱셈 유닛의 수를 4개로 하였다. 전체적으로 그룹화에 의해서 성능 변화가 별로 없다. 특히 그룹화 1 또는 그룹화 2가 적용된 구조는 그룹화가 적용되지 않은 구조와 비교해서 비슷한 또는 약간 더 좋은 성능을 보인다. 반면에 그룹화 3을 적용하였을 경우에는 그렇지 않은 경우보다 낮은 성능을 보인다.

표 6-5에 나와있듯이 예상과는 달리 그룹화 1을 적용해도 성능 저하가 거의 없다. 이것은 ICOUNT_Q의 페치 우선권 정책이 좋은 성능을 보이는 것과 비슷한 이유다. 즉 그룹화 1에 의해서 명령어 페치 자원의 사용이 특정 스레드들에게 집중되는 것을 막을 수 있고, 이로 인해 프로세서의 자원을 여러 스레드들에게 균등하게 배분하게 된다. 이것이 페치 자원을 스레드들간에 정적으로 배분하는 그룹화 1의 단점을 보완하여 그룹화로 인한 성능 저하를 없앨 수 있었던 것이다. 따라서 그룹화 1은 최대한 적용하여 설계의 복잡도를 줄이는 것이 좋다. 그런데 그룹화 1에 의한 최대 그룹 수는 스레드 수가 아닌 명령어 캐쉬, ITLB 및 BTB의 포트 수에 의해서 제한된다. 즉 단일 사이클에 페치할 수 있는 스레드의 수는 이러한 포트 수에 의해서 제한되고, 이것은 또한 그룹화 1에 의한 최대 그룹 수를 제한하는 것이다. 따라서 그룹화 1을 더 많이 적용하기 위해서는 포트 수를 늘려야 하는데, 이것은 비용에 있어서 부담이 크기 때문에 그룹화 1의 더 많은 적용을 위해서 포트 수를 증가시키는 것은 설계의 복잡도 감소라는 그룹화의 목적에 부합하지 않는다. 이러한 이유로 명령어 캐쉬, ITLB 및 BTB의 포트 수는 비용과 성능을 고려해서 결정하고, 그 한도 내에서 그룹화 1을 최대한 적용하여 명령어 페치 자원들을 스레드들 사이에 정적으로 배분하는 것이 적절하다.

그룹화 2가 적용된 구조에서의 높은 성능은 명령어 페치 큐와 명령어 이슈 큐의 엔트리 수가 충분하기 때문에 가능하다. 표 6-5의 결과를 얻기 위한 시뮬레이

선에서 두 큐의 엔트리 수로서 표 6-1에 주어진 값을 사용하였는데, 이것은 64개이다. 그런데 64개는 그림 6-32 및 6-34의 결과에서 주어진 16개 또는 32개보다 훨씬 큰 값이기 때문에 그룹화에 의해서 그 큐를 분할하더라도 성능 저하가 발생하지 않은 것이다. 이러한 결과는 4-3절에서 그룹화 2를 제안할 때에 이미 예상된 결과이다. 즉, 그룹화 2를 적용할 때에 각 명령어 페치 큐 및 명령어 이슈 큐의 엔트리 수가 부족할 경우는 전체적으로 같은 수의 엔트리를 가지면서 그룹화 2를 적용하지 않은 경우보다 낮은 성능을 보이게 된다. 그러나 각 큐의 엔트리 수를 적절하게 설정함으로써 그룹화 2의 적용으로 인한 성능 저하를 없앨 수 있다. 이러한 결과는 그룹화 2는 설계의 복잡도를 낮추는 역할을 하지만 전체적으로 명령어 페치 큐와 명령어 이슈 큐의 엔트리 수를 증가시킨다는 단점이 있다는 것을 말한다. 그러나 단순한 엔트리 수의 증가는 4-3절에서 설명한 그룹화 2의 적용에 의한 설계의 단순화라는 장점을 고려하였을 때 큰 의미가 없다. 또한 그룹화 2는 그룹화 1처럼 명령어 이슈 큐와 명령어 페치 큐의 자원을 스레드간에 균등하게 배분할 수 있게 하였고, 이것 또한 성능 저하를 억제하는 역할을 하였다. 전체적으로 개개의 명령어 페치 큐 및 명령어 이슈 큐의 엔트리 수가 충분하다고 하면, 그룹화 2의 적용은 성능 저하를 유발하지 않는다는 것을 알 수 있다. 따라서 그룹화 2는 그룹화 1처럼 최대한 적용하는 것이 좋다. 즉, 그룹화 2에 의해서 스레드 수만큼의 그룹들로 명령어 페치 큐, 디코드 슬롯, 및 명령어 이슈 큐를 나눔으로써 설계의 복잡도를 줄이는 것이 좋다. 그룹화 2를 이와 같이 적용했을 때에는 매우 큰 장점이 생긴다. 스레드별로 명령어 페치 큐와 명령어 이슈 큐가 존재하기 때문에, 개개의 큐들은 슈퍼스칼라 구조의 큐와 같은 구조를 가지게 된다. 따라서 분기 예측 오류 또는 예외가 발생하였을 때의 명령어 플러쉬는 단일한 명령어 페치 큐 및 명령어 이슈 큐 전체의 플러쉬로 단순화된다. 또한 개개의 명령어 이슈 큐에서 명령어들은 순서대로 이슈되기 때문에 명령어 이슈 큐의 압축이 필요하지 않는데, 이것은 이슈 유닛의 설계를 매우 간단하게 할 뿐 아니라, 파이프라인 I 단계의 지연 시간을 상당히 줄일 수 있게 한다.

반면에 표 6-5에 나와 있듯이 그룹화 3을 적용한 구조는 그렇지 않은 구조에 비해서 성능 저하가 뚜렷이 나타난다. 그룹화 3을 적용하였을 때에는 그룹당 기능

유닛의 종류별 수가 제한되기 때문에, 그룹들의 종류별 명령어 수의 분포가 차이가 많이 나는 시간대에는 명령어 이슈가 비효율적으로 이루어진다. 이러한 경우는 프로그램의 전체 수행 중에 자주 발생하기 때문에 성능이 낮아지는 것이다. 예를 들어서 그룹 수가 2개이고 그룹당 로드/스토어 유닛의 수가 2개인 경우, 그룹 0에는 이슈할 수 있는 로드 또는 스토어 명령어의 수가 3개이고 그룹 1에는 그러한 명령어가 존재하지 않는 사이클에, 전체적으로 이슈되는 로드 또는 스토어 명령어의 수는 2개이다. 그룹 1에 사용되지 않는 로드/스토어 유닛이 존재함에도 불구하고 그룹 0의 로드/스토어 명령어 1개가 이슈되지 못하는 것이다. 트레이스 파일을 검사한 결과 이러한 사이클이 자주 발생하였다. 그 외에도 레지스터 파일의 포트와 관련해서 비효율성이 나타난다. 본 논문의 명령어 세트 구조에서는 3개의 레지스터를 읽는 명령어들이 다수 존재한다. 이러한 이유로 단일 그룹에 속하는 명령어들이 필요로 하는 레지스터 파일의 읽기 포트 수가 그 그룹의 포트 수보다 클 경우가 자주 발생한다. 이 경우 다른 그룹에 여분의 포트가 존재하더라도 그 포트를 사용하지 못하고 이슈되지 못하는 명령어들이 생기게 된다. 이와 같은 이유로 그룹화 3의 적용은 성능 저하에 비교적 많은 영향을 미친다. 따라서 그룹화 3에 의한 그룹 수가 지나치게 클 경우에는 그로 인한 성능 저하가 설계 복잡도의 감소라는 장점보다 더 크게 작용하게 된다. 전체적으로 그룹화 3은 비용과 성능을 고려한 적절한 수준에서 적용할 것이 권장되며, 표 6-5의 결과는 8-issue, 8-thread 구조에 대해 그룹화 3에 의한 그룹 수는 2개 정도가 적당함을 보여준다.

SMT 구조는 기본적으로 모든 스레드들이 프로세서의 자원을 공유함으로써, ILP뿐 아니라 TLP를 충분히 활용하는 구조이다. 그러나 이러한 공유 철학은 SMT의 설계를 어렵게 만드는 요인이기도 하다. SMT는 시뮬레이션 결과에서 보듯이 성능에 있어서 놀랄만한 잠재력을 가지고 있지만 이러한 설계 복잡도는 실제 구현을 어렵게 만든다. 그룹화는 이러한 SMT 구조의 단점을 보완할 수 있는 방법이 된다. 상기의 결과에서 볼 수 있듯이 적절한 그룹화의 적용은 설계의 복잡도는 줄이면서 성능에는 큰 영향을 미치지 않는다. 따라서 SMT 프로세서를 설계할 때에는 그룹화에 대한 철저한 연구를 수행하여야 한다. 상기의 그룹화에 의한 성능 변화 결과는 그러한 연구의 척도가 된다.

표 6-5 그룹화에 따른 8-issue, 8-thread 구조의 성능 변화

Table 6-5 Performance of 8-issue, 8-thread architectures as a function of grouping

그룹 수 (그룹화 1 / 그룹화 2 / 그룹화 3)	페치율 (IPC)	이슈율 (IPC)	실행률 (IPC)
1 / 1 / 1	6.2379	5.2505	4.5767
2 / 1 / 1	6.2652	5.1376	4.6253
1 / 2 / 1	6.3315	5.1821	4.5418
1 / 1 / 2	6.3572	5.1242	4.5197
1 / 2 / 2	6.4501	5.1435	4.5700
2 / 1 / 2	6.2840	4.9627	4.4394
2 / 2 / 1	6.2578	5.1072	4.5511
4 / 1 / 1	6.4434	5.3102	4.7189
1 / 4 / 1	6.3739	5.2104	4.6557
1 / 1 / 4	6.2351	4.5117	4.0372
1 / 4 / 4	6.3986	4.6972	4.2078
4 / 1 / 4	6.2020	4.5809	4.1297
4 / 4 / 1	6.6097	5.5041	4.8795
2 / 2 / 2	6.3161	5.1579	4.6588
4 / 2 / 2	6.2344	5.1513	4.5969
2 / 4 / 2	6.4685	5.0302	4.5117
2 / 2 / 4	6.3051	4.6212	4.1412
2 / 4 / 4	6.4498	4.7570	4.2848
4 / 2 / 4	6.2633	4.6247	4.1243
4 / 4 / 2	6.5315	5.2886	4.6911
4 / 4 / 4	6.4341	4.8106	4.2905
1 / 8 / 1	6.3823	5.1268	4.5882
2 / 8 / 2	6.5442	5.1720	4.6538
4 / 8 / 2	6.5762	5.2310	4.6930
4 / 8 / 4	6.3721	4.8765	4.4345

캐쉬의 블록 크기와 관련해서는 블록 크기를 너무 작게 또는 너무 크게 설정하지 않는 한도에서는 블록 크기의 변화는 성능에 별로 영향을 주지 못한다. 그리고, 캐쉬, BTB 및 ITLB의 대체 알고리즘과 관련해서는 RANDOM과 LRU가 대체로 비슷한 성능을 보였으며, RANDOM이 LRU보다 더 높은 성능을 보이는 경우도 있다. 따라서 구현이 간단한 RANDOM을 대체 정책으로 사용하는 것이 적당하다. 본 논문의 시뮬레이션에서는 가능한 페이지의 크기를 크게 설정하였기 때문에 TLB 미스는 시뮬레이션 초기에 조금 발생하고, 더 이상 발생하지 않는다. 실제로 TLB 미스율은 시스템 프로그램의 페이징 알고리즘의 영향을 많이 받기 때문에 본 논문에서는 TLB 미스로 인한 성능 변화를 최소화하기 위해서 TLB 미스 처리 사이클 수를 1사이클로 설정하였다. 이러한 이유로 ITLB와 DTLB의 엔트리 수 변화에 따른 성능 변화는 측정하지 않았다.

6-3절 최적의 순차적 SMT 구조 제안

본 절에서는 6-1절에서의 결과를 토대로 적당한 SMT 구조를 제안한다. 그런데 마이크로프로세서의 구조는 그 프로세서가 응용되는 분야에 따라서 달라져야 한다. 따라서 본 논문에서는 두 가지 종류의 SMT 구조를 제안한다.

첫째는 내장형 프로세서(embedded processor)^[93-95]에 적당한 SMT 구조이다. 내장형 프로세서는 일반 컴퓨터 시스템에 사용되는 마이크로프로세서보다는 작은 성능을 가져도 되며, 비용과 성능 및 응용 분야에서의 요구 등을 고려해서 가장 적절한 구조로 구현되어야 한다. 그런데 SMT 구조를 극히 저비용의 저성능 프로세서에 적용하는 것은 적당하지 않다. SMT 구조는 단일 사이클에 다수의 스레드로부터의 명령어들을 이슈하는 구조이므로, 1-issue 또는 2-issue 구조는 SMT에 적당하지 않다. 그리고 이슈 폭이 4개의 명령어 이상인 고성능의 내장형 프로세서들이 출시되고 있다^[96]. 이러한 점들을 고려하여 본 논문에서는 비교적 고성능의 내장형 프로세서에 적당한 SMT 구조를 제시한다. 표 6-6은 6-1절의 결과를 토대로 내장형 SMT 구조의 적절한 구성을 제시한다. 표 6-6의 구조 변수들 중에서 명령어 페치 큐와 명령어 이슈 큐의 엔트리 수가 6-1절에서 제시한 수보다 큰데 이것은 그룹화 2의 영향이다. 즉 그룹화 2로부터 얻는 이득의 대가로 명령어 이슈 큐와 명령어 페치 큐의 엔트리 수가 약간 많아진 것이다.

시뮬레이션 결과 제안된 내장형 SMT 구조는 IPC가 2.5812인 반면에, 비슷한 구조의 슈퍼스칼라 구조는 0.9003의 IPC를 보였다. 즉 208%의 IPC 향상을 보인 것이다. 이와 같은 IPC의 향상을 위해서 SMT에서 추가되는 비용은 레지스터 파일과 스코어보드 어레이의 면적 증가, 그리고 각 파이프라인 단계에서 다수의 스레드를 지원하기 위한 제어 로직의 복잡화 정도이다. 제안된 내장형 SMT 구조와 슈퍼스칼라 구조에 있어서 면적 및 주파수에 대한 정량적인 비교는 실제 칩으로 구현하기 전에는 힘들다. 그러나 같은 슈퍼스칼라 구조에서는 트랜지스터 수를 10배로 증가시켜도 IPC를 2배 이상 증가시키지 힘들다는 것을 고려하면 SMT가 보이는 IPC에 있어서의 큰 성능 향상은 ILP뿐 아니라 TLP까지 이용하는 본 논문의

SMT 구조가 가지는 성능에 있어서의 잠재력을 충분히 보여준다.

표 6-6 제안된 내장형 순차적 SMT 구조

Table 6-6 Proposed embedded in-order SMT architecture

구조 변수	변수 값
GRP1_NUM	2
GRP2_NUM	4
GRP3_NUM	1
THREAD_NUM	4
FETCH_WIDTH	4
DECODE_WIDTH	4
ISSUE_WIDTH	4
IFQ_SIZE	16 (4 entries per instruction fetch queue)
IIQ_SIZE	32 (8 entries per instruction issue queue)
REGR_NUM	8
REGW_NUM	4
ALUF_NUM	3
MULF_NUM	1
LSUF_NUM	2
ICP_NUM	2
IC_SIZE	16 Kbytes
ICWAY_NUM	4
ICBLK_SIZE	32 bytes
DC_SIZE	16 Kbytes
DCWAY_NUM	4
DCBLK_SIZE	32 bytes
BTB_SIZE	128
ITLB_SIZE	64
DTLB_SIZE	64
CACHEREP	RANDOM
BTBREP	RANDOM
TLBREP	RANDOM
FETCH_PRI	ICOUNT_Q
ISSUE_PRI	OLDEST

두 번째로 제시하는 SMT 구조는 고성능의 응용에 적당한 것이다. 고성능이라고 하더라도 하드웨어 비용을 무조건 크게 할 수는 없다. 더구나, 설계의 복잡도 증가로 인한 클락 주파수의 감소도 고려해야 한다. 이러한 점들과 현재 8-issue 프로세서들이 출시되고 있다는 점을 고려하여 고성능 SMT 구조는 8-issue 구조로서 제시한다. 표 6-7은 본 논문에서 제안하는 고성능 SMT 구조의 구성을 제시하고 있다. 표 6-7의 구조 변수 중에서 6-1절에서 제시한 값과 다른 것들이 있는데, 그룹화의 적용 때문이다. 시뮬레이션 결과 제안된 내장형 SMT 구조는 IPC가 4.5417인 반면에, 비슷한 구조의 슈퍼스칼라 구조는 0.9288의 IPC를 보였으며, 이것은 389%의 IPC 향상에 해당한다.

제 1 장에서 언급하였듯이, 현재 네트워크 상의 패킷들을 라우팅하기 위한 네트워크 프로세서에 대한 연구가 활발히 이루어지고 있는데, SMT 구조는 이러한 네트워크 프로세서에 적당한 구조이다. 이것은 SMT 구조의 네트워크 프로세서는 각 스레드에 패킷을 대응하여 독립적인 패킷들을 병렬적으로 처리할 수 있기 때문이다. 이러한 이유로 제안된 SMT 구조가 네트워크 프로세서의 코어로서 사용되었을 때에 보일 성능을 평가하였으며, 표 6-8은 본 논문에서 제안된 고성능 SMT 구조를 네트워크 프로세서에 적용하였을 때의 성능을 제시한다. 표 6-8에서는 단일 패킷을 처리하기 위해서 실행되는 명령어의 수를 100개로 하였는데, 패킷 스케줄러(packet scheduler)^[97]와 같이 네트워크 프로세서에 필요한 유닛들을 추가할 경우에 대한 것이다^[98]. 그리고, 현재 1 GHz 이상의 클락 주파수를 갖는 마이크로프로세서들이 출시되고 있다는 것^[99]을 고려하면 500 MHz의 클락 주파수는 충분히 구현 가능하다. 그리고 패킷의 평균 크기를 64 바이트로 가정하고 있다^[2]. 표 6-8에서 제안된 고성능 SMT 구조를 채택한 네트워크 프로세서는 11.6 Gbps의 데이터 처리 능력을 가질 것으로 예상되는데, 이것은 현재 출시되고 있는 네트워크 프로세서들의 처리 능력이 5 내지 6 Gbps라는 것^{[2][4][100]}을 고려하면 상당히 높은 값이다. 따라서 본 논문에서 제안된 고성능 SMT 구조를 코어 구조로 가지고 있으면서 패킷 스케줄러와 같이 패킷 처리용 기능 유닛들을 추가한 네트워크 프로세서는 충분한 경쟁력을 가질 수 있다.

표 6-7 제안된 고성능 순차적 SMT 구조

Table 6-7 Proposed high-performance in-order SMT architecture

구조 변수	변수 값
GRP1_NUM	4
GRP2_NUM	8
GRP3_NUM	2
THREAD_NUM	8
FETCH_WIDTH	8
DECODE_WIDTH	8
ISSUE_WIDTH	8
IFQ_SIZE	32 (4 entries per instruction fetch queue)
IIQ_SIZE	64 (8 entries per instruction issue queue)
REGR_NUM	16
REGW_NUM	8
ALUF_NUM	6
MULF_NUM	2
LSUF_NUM	4
ICP_NUM	4
IC_SIZE	32 Kbytes
ICWAY_NUM	4
ICBLK_SIZE	32 bytes
DC_SIZE	32 Kbytes
DCWAY_NUM	4
DCBLK_SIZE	32 bytes
BTB_SIZE	256
ITLB_SIZE	64
DTLB_SIZE	64
CACHEREP	RANDOM
BTBREP	RANDOM
TLBREP	RANDOM
FETCH_PRI	ICOUNT_Q
ISSUE_PRI	OLDEST

표 6-8 제안된 고성능 SMT 구조를 사용한 네트워크 프로세서의 성능
 Table 6-8 Performance of a network processor adopting the proposed
 high-performance SMT architecture

IPC	4.5417
클럭 주파수	500 MHz
명령어 처리 능력	2271 MIPS
패킷당 소요되는 평균 명령어 수	100
패킷 처리 능력	22.71 MPPS(Million Packets Per Second)
패킷의 평균 크기	64 bytes
네트워크 데이터 교환 속도	11.6 Gbps

제 7 장 기존 SMT 연구와의 비교

본 논문은 앞의 장들에서 수퍼스칼라 구조를 기반으로 SMT 구조를 제안하고 SMT 구조에 그룹화를 적용함으로써 설계의 복잡도를 줄일 수 있는 방법을 모색하고 시뮬레이션에 의한 성능 평가를 수행하였다. 그런데 본 논문에서 제안된 SMT 구조는 기존의 SMT에 관한 연구들을 바탕으로 하고 있다. 본 장에서는 SMT에 관한 기존의 연구들을 소개하고 이를 본 논문의 연구와 비교한다.

SMT라는 용어는 Tullsen과 그의 동료들의 논문들^[34-36]에서 처음으로 만들어진 것이다. 그러나 단일 사이클에 다수의 스레드들로부터 명령어들을 이슈하는 구조에 관한 다른 연구들이 Tullsen의 논문^[34] 이전에도 존재했으며, 그 이후에도 이에 관한 많은 논문들이 발표되었다. 이러한 논문들에서의 구조들은 개념적으로 비슷하지만 세부적인 구조에서는 많은 차이를 보인다. 특히 스레드들이 자원을 공유하는 방식에 있어서 구조들 사이에서 많은 차이를 보인다.

우선, Tullsen과 그의 동료들의 논문^[34]은 SMT 구조의 이상적인 모델을 제시하고 다른 구조들과의 성능을 비교함으로써 SMT 구조가 성능 향상에 있어서 얼마나 큰 가능성을 가지고 있는지를 보여 주었다. 이 논문에서는 SMT 구조를 수퍼스칼라 구조 및 FMT 구조와 비교하였다. 이 논문에 의하면 수퍼스칼라 구조는 ILP의 부족으로 인해 이슈 자원의 수평적 낭비(horizontal waste)와 수직적 낭비(vertical waste)가 발생한다. FMT 구조는 수퍼스칼라 구조의 수직적 낭비를 줄일 수는 있지만 여전히 수평적 낭비를 보인다. 이에 반해서 SMT 구조는 단일 사이클에 다수의 스레드들로부터 명령어들을 이슈함으로써, 많은 양의 수평적 낭비 및 수직적 낭비를 제거하고 성능을 향상시킬 수 있음을 이 논문은 보여준다. 그 외에 이 논문에서는 명령어 캐쉬와 데이터 캐쉬의 공유에 따른 성능 변화를 측정하였다. 그리고 SMT를 비슷한 자원을 가지는 CMP와 비교하여, SMT의 융통성 있는 자원 공유로 인한 성능에 있어서의 우수성을 보여주었다. 이 논문에서는 본 논문의 그룹화 3과 비슷하게 이슈를 제한하는 구조를 제시하고 있다. 그러나, 본 논문

에서처럼 그룹화 3을 다양하게 적용하는 것이 아니라, 각 그룹에는 각 종류의 기능 유닛들이 1개씩만 존재하도록 구성된다는 차이점이 있다. 그리고, 본 논문의 그룹화 1과 그룹화 2에 대응할만한 방안을 제시하지 않는다. 그리고 이 논문은 개념적이고 이상적인 모델로서의 SMT를 제시하였지만, 구체적인 구현을 위한 구조를 제시하지 못했다.

Tullsen과 그의 동료들은 상기의 연구를 발전시켜서 또 하나의 논문^[35]을 발표하였다. 이 논문은 실제적인 구현을 위한 SMT 구조를 제시한다. 이 논문은 슈퍼스칼라 구조를 기반으로 약간의 변경을 통해서 SMT 구조를 제시한다는 점에서 본 논문과 비슷하다. 그러나 이 논문의 기반이 되는 슈퍼스칼라 구조는 비순차적 명령어 이슈 및 완료에 기초한다. 따라서 SMT 구조에서도 각 스레드별로 비순차적 이슈 및 완료를 사용하고, 이를 위해서 레지스터 리네이밍과 순차적 상태의 추적 및 복구라는 복잡한 동작에 대한 지원이 필요하다. 그리고, 이 논문에서는 명령어 페치와 이슈에 관한 다양한 구조 변수들을 사용하여 적절한 페치 및 이슈 정책을 제시하고 있다. 그러나, 본 논문에서처럼 다양한 구성 요소들의 변화에 따른 성능 변화를 측정하지는 않았고 성능 평가가 8-issue, 8-thread 구조에 대해서만 이루어졌다. 그리고 자원들의 융통성 있는 공유에 초점을 맞추고 있는 이유로 그룹화에 대한 고려를 하지 못했다. 이 논문의 SMT 구조는 8-issue, 8-thread로 구성되었을 때, 5.4의 IPC를 보인다. 이것은 본 논문의 대응되는 SMT 구조가 보이는 4.5의 IPC보다 좋다. 그 이유는 첫째 채택된 명령어 세트 구조와 입력으로 사용된 프로그램이 다르다는 것이고, 둘째는 이 논문의 SMT 구조는 비순차적 명령어 이슈 및 완료 정책을 사용하고 그룹화를 적용시키지 않았다는 점에서 그 이유를 찾을 수 있다. 그러나 비순차적 명령어 이슈 및 완료로 인한 설계의 복잡도 및 하드웨어 비용 증가, 그리고 그룹화로 인한 설계의 단순화를 고려한다면 약간의 성능 증가는 큰 장점이 되지 못한다. 실제로 이 논문에서는 SMT 구조가 슈퍼스칼라 구조와 비교했을 때에 2.5배의 IPC 향상을 보이는데, 이것은 본 논문의 4.9배의 IPC 향상보다 매우 낮은 수치이다.

Lo와 그의 동료들이 쓴 논문^[36]은 Tullsen의 논문들^{[34][35]}에서의 연구를 바탕으로 SMT 구조와 CMP 구조를 비교했다. 이 논문에서 CMP 구조는 실행 자원들을

정적으로 분할하기 때문에 모든 자원들을 스레드들이 동적으로 공유하는 SMT 구조보다 성능이 나쁘다고 제시하고 있다. 그 이유는 CMP 구조는 정적인 자원 분할로 인해 변화하는 ILP와 TLP의 양에 적응하지 못하는 단점을 가지고 있기 때문이다. 그 외에 이 논문에서는 캐쉬와 분기 예측 자원에서의 스레드간의 간섭(inter-thread interference)의 양이 스레드 수가 증가할수록 증가하지만 SMT의 성능 향상에 큰 영향을 미치지 않는다는 것을 보여주었다.

Hirata^[31]와 그의 동료들의 논문은 Tullsen의 SMT 이전에 개념적으로 매우 비슷한 구조를 제시하였다. 이 논문에서의 다중 스레딩 구조는 병렬 다중스레딩(parallel multithreading)과 공존 다중스레딩(concurrent multithreading)을 동시에 지원하는 것으로서, 단일 프로세서가 아닌 다중 프로세서의 일부분이 될 프로세서의 구조로서 제시되었다. 이 논문의 병렬 다중스레딩은 본 논문의 SMT와 비슷한 개념을 가진다. 즉 이 논문의 프로세서는 병렬 다중스레딩에 의해서 다수의 스레드로부터의 명령어들을 동시에 이슈하고 실행할 수 있다. 이 논문에서는 8개의 스레드를 지원하는 구조는 비슷한 자원의 슈퍼스칼라 구조보다 3.5배의 IPC를 보이는데, 이것은 본 논문에서의 성능 향상보다 작은 값이다. 이 논문의 구조에서 명령어 큐와 디코드 유닛은 스레드별로 존재하는데, 이것은 본 논문에서의 그룹화 2를 적용한 것과 비슷하다. 그러나 이 논문은 명령어 큐와 디코드 유닛을 스레드들이 공유하는 것에 대해서는 고려하지 않고 있다. 그리고, 이 논문에서는 캐쉬의 구성에 따른 영향을 측정하지 않았다. 즉, 모든 메모리 액세스에 대해서는 캐쉬 히트되는 것으로 가정하고서 시뮬레이션을 수행하였다. 그리고, 다양한 구조 변수에 의한 영향을 고려하지 않고, 스레드의 수와 이슈 폭에 의한 영향만을 측정하였다.

Daddis와 Torng의 논문^[101]은 비순차적 실행과 다중스레딩을 결합한 구조를 제시하였다. 이 구조에서는 비순차적 수행을 위해서 명령어들은 폐치된 후에 명령어 창(instruction window)에 일차적으로 저장되는 방법이 채택되었다. 이 논문의 구조에서 2개 이상의 스레드를 지원할 수 있지만 실험은 2개의 스레드를 지원하는 구조로 제한되었다. 그리고 이 논문에서는 기능 유닛의 수가 무한하고 데이터 캐쉬에서의 미스가 존재하지 않는다고 가정하고 있다. 이 구조는 명령어 이슈 폭이 8개이고 32-entry의 명령어 창을 사용할 때에 슈퍼스칼라 구조에 비해서 100%의

IPC 향상을 보였는데 이것은 본 논문의 성능 향상에 비해서 매우 작은 값이다.

Yamamoto와 그의 동료들에 의한 논문^[102]은 슈퍼스칼라 구조에서 다중스레딩을 지원하는 것의 효과를 평가하기 위한 분석적 모델(analytic model)을 제시하고 시뮬레이터에 의해서 그 모델을 검증하였다. 이 모델은 완벽한 메모리 시스템, 완벽한 분기 예측을 가정하고 있으며, 분기 지연 슬롯이 존재하지 않는다. 그리고, 시뮬레이션에서 각 스레드는 모두 같은 프로그램을 수행하도록 하였다. 이 모델에서 4개의 스레드를 수행할 때에 단순한 슈퍼스칼라 구조보다 3배의 IPC를 보이는데 이것은 본 논문의 수치와 비슷하다.

Gulati와 Bagherzadeh의 논문^[103]은 다중스레딩 기법을 비순차적 슈퍼스칼라 구조에 적용하는 방식을 제시하고 있다. 즉 본 논문과 같이 ILP와 TLP를 동시에 활용하는 구조를 이 논문에서는 제시하고 있으며, Tullsen의 논문과 비슷하게 비순차적 명령어 실행 방법을 사용한다. 그런데, 이 논문에서 제시한 20-55%의 IPC 향상은 본 논문에서의 IPC 향상에 비해 매우 작은 값이다.

Li와 Chu의 논문^[104]은 FPMP(Finely Parallel Multithreaded Processor) 구조를 제시하고 있는데, 이 구조는 SMT와 개념적으로 같은 것이다. 이 구조에서는 각 하드웨어 콘텍스트(hardware context)가 각각의 명령어 캐쉬를 가지고 있는데, 여기에서 콘텍스트는 스레드에 해당한다. 이 논문은 콘텍스트의 수 및 기능 유닛의 수와 같은 구조 변수들을 변화시키면서 그 효과를 측정하였으나, 본 논문에서처럼 다양한 구성 요소들의 변화에 의한 영향을 측정하지는 않았다.

Keckler와 Dally의 논문^[105]에서는 프로세서 연합(processor coupling)을 제시하였다. 프로세서 연합은 ILP와 TLP를 활용하기 위해서 다수의 ALU를 관리하는 방법이다. 이 구조에서는 ALU들은 사이클 단위로 스레드들에 할당된다. 그리고, 다수의 스레드는 동시에 활동적(active)일 수 있다. 이 논문의 결과는 프로세서 연합에 의해서 60-65% 정도의 성능 향상이 발생하는 것을 보여 준다.

Wolfe와 Shen의 논문^[106]은 XIMD 구조를 제시하였다. XIMD 구조는 VLIW 구조를 확장하여 다수의 스레드를 동시에 지원할 수 있도록 한 것이다. 이 구조에서는 컴파일러가 실행 자원들을 스레드들에 분배하고, 사이클 단위로 기능 유닛들을 스레드들에게 분할한다. 이 구조에서는 컴파일러가 실행 자원들을 제어하기 위해

서 모든 스레드들은 같은 응용 프로그램에 속하는 것이어야 한다.

Espasa와 Valero의 논문^[107]은 SMT 구조를 벡터 명령어들을 지원하기 위해 확장시킨 구조로서 SMV(Simultaneous Multithreaded Vector) 구조를 제시하였다. SMV 구조는 DLP(Data Level Parallelism)를 기존의 병렬성에 추가로 활용함으로써, 성능을 높이는 것이다.

Govindarajan과 그의 동료들의 논문^[108]에서도 SMT와 비슷한 구조를 제시하고 있다. 그러나, 이 구조에서는 명령어들의 이슈 및 실행 자원들이 스레드간에 공유되지 않고, 고정적으로 분할되어 있다. 따라서 이 구조는 ILP와 TLP의 양이 프로세서의 구조와 잘 맞지 않는 경우 낮은 성능을 보인다. 이것은 CMP 구조의 단점이기도 한 것이다. Beckmann과 Polychronopoulos의 논문^[109]도 Govindarajan의 논문처럼 실행 자원의 고정적 분할 방식을 사용하는 구조를 제시하였다.

이 장에서는 상기에서 기술한 바와 같이 SMT 또는 SMT와 비슷한 구조를 제시한 이전의 논문들을 살펴보았다. 본 논문이 이러한 이전의 논문들과 다른 점은 우선 그룹화에 대한 연구이다. SMT 구조는 다수의 스레드가 프로세서 내의 자원들을 공유함으로써 성능을 향상시키는 구조이다. 그런데 자원의 공유는 설계의 복잡도를 증가시키기 때문에 실제 구현에 있어서 많은 장애가 된다. 따라서 프로세서 자원의 공유 정도를 변화시키면서 성능에 어떠한 영향을 미치는 지에 대한 연구를 토대로 적절한 자원 공유 형식을 결정한 필요가 있다. 본 논문은 프로세서의 모든 자원들에 대해서 다양한 그룹화 방안들을 체계적으로 제시하고, 그룹화의 적용 정도에 따라서 성능에 어떠한 영향이 발생하는지를 제시하고 있다. 반면에 이전의 논문들은 이러한 그룹화가 성능에 미치는 영향에 대한 연구가 전무하거나 또는 있다고 하더라도 제한된 자원과 그룹화 방법에 대해서만 다루고 있다.

이전의 SMT에 관한 연구 논문들은 높은 성능에 중점을 두는 관계로 비순차적 명령어 실행 방법을 사용한 구조에 대해서만 연구하고 있다. 그러나, SMT의 경우에는 TLP를 이용해서 ILP의 부족을 극복할 수 있기 때문에 비순차적 명령어 이슈 및 완료의 아니더라도 높은 성능을 보일 수가 있다. 본 논문은 순차적 이슈 및 완료 방법을 사용하더라도 이전의 논문에서 보여준 비순차적 SMT 구조와 비교하였을 때 비슷한 또는 더 높은 성능을 달성할 수 있음을 보여준다.

SMT 구조는 다수의 스레드가 동시에 수행되는 구조이기 때문에 프로세서의 각 구성 요소들의 변화에 따른 성능 변화가 기존의 슈퍼스칼라 구조와는 다른 방식으로 이루어진다. 본 논문은 프로세서의 거의 모든 자원에 대해서 그 구성의 변화에 따른 영향을 측정하고 있다. 이러한 연구는 실제 구현에 있어서 목표하는 성능에 적절한 SMT 구조를 정하는데 많은 도움이 된다. 반면에 이전의 논문들은 이러한 구성 요소들의 변화에 따른 성능 변화를 종합적으로 제시하지 못하고 있다. 대부분의 경우에 기능 유닛의 수, 이슈 폭 및 스레드 수에 따른 성능 변화만을 제시할 뿐이며, Tullsen 및 그의 동료들이 쓴 논문에서만 비교적 많은 구성 요소들의 영향을 다루고 있지만, 본 논문과 같이 포괄적이지 못하다.

표 7-1은 상기 설명한 기존의 SMT 구조들과 본 논문에서 제안된 두 개의 SMT 구조들을 비교한 것이다. 표 7-1에 나와 있듯이, 순차적 이슈 방법을 사용한 본 논문의 SMT 구조들은 기존의 SMT 구조에 비해 슈퍼스칼라 구조 대비 성능 향상 비율이 높게 나오고 있다. 이것으로부터 SMT 구조에서는 비순차적 이슈보다는 순차적 이슈를 사용하였을 때 더 많은 이득을 얻을 수 있고, 비용과 성능을 고려했을 때 순차적 SMT 구조가 더 효율적임을 알 수 있다.

표 7-1 기존의 SMT 구조들과 본 논문의 제안된 SMT 구조들의 비교

Table 7-1 Comparison of the previous SMT architectures and the proposed SMT architectures in this dissertation

비교 사항	Tullsen ^[35]	Hirata ^[31]	Daddis ^[101]	Yamamoto ^[102]	내장형 (본 논문)	고성능 (본 논문)
이슈 방법	비순차적	비순차적	비순차적	비순차적	순차적	순차적
이슈 폭	8	8	8	-	4	8
스레드 수	8	8	2	4	4	8
슈퍼스칼라 대비 IPC 향상	2.5배	3.5배	2배	3배	3.1배	4.9배

제 8 장 결 론

본 논문에서는 단일 사이클에 다수의 스레드로부터의 명령어들을 이슈하고 실행할 수 있는 SMT 구조를 연구하고 그 결과를 바탕으로 최적의 SMT 구조를 제안하였다.

SMT 구조에서는 TLP를 이용하여 각 스레드의 ILP 부족을 보충할 수 있기 때문에 비순차적인 명령어 이슈 및 완료 정책은 사용할 필요가 없다. 이런 이유로 제안된 SMT 구조는 순차적으로 명령어를 이슈하고 실행하는 구조를 가지며, 이는 명령어의 이슈를 간단히 하고, 분기 예측 오류 또는 예외 시에 프로세서 상태의 복구를 간략하게 한다. SMT 구조는 순차적 명령어 이슈 및 완료 정책을 사용하는 슈퍼스칼라 구조를 바탕으로 약간의 변경에 의해서 제시되었다.

SMT 구조의 파이프라인은 S, F, D, I, R, E, M 및 W의 8 단계로 구성되는데, 이 중에서 S 단계는 슈퍼스칼라 구조에는 없는 SMT 특유의 단계이다. S 단계에서는 다음 사이클에 명령어를 폐치할 스레드들을 선택하고, F 단계에서는 S 단계에서 선택된 스레드들로부터 명령어 폐치를 수행하여 명령어 폐치 큐에 저장한다. D 단계에서는 명령어들을 디코드하여 명령어 이슈 큐에 저장하고, I 단계에서는 명령어 이슈 큐의 명령어들을 기능 유닛들로 이슈한다. 기능 유닛들은 R 단계부터 W 단계까지 각 명령어의 주어진 동작을 수행한다.

명령어 폐치와 관련된 유닛들로는 명령어 캐쉬, IMMU, 명령어 폐치 큐 및 폐치 유닛이 있다. 명령어 캐쉬는 캐쉬 미스가 발생하였을 때 다른 스레드로부터 명령어 폐치를 계속할 수 있는 non-blocking 캐쉬이다. 명령어 캐쉬, IMMU 및 BTB는 같은 수의 액세스 포트를 가지며, 단일 사이클에 명령어들을 폐치할 수 있는 스레드의 수가 여러 개인 경우 다수의 포트들을 가지고 있다. 폐치 유닛은 명령어 폐치 주소 값을 가지고 있는 PC들, 스레드 선택기 및 BTB로 구성된다. 이 중에서 PC는 스레드당 한 개씩 존재하며, 스레드 선택기는 폐치 우선권 정책을 사용하여 폐치할 스레드를 선택하는 기능을 수행하는 것으로서 SMT 구조 특유의

유닛이다. 명령어 페치 큐는 최상위 포인터와 최하위 포인터를 사용한 선입선출 형식을 취한다.

SMT 구조의 디코드 유닛은 수퍼스칼라 구조의 것과 차이가 없다. 이슈 유닛은 명령어 이슈 큐의 명령어들을 기능 유닛들로 이슈하는데, 명령어간의 의존성 검사는 같은 스레드에 속하는 명령어들 사이에서만 이루어진다. 그리고 SMT 구조의 명령어 이슈는 수퍼스칼라 구조와 달리 어떤 명령어들을 우선적으로 이슈할지를 결정하기 위한 이슈 우선권 정책이 필요하다. 서로 다른 스레드에 속하는 명령어들에 대해서는 명령어 이슈 큐의 순서와 상관없이 이슈하기 때문에, 명령어 이슈 큐의 중간 엔트리들이 비어 있게 되는 현상이 발생한다. 이러한 빈 엔트리들을 없애기 위해서 명령어 이슈 큐에 대한 압축이 명령어 이슈와 동시에 이루어진다.

SMT 구조의 레지스터 파일은 스레드 수만큼의 레지스터 세트를 가지기 때문에 수퍼스칼라 구조와 비교해서 레지스터 파일의 크기가 스레드 수만큼 배가된다. 스코어보드 어레이는 레지스터당 한 개의 엔트리를 가지기 때문에, SMT 구조의 스코어보드 어레이 크기도 수퍼스칼라 구조와 비교해서 스레드 수만큼 배가된다. 레지스터 파일과 스코어보드 어레이에 있어서의 이러한 크기 증가는 SMT 프로세서의 면적 증가에 상당한 영향을 미친다. SMT 구조의 기능 유닛들은 수퍼스칼라 구조와 큰 차이가 없으며, 다만 기능 유닛들의 각 파이프라인 버퍼들이 해당 명령어의 스레드 식별자를 가지고 있어야 한다는 차이점이 존재한다. 그리고 SMT 구조에서 분기 예측 오류 및 예외가 발생할 경우에는 해당 스레드의 명령어들만을 플러쉬해야 하기 때문에 스레드별 명령어 플러쉬 기능이 필요하다. 분기 명령어의 실행에 사용되는 BHB는 스레드별로 한 개씩 존재한다.

SMT 구조는 기본적으로 모든 스레드들이 프로세서의 자원을 공유함으로써, ILP뿐 아니라 TLP를 충분히 활용하는 구조이다. 그러나 이러한 공유 철학은 SMT의 설계를 어렵게 만드는 요인이기도 하다. CMP 구조는 프로세서 자원을 스레드간에 정적으로 배분하는 이유로 SMT에 비해 프로세서의 활용도 및 성능이 낮지만 CMP 안의 각 프로세서 및 전체 칩의 설계가 SMT 구조에 비해 간단하다. 본 논문은 SMT와 CMP의 이러한 장단점을 고려하여 SMT 구조의 설계 복잡도

를 줄이면서 SMT 구조의 성능을 유지하기 위한 방법으로서 그룹화를 제안하였다. 설계 복잡도를 줄이기 위한 그룹화는 세 가지 종류로 분류된다. 그룹화 1은 명령어 캐쉬, ITLB 및 BTB의 포트와 같이 명령어 폐치를 위한 자원들에 적용된 것이다. 그룹화 2는 명령어 폐치 큐, 디코드 슬롯 및 명령어 이슈 큐에 대한 것이다. 마지막으로 그룹화 3은 명령어 이슈 로직, 레지스터 파일 포트 및 기능 유닛과 같은 명령어 이슈 및 실행 자원에 관한 것이다. 이러한 그룹화 방안의 적용은 약간의 성능 저하를 대가로 설계의 복잡도를 상당히 줄일 수 있다. SMT 구조의 설계가 매우 복잡하다는 것을 고려하면 제안된 그룹화는 SMT 프로세서의 설계에서 반드시 적용되어야 할 사항이다.

제시된 SMT 구조는 시뮬레이터로 모델링되었고, 구조 변수들을 변화시키면서 다양한 시뮬레이션이 수행되었다. 시뮬레이션 결과는 명령어 이슈 폭만큼의 스레드 수가 적당하다는 것을 보여준다. 그리고 스레드간의 충돌 미스를 최소화하기 위해 캐쉬의 way 수는 스레드 수 또는 스레드 수의 반으로 설정되는 것이 좋다. 그룹화와 관련해서는 그룹화 1과 그룹화 2를 최대한 적용경우 성능은 유지되면서 설계는 상당히 간단해진다. 그룹화 3의 적용은 설계의 간략화에 좋지만 성능 저하가 많이 일어나기 때문에 적당한 수준에서 이루어지는 것이 좋다. 전체적으로 그룹화의 적용은 설계를 복잡도를 상당히 줄이면서 SMT 본래의 성능을 유지하는 것을 시뮬레이션 결과 알 수 있다.

시뮬레이션 결과를 토대로 내장형 SMT 구조로서 4-issue, 4-thread 구조를 제안하였는데, 이 구조의 프로세서는 수퍼스칼라 구조와 비교해서 208%의 IPC 향상을 보였다. 그리고 고성능 SMT 구조로서 8-issue, 8-thread 구조를 제안하였는데, 이것은 수퍼스칼라 구조와 비교해서 389%의 IPC 향상을 보였다. 이러한 결과는 본 논문의 SMT 구조가 비교적 적은 비용 증가로 많은 성능 향상을 이루어낸다는 것을 보여준다. SMT 구조는 그 특성상 네트워크 프로세서의 코어에 적용되었을 경우 높은 성능을 발휘한다. 이러한 이유로 본 논문의 SMT 구조는 네트워크 프로세서로의 응용을 목적으로 한다. 제안된 고성능 SMT 구조를 적용한 네트워크 프로세서는 11.6 Gbps의 데이터 처리 능력을 가질 것으로 예상되는데, 이 값은 현재 출시되고 있는 네트워크 프로세서들과 비교해서 상당히 높은 값이다. 이러한

결과는 제안된 SMT 구조를 적용한 네트워크 프로세서가 충분한 경쟁력을 가지고 있는 것을 보여준다.

참 고 문 헌

- [1] Pierre G. Paulin, Faraydon Karim and Paul Bromley, "Network Processors: A Perspective on Market Requirements, Processor Architectures and Embedded S/W Tools," *Proceedings of Design, Automation and Test in Europe, Conference and Exhibition 2001*, pp. 430-427, Munich, Germany, March 2001.
- [2] Tom R. Halfhill, "Intel Network Processor Targets Routers," *Microprocessor Report*, Vol. 13, No. 12, September 13, 1999.
- [3] Tom R. Halfhill, "Embedded Market Breaks New Ground," *Microprocessor Report*, Vol. 14, Archive 1, January 2000.
- [4] Tom R. Halfhill, "Sitera Samples Its First NPU," *Microprocessor Report*, Vol. 14, Archive 5, May 2000.
- [5] Kevin Krewell, "Agere's Pipelined Dream Chip," *Microprocessor Report*, Vol. 14, Archive 6, June 2000.
- [6] Tom R. Halfhill, "SiByte Reveals 64-Bit Core for NPUs," *Microprocessor Report*, Vol. 14, Archive 6, June 2000.
- [7] Linley Gwennap, "Cisco Rolls Its Own NPU," *Microprocessor Report*, Vol. 14, Archive 11, November 2000.
- [8] Peter N. Glaskowsky, "Networking Gets XStream," *Microprocessor Report*, Vol. 14, Archive 11, November 2000.
- [9] Kevin Krewell, "Rainier Leads PowerNP Family," *Microprocessor Report*, Vol. 15, Archive 1, January 2001.
- [10] Amir Eyal, "Challenges in Designing 40GB Network Processors," *Microprocessor Report*, Vol. 15, Archive 6, June 2001.
- [11] Peter N. Glaskowsky, "Lexra Readies Networking Chip," *Microprocessor*

- Report*, Vol. 15, Archive 6, June 2001.
- [12] Keith Diefendorff, "Compaq Chooses SMT for Alpha," *Microprocessor Report*, Vol. 13, No. 16, December 6, 1999.
- [13] Mike Johnson, *Superscalar Microprocessor Design*, Prentice Hall, pp. 1-163, 1991.
- [14] Moon Key Lee, Byeong Yoon Choi, Kwang Yub Lee, and Seong Ho Lee, "Data-stationary controller for 32-bit application-specific RISC," *Proceedings of 1993 IEEE International Symposium on Circuits and Systems*, Vol. 3, pp. 1933-1936, Chicago, Illinois, May 1993.
- [15] 이용석, "고성능 마이크로프로세서 구조의 개요," 비디오 및 온라인 강좌 시리즈, 2000, <http://mpu.yonsei.ac.kr/Lecture/nrl/Overview.htm>.
- [16] Manolis G. H. Katevenis, *Reduced Instruction Set Computer Architectures for VLSI*, The MIT Press, 1984.
- [17] K. C. Yeager, "The MIPS R10000 Superscalar Microprocessor," *IEEE Micro*, Vol. 16, No. 2, pp. 28-41, April 1996.
- [18] 문병인, 이용환, 안상준, 이용석, "명령어 버퍼를 이용한 최적화된 슈퍼스칼라 명령어 이슈 구조," *전자공학회논문지*, 제34권, C편, 제9호, pp. 43-52, 1997년 9월.
- [19] 문상국, 문병인, 이용환, 이용석, "64비트 4-way 슈퍼스칼라 마이크로프로세서의 효율적인 분기 예측을 수행하는 프리페치 구조," *한국통신학회논문지*, 제25권, 제11B호, pp. 1939-1947, 2000년 11월.
- [20] Alexander aiken, Alexandru Nicolau, "A Development Environment for Horizontal Microcode," *IEEE Transactions on Software Engineering*, Vol. 14, No. 5, pp. 584-594, May 1988.
- [21] A. Abnous and N. Bagherzadeh, "Special Features of a VLIW Architecture," *Proceedings of 5th International Parallel Processing Symposium*, pp. 224-227, Anaheim, California, April 1991.
- [22] Lynn Choi and Andrew A. Chien, "Integrating Networks and Memory

- Hierarchies in a Multicomputer Node Architecture,” *Proceedings of 8th International Parallel Processing Symposium*, pp. 10–17, Cancún, Mexico, April 1994.
- [23] R. M. Tomasulo, “An Efficient Algorithm for Exploiting Multiple Arithmetic Units,” *IBM Journal*, Vol. 11, pp. 25–33, January 1967.
- [24] S. Weiss and J. E. Smith, “Instruction Issue Logic in Pipelined Supercomputers,” *IEEE Transactions on Computers*, Vol. C-33, pp. 1013–1022, November 1984.
- [25] R. D. Acosta, J. Kjelstrup, H. C. Torng, “An Instruction Issuing Approach to Enhancing Performance in Multiple Functional Units Processors,” *IEEE Transactions on Computers*, Vol. 35, No. 9 pp. 815–828, September 1986.
- [26] Gurindar S. Sohi, “Instruction Issue Logic for High-Performance, Interruptible, Multiple Functional Unit, Pipelined Computers,” *IEEE Transactions on Computers*, Vol. 39, No. 3, pp. 349–359, March 1990.
- [27] John L. Hennessy and David A. Patterson, *Computer Architecture: A Quantitative Approach*, Second Edition, Morgan Kaufmann, pp. 221–240, 1996.
- [28] Norman P. Jouppi and David W. Wall, “Available Instruction-Level Parallelism for Superscalar and Superpipelined Machines,” *Proceedings of 3rd International Conference on ASPLOS*, pp. 272–282, 1989.
- [29] Michael D. Smith, Mike Johnson, and Mark A. Horowitz, “Limits on Multiple Instruction Issue,” *Proceedings of 3rd International Conference on ASPLOS*, pp. 290–302, 1989.
- [30] Anant Agarwal, “Performance Tradeoffs in Multithreaded Processors,” *IEEE Transactions on Parallel and Distributed Systems*, Vol. 3, No. 5, pp. 525–539, September 1992.
- [31] H. Hirata, K. Kimura, S. Nagamine, Y. Mochizuki, A. Nishimura, Y.

- Nakase, and T. Nishizawa, "An Elementary Processor Architecture With Simultaneous Instruction Issuing from Multiple Threads," *Proceedings of 19th Annual International Symposium on Computer Architecture*, pp. 136-145, Queensland, Australia, May 1992.
- [32] Peter Song, "Multithreading Comes of Age," *Microprocessor Report*, Vol. 11, No. 9, July 14, 1997.
- [33] Kenji Watanabe, Wanming Chu, and Yamin Li, "Exploiting Java Instruction/Thread Level Parallelism with Horizontal Multithreading," *Proceedings of 6th Computer Systems Architecture Conference*, pp. 122-129, Queensland, Australia, January 2001.
- [34] Dean M. Tullsen, Susan J. Eggers, and Henry M. Levy, "Simultaneous Multithreading: Maximizing On-Chip Parallelism," *Proceedings of 22nd Annual International Symposium on Computer Architecture*, pp. 392-403, Santa Margherita Ligure, Italy, May 1995.
- [35] Dean M. Tullsen, Susan J. Eggers, Joel S. Emer, and Henry M. Levy, "Exploiting Choice: Instruction Fetch and Issue on an Implementable Simultaneous Multithreading Processor," *Proceedings of 23rd Annual International Symposium on Computer Architecture*, pp. 191-202, Philadelphia, Pennsylvania, May 1996.
- [36] Jack L. Lo, Susan J. Eggers, Joel S. Emer, Henry, M. Levy, Rebecca L. Stamm, and Dean M. Tullsen, "Converting Thread-Level Parallelism to Instruction-Level Parallelism via Simultaneous Multithreading," *ACM Transactions on Computer Systems*, Vol, 15, No. 3, pp. 322-354, August 1997.
- [37] Susan J. Eggers, Joel S. Emer, Henry, M. Levy, Jack L. Lo, Rebecca L. Stamm, and Dean M. Tullsen, , "Simultaneous Multithreading: A Platform for Next-Generation Processors," *IEEE Micro*, Vol, 17, No. 5, pp. 12-19, September-October 1997.

- [38] John L. Hennessy and David A. Patterson, *Computer Architecture: A Quantitative Approach*, Second Edition, Morgan Kaufmann, pp. 373-474, 1996.
- [39] Keith Diefendorff, "Power4 Focuses on Memory Bandwidth," *Microprocessor Report*, Vol. 13, No. 13, October 6, 1999.
- [40] Brian Case, "Sun Makes MAJC with Mirrors," *Microprocessor Report*, Vol. 13, No. 14, October 25, 1999.
- [41] *ARM Architecture Reference Manual*, ARM Limited, 2000.
- [42] Steve Furber, *ARM System Architecture*, Addison Wesley Longman, 1996.
- [43] Alex van Someren and Carol Atack, *The ARM RISC Chip: A Programmer's Guide*, The University Press, Cambridge, 1993.
- [44] Dan Picker and Ronald D. Fellman, "A VLSI Priority Packet Queue with Inheritance and Overwrite," *IEEE Transactions on VLSI Systems*, Vol. 3, No. 2, pp. 245-253, June 1995.
- [45] S.-H. Gary Chan and Hisashi Kobayashi, "Packet Scheduling Algorithms and Performance of Buffered Shufflenet with Deflection Routing," *Journal of Lightwave Technology*, Vol. 18, No. 4, pp. 490-501, April 2000.
- [46] Sung-Whan Moon, Jennifer Rexford, and Kang G. Shin, "Scalable Hardware Priority Queue Architectures for High-Speed Packet Switches," *IEEE Transactions on Computers*, Vol. 49, No. 11, pp. 1215-1227, November 2000.
- [47] Nen-Fu Huang and Shi-Ming Zhao, "A Novel IP-Routing Scheme and Hardware Architecture for Multigigabit Switching Routers," *IEEE Journal on Selected Areas in Communications*, Vol. 17, No. 6, pp. 1093-1104, June 1999.
- [48] Butler Lampson, Venkatachary Srinivasan, and George Varghese, "IP Lookups Using Multiway and Muticolumn Search," *IEEE/ACM Transactions on Networking*, Vol. 7, No. 3, pp. 325-334, June 1999.

- [49] Pi-Chung Wang, Chia-Tai Chan, and Yaw-Chung Chen, "Performance Enhancement of IP Forwarding by Reducing Routing Table Construction Time," *IEEE Communications Letters*, Vol. 5, No. 5, pp. 230-232, May 2001.
- [50] Jan M. Rabaey, Miodrag Potkonjak, Farinaz Koushanfar, Suet-Fei Li, and Tim Tuan, "Challenges and Opportunities in Broadband and Wireless Communication Designs," *Proceedings of 2000 IEEE/ACM International Conference on Computer Aided Design*, pp. 76-82, San Jose, California, November 2000.
- [51] Tilman Wolf and Jonathan S. Turner, "Design Issues for High-Performance Active Routers," *IEEE Journal on Selected Areas in Communications*, Vol. 19, No. 3, pp. 404-409, March 2001.
- [52] Hideyuki Shimonishi and Tutomu Murase, "A Network Processor Architecture for Flexible QoS Control in Very High-Speed Line Interfaces," *Proceedings of 2001 IEEE Workshop on High Performance Switching and Routing*, pp. 402-406, Dallas, Texas, May 2001.
- [53] Patrick Crowley, Marc E. Fiuczynski, Jean-Loup Baer, and Brian N. Bershad, "Characterizing Processor Architecture for Programmable Network Interfaces," *Proceedings of 2000 International Conference on Supercomputing*, pp. 54-65, Santa Fe, New Mexico, May 2000.
- [54] Kemathat Vibhatavanij, Nian-Feng Tzeng, and Angkul Kongmunvattana, "Simultaneous Multithreading-Based Routers," *Proceedings of 2000 International Conference on Parallel Processing*, pp. 362-369, Toronto, Canada, August 2000.
- [55] 이용석, "고성능 마이크로프로세서 캐쉬 메모리의 구조," 비디오 및 온라인 강좌 시리즈, 2000, <http://mpu.yonsei.ac.kr/Lecture/nrl/Cache.htm>.
- [56] J. K. F. Lee and A. J. Smith, "Branch Prediction Strategies and Branch Target Buffer Design," *IEEE Computer Magazine*, Vol. 17, pp. 6-22,

January 1984.

- [57] C. H. Perleberg and A. J. Smith, "Branch Target Buffer Design and Optimization," *IEEE Transactions on Computers*, Vol, 42, No. 4, pp. 396-412, April 1993.
- [58] 이용석, "고성능 마이크로프로세서 분기명령어의 수행방법," 비디오 및 온라인 강좌 시리즈, 2000, <http://mpu.yonsei.ac.kr/Lecture/nrl/Branch.htm>.
- [59] James E. Smith, "A Study of Branch Prediction Strategies," *Proceedings of 8th Annual International Symposium on Computer Architecture*, pp. 135-148, May 1981.
- [60] Yong-Hwan Lee, Woo-Kyeong Jeong, Sang-Jun An, and Yong-Surk Lee, "An On-Chip Multiprocessor Microprocessor with Shared MMU and Cache," *Journal of Electrical Engineering and Information Science*, Vol. 2, No. 4, pp. 1-7, 1997.
- [61] Yong Hwan Lee, Woo Kyeong Jeong, and Yong Surk Lee, "Unified Tag Memory Architecture with Snoop Support," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, Vol. E81-A, No. 6, pp. 1172-1175, June 1998.
- [62] 이용석, "고성능 마이크로프로세서 ALU와 레지스터 파일의 구조," 비디오 및 온라인 강좌 시리즈, 2000, <http://mpu.yonsei.ac.kr/Lecture/nrl/ALURegisterFile.htm>.
- [63] 이용석, "고성능 마이크로프로세서 곱셈기의 구조," 비디오 및 온라인 강좌 시리즈, 2000, <http://mpu.yonsei.ac.kr/Lecture/nrl/multiplier.htm>.
- [64] Byung In Moon, Dong Ryul Ryu, Jong Wook Hong, Tae Young Lee, Sangook Moon, and Yong Surk Lee, "A 32-bit RISC Microprocessor with DSP Functionality: Rapid Prototyping," *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, Vol. E84-A, No. 5, pp. 1339-1347, May 2001.
- [65] 이용석, "고성능 마이크로프로세서 부동 소수점 연산기의 구조," 비디오 및

- 온라인 강좌 시리즈, 2000, <http://mpu.vonsei.ac.kr/Lecture/nrl/fpu.htm>.
- [66] 이용석, “고성능 마이크로프로세서 부동 소수점 연산기의 구조(2),” 비디오 및 온라인 강좌 시리즈, 2000, <http://mpu.vonsei.ac.kr/Lecture/nrl/FPU2.htm>.
- [67] Jong Keun Ahn, Byung In Moon, Sang Kook Moon, Yong Hwan Lee, and Yong Surk Lee, “VLSI Design of the Control Unit of a 32-bit RISC Integer Unit,” *Proceedings of ITC-CSCC*, pp. 330-333, 1996.
- [68] James E. Smith and Adrew R. Pleszkun, “Implementing Precise Interrupts in Pipelined Processors,” *IEEE Transactions on Computers*, Vol. 37, No. 5, pp. 562-573, May 1988.
- [69] Mayan Moudgill and Stamtis Vassiliadis, “Precise Interrupts,” *IEEE Micro*, Vol. 16, No. 1, pp. 58-67, February 1996.
- [70] John L. Hennessy and David A. Patterson, *Computer Architecture: A Quantitative Approach*, Second Edition, Morgan Kaufmann, pp. 262-278, 1996.
- [71] Michael J. Flynn, *Computer Architecture: Pipelined and Parallel Processor Design*, Jones and Bartlett Publishers, pp. 328-329, 1995.
- [72] Keith I. Farkas and Norman P. Jouppi, “Complexity/Performance Tradeoffs with Non-Blocking Loads,” *Proceedings of 21st Annual International Symposium on Computer Architecture*, pp. 211-222, Chicago Illinois, April 1994.
- [73] G. S. Sohi and M. Flanklin, “High-Bandwidth Interleaved Memories for Vector Processors-A Simulation Study,” *IEEE Transactions on Computers*, Vol. 42, No.1, pp. 34-44, January 1993.
- [74] Bryan Black and John Paul Shen, “Calibration of Microprocessor Performance Models,” *IEEE Computer*, Vol. 31, No. 5, pp. 59-65, May 1998.
- [75] A. Borg and E. E. Kessler, and D. W. Wall, “Generation and Analysis of Very Long Address Traces,” *Proceedings of 17th Annual International*

- Symposium on Computer Architecture*, pp. 270–279, 1990.
- [76] Moon Gyung Kim, Byung In Moon, Sang Jun An, Dong Ryul Ryu, and Yong Surk Lee, “Implementation of a Cycle-Based Simulator for the Design of a Processor Core,” *Proceedings of 1st IEEE Asia Pacific Conference on ASICs*, pp. 108–111, Seoul, Korea, August 1999.
 - [77] Donald E. Thomas and Philip R. Moorby, *The Verilog Hardware Description Language*, Kluwer Academic Publishers, 1991.
 - [78] Douglas J. Smith, *HDL Chip Design: A Practical Guide for Designing, Synthesizing and Simulating ASICs and FPGAs using VHDL or Verilog*, Doone Publications, 1996.
 - [79] Al Kelley and Ira Pohl, *A Book on C: Programming in C*, Second Edition, The Benjamin/Cummings Publishing Company, Inc., 1989.
 - [80] David J. Kruglinski, *Inside Visual C++*, Fourth Edition, Microsoft Press, 1997.
 - [81] Kevin Krewell, “SPEC CPU2000 Released,” *Microprocessor Report*, Vol. 14, Archive 4, April 2000.
 - [82] J .L. Henning, “SPEC CPU2000: Measuring CPU Performance in the New Millennium,” *IEEE Computer*, Vol. 33, No. 7, pp. 28–35, July 2000.
 - [83] *ARM Developer Suite: Developer Guide*, ARM Limited, 2000.
 - [84] *ARM Developer Suite: CodeWarrior IDE Guide*, ARM Limited, 2000.
 - [85] *ARM Developer Suite: Compiler, Linker, and Utilities Guide*, ARM Limited, 2000.
 - [86] *ARM Developer Suite: Assembler Guide*, ARM Limited, 2000.
 - [87] *ARM Developer Suite: Debuggers Guide*, ARM Limited, 2000.
 - [88] *ARM Developer Suite: Debug Target Guide*, ARM Limited, 2000.
 - [89] Dean M. Tullsen, Jack L. Lo, Susan J. Eggers, and Henry M. Levy, “Supporting Fine-Grained Synchronization on a Simultaneous Multithreading Processor,” *Proceedings of 5th International Symposium on*

- High-Performance Computer Architecture*, pp. 54-58, Orlando, Florida, January 1999.
- [90] William Stallings, *Operating Systems: Internals and Design Principles*, Third Edition, Prentice Hall, pp. 47-49, 1998.
- [91] *ARM ELF*, ARM Limited, 1999.
- [92] The file “readme.txt” for the Dhrystone benchmark program in ARM Developer Suite, ARM Limited, 2000.
- [93] Ron Cates, “Processor Architecture Considerations for Embedded Controller Applications,” *IEEE Micro*, Vol. 8, No. 3, pp. 28-38, June 1988.
- [94] Kazumasa Suzuki, Tomohisa Arai, Kouhei Nadehara, and Ichiro Kuroda, “V830R/AV: Embedded Multimedia Superscalar RISC Processor,” *IEEE Micro*, Vol. 18, No. 2, pp. 36-37, March-April 1998.
- [95] Richard F. Hobson and P. S. Wong, “A Parallel Embedded-Processor Architecture for ATM Reassembly,” *IEEE/ACM Transactions on Networking*, Vol. 7, No. 1, pp. 23-37, February 1999.
- [96] Tom R. Halfhill, “Intel Embeds Coppermine,” *Microprocessor Report*, Vol. 14, Archive 6, June 2000.
- [97] 박재민, *SMT 방식 네트워크 프로세서의 패킷 스케줄러 설계*, 연세대학교 대학원 전기전자공학과, 공학석사학위논문, 2001년 12월 예정.
- [98] “Network Processor Designs for Next-Generation Networking Equipment,” White Paper of EZchip Technologies, http://www.ezchip.com/html/tech_nsppaper.html.
- [99] Peter N. Glaskowsky, “Pentium 4 (Partially) Previewed,” *Microprocessor Report*, Vol. 14, Archive 8, August 2000.
- [100] “Performance Routing Engine,” Documentation of Cisco Systems, http://www.cisco.com/univercd/cc/td/doc/pcat/prte__il.htm.
- [101] G. Daddis, Jr. and H. Torng, “The Concurrent Execution of Multiple Instruction Streams on Superscalar Processors,” *Proceedings of*

- International Conference on Parallel Processing*, Vol. I, pp. 76-83, August 1991.
- [102] Wayne Yamamoto, Mauricio J. Serrano, Adam R. Talcott, Roger C. Wood, and Mario Nemirovsky, "Performance Estimation of Multistreamed, Superscalar Processors," *Proceedings of 27th Annual Hawaii International Conference on System Sciences*, Vol. I, pp. 195-204, January 1994.
- [103] Manu Gulati and Nader Bagherzadeh, "Performance Study of a Mutithreaded Superscalar Microprocessor," *Proceedings of 2nd International Symposium on High-Performance Computer Architecture*, pp. 291-301, San Jose, California, February 1996.
- [104] Yamin Li and Wanming Chu, "The Effects of STEF in Finely Parallel Mutithreaded Processors," *Proceedings of 1st International Symposium on High-Performance Computer Architecture*, pp. 318-325, Raleigh, North Carolina, January 1995.
- [105] Stephen W. Keckler and William J. Dally, "Processor Coupling: Integrating Compile Time and Runtime Scheduling for Parallelism," *Proceedings of 19th Annual International Symposium on Computer Architecture*, pp. 202-213, Queensland, Australia, May 1992.
- [106] Andrew Wolfe and John P. Shen, "A Variable Instruction Stream Extension to the VLIW Architecture," *Proceedings of 4th International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 2-14, April 1991.
- [107] Roger Espasa and Mateo Valero, "Exploiting Instruction- and Data-Level Parallelism," *IEEE Micro*, Vol. 17, No. 5, pp. 20-27, October 1997.
- [108] R. Govindarajan, S. S. Nemawarkar, and Philip LeNir, "Design and Performance Evaluation of a Mutithreaded Architecture," *Proceedings of 1st International Symposium on High-Performance Computer Architecture*,

pp. 298-307, Raleigh, North Carolina, January 1995.

- [109] Carl J. Beckmann and Constantine D. Polychronopoulos,
“Microarchitecture Support for Dynamic Scheduling of Acyclic Task
Graphs,” *Proceedings of 25th International Symposium on
Microarchitecture*, pp. 140-148, Portland, Oregon, December 1992.

부 록

부록 1. 시뮬레이터의 main 함수

부록 2. 구조 변수들을 정의하는 헤더 파일의 내용

부록 3. 트레이스 파일 내용의 일부분

부록 4. 결과 파일의 예

부록 1 시뮬레이터의 main 함수

```
void main()
{
    THREAD_TYPE i, j, k;
    BOOL sim_enable;
    char process_name[80];
    FILE *ftemp;

    sim_enable = TRUE;
    initialize(); // make and initialize system components, and load programs
    /* flush itlb, dtlb, icache, dcache */
    flush_tlb(TRUE,255);
    flush_tlb(FALSE,255);
    flush_cache(ichache,255);
    flush_cache(dcache,255);

    /* initialize busy cycles for external memory access resources */
    bus_busy_cycle = 0;
    ic_repl_port_busy = 0;
    dc_repl_port_busy = 0;

    printf("Simulation start:\n");

    /* simulator body */
    while(sim_enable)
    {
        if(sim_cycle == TERMINATE_SIM_CYCLE)
        {
            break;
        }
        if(sim_cycle == TRACE_START_CYCLE)
        {
            trace_enable = TRACE_ENABLE;
        }
        else if(sim_cycle == TRACE_END_CYCLE)
        {
            trace_enable = FALSE;
        }
        if(sim_cycle == CHECK_POINT)
        {
            sim_cycle++;
            sim_cycle--;
        }
        system_operation(); // system management
        writeback(); // W-stage operation
        ext_memory(); // EM-stage operation
        memory(); // M-stage operation
        execute(); // E-stage operation
        reg_read(); // R-stage operation
        forder_switch(); // switch FU order lists
    }
}
```

```

issue();          // I-stage operation
decode();         // D-stage operation
fetch();         // F-stage operation
fetch_sel(FETCH_PRI); // S-stage operation
if(trace_enable)
{
    report_state(); // output trace data each cycle
}
sim_cycle++;
clear_ex_cnt();
clear_fetch_cnt();
/* check process finishes and start new processes */
for(i=0,j=0,sim_enable=FALSE;i<THREAD_NUM;i++)
{
    if(thread_active[i])
    {
        sim_enable |= TRUE;
        j++;
    }
    else
    {
        if(current_process[i] != 255)
        {
            waiting_process[i][current_process[i]] = FALSE;
            left_process_num[i] = (left_process_num[i] > 0) ? left_process_num[i]
- 1 : 0;
            fprintf(gen_trace_file,"Cycle%10llu:",sim_cycle);
            fprintf(gen_trace_file,"Process %s%02d.axf is
finished.\n",filename[current_process[i]],i);
        }
        while(left_process_num[i])
        {
            k = rand() % PROCESS_NUM;
            if(waiting_process[i][k])
            {
                sprintf(process_name,"%s%02d.axf",filename[k],i);
                if(ftemp = fopen(process_name,"rb"))
                {
                    fclose(ftemp);
                    get_elf_information(process_name,i);
                    load_elf_file(i);
                    set_fpc(i,prog_start_point);
                    set_cpsr(i,0x00000010);
                    flush_btb(i);
                    flush_tlb(TRUE,i);
                    flush_tlb(FALSE,i);
                    flush_cache(icache,i);
                    flush_cache(dcache,i);
                    thread_active[i] = TRUE;
                    current_process[i] = k;
                    sim_enable |= TRUE;
                    j++;
                    fprintf(gen_trace_file,"Cycle%10llu:",sim_cycle);
                    fprintf(gen_trace_file,"Process %s%02d.axf is
started.\n",filename[current_process[i]],i);

```



```

        if (TRACE_ENABLE)
        {
            report_tlb_master();
        }
        break;
    }
    else
    {
        waiting_process[i][k] = FALSE;
        left_process_num[i] = (left_process_num[i] > 0) ?
left_process_num[i] - 1 : 0;
        fprintf(gen_trace_file, "Cycle%10llu:", sim_cycle);
        fprintf(gen_trace_file, "Process %s%02d.axf is not
found.\n", filename[k], i);
    }
}
}
}
}
if (j == THREAD_NUM - 1)
{
    final_report(FALSE);
}
}
final_report(TRUE);    // output performance data to the result file
fcloseall();
printf("Simulation is terminated in %llu cycles.\n", sim_cycle);
}

```

부록 2 구조 변수들을 정의하는 헤더 파일의 내용

```
#ifndef OPTION_H
#define OPTION_H

#include <stdio.h>
#include <stdlib.h>

/* Fetch priority options
   ICOUNT_IFQ gives the highest priority to threads with the
   fewest instructions
               in instruction fetch queue (IFQ).
   ICOUNT_Q gives the highest priority to threads with the
   fewest instructions
               in IFQ and instruction issue queue (IIQ).
   ICOUNT_ALL gives the highest priority to threads with the
   fewest instructions
               in IFQ, IIQ and all pipe stages in the processor.
   ICOUNT_BR gives the lowest priority to threads with the
   most instructions
               in each branch history buffer (BHB).
   ICOUNT_MISS gives the highest priority to threads with the
   fewest instructions
               in load buffer (LB).
   IQOL gives the lowest priority to threads with instructions
   closest to the
               head of IIQ.
   In RR, threads are always selected using a round-robin
   priority scheme.
   * Note: If two or more threads are equally matched in a
   given fetch priority algorithm,
               then the thread with the lowest thread id is given the
   highest priority. */
typedef enum {
    ICOUNT_IFQ,
    ICOUNT_Q,
    ICOUNT_ALL,
    ICOUNT_BR,
    ICOUNT_MISS,
    IQOL,
    RR
} FETCHPRI_TYPE;

/* Issue priority options
   OLDEST gives the highest priority to the instruction closest
   to the head-pointed location,
               and if two or more instructions belong to different
   IIQs and are equally matched
               in entry order, gives the highest priority to the
   instruction with the lowest thread id.
```

ICNT_FU gives the highest priority to the instructions belonging to the thread with the fewest instructions in functional units including the load and store buffers.

ICNT_MISS gives the highest priority to the instructions belonging to the thread with the fewest instructions in the load buffer.

* Note: If two or more threads are equally matched in a given issue priority algorithm, then the instructions with the lowest thread id are given the highest priority. */

```
typedef enum {
    OLDEST,
    ICNT_FU,
    ICNT_MISS
} ISSUEPRI_TYPE;
```

/* Replacement option types */

```
typedef enum {
    RANDOM = 0,          // Relace a random entry
    LRU = 1              // Least Recently Used (LRU) Method
} REPLACE_TYPE;
```

/* Grouping options

Grouping 1: read ports of I\$, read ports of ITLB, and read ports of BTB

Grouping 2: IFQ, decode slots, and IIQ

Grouping 3: instruction issue slots, read and write ports of register file(s), read and write ports of scoreboard array(s), functional units, read ports of D\$, read ports of DTLB */

```
#define GRP1_NUM          1          // The number of groups by
grouping 1
#define GRP2_NUM          8          // The number of groups by
grouping 2
#define GRP3_NUM          1          // The number of groups by
grouping 3
```

/* The number of threads */

```
#define THREAD_NUM        8          // The
number of supported threads
#define THREAD_G1NUM      ((THREAD_NUM)/(GRP1_NUM))//
The number of supported threads per group by grouping 1
#define THREAD_G2NUM      ((THREAD_NUM)/(GRP2_NUM))//
The number of supported threads per group by grouping 2
#define THREAD_G3NUM      ((THREAD_NUM)/(GRP3_NUM))//
The number of supported threads per group by grouping 3
```

/* Fetch options */

```
#define FETCH_WIDTH        8          // The
maximum number of instructions fetched per cycle
#define ICPT_NUM           4          // The total
number of read ports of I$,
#define ICPG_NUM           ((ICPT_NUM) / (GRP1_NUM)) // The
```

```

number of read ports of I$ per group by grouping 1
#define FETG_WIDTH ((FETCH_WIDTH) / (GRP1_NUM)) //
The maximum number of instructions fetched per cycle per group by grouping
1
#define ICP_WIDTH ((FETCH_WIDTH) / (ICPT_NUM)) //
The number of instructions per I$ port
#define ITLBPT_NUM (ICPT_NUM) // The total
number of read ports of ITLB
#define ITLBPG_NUM (ICPG_NUM) // The total
number of read ports of ITLB per group by grouping 1
#define BTBPT_NUM (ICPT_NUM) // The total
number of read ports of BTB
#define BTBPG_NUM (ICPG_NUM) // The total
number of read ports of BTB per group by grouping 1
#define FETCH_PRI (RR) // The rule of fetch
selection priority among threads

/* Decode options */
#define DECODE_WIDTH (FETCH_WIDTH) //
The maximum number of instructions decoded per cycle
#define DECG_WIDTH ((DECODE_WIDTH) / (GRP2_NUM)) // The
maximum number of instructions decoded per cycle per group grouping 2

/* Issue options */
#define ISSUE_WIDTH (FETCH_WIDTH) // The
maximum number of instructions issued per cycle
#define ISSG_WIDTH ((ISSUE_WIDTH) / (GRP3_NUM)) // The
maximum number of instructions issued per cycle per group grouping 3
#define ISSUE_PRI OLDEST // The rule
of issue selection priority among threads

/* Register port options */
#define REGRT_NUM ((ISSUE_WIDTH) * 2) // The number of
register file read ports
#define REGRG_NUM ((REGRT_NUM) / (GRP3_NUM)) // The number of
register file read ports per group by grouping 3
#define REGWT_NUM (ISSUE_WIDTH) // The
number of register file write ports
#define REGWG_NUM ((REGWT_NUM) / (GRP3_NUM)) // The
number of register file write ports per group by grouping 3

/* Register file size options */
#define LREG_NUM 30 // The number of registers in the
register file per thread
// 0..14 (R0..R14, User or System
mode)
// 15..21 (R8_fiq..R14_fiq, Fast
interrupt mode)
// 22..23 (R13_irq..R14_irq, Interrupt
mode)
// 24..25 (R13_svc..R14_svc,
Supervisor mode)
// 26..27 (R13_abt..R14_abt, Abort
mode)
// 28..29 (R13_und..R14_und,

```

```

Undefined mode)
#define PREG_NUM ((LREG_NUM) * (THREAD_NUM)) // The
total number of registers in the register file

/* The total number of system registers
0: FPC (Fetch PC)
1: CPSR (Current Processor State Register)
2: SPSR_fiq (Saved Processor State Register, Fast interrupt mode)
3: SPSR_irq (Saved Processor State Register, Interrupt mode)
4: SPSR_svc (Saved Processor State Register, Supervisor mode)
5: SPSR_abt (Saved Processor State Register, Abort mode)
6: SPSR_und (Saved Processor State Register, Undefined mode) */
#define LSYSREG_NUM 7
#define SYSREG_NUM ((THREAD_NUM) * (LSYSREG_NUM))

/* Queues options */
#define IFQT_SIZE 64 // The total number of IFQ
entries
#define IFQG_SIZE ((IFQT_SIZE) / (GRP2_NUM)) //
The number of IFQ entries of each group by grouping 2
#define IIQT_SIZE 64 // The total number of IIQ
entries
#define IIQG_SIZE ((IIQT_SIZE) / (GRP2_NUM)) //
The number of IIQ entries of each group by grouping 2

/* The number of entries of Branch History Buffer (BHB) of each thread is
infinite (do not use these options, always infinite) */
#define BHB_SIZE infinite // The number of BHB entires of
each thread
#define BHBP_WIDTH infinite // The number of entries read per
cycle from a BHB

/* Branch Target Buffer (BTB) options */
#define BTB_DIVIDE FALSE // whether BTB is
divided for each thread or not
#define BTB_SIZE 128 // The
number of BTB entries
#define BTBWAY_NUM 128 // The
number of Branch Target Buffer Ways
#define BTBSET_NUM ((BTB_SIZE) / (BTBWAY_NUM)) //
The number of Branch Target Buffer Sets
#define BTBPINF_NUM 2 // The
number of instructions per which one data entry is assigned to BTB
#define BTBENTD_NUM (((ICP_WIDTH) / (BTBPINF_NUM)) ?
((ICP_WIDTH) / (BTBPINF_NUM)) : 1) // The number of BTB data entries
per one BTB entry

/* Translatioin Lookaside Buffer (BTB) options */
#define ITLB_SIZE 128 // The number of ITLB entries
(always fully-associative)
#define DTLB_SIZE 128 // The number of DTLB entries
(always fully-associative)

/* The number of Functional units options */

```

```

#define ALUFG_NUM          (ISSG_WIDTH)                // The
number of ALUs per group by grouping 3
#define ALUFT_NUM          ((ALUFG_NUM) * (GRP3_NUM))    //
The total number of ALUs
#define MULFT_NUM          4
// The total number of Multiply Units
#define MULFG_NUM          ((MULFT_NUM) / (GRP3_NUM))    //
The number of Multiply Units per group by grouping 3
#define MEMFG_NUM          ((ISSG_WIDTH) / 2)          // The
number of Load/Store Units per group by grouping 3
#define MEMFT_NUM          ((MEMFG_NUM) * (GRP3_NUM))    //
The total number of Load/Store Units
#define CP15FG_NUM         1                          // The
number of CP15 functional Units per group by grouping 3 (currently not
useless)
#define CP15FT_NUM         ((CP15FG_NUM) * (GRP3_NUM))    //
The total number of CP15 functional Units (currently not useless)
#define FUG_NUM             ((ALUFG_NUM) + (MULFG_NUM) +
(MEMFG_NUM) + (CP15FG_NUM))    // The total number of functional
units per group by group 3
#define FUT_NUM             ((FUG_NUM) * (GRP3_NUM))    //
The total number of functional units

/* I$ options */
#define IC_SIZEFLD         7                          // The
"size" field of the Isize field of the Cache Type register
#define IC_ASSFLD          3                          // The
"assoc" field of the Isize field of the Cache Type register
#define IC_MFLD            0                          //
The "M" field of the Isize field of the Cache Type register
#define IC_LENFLD          2                          // The "len"
field of the Isize field of the Cache Type register
#define ICBLOCK_SIZE       (1 << ((IC_LENFLD) + 3))    // The I$
block size (in bytes) (32 bytes in the above case)
#define ICACHE_SIZE        ((2 + (IC_MFLD)) << ((IC_SIZEFLD) + 8)) //
The number of I$ entries (in bytes) (32Kbtes in the above case)
#define ICWAY_NUM          ((IC_ASSFLD==0) ? ((2 + (IC_MFLD)) / 2) : ((2 +
(IC_MFLD)) << ((IC_ASSFLD) - 1)))    // The number of I$ Ways (4-way
in the above case)
#define ICSET_NUM          (1 << ((IC_SIZEFLD) + 6 - (IC_ASSFLD) -
(IC_LENFLD))) // The number of I$ Sets (256 in the above case)

/* D$ options */
#define DC_SIZEFLD         7                          // The
"size" field of the Dsize field of the Cache Type register
#define DC_ASSFLD          3                          // The
"assoc" field of the Dsize field of the Cache Type register
#define DC_MFLD            0                          //
The "M" field of the Dsize field of the Cache Type register
#define DC_LENFLD          2                          // The "len"
field of the Dsize field of the Cache Type register
#define DCBLOCK_SIZE       (1 << ((DC_LENFLD) + 3))    // The D$
block size (in bytes) (32 bytes in the above case)
#define DCACHE_SIZE        ((2 + (DC_MFLD)) << ((DC_SIZEFLD) + 8)) //
The number of D$ entries (in bytes) (32Kbtes in the above case)

```

```

#define DCWAY_NUM ((DC_ASSFLD==0) ? ((2 + (DC_MFLD)) / 2) : ((2 + (DC_MFLD)) << ((DC_ASSFLD) -1))) // The number of D$ Ways (4-way
in the above case)
#define DCSET_NUM (1 << ((DC_SIZEFLD) + 6 - (DC_ASSFLD) - (DC_LENFLD))) // The number of D$ Sets (256 in the above case)
#define DCPG_NUM (MEMFG_NUM) // The
number of D$ ports per group by grouping 3
#define DCPT_NUM ((DCPG_NUM) * (GRP3_NUM)) // The total
number of D$ ports

/* Masks of the fields of the Cache Type register */
#define IC_SIZE_MSK 0x0000001c0 // the
"size" field mask of the Isize field of the Cache Type register
#define IC_ASS_MSK 0x000000038 // the
"assoc" field mask of the Isize field of the Cache Type register
#define IC_M_MSK 0x000000004 //
the "M" field mask of the Isize field of the Cache Type register
#define IC_LEN_MSK 0x000000003 // the "len"
field mask of the Isize field of the Cache Type register
#define DC_SIZE_MSK 0x0001c0000 // the
"size" field mask of the Dsize field of the Cache Type register
#define DC_ASS_MSK 0x000038000 // the
"assoc" field mask of the Dsize field of the Cache Type register
#define DC_M_MSK 0x000004000 //
the "M" field mask of the Dsize field of the Cache Type register
#define DC_LEN_MSK 0x000003000 // the "len"
field mask of the Dsize field of the Cache Type register

/* Other System options */
#define LITTLE_ENDIAN TRUE // indicates that target
system is little-endian
#define CACHEREP RANDOM // Cache replacment
policy (applies to both I$ and D$)
#define RR_MASK 0x00004000 // RR (CP15 register bit[14],
cache replacement policy) mask
#define TLBREP RANDOM // TLB replacement
policy (applies to both ITLB and DTLB)
#define BTBREP RANDOM // BTB replacement
policy
#define SBUS_WIDTH 32 // The width of the system bus (in
bytes)

/* Overhead options */
#define EXCEPT_CYC 1 // Exception processing cycles (in
cycles) (can fetch instructions the next cycle in this case)
#define MISBR_CYC 1 // Branch misprediction processing
cycles (in cycles) (can fetch instructions the next cycle in this case)
#define ITLBMISS_OH 1 // ITLB miss exception routine
processing cycles in addition to EXCEPT_CYC
#define DTLBMISS_OH 1 // DTLB miss exception routine
processing cycles in addition to EXCEPT_CYC
#define EXTMEMLAT_CYC 5 // The number of cycles
from address output to data return or data write
#define EXTMEMTRF_CYC EXTMEMLAT_CYC // The number of
cycles between consecutive memory accesses

```

```

#define EXTMEMBST_CYC      1                // The number of cycles
between consecutive data transfers in burst mode
#define ICFILL_OH          ((EXTMEMLAT_CYC) + (((ICBLOCK_SIZE) /
(SBUS_WIDTH)) - 1) * (EXTMEMBST_CYC))) // The number of cycles
taken to fill one I$ block
#define DCFILL_OH          ((EXTMEMLAT_CYC) + (((DCBLOCK_SIZE) /
(SBUS_WIDTH)) - 1) * (EXTMEMBST_CYC))) // The number of cycles
taken to fill one D$ block
#define DCEVCT_OH          ((EXTMEMLAT_CYC) + (((DCBLOCK_SIZE) /
(SBUS_WIDTH)) - 1) * (EXTMEMBST_CYC))) // The number of cycles
taken to evict one D$ block

#define TRACE_ENABLE      FALSE

#endif

```


부록 3 트레이스 파일 내용의 일부분

```
***** Turn #      10000:*****
Thread 0:
SREG 0=00008090 SREG 1=20000010 SREG 2=00000000 SREG 3=00000000
SREG 4=00000000 SREG 5=00000000 SREG 6=00000000
REG 0:00022528(0) REG 1:00022528(0) REG 2:00022540(0) REG
3:00022540(0)
REG 4:000253AC(1) REG 5:000286D4(1) REG 6:00000000(0) REG
7:00000000(0)
REG 8:00008098(0) REG 9:00000000(0) REG 10:00000000(0) REG
11:00022503(0)
REG 12:00022527(0) REG 13:00000000(0) REG 14:00000000(0) REG
15:00000000(0)
REG 16:00000000(0) REG 17:00000000(0) REG 18:00000000(0) REG
19:00000000(0)
REG 20:00000000(0) REG 21:00000000(0) REG 22:00000000(0) REG
23:00000000(0)
REG 24:00000000(0) REG 25:00000000(0) REG 26:00000000(0) REG
27:00000000(0)
REG 28:00000000(0) REG 29:00000000(0) ConditionFlags:nzCvq(1)
Thread 1:
SREG 0=00008088 SREG 1=20000010 SREG 2=00000000 SREG 3=00000000
SREG 4=00000000 SREG 5=00000000 SREG 6=00000000
REG 0:00022528(0) REG 1:00022528(0) REG 2:00022540(0) REG
3:00022540(0)
REG 4:000240AC(0) REG 5:000299D4(0) REG 6:00000000(0) REG
7:00000000(0)
REG 8:00008098(0) REG 9:00000000(0) REG 10:00000000(0) REG
11:00022503(0)
REG 12:00022527(0) REG 13:00000000(0) REG 14:00000000(0) REG
15:00000000(0)
REG 16:00000000(0) REG 17:00000000(0) REG 18:00000000(0) REG
19:00000000(0)
REG 20:00000000(0) REG 21:00000000(0) REG 22:00000000(0) REG
23:00000000(0)
REG 24:00000000(0) REG 25:00000000(0) REG 26:00000000(0) REG
27:00000000(0)
REG 28:00000000(0) REG 29:00000000(0) ConditionFlags:nzCvq(0)
Thread 2:
SREG 0=00008090 SREG 1=20000010 SREG 2=00000000 SREG 3=00000000
SREG 4=00000000 SREG 5=00000000 SREG 6=00000000
REG 0:00010314(0) REG 1:00010314(0) REG 2:0001032C(0) REG
3:0001032C(0)
REG 4:00011664(1) REG 5:00000910(1) REG 6:00000000(0) REG
7:00000000(0)
REG 8:00008098(0) REG 9:00000000(0) REG 10:00000000(0) REG
11:000102EF(0)
REG 12:00010313(0) REG 13:00000000(0) REG 14:00000000(0) REG
15:00000000(0)
REG 16:00000000(0) REG 17:00000000(0) REG 18:00000000(0) REG
```

19:00000000(0)
 REG 20:00000000(0) REG 21:00000000(0) REG 22:00000000(0) REG
 23:00000000(0)
 REG 24:00000000(0) REG 25:00000000(0) REG 26:00000000(0) REG
 27:00000000(0)
 REG 28:00000000(0) REG 29:00000000(0) ConditionFlags:nzCvq(1)
 Thread 3:
 SREG 0=00008090 SREG 1=20000010 SREG 2=00000000 SREG 3=00000000
 SREG 4=00000000 SREG 5=00000000 SREG 6=00000000
 REG 0:00010314(0) REG 1:00010314(0) REG 2:0001032C(0) REG
 3:0001032C(0)
 REG 4:00011C3C(0) REG 5:00000338(1) REG 6:00000000(0) REG
 7:00000000(0)
 REG 8:00008098(0) REG 9:00000000(0) REG 10:00000000(0) REG
 11:000102EF(0)
 REG 12:00010313(0) REG 13:00000000(0) REG 14:00000000(0) REG
 15:00000000(0)
 REG 16:00000000(0) REG 17:00000000(0) REG 18:00000000(0) REG
 19:00000000(0)
 REG 20:00000000(0) REG 21:00000000(0) REG 22:00000000(0) REG
 23:00000000(0)
 REG 24:00000000(0) REG 25:00000000(0) REG 26:00000000(0) REG
 27:00000000(0)
 REG 28:00000000(0) REG 29:00000000(0) ConditionFlags:nzCvq(1)
 Scoreboard
 Thread 0:
 REG 0:00008028:(A|00)(R: 0)E REG 1:0000800C:(A|01)(R: 0)E
 REG 2:00008070:(A|00)(R: 0)E REG 3:00008014:(A|01)(R: 0)E
 REG 4:0000808C:(A|01)M(R: 0)E REG 5:00008088:(A|00)M(R: 0)E
 REG 6:00008028:(L|00)(R: 0) REG 7:0000806C:(A|03)(R: 0)E
 REG 8:00008000:(A|00)(R: 0)E REG 9:00000000:(A|00)(R: 0)
 REG 10:00008098:(A|00)(R: 0)E REG 11:00008018:(A|02)(R: 0)E
 REG 12:0000801C:(A|00)(R: 0)E REG 13:00000000:(A|00)(R: 0)
 REG 14:00008068:(A|02)(R: 0)E REG 15:00000000:(A|00)(R: 0)
 REG 16:00000000:(A|00)(R: 0) REG 17:00000000:(A|00)(R: 0)
 REG 18:00000000:(A|00)(R: 0) REG 19:00000000:(A|00)(R: 0)
 REG 20:00000000:(A|00)(R: 0) REG 21:00000000:(A|00)(R: 0)
 REG 22:00000000:(A|00)(R: 0) REG 23:00000000:(A|00)(R: 0)
 REG 24:00000000:(A|00)(R: 0) REG 25:00000000:(A|00)(R: 0)
 REG 26:00000000:(A|00)(R: 0) REG 27:00000000:(A|00)(R: 0)
 REG 28:00000000:(A|00)(R: 0) REG 29:00000000:(A|00)(R: 0)
 CondFl:00008088:(A|00)M(R: 0)E
 Thread 1:
 REG 0:00008028:(A|00)(R: 0)E REG 1:0000800C:(A|01)(R: 0)E
 REG 2:00008070:(A|01)(R: 0)E REG 3:00008014:(A|01)(R: 0)E
 REG 4:0000808C:(A|01)(R: 0)E REG 5:00008088:(A|00)(R: 0)E
 REG 6:00008028:(L|00)(R: 0) REG 7:0000806C:(A|01)(R: 0)E
 REG 8:00008000:(A|00)(R: 0)E REG 9:00000000:(A|00)(R: 0)
 REG 10:00008098:(A|02)(R: 0)E REG 11:00008018:(A|02)(R: 0)E
 REG 12:0000801C:(A|00)(R: 0)E REG 13:00000000:(A|00)(R: 0)
 REG 14:00008068:(A|00)(R: 0)E REG 15:00000000:(A|00)(R: 0)
 REG 16:00000000:(A|00)(R: 0) REG 17:00000000:(A|00)(R: 0)
 REG 18:00000000:(A|00)(R: 0) REG 19:00000000:(A|00)(R: 0)
 REG 20:00000000:(A|00)(R: 0) REG 21:00000000:(A|00)(R: 0)
 REG 22:00000000:(A|00)(R: 0) REG 23:00000000:(A|00)(R: 0)

```

REG 24:00000000:(A|00)(R: 0) REG 25:00000000:(A|00)(R: 0)
REG 26:00000000:(A|00)(R: 0) REG 27:00000000:(A|00)(R: 0)
REG 28:00000000:(A|00)(R: 0) REG 29:00000000:(A|00)(R: 0)
CondFl:00008088:(A|00)(R: 0)E
Thread 2:
REG 0:00008028:(A|02)(R: 0)E REG 1:0000800C:(A|01)(R: 0)E
REG 2:00008070:(A|01)(R: 0)E REG 3:00008014:(A|01)(R: 0)E
REG 4:0000808C:(A|00)R(R: 0)EC REG 5:00008088:(A|01)E(R: 0)E
REG 6:00008028:(L|01)(R: 0) REG 7:0000806C:(A|01)(R: 0)E
REG 8:0000809C:(A|01)(R: 0)E REG 9:00000000:(A|00)(R: 0)
REG 10:00000000:(A|00)(R: 0) REG 11:00008018:(A|02)(R: 0)E
REG 12:0000801C:(A|00)(R: 0)E REG 13:00000000:(A|00)(R: 0)
REG 14:00008068:(A|00)(R: 0)E REG 15:00000000:(A|00)(R: 0)
REG 16:00000000:(A|00)(R: 0) REG 17:00000000:(A|00)(R: 0)
REG 18:00000000:(A|00)(R: 0) REG 19:00000000:(A|00)(R: 0)
REG 20:00000000:(A|00)(R: 0) REG 21:00000000:(A|00)(R: 0)
REG 22:00000000:(A|00)(R: 0) REG 23:00000000:(A|00)(R: 0)
REG 24:00000000:(A|00)(R: 0) REG 25:00000000:(A|00)(R: 0)
REG 26:00000000:(A|00)(R: 0) REG 27:00000000:(A|00)(R: 0)
REG 28:00000000:(A|00)(R: 0) REG 29:00000000:(A|00)(R: 0)
CondFl:00008088:(A|01)E(R: 0)E
Thread 3:
REG 0:00008028:(A|00)(R: 0)E REG 1:0000800C:(A|01)(R: 0)E
REG 2:00008070:(A|01)(R: 0)E REG 3:00008014:(A|01)(R: 0)E
REG 4:0000808C:(A|01)(R: 0)E REG 5:00008088:(A|01)R(R: 0)E
REG 6:00008028:(L|00)(R: 0) REG 7:0000806C:(A|01)(R: 0)E
REG 8:00008098:(A|01)(R: 0)E REG 9:00000000:(A|00)(R: 0)
REG 10:00000000:(A|00)(R: 0) REG 11:00008018:(A|00)(R: 0)E
REG 12:0000801C:(A|00)(R: 0)E REG 13:00000000:(A|00)(R: 0)
REG 14:00008068:(A|01)(R: 0)E REG 15:00000000:(A|00)(R: 0)
REG 16:00000000:(A|00)(R: 0) REG 17:00000000:(A|00)(R: 0)
REG 18:00000000:(A|00)(R: 0) REG 19:00000000:(A|00)(R: 0)
REG 20:00000000:(A|00)(R: 0) REG 21:00000000:(A|00)(R: 0)
REG 22:00000000:(A|00)(R: 0) REG 23:00000000:(A|00)(R: 0)
REG 24:00000000:(A|00)(R: 0) REG 25:00000000:(A|00)(R: 0)
REG 26:00000000:(A|00)(R: 0) REG 27:00000000:(A|00)(R: 0)
REG 28:00000000:(A|00)(R: 0) REG 29:00000000:(A|00)(R: 0)
CondFl:00008088:(A|01)R(R: 0)E
< Queues >
No entry of the corresponding thread is examined
No entry of the corresponding thread is examined
no reason for blocking instructio issue (instruction issued)
no reason for blocking instructio issue (instruction issued)
(Group2:0)
#Fetch queue( 2 entries)#
Entry# 0( 0):V(0000808C)=24847004 TEntry# 1( 0):(00008090)=8AFFFFFC
Entry# 2( 3):(0000808C)=24847004 HEntry# 3( 0):V(00008088)=E255004

#Issue queue( 4 entries)#
Entry# 0( 0):V(00008090)=BHI 0xFFFFFFFF0
Entry# 1( 3):V(0000808C)=STRCS R07,[R04],#+004
TEntry# 2(255):(00008088)=
Entry# 3(255):(00008088)=
Entry# 4(255):(00008090)=
Entry# 5(255):(00008090)=

```

```

HEntry# 6(255):(0000808C)=
Entry# 7(255):(00008088)=

{ALU#000#}
Entry#0( 2)( 0):V(0000808C)=STRCS R07,[R04],#+004
Entry#1( 1)( 0):V(00008090)=BHI 0xFFFFFFFF0
Entry#2( 0)( 0):V(00008088)=SUBS R05,R05,#00000004
Entry#3( 0)( 0):(00008090)=

{ALU#001#}
Entry#0( 3)( 0):V(00008088)=SUBS R05,R05,#00000004
Entry#1( 2)( 0):V(00008088)=SUBS R05,R05,#00000004
Entry#2( 0)( 0):V(0000808C)=STRCS R07,[R04],#+004
Entry#3( 3)( 0):(00008090)=

{ALU#002#}
Entry#0( 3)( 0):(00008088)=
Entry#1( 3)( 0):(00008088)=
Entry#2( 3)( 0):(00008088)=
Entry#3( 3)( 0):(00008088)=

{ALU#003#}
Entry#0( 0)( 0):(0000808C)=
Entry#1( 0)( 0):(0000808C)=
Entry#2( 0)( 0):(0000808C)=
Entry#3( 0)( 0):(0000808C)=

{MUL#000#}
Entry#0(255)( 0):(00000000)=
Entry#1(255)( 0):(00000000)=
Entry#2(255)( 0):(00000000)=
Entry#3(255)( 0):(00000000)=

{LSU#000#}
Entry#0( 2)( 0):V(0000808C)=STRCS R07,[R04],#+004
Entry#1( 0)( 0):(0000808C)=
Entry#2( 0)( 0):V(0000808C)=STRCS R07,[R04],#+004
Entry#3( 3)( 0):(0000808C)=

{LSU#001#}
Entry#0( 3)( 0):(0000808C)=
Entry#1( 3)( 0):(0000808C)=
Entry#2( 3)( 0):(0000808C)=
Entry#3( 3)( 0):(0000808C)=

{COP#000#}
Entry#0(255)( 0):(00000000)=
Entry#1(255)( 0):(00000000)=
Entry#2(255)( 0):(00000000)=
Entry#3(255)( 0):(00000000)=

***** fd_except 0(0) *****
Excepted instruction address : 00008000
Fault address                : 00008000
Exception type flags         :

```

```

Abort type flags          : 00000003
***** fd_except 1(0) *****
Excepted instruction address : 00008000
Fault address              : 00008000
Exception type flags      :
Abort type flags          : 00000003
***** fd_except 2(0) *****
Excepted instruction address : 00008000
Fault address              : 00008000
Exception type flags      :
Abort type flags          : 00000003
***** fd_except 3(0) *****
Excepted instruction address : 00008000
Fault address              : 00008000
Exception type flags      :
Abort type flags          : 00000003
*****

```

```

***** exception_manage 0(0)(1) *****
Excepted instruction address : 0000808C
Fault address                : 00025000
Exception type flags         : D
Abort type flags             : 00000003
***** exception_manage 1(0)(1) *****
Excepted instruction address : 0000808C
Fault address                : 00024000
Exception type flags         : D
Abort type flags             : 00000003
***** exception_manage 2(0)(1) *****
Excepted instruction address : 0000808C
Fault address                : 00011400
Exception type flags         : D
Abort type flags             : 00000003
***** exception_manage 3(0)(1) *****
Excepted instruction address : 0000808C
Fault address                : 00011C00
Exception type flags         : D
Abort type flags             : 00000003
*****

```

```

***** unpb 0(0) *****
inst address : 0000809C
instruction :
***** unpb 1(0) *****
inst address : 0000809C
instruction :
***** unpb 2(0) *****
inst address : 00000000
instruction :
***** unpb 3(0) *****
inst address : 00000000
instruction :
*****

```

```

*****

```

```

bus_busy_cycle      :    0
*****
ic_repl_port_busy  :    0
*****
dc_repl_port_busy   :    1
*****
tlb_latency[00]    :    0
tlb_latency[01]    :    0
tlb_latency[02]    :    0
tlb_latency[03]    :    0
*****
icache_latency[00]:    0
icache_latency[01]:    0
icache_latency[02]:    0
icache_latency[03]:    0
*****
*** Flag Report ***
  Thread No. :0123
itlb missed :
icache missed:
branch missed:
except taken :
fd excepted  :
unps occur   :
unbs occur   :
unbs pend    :
moddec pend  :
moddec clear :
pload pend   :

```

부록 4 결과 파일의 예

```

=====
=====+
|*****|
|***** FINAL REPORT*****|
|*****|
|*****|
=====
=====+
|
|                                     |
|                               < Option Setting >                               |
|                                     |
+-----+
+-----+-----+-----+-----+
+-----+
| Supported thread # : 8 | Grp1 num : 1 | Grp2 num : 1 | Grp3 num : 1 |
+-----+-----+-----+-----+
+-----+
| Thread Number | 8 | 8 | 8 |
+-----+-----+-----+
+-----+
+-----+
|                                     *** Fetch Option ***                               |
+-----+-----+-----+
+-----+
| Fetch width : ( 8/ 8(G)) | 2 inst/port | Fetch Priority :RR |
+-----+-----+-----+
+-----+
| I$ rdport ( 4/ 4(G)) | ITLB rdport( 4/ 4(G)) | BTB rdport ( 4/ 4(G)) |
+-----+-----+-----+
+-----+
+-----+
|                                     *** Decode Option ***                               |
+-----+-----+-----+
+-----+
| Decode width : ( 8/ 8(G2)) | BTB for each thread : FALSE |
+-----+-----+-----+
+-----+
+-----+
|                                     *** Issue Option ***                               |
+-----+-----+-----+
+-----+
| Issue width : ( 8/ 8(G3)) | Issue Priority :OLDEST |
+-----+-----+-----+
+-----+

```

```

-----+
|                                     *** Register file port Option ***                                     |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
| Reg.File Readport : (16/16(G3)) | Reg.File Writeport : ( 8/ 8(G3)) |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
|                                     *** Queue Option ***                                     |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
| Inst. Fetch Queue : ( 64/ 64(G2)) | Inst. Issue Queue : ( 64/ 64(G2)) |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
|                                     *** BTB Option ***                                     |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
| Size : 128 | Way : 128 | Set : 1 | entry/inst : 2 | entry : 1 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
|                                     *** TLB Option ***                                     |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
| ITLB Size : 128 | DTLB_SIZE : 128 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
|                                     *** Functional Unit Option ***                                     |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
| ALU ( 8/ 8(G3)) | MUL ( 1/ 1(G3)) | LSU ( 4/ 4(G3)) | CP ( 1/ 1(G3)) |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
|                                     *** Instruction Cache Option ***                                     |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
| Way : 4 | Set: 512 | Block : 32 | Rdport( 4/ 4(G3)) | Size: 64kB |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
|                                     *** Data Cache Option ***                                     |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
| Way : 4 | Set: 512 | Block : 32 | Rdport( 4/ 4(G3)) | Size: 64kB |

```



```

+-----+-----+-----+-----+-----+
+-----+
+-----+
+-----+
|                                     *** Miscellaneous Option ***                                     |
+-----+-----+-----+-----+-----+
+-----+
| Endian : Little|Cache Repl.: RANDOM|TLB Repl: RANDOM|BTB Repl:
RANDOM|
+-----+-----+-----+-----+-----+
+-----+
+-----+
+-----+
| System Bus Width : 32| ITLB miss penalty : 1| DTLB miss penalty : 1|
+-----+-----+-----+-----+-----+
+-----+
+-----+
+-----+
| I$ refill penalty : 5| D$ refill penalty : 5| D$ eviction penalty : 5|
+-----+-----+-----+-----+-----+
+-----+
+-----+
+-----+
|                                     < Simulation Result >                                     |
|                                     |                                     |
| Total simulation cycle :           10000000 cycle                                     |
+-----+-----+-----+-----+-----+
+-----+
+-----+-----+-----+-----+-----+-----+-----+
+-----+
| Thread | fetched inst | Ratio | Flushed | ITLB missed | DTLB missed |
+-----+-----+-----+-----+-----+-----+-----+
+-----+
| 0 | 8006368 | 0.8006 | 378898 | 10 | 250 |
| 1 | 7894385 | 0.7894 | 376213 | 14 | 262 |
| 2 | 7778778 | 0.7779 | 374086 | 159 | 230 |
| 3 | 8437855 | 0.8438 | 390532 | 120 | 216 |
| 4 | 7807714 | 0.7808 | 372630 | 9 | 251 |
| 5 | 8366650 | 0.8367 | 388838 | 131 | 183 |
| 6 | 7313467 | 0.7313 | 362962 | 81 | 265 |
| 7 | 7264398 | 0.7264 | 358366 | 79 | 274 |
+-----+-----+-----+-----+-----+-----+
+-----+
| Group2 | fetched inst | Ratio | Flushed | ITLB missed | DTLB missed |
+-----+-----+-----+-----+-----+-----+
+-----+
| 0 | 62869615 | 6.2870 | 3002525 | 603 | 1931 |
+-----+-----+-----+-----+-----+-----+
+-----+
| Total | fetched inst | Ratio | Flushed | ITLB missed | DTLB missed |
+-----+-----+-----+-----+-----+-----+
+-----+
| 0 | 62869615 | 6.2870 | 3002525 | 603 | 1931 |
+-----+-----+-----+-----+-----+-----+

```

Thread	issued inst	Ratio	executed inst	Ratio		
0	6682183	0.6682	5861653	0.5862		
1	6611758	0.6612	5798020	0.5798		
2	6279159	0.6279	5471198	0.5471		
3	6849966	0.6850	6110692	0.6111		
4	6552192	0.6552	5745105	0.5745		
5	6807081	0.6807	6077628	0.6078		
6	6142275	0.6142	5296547	0.5297		
7	6075446	0.6075	5245286	0.5245		
Group3	issued inst	Ratio	executed inst	Ratio		
0	52000060	5.2000	45606129	4.5606		
Total	issued inst	Ratio	executed inst	Ratio		
	52000060	5.2000	45606129	4.5606		
Instruction Cache	Accessed	Missed	Ratio			
	36070982	143781	0.0040			
Data Cache	Accessed	Missed	Ratio			
	15756379	1404425	0.0891			
Thread	Branch	Taken Hit	Ratio	Taken Miss	Ratio	
0	1712275	836721	0.4887	80348	0.0469	
1	1688651	821748	0.4866	79138	0.0469	
2	1210235	452208	0.3737	99074	0.0819	
3	831874	218578	0.2628	54191	0.0651	
4	1669285	810926	0.4858	78106	0.0468	
5	828298	217382	0.2624	53952	0.0651	

	6		1353806		594632		0.4392		98428		0.0727
	7		1340179		592129		0.4418		97342		0.0726
+-----+											
-----+											
	Total		Branch		Taken Hit		Ratio		Taken Miss		Ratio
+-----+											
			10634603		7636931		0.4273		640579		0.0602
+-----+											
+-----+											
	Thread		Branch		Not Taken Hit		Ratio		Not Taken Miss		Ratio
+-----+											
	0		1712275		497334		0.2905		297872		0.1740
	1		1688651		491384		0.2910		296381		0.1755
	2		1210235		384539		0.3177		274414		0.2267
	3		831874		223265		0.2684		335840		0.4037
	4		1669285		486407		0.2914		293846		0.1760
	5		828298		222562		0.2687		334402		0.4037
	6		1353806		396798		0.2931		263948		0.1950
	7		1340179		390318		0.2912		260390		0.1943
+-----+											
-----+											
	Total		Branch		Not Taken Hit		Ratio		Not Taken Miss		Ratio
+-----+											
			10634603		3092607		0.2908		2357093		0.2216
+-----+											
+-----+											
	Thread		Predicted		Ratio		Predict Missed		Ratio		
+-----+											
	0		917071		0.0917		156502		0.1707		
	1		900889		0.0901		153858		0.1708		
	2		551282		0.0551		185215		0.3360		
	3		272769		0.0273		98789		0.3622		
	4		889033		0.0889		152230		0.1712		
	5		271334		0.0271		97882		0.3607		
	6		693061		0.0693		192705		0.2780		
	7		689471		0.0689		190666		0.2765		
+-----+											
-----+											
	Total		Predicted		Ratio		Predict missed		Ratio		BTB Repl.
+-----+											
			5184910		0.5185		1227847		0.2368		2084911
+-----+											
+-----+											
			Branch Hit Ratio		Branch Miss Ratio		BTB Replacement Ratio				

ABSTRACT

Study of an In-order SMT Architecture and Grouping Schemes

Moon, Byung In

Dept. of Electrical and Electronic Eng.

The Graduate School

Yonsei University

This dissertation proposes a simultaneous multithreading (SMT) architecture that improves instruction throughput by exploiting instruction level parallelism (ILP) and thread level parallelism (TLP). The proposed architecture issues and completes instructions belonging to the same thread in exact program order. That policy greatly reduces the design complexity and hardware cost of the SMT architecture, compared with the one adopting out-of-order issue and completion. On the other hand, when the instructions belong to the different threads, the issue and completion orders for those instructions may not necessarily be the same with the fetch order. The architecture issues instructions simultaneously from multiple threads to functional units by exploiting ILP and TLP, and by dynamic resource sharing. That parallel execution notably improves performance and resource utilization with minimal additional hardware cost over the superscalar conventional architecture.

While SMT has excellent potential to improve processor utilization and performance, it can add substantial complexity to the design. To reduce design complexity, the dissertation presents grouping schemes. Without grouping, all

threads dynamically and flexibly share most resources. On the other hand, in the SMT architecture with grouping, in which resources and threads are divided into several groups for the purpose of reducing design complexity, resources are shared only among threads belonging to the same group as those resources.

An execution-driven, cycle-based simulator written in C language is implemented to model in detail the SMT architecture. It has a variety of configuration parameters in order to simulate SMT processors with various organizations. The simulator adopts a 32-bit instruction set architecture (ISA), and the workload is from the SPEC CPU2000 benchmark suite and the ADS (ARM Developer Suite).

The simulation results show that SMT processors are cost-effective when the number of threads is equal to that of instructions that can be issued in one cycle. To minimize inter-thread conflict misses, the number of cache ways which is larger than half that of supported threads is recommended in the SMT architecture. And reasonable grouping reduces the design complexity of SMT processors with little performance decrease.

Using the simulation results, this dissertation proposes two SMT architectures for embedded and high-performance applications, respectively. With small hardware overhead, the two SMT architectures improve IPC (Instructions Per Cycle) by three or more times over the superscalar architectures with comparable resources and policies. Also, network processors adopting the SMT architecture proposed for high-performance applications are expected to have sufficient competitiveness compared with network processors already released in the market.

Key words : multithreading, SMT, superscalar, ILP, TLP, dynamic resource sharing, in-order issue and completion, grouping